代码随想录知识星球精华（最强八股文）第四版（Java篇））

代码随想录知识星球精华（最强八股文）第四版为九份PDF，分别是：

- 代码随想录知识星球八股文概述
- C++篇
- go篇
- Java篇
- 前端篇
- 算法题篇
- 计算机基础篇
- 问答精华篇
- 面经篇

本篇为最强八股文之Java篇。

Java基础

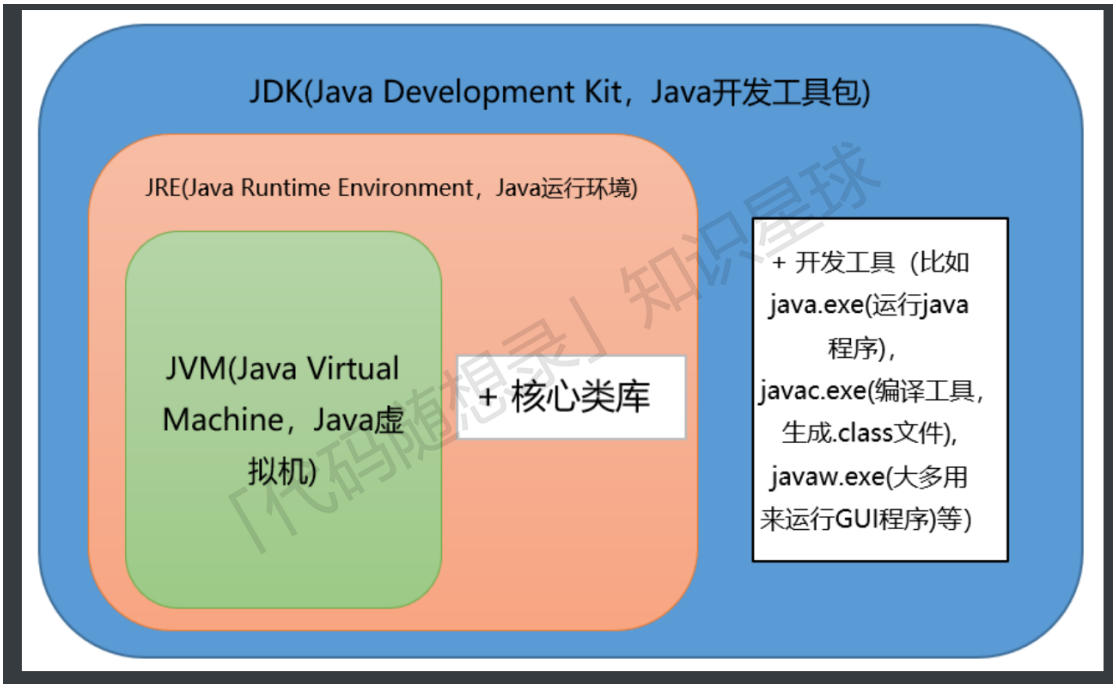
Java概述

1、JVM、JRE、JDK之间的关系

我们知道Java一次编写到处运行，可移植性好，保证这一点的就是java虚拟机JVM

JRE是运行环境，不能创建新程序。他是包括JVM的

JDK是功能最齐全的，包括编译器和各种工具，我们写代码就需要这个了。



2、public、protected、default、private的区别

（1）default是默认的，什么都不写，在同一个包内是可见的，不使用任何修饰符。

（2）public能用来修饰类，在一个java源文件中只能有一个类被声明为public，而且一旦有一个类为public，那这个java源文件的文件名就必须要和这个被public所修饰的类的类名相同，否则编译不能通过。public用来修饰类中成员（变量和方法），被public所修饰的成员可以在任何类中都能被访问到。

（3）protected是受保护的，受到该类所在的包所保护，被protected所修饰的成员会被位于同一package中的所有类访问到。同时，被protected所修饰的成员也能被该类的所有子类继承下来。

（4）private，private是私有的，即只能在当前类中被访问到，它的作用域最小。

修饰符	当前类	同一包内	子孙类(同一包)	子孙类(不同包)	其他包
public	Y	Y	Y	Y	Y
protected	Y	Y	Y	Y/N（说明）	N
default	Y	Y	Y	N	N
private	Y	N	N	N	N

3、final、finally、finalize的区别

（1）final就是不可变的意思，可以修饰变量、方法和类。修饰变量时，这个变量必须初始化，所以也称为常量。

（2）finally是异常处理的一部分，只能用在try/catch中，并且附带一个语句块表示这段语句一定会被执行，无论是否抛出异常。

（3）finalize是java.lang.Object中的方法，也就是每一个对象都有这个方法，一个对象的finalize方法只会调用一次，调用了不一定被回收，因为只有对象被回收的时候才会被回收，就会导致前面调用，后面回收的时候出现问题，不推荐使用。

4、static关键字啥作用?

这就要提到new对象，只有new对象之后，数据存储空间才会被分配，方法才能供外界调用。但是当没有创建对象的时候也想要调用方法或者就是想为特定分配存储空间的时候，就需要用static。所以有了static，成员变量或者方法就可以在没有所属类的时候被访问了。

5、面向对象、面向过程

面向过程的性能比较高，因为没有实例化等操作，开销比较小。

面向对象因为有了封装继承多态的特性，可以设计出低耦合的系统，使得系统更灵活、容易维护。

封装是指封装成抽象的类，并且对于可信的类或者对象，是可以操作的，对于不可信的进行隐藏。

继承是指可以使用现有类的所有功能，而且还可以在现有功能的基础上做拓展。

多态是基于继承的，他是指父类中定义的属性和方法被子类继承之后，可以具有不同的数据类型或者表现出不同的行为，使得同一个属性在父类及其子类中具有不同的含义。

重载就是多态的一个例子，是编译时的多态。其实我们所说的多态是运行时多态，也就是说编译的时候不确定调用哪个具体方法，一直延迟到运行时才可以确定，所以多态又叫延迟方法。

Java实现多态有 3 个必要条件：继承、重写和向上转型。只有满足这 3 个条件，开发人员才能够在同一个继承结构中使用统一的逻辑实现代码处理不同的对象，从而执行不同的行为。

- 继承：在多态中必须存在有继承关系的子类和父类。
- 重写：子类对父类中某些方法进行重新定义，在调用这些方法时就会调用子类的方法。
- 向上转型：在多态中需要将子类的引用赋给父类对象，只有这样该引用才既能可以调用父类的方法，又能调用子类的方法。

那，重载和重写都是实现多态的方式，区别是啥?

区别在于重载是编译时多态，重写是运行时多态。

- 重载(overloading) 是在一个类里面，方法名字相同，而参数不同。返回类型可以相同也可以不同。每个重载的方法（或者构造函数）都必须有一个独一无二的参数类型列表。最常用的地方就是构造器的重载。

- 重写发生在子类与父类之间，重写方法返回值和形参都不能改变，与方法返回值和访问修饰符无关，即重载的方法不能根据返回类型进行区分。**即外壳不变，核心重写！**

ArrayList

ArrayList 底层就是一个 Object[] 数组

ArrayList 底层数组默认初始化容量为 10

1、jdk1.8 中 ArrayList 底层先创建一个长度为 0 的数组

2、当第一次添加元素（调用 add() 方法）时，会初始化为一个长度为 10 的数组

当 ArrayList 中的容量使用完之后，则需要对容量进行扩容：

1、ArrayList 容量使用完后，会“自动”创建容量更大的数组，并将原数组中所有元素拷贝过去，这会导致效率降低

2、优化：可以使用构造方法 ArrayList (int capacity) 或 ensureCapacity(int capacity) 提供一个初始化容量，避免刚开始就一直扩容，造成效率较低

ArrayList 构造方法：

1. ArrayList()：创建一个初始化容量为 10 的空列表
2. ArrayList(int initialCapacity)：创建一个指定初始化容量为 initialCapacity 的空列表
3. ArrayList(Collection<? extends E> c)：创建一个包含指定集合中所有元素的列表

ArrayList 特点：

优点：

1. 向 ArrayList 末尾添加元素（add() 方法）时，效率较高
2. 查询效率高

缺点：

1. 扩容会造成效率较低（可以通过指定初始化容量，在一定程度上对其进行改善）
2. 另外数组无法存储大数据量（因为很难找到一块很大的连续的内存空间）
3. 向 ArrayList 中间添加元素（add(int index)），需要移动元素，效率较低
    1. 但是，向 ArrayList 中国位置增/删元素的情况较少时不影响；
    2. 如果增/删操作较多，可考虑改用链表

Linklist

LinkedList 特点

数据结构： LinkedList 底层是一个双向链表

优点： 增/删效率高

缺点： 查询效率较低

LinkedList 也有下标，但是内存不一定是连续的（类似C++重载[]符号，将循位置访问模拟为循秩访问）

LinkedList 可以调用 get(int index) 方法，返回链表中第 index 个元素

但是，**每次查找都要从头结点开始遍历**

LinkedList 部分源码解读

add()方法

```java
public class LinkedList<E> extends ... implements ... {
    transient int size = 0;  // 链表长度
    transient Node<E> first; // 指向链表第一个节点
    transient Node<E> last;  // 指向链表最后一个节点

    public boolean add(E e) { // 添加元素
        linkLast(e); // 向链表末尾添加元素 e
        return true;
    }

    void linkLast(E e) { // 向链表末尾添加元素 e
        final Node<E> l = last; // 暂时保存最后一个元素的指针
        final Node<E> newNode = new Node<>(l, e, null);
        last = newNode; // newNode 作为最后一个节点
        if (l == null) // 当前链表为空
            first = newNode; // 第一个添加的节点，即为 first
        else // 链表不空
            l.next = newNode; // 当前链表的最后一个节点next 指向newNode
        size++;
        modCount++;
    }

    // LinkedList 底层是一个双向链表
    private static class Node<E> { // LinkedList 的节点静态内部类 Node
        E item;
        Node<E> next; // 后继
        Node<E> prev; // 前驱

        Node(Node<E> prev, E element, Node<E> next) {
            this.item = element;
            this.next = next;
            this.prev = prev;
        }
```

posper

ListIterator 接口

1、LinkedList.add 方法只能将数据添加到链表的末尾

2、如果要将对象添加到链表的中间位置

则需要使用 ListIterator 接口的 add 方法

1. ListIterator 是 Iterator 的一个子接口

Modifier and Type	Method and Description
void	add(E e) 将指定的元素插入列表（可选操作）。
boolean	hasNext() 返回 true如果遍历正向列表，列表迭代器有多个元素。
boolean	hasPrevious() 返回 true如果遍历反向列表，列表迭代器有多个元素。
E	next() 返回列表中的下一个元素，并且前进光标位置。
int	nextIndex() 返回随后调用 next()返回的元素的索引。
E	previous() 返回列表中的上一个元素，并向后移动光标位置。
int	previousIndex() 返回由后续调用 previous()返回的元素的索引。
void	remove() 从列表中删除由 next()或 previous()返回的最后一个元素（可选操作）。
void	set(E e) 用 指定的元素替换由 next()或 previous()返回的最后一个元素（可选操作）。

posper

3、Iterator 中 remove 方法

1. 调用 next 之后，remove 方法删除的是迭代器左侧的元素（类似键盘的 backspace）
2. 调用 previous 之后，remove 删除的是迭代器右侧的元素

4、ListIterator 中 add 方法

1. 调用 next 之后，在迭代器左侧添加一个元素
2. 调用 previous 之后，add 是在迭代器右侧添加元素

Vector

Vector 底层是数组

初始化容量为 10

扩容： 原容量使用完后，会进行扩容。新容量扩大为原始容量的 2 倍

Vector 是线程安全的（里面方法都带有 synchronized 关键字），效率较低，现在使用较少

如何将 ArrayList 变成线程安全的?

调用 Collections 工具类中的 static List synchronizedList(List list) 方法

Set

泛型：

1、jdk 1.5 引入，之前都是使用 Object[]

2、使用 Object[] 的缺点（2个）

1. 获取一个值时必须进行强制类型转换
2. 调用一个方法前必须使用 instanceof 判断对象类型

泛型的好处：

1、减少了强制类型转换的次数

获取数据值更方便

2、类型安全

调用方法时更安全

3、泛型只在编译时期起作用

运行阶段 JVM 看不见泛型类型（JVM 只能看见对应的原始类型，因为进行了类型擦除）

4、带泛型的类型

在使用时没有指定泛型类型时，默认使用 Object 类型

```
List list = new HashTestrrayList(); // 默认可以放任意 Object 类型
```

5、lambda 表达式

HashSet

特点：HashSet 无序（没有下标），不可重复

TreeSet

TreeSet 无序（没下标），不可重复，但是可以排序

HashSet 为 HashMap 的 key 部分；TreeSet 为 TreeMap 的 key 部分。

所以，这里没有重点将。重点掌握 HashMap 和 TreeMap。

Map

1. Map 和 Collection 没有继承关系
2. Map 以 (key ,value) 的形式存储数据：键值对
   - key 和 value 存储的都是对象的内存地址（引用）

Map 接口常见方法：

方法签名	功能
Set<K> keySet()	返回 map 中得 key 集合（Set）
Collection<V> values()	返回 map 中的 value 集合(Collection)
Set<Map.Entry<K, V>> entrySet()	返回 map 对应的 Map.Entry 集合（set）
V put(K key, V value)	向 map 中添加键为 key，值为 value的元素（map中有key，则更新）
V get(Object key)	返回 map 中 key 对应的 value；如果 map 中不含 key，则返回 null
default V **getOrDefault**(Objectl) key, V] defaultValue)	返回 map 中 key 对应的 value；如果 map 中不含 key，则返回 defaultValue
V remove(Object key)	移除 map 中 key
void clear()	清空 map
boolean isEmpty()	判空（判断 map 中 size 是否为 0）
boolean containsKey(Object key)	查看 map 中是否包含 key
boolean containsValue(Object value)	查看 map 中是否包含 value

「代码随想录」知识星球

posper

注意：

Map.Entry<K, V> 是 Map 的一个接口。接口中的内部接口默认是 public static 的。

```
public class HashMap extends  AbstractMap<K,V> implements Map<K,V>, Cloneable, Serializable {
    static class Node<K,V> implements Map.Entry<K,V> { // 静态内部类 Node
        final int hash;     // 哈希值（由 key 经过哈希函数计算得到）
        final K key;        // key 是 final 修饰
        V value;            // value 和 key 不能用泛型类型在 Map.Etry<K, V> 接口定义
        Node<K,V> next;     // 下一个节点的地址

        transient Node<K,V>[] table; // 哈希表

        static final int DEFAULT_INITIAL_CAPACITY = 1 << 4; // 默认初始化容量（必须是 2 的次幂）
        static final float DEFAULT_LOAD_FACTOR = 0.75f;     // 默认加载因子
        static final int TREEIFY_THRESHOLD = 8;             // 单链表元素超过8个，则转变为红黑树
        static final int UNTREEIFY_THRESHOLD = 6;           // 红黑树节点数量小于6时，会重新变为单链表

        Node(int hash, K key, V value, Node<K,V> next) {
            this.hash = hash;
            this.key = key;
            this.value = value;
            this.next = next;
        }

        public final K getKey()        { return key; }
        public final V getValue()      { return value; }

        ..... // 省略
    }
}
```

posper

Map的遍历方式

第一类方法

先获取map 的 keySet，然后取出 key 对应的 value

特点：

效率相对较低。（因为还要根据 key 从哈希表中查找对应的 value）

方法1：通过 foreach 遍历 map.keySet()，取出对应的 value

```
public static void printMap1(Map<Integer, String> map) {
    if (map.size() == 0) {
        System.out.println("the map is empty...");
        return;
    }
    System.out.println("the elements in the map are like below...");
    for (Integer key : map.keySet()) {
        System.out.println(key + " : " + map.getOrDefault(key, ""));
    }
}
```

posper

方法2：通过迭代器迭代 map.keySet()，来取出对应的 value

```java
public static void printMap2(Map<Integer, String> map) {
    Set<Integer> keySet = map.keySet();
    Iterator<Integer> it = keySet.iterator();
    while (it.hasNext()) {
        Integer cntKey = it.next();
        System.out.println(cntKey + " : " + map.getOrDefault(cntKey, ""));
    }
}
```

posper

第二类方法

调用 map.entrySet()方法，获取 entrySet，然后直接从 entrySet 中获取 key 和 value。

特点：

1. 效率较高（直接从 node 中获取key，value）
2. 适用于大数据量 map 遍历

方法3：调用 map.entrySet()，然后使用 foreach 遍历 entrySet

```java
public static void printMap3(Map<Integer, String> map) {
    Set<Map.Entry<Integer, String>> entry = map.entrySet();
    for (Map.Entry<Integer, String> it : entry) {
        System.out.println(it.getKey() + " : " + it.getValue());
    }
}
```

posper

方法4：调用 map.entrySet()，然后使用迭代器遍历 entrySet

```java
public static void printMap4(Map<Integer, String> map) {
    Set<Map.Entry<Integer, String>> entrySet = map.entrySet();
    Iterator<Map.Entry<Integer, String>> it = entrySet.iterator();
    while (it.hasNext()) {
        Map.Entry<Integer, String> node = it.next();
        System.out.println(node.getKey() + " : " + node.getValue());
    }
}
```

posper

HashMap

HashMap 概述

1. HashMap 底层是一个数组
2. 数组中每个元素是一个单向链表（即，采用拉链法解决哈希冲突）
   - 单链表的节点每个节点是 Node<K, V> 类型（见下源码）
3. 同一个单链表中所有 Node 的 hash值不一定一样，但是他们对应的数组下标一定一样
   - 数组下标利用哈希函数/哈希算法根据 hash值计算得到的
4. HashMap 是数组和单链表的结合体
   1. 数组查询效率高，但是增删元素效率较低
   2. 单链表在随机增删元素方面效率较高，但是查询效率较低
   3. HashMap 将二者结合起来，充分它们各自的优点
5. HashMap 特点
   1. 无序、不可重复
   2. 无序：因为不一定挂在那个单链表上了
6. 为什么不可重复？
   - 通过重写 equals 方法保证的

HashMap 部分源码解析

```
public class HashMap extends  AbstractMap<K,V> implements Map<K,V>, Cloneable, Serializable {
    static class Node<K,V> implements Map.Entry<K,V> { // 静态内部类 Node
        final int hash;     // 哈希值（由 key 经过哈希函数计算得到）
        final K key;        // key 是 final 修饰
        V value;            // value 和 key 不能用泛型类型在 Map.Etry<K, V> 接口定义
        Node<K,V> next;     // 下一个节点的地址

        transient Node<K,V>[] table; // 哈希表

        static final int DEFAULT_INITIAL_CAPACITY = 1 << 4; // 默认初始化容量（必须是 2 的次幂）
        static final float DEFAULT_LOAD_FACTOR = 0.75f;     // 默认加载因子
        static final int TREEIFY_THRESHOLD = 8;             // 单链表元素超过8个，则转变为红黑树
        static final int UNTREEIFY_THRESHOLD = 6;           // 红黑树节点数量小于6时，会重新变为单链表

        Node(int hash, K key, V value, Node<K,V> next) {
            this.hash = hash;
            this.key = key;
            this.value = value;
            this.next = next;
        }

        public final K getKey()        { return key; }
        public final V getValue()      { return value; }

        ..... // 省略
    }
}
```

posper

1. HashMap 默认初始化容量： 16

    1. 必须是 2 的次幂，这也是 jdk 官方推荐的
    2. 这是因为达到散列均匀，为了提高 HashMap 集合的存取效率，所必须的
2. HashMap 默认加载因子：0.75
    - 数组容量达到 3/4 时，开始扩容
3. JDK 8 之后，对 HashMap 底层数据结构（单链表）进行了改进
    1. 如果单链表元素超过8个，则将单链表转变为红黑树；
    2. 如果红黑树节点数量小于6时，会将红黑树重新变为单链表。

这种方式也是为了提高检索效率，二叉树的检索会再次缩小扫描范围。提高效率。

put() 方法原理

1. 先将 key, value 封装到 Node 对象中
2. 底层会调用 key 的 hashCode() 方法得出 hash 值
3. 通过哈希函数/哈希算法，将 hash 值转换为数组的下标
    - 如果下标位置上没有任何元素，就把 Node 添加到这个位置上；
    - 如果下标位置上有但链表，此时会将当前 Node 中的 key 与链表上每一个节点中的 key 进行 equals 比较
        - 如果所有的 equals 方法返回都是 false，那么这个新节点 Node 将被添加到链表的末尾；
        - 如果其中有一个 equals 返回了 true，那么链表中对应的这个节点的 value 将会被新节点 Node 的 value 覆盖。（保证了不可重复）

注：

1. HashMap 中允许 key 和 value 为 null，但是只能有一个（不可重复）！
2. HashTable 中 key 和 value 都不允许为 null。

get() 方法原理

1. 先调用 key 的 hashCode() 方法得出 hash 值
2. 通过哈希函数/哈希算法，将 hash 值转换为数组的下标
3. 通过数组下标快速定位到数组中的某个位置：
    - 如果这个位置上什么也没有（没有链表），则返回 null；
    - 如果这个位置上有单链表，此时会将当前 Node 中的 key 与链表上每一个节点中的 key 进行 equals 比较。
        - 如果所有的 equals 方法返回都是 false，那么 get 方法返回 null；
        - 如果其中有一个 equals 返回了 true，那么这个节点的 value 便是我们要找的 value，此时 get 方法最终返回这个要找的 value。

注：

1. 放在 HashMap 中 key 的元素（或者放在 HashSet 中的元素）需要同时重写 hashCode() 和 equals() 方法！！！

同时重写 hashCode() 和 equals() 方法

- 重写 hashCode() 方法时要达到散列分布均匀！！！
    - 如果 hashCode() 方法返回一个固定的值，那么 HashMap 底层则变成了一个单链表；
    - 如果 hashCode() 方法所有返回的值都不同，此时 HashMap 底层则变成了一个数组。

这两种情况称之为，散列分布不均匀

equals 和 hashCode方法一定要同时重写（直接用 eclipse 生成就行）

```
public class Student {
    String name;

    public Student(String name) {
        this.name = name;
    }

    ... // 重写 equals() 方法

    ... // 重写 hashCode() 方法
}

main() {
    Set<Student> set = new HashSet<>();
    Student stu1 = new Student("Amy");
    Student stu2 = new Student("Amy");
    set.add(stu1);
    set.add(stu2);
    // 如果只从重写 equals，但是未重写 hashCode 方法，这里 size 将会是 2
    // 所以，equals 和 hashCode方法一定要同时重写（直接用 eclipse 生成就行）
    System.out.println("size = " + set.size());
    // System.out.println(set); // HashSet 没有重写 toString
}
```

「代码随想录」知识星球

posper

HashTable & Properties

1. HashTable
2. Properties

Properties 的 key 和 values 都是 String
常用方法
String getProperty(String key)
Object setProperty(String key, String value)

TreeMap

TreeMap 概述

1. TreeSet/TreeMap 是自平衡二叉树
2. TreeSet/TreeMap 迭代器采用的是中序遍历方式

TreeMap 特点

无序，不可重复，但是可排序

排序规则

TreeSet/TreeMap中key 可以自动对 String 类型或8大基本类型的包装类型进行排序

TreeSet 无法直接对自定义类型进行排序

直接将自定义类型添加到 TreeSet/TreeMap中key 会报错 java.lang.ClassCastException

原因： 是因为自定义没有实现 java.lang.Comparable 接口（此时，使用的是 TreeSet 的无参构造器）

对 TreeSet/TreeMap 中 key部分 元素，必须要指定排序规则。主要有两种解决方案：

方法一： 放在集合中的自定义类型实现 java.lang.Comparable 接口，并重写 compareTo 方法

方法二： 选择 TreeSet/TreeMap 带比较器参数的构造器，并从写比较器中的 compare 方法

此时，在传递比较器参数给 TreeSet/TreeMap 构造器时，有 3 种方法：

1. 定义一个 Comparator 接口的实现类
2. 使用匿名内部类
3. lambda 表达式（Comparator 是函数式接口）
   - 利用 -> 的 lambda表达式 重写 compare 方法
   - 利用 Comparator.comparing 方法

两种解决方案如何选择呢?

1. 当比较规则不会发生改变的时候，或者说比较规则只有一个的时候，建议实现 Comparable 接口
2. 当比较规有多个，并且需要在多个比较规则之间频繁切换时，建议使用 Comparator 接口

方法一代码：

```java
// 利用 TreeSet 对自定义类型排序
/**
 * 方法1：自定义类型实现 Comparable 接口
 * @date 2021-06-25 14:33
 * @author preci
 * @version 1.0
 *
 */
public class Person implements Comparable<Person> {
    int age;

    public Person(int age) {
        this.age = age;
    }

    @Override
    public int compareTo(Person o) {
        return this.age - o.age; // 按照年龄升序排序
    }
}

public static void main(String[] args) {
    Set<Person> persons = new TreeSet<>();
    persons.add(new Person(1));
    persons.add(new Person(25));
    persons.add(new Person(10));
    persons.add(new Person(8));  // output:1, 8, 10, 25 (年龄升序)
}
```

「代码随想录」知识星球

posper

方法二代码：

```java
/**
 * 方法2：利用比较器 Comparator
 * @date 2021-06-25 14:08
 * @author preci
 * @version 1.0
 *
 */
class Cat { // 没有实现 Comparable 接口
    int age;
    public Cat(int age) {
        this.age = age;
    }
}
public static void main(String[] args) {
    // 1、使用接口实现类
    // Set<Cat> set = new TreeSet<>(new MyCmp()); // 传递一个比较器对象给 TreeSet 构造器
    // 2、使用匿名内部类
```

```
    Set<Cat> set = new TreeSet<>(new Comparator<Cat>() {
        @Override
        public int compare(Cat o1, Cat o2) {
            return o1.age - o2.age;
        }
    });
    // (3) 使用 lambda 表达式，传递一个比较器对象
    // Set<Cat> set = new TreeSet<>((o1, o2) -> o1.age - o2.age);
    set.add(new Cat(1));
    set.add(new Cat(15));
    set.add(new Cat(12)); // output:1,12,15
}
// 创建一个比较器类
class MyCmp implements Comparator<Cat> {
    @Override
    public int compare(Cat o1, Cat o2) {
        return o1.age - o2.age;
    }
}
```

Collections 工具类

Collections.sort(List list)
Collections.sort(List list, Compataor cmp)
如果比较自定义类型，需要传入比较器或者自定义类型实现 Comparable 接口

HashSet 可以利用 ArrayList 构造器转换为 list

```
Set<String> set = new HashSet<>();
set.add("aa");
set.add("b");
ArrayList list = new ArrayList(set); // 将 set 转化为 list
Collections.sort(list);
```

「代码随想录」知识星球
posper

String存储原理

1、String 类型是不可变的

2、Java 中用双引号括起来的字符串，例如："abc"、"def"，都是直接存储在“方法区”的“字符串常量池”当中的。

3、为什么把字符串存储在一个“字符串常量池”当中呢?

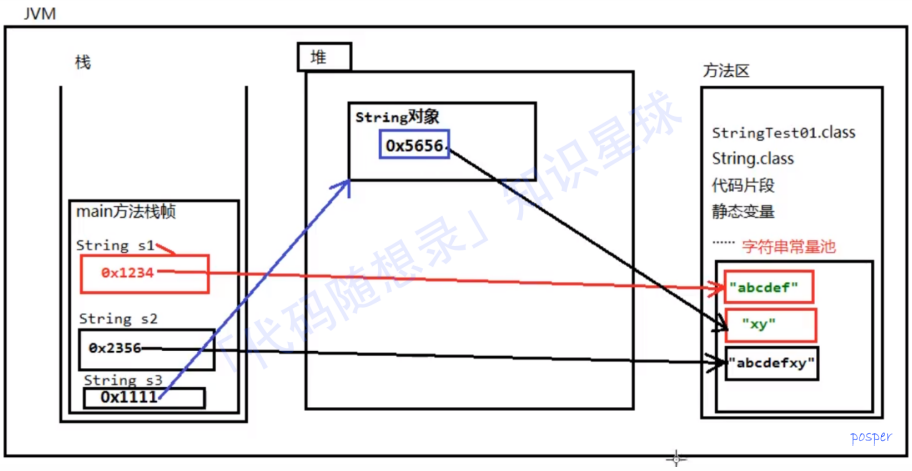
1. 因为字符串在实际的开发中使用太频繁

2. 为了提高执行效率，所以把字符串放到了方法区的“字符串常量池”当中

```
public class StringTest02 {
    public static void main(String[] args) {
        /**
         * 这里会创建 3 个对象，均在“字符串常量池”中。
         * "abcdef"仍然在“字符串常量池”中，且只会存一份。
         * “字符串常量池”中还有 "xy"，以及新结果 "abcefxy"
         */
        String s1 = "abcdef";
        String s2 = "abcdef" + "xy";
    }
}
```

posper

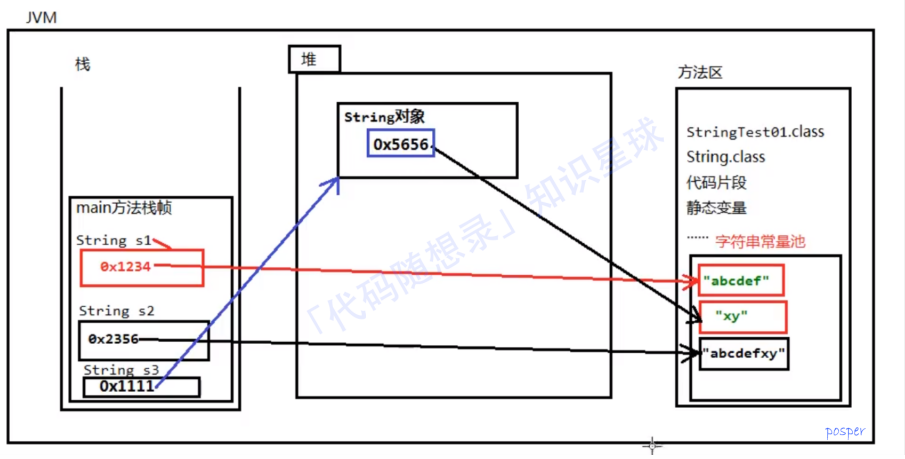
以上代码在 JVM 中加载过程如下：




```
public class StringTest01 {
    public static void main(String[] args) {
        /**
         * "abcef"仍然在“字符串常量池”中。
         * 但是，new 的话，会在堆中开辟空间，其中存的是指向“字符串常量池”的内存地址。
         * 然后，在栈中存的是指向堆的引用地址。
         */
        /**
         * 这里一共创建了 3 个对象：
         *  方法区“字符串常量池”中有一个："abc"
         *  堆内存中有两个 String 对象。
         *  一共 3 个。
         */
        String s3 = new String("abcdef"); //
        String s4 = new String("abcdef"); //
        System.out.println(s3 == s4);       // false，比较地址
        System.out.println(s3.equals(s4));  // true，比较内容
    }
}
```

posper

以上代码在 JVM 中加载过程如下：



String、StringBuilder、StringBuffer 部分源码解读

String 底层数组用 final 修饰，不可变。
StringBuilder 底层数组没有用 final 修饰，可变；线程不安全，效率高（一般用的多）
StringBuffer 底层数组没有用 final 修饰，可变；线程安全，效率低（一般用的少）

方法都采用 了 synchronized 修饰

JDK1.8 中 String 部分源码解读

```
// jdk1.8 中 String 部分源码解读
public final class String implements ... {
    private final char value[];
    /** Cache the hash code for the string */
    private int hash; // Default to 0

    public String() {
        this.value = "".value;
    }

     public String(String original) {
        this.value = original.value;
        this.hash = original.hash;
    }
}
```

JDK1.8 中 StringBuilder 部分源码解读

```java
// jdk1.8 中 StringBuilder 部分源码解读
package java.lang;

public final class StringBuilder extends AbstractStringBuilder implements ... {
    // 字段 value 和 count 使用的是父类 AbstractStringBuilder 中的...

    // 无参构造器
    public StringBuilder() {
        super(16);
    }

    public StringBuilder(int capacity) {
        super(capacity);
    }

    // 待参数 string 的构造器
    public StringBuilder(String str) {
        super(str.length() + 16);
        append(str);
    }
}
```

posper

JDK1.8 中 AbstractStringBuilder 部分源码解读

```java
// jdk1.8 中 AbstractStringBuilder 部分源码解读
abstract class AbstractStringBuilder implements Appendable, CharSequence {
    char[] value; // 没有用 final修饰，可变。
                  // 包访问权限，AbstractStringBuilder 和 StringBuilder 都在 java.lang 包中
    int count;

    // 无参构造器
    AbstractStringBuilder() { }

    AbstractStringBuilder(int capacity) {
        value = new char[capacity];
    }

    @Override
    public int length() {
        return count;
    }

    @Override
    public StringBuilder append(String str) {
        super.append(str);
        return this;
    }
}
```

posper

JDK1.8 中 StringBuffer 部分源码解读

```java
// jdk1.8 中 StringBuffer 部分源码解读
package java.lang;

public final class StringBuffer extends AbstractStringBuilder implements ... {
    private transient char[] toStringCache;

    // 无参构造器
    public StringBuffer() {
        super(16);
    }

    // 参数为 String的构造器
    public StringBuffer(String str) {
        super(str.length() + 16);
        append(str);
    }

    @Override
    public synchronized int length() { // 使用 synchronized 修饰
        return count; // 这里 count 是父类 AbstractStringBuilder 中的
    }
}
```

posper

「代码随想录」知识星球

方法参数

Java 中方法的参数传递只有按值调用，没有c++中的按引用调用。也就是说， 方法得到的是所有参数值的一个副本。

方法参数共有两种类型：

1. 基本数据类型（8种）
2. 引用类型

Java 中方法参数的使用情况：

1. 一个方法不能修改一个基本数据类型的参数（即数值型或布尔型）。
2. 一个方法可以改变一个对象参数的状态。
3. 一个方法不能让对象参数引用一个新的对象。

Java面向对象

面向对象程序设计概述

面向过程：

先确定如何操作数据，再决定数据的结构。适用于小规模问题;

面向对象OOP：

先决定数据的结构，在考虑操作数据的算法。适用于大规模问题;

1. 类

1、类是构造对象的模板或蓝图

2、封装是处理对象的一个重要概念

就是将数据和行为组合在一个包中，并对对象的使用者隐藏具体的实现方式

1. 实例字段：对象中的数据数据;
2. 方法：操作数据的过程;
3. 对象的状态：特定对象有一组特定的实例字段值，这些值的集合就是这个对象的当前状态。

3、OOP原则：

1. 封装：绝对不能让类中的方法直接访问其他类的实例字段;
2. 扩展：可以通过扩展其他类扩展来构建新类。

4、类、超类和子：

1. 继承：基于已有的类创建新的类。继承一个类就是复用（继承）这些类的方法，而且可以增加一些新的方法和字段
2. 继承：is-a 关系

5、定义子类

1. extends 关键字
2. 父类，又叫基类、超类
3. 子类，又叫派生类、孩子类
4. 子类比父类拥有的功能更多
    1. 通过扩招父类定义子类时，只用指出子类与父类的不同之处
    2. 一般的方法放在父类中，更特殊的方法放在子类中

6、覆盖方法

1. 超类中的有些方法对子类并不一定适用，此时需要在子类中覆盖（重写）父类方法
2. 使用 super 关键字可以调用父类方法，避免覆盖父类方法时调用父类方法造成不必要的递归
3. super 不是一个对象的引用，不能将super 赋给另一个对象变量

7、子类构造器

1. super(...) 调用父类构造器
2. 必须放在子类构造器的第一条
3. 若子类构造器没有显示调用父类构造器，将自动调用父类的无参构造器；
4. 若父类没有无参构造器，必须要在子类构造器中明确指明调用父类哪个构造器；否则，Java编译器就会报错

一个对象变量可以指示多种实际类型的现象成为多态。

在运行时能够自动选择适当的方法，成为动态绑定。

8、this 与 super 关键字

1. this 关键字
    - 隐式参数的引用
    - 调用该类的其他构造器
2. super 关键字
    - 调用父类方法
    - 调用父类构造器

注意：this 可以作为当前对象的引用，但是 super 却不可以作为父类对象的引用

2. 对象

1. 对象的状态改变必须通过调用方法实现;（如果不经过方法改变对象状态，说明破坏了封装性）
2. 对象的状态不能完全描述一个对象，每个对象有一个唯一标识;
3. 作为同一个类的实例，每个对象的标识总是不同的，状态往往也存在着差异。

3. 类之间的关系

- 依赖(uses-a)：一个类的方法使用另一个类的对象;
  ------->
- 聚合(has-a)：类A的对象包含类B的对象;
  ◇————
- 继承(is-a)：一个更特殊的类和一个更一般的类之间的关系;
  ——▷

4. 继承与多态

1、继承层次

1. 由一个公共超类派生出来的所有类的集合成为继承层次。
2. 在继承层次中，从某个特定的类到其祖先的路径成为该类的继承链（子到父）

Java中不支持多继承，但是可以通过接口来实现

2、Java中，对象变量是多态的

1. 父类类型的变量既可以引用自身类型的变量，还可以引用子类类型的变量
2. 但是，子类类型的变量不可以引用父类类型的变量

3、子类可以调用父类public、protected、包权限的方法

但是父类不可以调用子类的特有方法（子类的特殊性）

1. 纯父类变量（左右都是父类）不可以调用子类的任何方法
2. 上转性变量（左边是父类，右边是子类）可以调用子类重写父类的方法（多态），但是仍然不能调用子类独有的方法

```
public class Manager extends Employee {
    private double bonus;

    public void setBonus(double bonus) { // Manger 中独有的方法，Employee 类中没有
        this.bonus = bonus;
    }
}

Manager boss = new Manager(); // 纯子类变量（引用、对象）
Employee staff = boss; // 上转型对象。staff与boss引用同一个对象，但是编译器将 staff 看成一个 Employee 对象
boss.setBonus(100);   // OK。子类调用子类方法，没毛病
staff.setBonus(100);  // ERROR。setBonus 方法是子类独有的，不能通过上转型对象和父类对象调用，只能通过子类对象调用
```

「代码随想录」知识星球
posper

4、理解方法调用

以调用 x.f(arg) 为例，隐式参数 x 为类 C 的一个对象：

（1）编译器查看对象的声明类型和方法名

编译器查找 C 类中所有名为 f 的方法和父类中名为 f 且可访问的方法（父类 private 方法不可访问）

此时，编译器知道所有可能被调用的候选方法

（2）编译器确定方法调用中提供的参数类型

重载解析： 在所有名为 f 的方法中，找到一个与所提供参数类型完全匹配的方法

此时，编译器已经知道需要调用的方法名字和参数类型

（3）静态绑定(编译阶段绑定)

如果是 private 方法、static 方法、final 方法或者构造器， 那么编译器将可以准确地知道应该调用哪个方法。

此时调用方法只用考虑 x 的类型（若在类 C 中找不到 f 方法，则向其父类中找），不需要考虑类 C 的子类（因为这几种修饰的方法都不能被继承）

只有静态绑定成功，即编译通过了，才能进入运行阶段

（4）**动态绑定**(运行阶段绑定)

如果调用的方法依赖于隐式参数的实际类型，那么必须在运行时使用动态绑定。

虚拟机必须调用与 x 所引用对象的实际类型对应的那个方法

例如：x 的实际类型是 D，它是 C 类的子类。

如果：D 类定义了方法 f(String)，就直接调用它；
否则：将在 D 类的超类中寻找 f(String)，以此类推。

在覆盖一个方法的时候，子类方法不能低于父类方法的可见性。
比如，父类方法为 public，子类必须为 public。
如果子类遗漏了 public，编译器就会报错。

5、阻止继承：final 类和方法

（1）**final 关键字**

```
1. final 修饰字段

   * 基本类型：不可更改
   * 引用类型：不可指向新的引用，但是对象状态可能会改变

2. final 修饰类：该类不可以被继承

3. final 修饰方法：子类不能覆盖这个方法
```

（2）**将方法或类声明为 final 主要原因**

确保它们不会在子类中改变语义

6、强制类型转换

7、向上转型（upcasting）

- 子类型 -> 父类型
- 又被称为：自动类型转换

8、向下转型（downcasting）

- 父类型 -> 子类型
- 又被称为：强制类型转换

进行强制类型转换的唯一原因是：

在暂时忽视对象的实际类型之后， 使用对象的全部功能。

无论是向上转型还是向下转型，这两种类型之间必须要有继承关系；否则，编译器报错。

```
Employee[] staff = new Employee[3]; // 统一定义为父类变量，暂时忽视子类的实际类型

staff[0] = new Manager(); // 上转型对象
staff[1] = new Employee(); // 父类对象
staff[2] = new Employee();

Manager boss = (Manager) staff[0]; // 强制类型转换。使用对象的全部功能（因为上转型对象 staff[0] 不能使用 Manager 类中的 setBonus 方法）
```

- **可以**将上转型对象强制转化为子类变量，但是**不能**将父类对象强制类型转换为子类对象

```
// 同上代码
Manager boss = (Manager) staff[0]; // OK。上转型 to 子类
Manager boss = (Manager) staff[1]; // ERROR。父类 不可to 子类。此时，会产生 ClassCastException 异常

// 在将超类转换成子类之前，应该使用instanceof 进行检查
if (staff[1] instanceof Manager) {
    boss = (Manager) staff[1];
}
```

posper

小结：

1. 只能在继承层次内进行类型转换
2. 在将超类转换成子类之前，应该使用 instanceof 进行检查。

通过类型转换调整对象的类型并不是一种好的做法。在一般情况下，应该尽量少用强制类型转换和 instanceof 运算符。

大多情况下，因为多态性的动态绑定机制，我们不用将 Employee 转换为 Manager 也能正确调用方法；

只有在使用 Manager 特有方法才需要进行强制类型转换。例如，setBonus 方法。

但是，此时应该自问超类设计是否合理。是否需要重新设计超类，并添加 setBonus 方法，这才是更合适的选择。

抽象类

abstract 关键字：

1、abstract 修饰方法

1. 抽象方法，不用实现，在具体的子类中实现
2. 若子类未实现抽象方法，仍然要定义为抽象类

2、abstract 修饰类

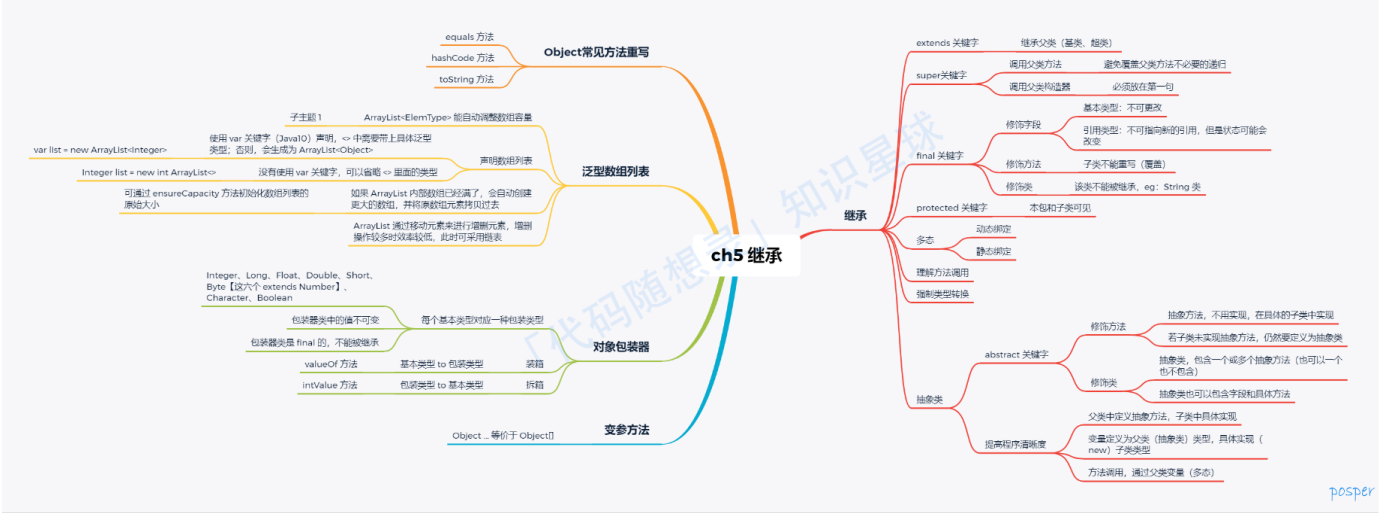
1. 抽象类，包含一个或多个抽象方法（也可以一个也不包含）
2. 抽象类也可以包含字段和具体方法

3、提高程序清晰度

1. 父类中定义抽象方法，子类中具体实现
2. 变量定义为父类（抽象类）类型，具体实现（new）子类类型（上转型）
3. 方法调用，通过父类变量（多态，动态绑定）

思维导图小结：

原图链接：「代码随想录」知识星球



使用预定义的类

1. 对象与对象变量

1. 对象变量并没有时间包含一个对象，它只是引用一个对象
2. 在Java中，任何对象变量的值都是对存储在另一个地方某个对象的引用
3. new操作符的返回值也是一个引用
4. Java对象都存储在堆中，当一个对象包含另一个对象变量时，它只是包含着另一个堆对象的指针
5. Java中如果使用一个没有初始化的指针，运行时系统将会产生一个运行时错误，同时不必担心内存管理问题，垃圾回收器会处理相关的事宜
6. Java中必须使用clone方法获得对象的完整副本

2. Java中的LocalDate类

1、Java类库包含两种保存时间的类

1. Date类：表示时间点
2. LocalDate类：日历表示法表示日期

2、一般通过这三种静态工厂方法构造一个LocalDate

```
LocalDate.of(2020,4,20);
LocalDate.now();
LocalDate.parse("2021-04-20");
```

3、使用方法

```
isLeapYear();                  //此LocalDate是否闰年
minusWeeks(); minusDays(); //减去星期/天之后的表示
plusWeeks(); plusDays(); //加上星期/天之后的表示
getMonthValue();           //获得月的值
getDayOfMonth();           //获得该LocalDate的day
getDayOfWeek(); getMonth(); //返回枚举值，获得该天的枚举值
// (1-7)，获得该月的枚举值（1~12）
```

1. 访问器方法：只访问对象而不修改对象的方法;（如plusDays返回一个新的对象，原对象还在）
2. 更改器方法：调用方法之后修改对象的内容。

在C++中，带有const后缀的方法是访问器方法，没有的是更改器方法。而在Java中，这两种方法没有明面上的语法区别

用户自定义类

1. 主力类：通常没有main方法，却有自己的实例字段和实例方法;（例如：自定义User类）
2. 一个源文件中只能有一个公共类，但是可以有任意数量的非公共类。

1. 多个源文件的使用

将两个类存放在一个单独的类文件中

例如：Employee类存放在文件EmployeeTest.java中，然后键入以下指令javac EmployeeTest.java 此时并没有显示的编译Employee.java，不过编译器会发发现需要使用这两个类时，会自动搜索Employee.java，并编译。

如果编译器发现Employee.java较已有的Employee.java已有更新，那么Java编译器会自动的编译这个文件。

2. 构造器方法

构造器与类名同名，构造器总是结合new运算符来调用，不能对一个已存在的对象调用构造器来达到重新设置实例字段的目的。

3. 用var声明局部变量（在Java10中）

在Java10中，如果可以从变量的初始值推导出他们的类型，那么可以使用var关键字声明局部变量，而无需指定类型

var关键字只能用于方法中的局部变量，参数和字段的类型必须声明

4. 使用null引用

定义一个类时，最好清楚哪些字段可能为null，对于初始化时字段可能null，一般有两种解决方案:

1、宽容型

将null参数转换成一个非null值

```
if(n == null) name = "unknown"; else name = n;
/* Java9中有一个方法
name = Object.requireNonNull(n,"unknown");
达到同样的效果 */
```

2、严格型

直接拒绝null参数

```
Object.requireNonNull(n, "the name cannot be null");
```

5. 隐式参数和显式参数

1、隐式参数：

出现在方法名前的对象的参数;

2、显示参数：

位于方法名后面括号里的数值;

每个方法中，用this关键字指示隐式参数，这样可以将 实例字段 和 局部变量 明显的区分开来。

6. 封装的优点

注意不要编写返回可变对象引用的访问器方法（getter方法）

例如返回一个Date类：

因为Date类中有setDate()方法，所以Date对象可变，也就意味着破坏了封装性，如果需要返回一个可变对象的引用，首先应该对他进行clone，然后再返回克隆的对象副本。

7. 私有方法

只要方法是私有的，类设计者就可以确信它不会在别处使用，所以可以删去，而如果是公共的，那么可能会因为其他其他代码依赖这个方法。

8. final字段

1、final修饰符对于 基本类型 或者不可变类

类中所有方法都不会改变其对象，如String类的字段尤其有用

2、对于可变的类

使用final会造成混乱，final关键字只表示存储在该 变量 中的 对象引用 不会再指向另一个不同的对象，而这个对象还是可以更改

静态字段与静态方法

static的含义

1. 静态字段

1、静态字段：

静态字段属于类(即使没有创建该类的对象，静态字段也存在)，每个类只有一个这样的字段。创建的多个对象共享这同一个静态字段;

2、非静态字段：

每个对象都有自己的一个副本。

2. 静态常量

静态常量：设置为公共没问题，因为不允许再将它赋值为另一个值（System类中的setOut()方法可以将System.out设置为不同流，这是因为setOut方法是一个原生方法，不是在Java语言中实现的，因此可以绕开Java的访问控制机制。）

3. 静态方法

静态方法是一个没有this参数的方法;（可直接通过类名调用）

可以使用对象来调用静态方法，不过很容易造成混乱，因为方法计算的结果与该对象无关。建议使用类名直接调用静态方法;

两种情况下使用静态方法：

1. 方法不需要访问对象状态，如Math.pow
2. 方法只需要访问类的静态字段

4. 工厂方法

静态方法还有一种，类似LocalDate和NumberFormat 类使用静态工厂方法来构造对象。

使用这种方式构造对象的原因有两个:

1、无法命名构造器：

构造器名字必须与类名相同，但是有事希望有两个不同的名字，分别得到货币实例和百分比实例

2、使用构造器时：

无法改变所构造对象的类型。静态工厂方法可以，如工厂方法实际将返回DecimalFormat类对象，这是NumberFormat的一个子类

5. 方法参数

Java中，方法参数：

1. 方法不能修改基本数据类型的参数;（数值型或布尔型）
2. 方法可以改变对象参数的状态;
3. **方法不能让一个对象参数引用一个新的对象。**

对象构造

1. 重载

多个方法有相同的名字，不同的参数，便出现了重载

重载解析： 编译器必须挑选出具体调用那一个方法，它用各个方法首部中的 参数类型 和 值类型来匹配

方法签名：**方法名 + 参数类型** 注意返回类型不是方法签名的一部分

2. 默认字段初始化

如果在构造器中没有显式的为字段设置初始值，会自动赋为初始值（0，false，null）

方法中的局部变量必须要明确的初始化，否则会报错

类中的字段，没有初始化会默认初始化

3. 无参数的构造器

无参构造默认将所有字段设置为默认值

当且仅当类没有任何其他构造器的时候，才会得到一个默认的无参构造器

4. 显示字段初始化

可以在类定义时，为每个实例字段设置一个有意义的初始值（如果一个类的所有构造器希望把某个特定的实例字段设置为同一个值，这个就有用）

初始值也可以不是常量值，利用方法调用初始化一个字段，使用类中的静态变量，每次调用改变这个静态变量的值即可。

5. 参数名

参数变量会遮蔽同名的实例字段，但是给实例字段加上this就可以避免这种情况。

6. 调用另一个构造器

如果构造器的第一个语句形如`this(...)`，这个构造器将会调用另一个构造器，采用这种方式使用this关键字非常有用，对公共的构造器代码只需要编写一次即可。

C++中一个构造器不能调用另一个构造器。

7. 初始化块

一个类的生命中，可以包含任意多个代码块，只要构造这个类的对象，这些块就会被执行

（这种机制不常见，通常使用构造器初始化对象。同时，为了避免混淆，初始化块放在字段定义之后，避免重复定义带来的麻烦）

而静态初始化在类的第一次加载的时候，会进行初始化。

包

Java 允许使用包（ package） 将类组织起来

使用包的主要原因是**确保类名的唯一性**

包名通常是域名逆序来写，为了保证包名的绝对唯一性，将公司的因特网域名（这显然是独一无二的） 以逆序的形式作为包名

1. 类的导入

1、一个类可以使用所属包中的所有类，以及其他包中的公共类

2、两种方法访问另一个包中的公共类

（1）完全限定名（比较繁琐）

包名之后跟着类名`java.time.LocalDate today = java.time.LocalDate.now();`

（2）import导入（简单常用）

import应该位于源文件的顶部，package语句的后面

只能使用星号（*) 导入一个包， 而不能使用 import java.* 导入以java 为前缀的所有包

3、如果 import 两个包中包含了同名的类，则会发生命名冲突。

主要有以下两种解决方案：

1. 如果只是使用用一个包中的同名类时，可以考虑增加一个特定的 import 语句来解决这个问题
2. 如果两个个包中的同名类都被使用时，则需要在每个类名前加上完整的包名

其中，方法2通用，但是相对可能繁琐一些

2. 静态导入

使用import语句导入静态方法和静态字段，而不只是类

1. `import static java.lang.System.*;`
2. 就可以使用System 类的静态方法和静态域，而不必加类名前缀：
3. `out.println("Goodbye, World!");` / / i.e., System.out

但是，这种编写形式不利于代码的清晰度。最好少用或者不用

3. 在包中增加类

1. 要想将一个类放入包中，就必须将包的名字放在源文件的开头，包中定义类的代码之前
2. 如果没有在源文件中放置 package 语句，这个源文件中的类就属于无名包
3. 编译器在编译源文件的时候不检查目录结构
4. 如果包与目录不匹配，或许可以成功编译（如果不依赖于其他包时），但是无法成功运行（虚拟机就找不到类）

1、包作用域：

1. 标记为 public 的部分可以被任意的类使用；
2. 标记为 private 的部分只能被定义它们的类使用
3. 如果没有指定public 或 private, 这个部分（类、方法或变量）可以被同一个包中的所有方法访问

注意：变量必须显式地标记为 private, 不然的话将默认为包可见。显然，这样做会破坏封装性。

2、类路径：

（1）类路径是所有包含类文件的路径的集合。

（2）javac 编译器总是在当前的目录中查找文件，

但 Java 虚拟机仅在类路径中有『.』目录的时候才查看当前目录

1. 如果没有设置类路径，默认的类路径包含『.』目录
2. 然而如果设置了类路径却忘记了包含 『.』 目录，则程序仍然可以通过编译， 但却不能运行（因为此时JVM找不到当前目录的文件）

4. Jar文件

在将应用程序打包时，希望**只向用户提供一个单独的文件**，而不是一个包含大量文件的目录结构。

JAR文件便是为此而设计的，一个JAR 文件可以包含文件、图像、声音等其他类型的文件。

JAR文件是压缩的，它使用了我们熟悉的zip压缩格式。

5. 清单文件

除了类文件，图像的其他资源外，每个JAR文件还包含一个清单文件(manifest) ，用于描述文档的特殊特性

清单文件位于JAR文件的 META-INF 子目录中

6. 可执行JAR文件

jar命令中的e指定程序的入口点，指定执行jar文件时启动的类。

执行命令：`java –jar jarFileName.jar`

文档注释

JDK包含一个很有用的工具，javadoc，它可以由源文件生成一个HTML文档

ch3中联机API 文档就是通过对标准Java 类库的源代码运行javadoc 生成的

1. 注释的插入

1、javadoc从以下项中抽取信息

1. 模块
2. package
3. public 的 class 和 interface
4. public 和 protected 的字段，构造器，方法

2、应该为以上各个部分添加注释/ */**

注释中的内容是 自由格式文本，标记以@开始，如@param

第一句为概要性句子，javadoc将抽取第一句生成概要页

自由格式文本中可以使用HTML标签，如`<strong></strong>`表强调，`<img>`插入图片

其他文件图片的文件，应该放到包含源文件目录的一个子目录doc-files中

2. 类注释

1. @param 可以占据多行，可以使用HTML，一个方法的param必须放在一起
2. @return 可以多行，使用HTML
3. @throws 添加异常抛出注释

3. 字段注释

只需要对公共字段（静态常量static final）建立文档。

4. 通用注释

1. @since 引入该条目 始于版本;
2. @author 可用多个;
3. @version 当亲版本。

类设计技巧

1、保证数据私有

1. 绝对不要破坏封装性
2. 当数据保持私有时， 它们的表示形式的变化不会对类的使用者产生影响， 即使出现bug 也易于检测。

2、对数据初始化（避免错误）

1. Java 不对局部变量进行初始化， 但是会对对象的实例域进行初始化
2. 最好不要依赖于系统的默认值， 而是应该显式地初始化所有的数据

3、不要再类中使用过多基本类型（不易重复利用）

1. 就是说，用其他的类代替多个相关的基本类型的使用
2. 这样会使类更加易于理解且易于修改

4、不是所有的字段都需要单独的getter和setter

字段访问器和字段更改器（有一些实例字段不希望其他人修改）

在对象中，常常包含一些不希望别人获得或设置的实例域

5、分解职责过多的类

6、类名和方法名要充分体现职责（见名知意）

命名类名的良好习惯是采用一个名词（ Order)、前面有形容词修饰的名词（ RushOrder)或动名词（有“ -ing” 后缀）修饰名词（例如， BillingAddress)。

对于方法来说，习惯是访问器方法用小写get 开头, 更改器方法用小写的set 开头

7、优先使用不可变的类（避免多线程并发同时更新一个对象带来的麻烦）

更改对象的问题在于，如果多个线程试图同时更新一个对象，就会发生并发更改。其结果是不可预料的。如果类是不可变的，就可以安全地在多个线程间共享其对象。

因此，要尽可能让类是不可变的

当然，并不是所有类都应当是不可变的。

内存中对象

前言

“对象在内存中是怎么布局的？”

“对象头具体包括什么？”

“锁在对象的哪里？”

“对象怎么定位？”

面试过程中，你是否被问到过这些问题，如果不能完全回答出来，那么这篇文章一定要看下去。

对象的内存布局

对象实例中主要包含三部分的内容，分别为对象头、示例数据和对齐填充padding。下面分别进行介绍。

1、对象头

对象头中的内容主要是**运行时元数据和类型指针**。

其中运行时元数据主要存储的是以下六个内容：

1. 哈希值
2. GC分代年龄
3. 锁状态标志
4. 线程持有的锁
5. 偏向线程ID
6. 偏向时间戳

所以我们知道，锁是在对象头中的。

那么类型指针又是什么呢？

实际上，类型指针是指向元数据类型的InstanceKlass，用来确定该对象所属的类型。比如通过getInstance()得到对象实例时就需要用到该指针。这里需要注意的是，不是所有对象都会保留该指针的。

以上我们说的对象指的是自定义对象，如果是数组对象，那么还需要记录数据的长度。

2、示例数据

示例数据中是对象真正存储的有效信息，包括程序中定义的各种字段，这里的字段也包括从父类、Object类中定义的各种字段。

那么这么多字段，是怎么放在一起的呢？所以就不得不提存放规则了。

示例数据的存放规则遵循以下三个原则

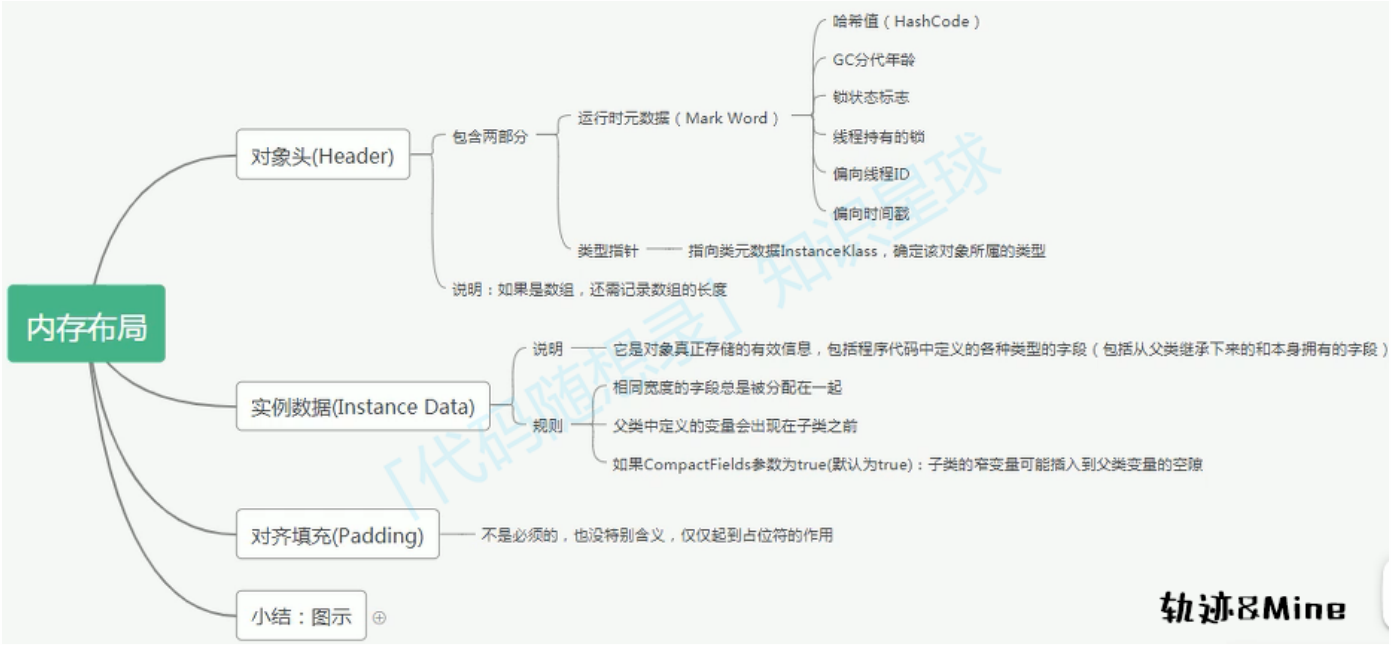
1. 父类中定义的变量在子类之前。
2. 同一个类中，相同宽度的字段被分配在一起。如四个字节的数据一起。
3. 若 CompareFileds参数为true（默认为true）,子类的窄变量可能插入到父类变量的空隙。

对齐填充padding

由于HotSpot虚拟机的自动内存管理系统要求对象起始地址必须是8字节的整数倍，换句话说就是任何对象的大小都必须是8字节的整数倍。对象头部分已经被精心设计成正好是8字节的倍数（1倍或者2倍），因此，如果对象实例数据部分没有对齐的话，就需要通过对齐填充来补全。

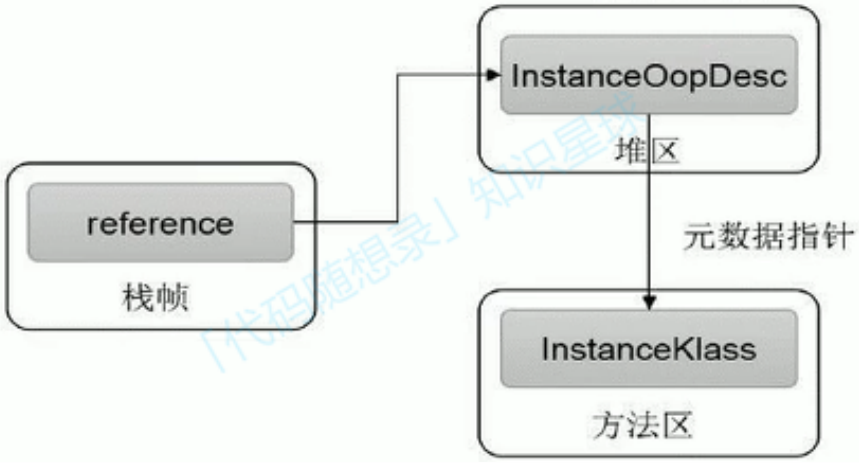
总的一句话来说，“数据项仅仅能存储在地址是数据项大小的整数倍的内存位置上（分别为偶地址、被4整除的地址、被8整除的地址）”比如int类型占用4个字节，地址仅仅能在0，4，8等位置上。

想讲好对齐填充，其实还是得去了解c++实现，这里不过多介绍。



对象的访问定位

堆中的对象是通过元数据指针找到方法区中的元数据信息的，那么栈帧中的对象引用又是如何访问找到对应的对象呢？



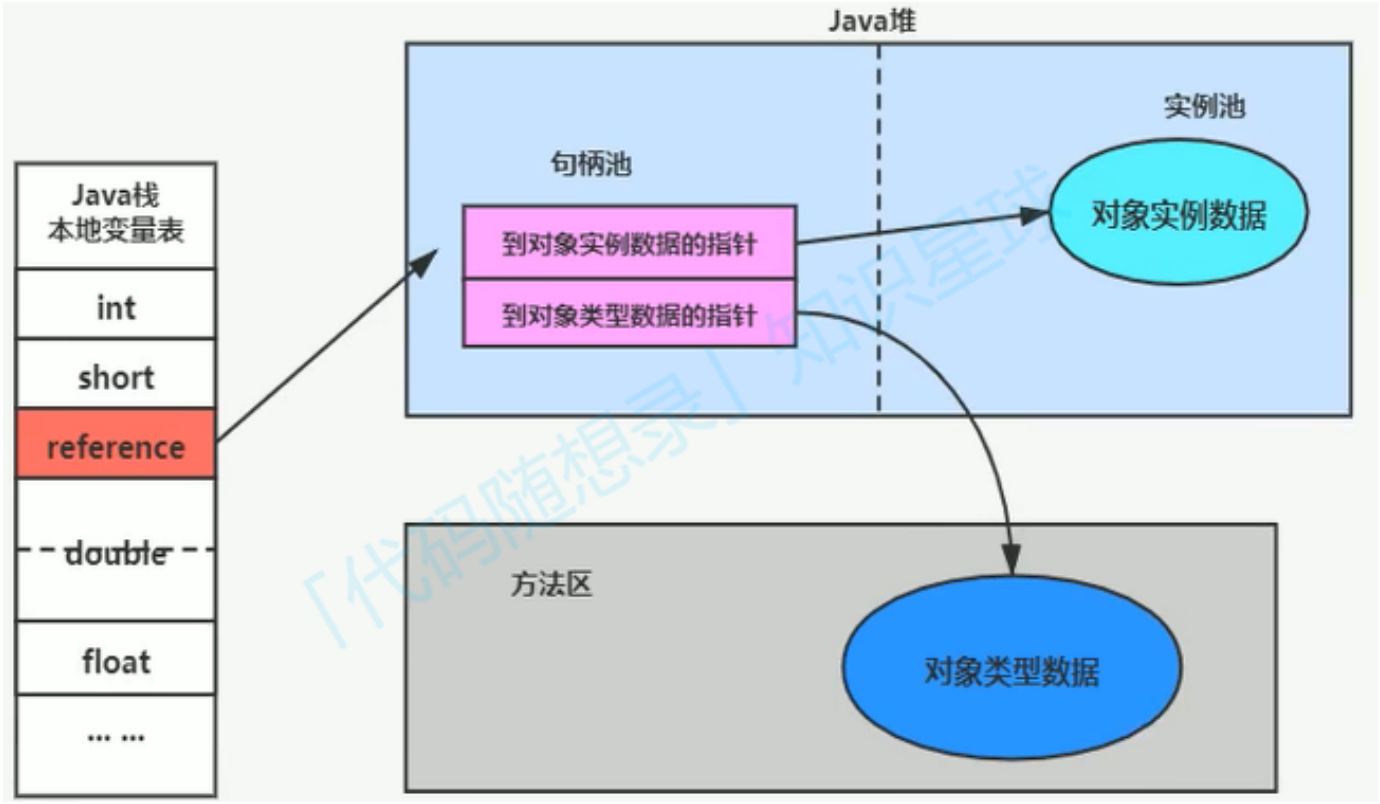
对象的访问定位方式主要有两种：

1. 句柄访问
2. 直接访问

句柄访问方式：

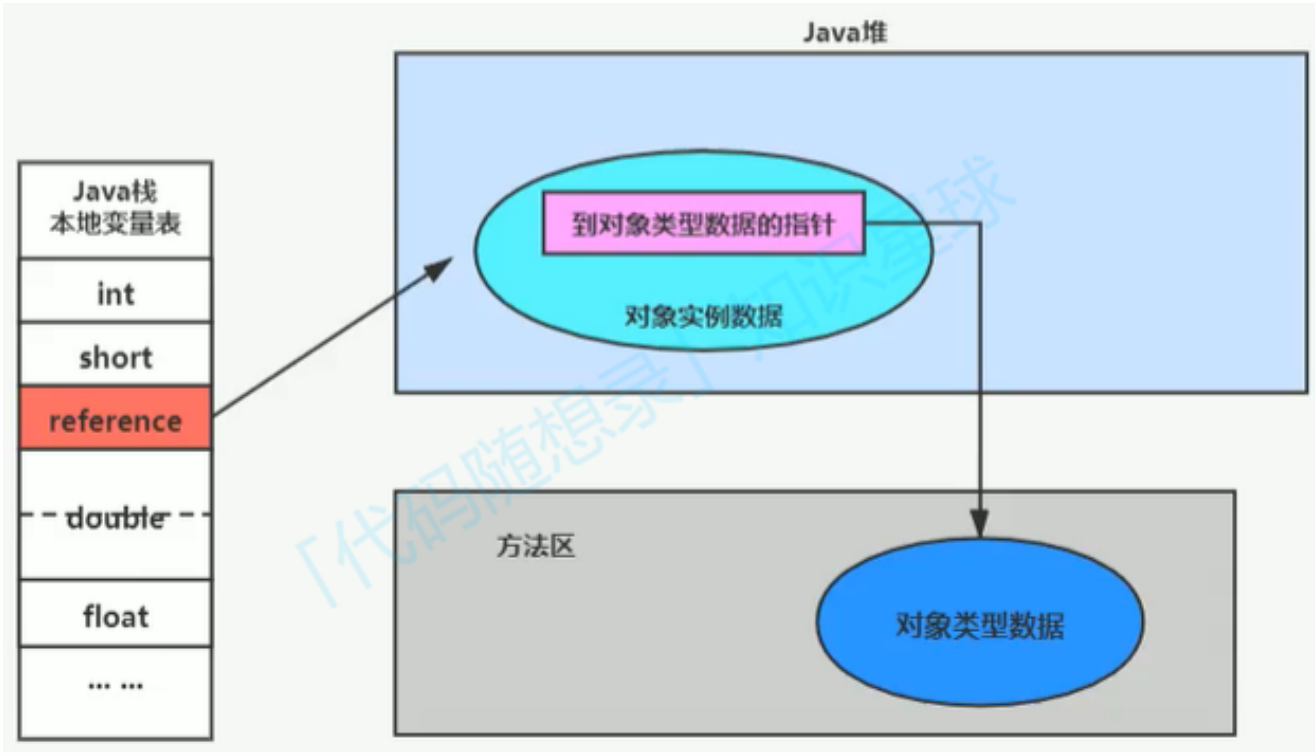
堆区中开辟了一块空间用于存储句柄，该空间称为句柄池。句柄池分别存储着两个句柄，分别是到对象实例数据的指针和到对象类型数据的指针。前者指向对象实例数据，后者指向对象类型数据。

栈帧的局部变量表中存储着到对象实例数据指针的位置，即指向句柄池中的第一个句柄。



直接访问方式：

直接访问方式相比句柄访问就简单多了，局部变量表中直接存储着对象实例数据的地址，而对象实例数据存储着方法区中对象类型数据的地址。这也是Hotspot虚拟机中的访问方式。



总结

关于对象的内部结构、内存布局和访问定位，值得深入的点还有很多，这里只是做一个初步的介绍，待深入学习后，会及时更新相应的内容。

Java 异常体系

Java的异常都是派生于Throwable类的一个实例，所有的异常都是由Throwable继承而来的。

Throwable有分为了Error类和Exception类。

Error（错误）

Error 类层次结构描述了 Java 运行时系统的内部错误和资源耗尽错误。

Error表示比较严重的问题，一般是JVM运行时出现了错误，如没有内存可分配抛出OOM错误、栈资源耗尽抛出StackOverflowError错误、Java虚拟机运行错误Virtual MachineError、类定义错误NoClassDefFoundError。

如果出现了这样的内部错误，除了通告给用户，并尽力使程序安全地终止之外，再也无能为力了。

Exception（异常）

异常又分为RuntimeException和其他异常：

由程序错误导致的异常属于RuntimeException，而程序本身没有问题，但由于像 I/O 错误这类问题导致的异常属于其他异常。

运行时异常RuntimeException：

顾名思义，运行时才可能抛出的异常，编译器不会处理此类异常。比如数组索引越界、使用的对象为空、强制类型转换错误、除0等等。出现了运行时异常，一般是程序的逻辑有问题，是程序自身的问题而非外部因素。

其他异常：

Exception中除了运行时异常之外的，都属于其他异常。也可以称之为编译时异常，这部分异常编译器要求必须处置。这部分异常常常是因为外部运行环境导致，因为程序可能运行在各种环境中，如打开一个不存在的文件，此时抛出FileNotFoundException。编译器要求Java程序必须捕获或声明所有的编译时异常，强制要求程序为可能出现的异常做准备工作。

不要被运行时异常的名称所迷惑，理论上所有的错误都是运行时发生的。包括Error、RuntimeException、编译时异常等等。所有的这些都只能在程序运行的过程中才能碰到。编译时异常指的是编译器要求必须处理的异常，并不是代码编译间发生的错误。

受检异常和非受检异常

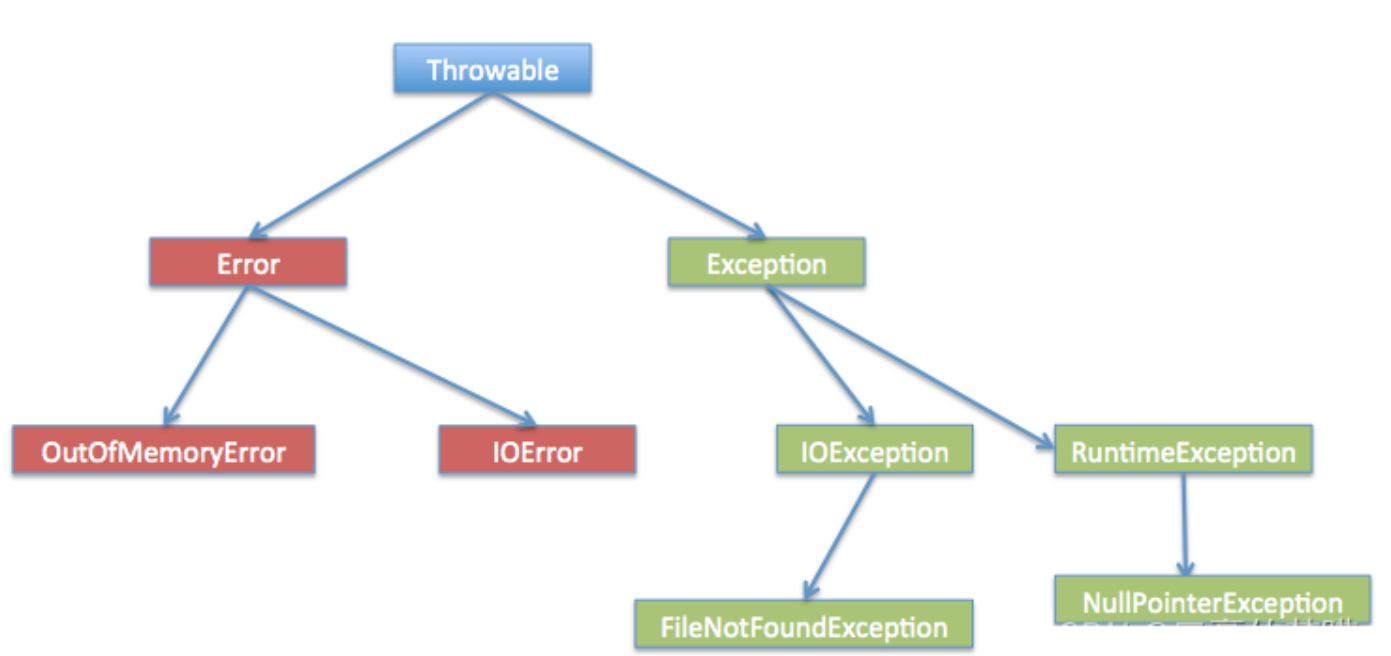
字面理解，接受检查的异常和不接受检查的异常。

根据上面的信息，Error和RuntimeException运行时异常都不能被检查。他们都是程序运行过程中所产生的，只是Error的错误比较严重，一般是JVM产生，而RuntimeException一般是程序逻辑自身的问题。

受检异常就是上面的编译时异常，这些异常在编译时被强制要求捕获或者声明。编译器将会为所有的受检异常提供异常处理器。

PS:其实异常发生了，除了更改程序或者配置等，没有其他的方法。只是对引起程序不正常工作的原因进行分类，就出现了Error和Exception（运行时异常和编译时异常）。

这种分类能让程序员更好的定位因此错误的原因，更方便更高效的进行开发，写出健壮性更好的代码。但是异常并不能改变当前运行的结果，因为从程序开始运行的那一刻，整个逻辑和数据都已经固定了。



异常处理机制

主要是try-catch-finally和throw、throws关键字。

异常都是派生于Throwable类的一个实例，这个实例可以由JVM产生，也可以在程序中手动创建，用throw手动抛出。throw的对象必须是派生于Throwable类的实例，其他类型无法通过编译。

受检异常只有两种选择。要么被捕获处理，要么被抛出，让调用者处理。

而非受检异常没有此强制要求。

throws是用来声明异常，只要是派生于Throwable类的都可以被声明。

方法应该在其首部声明所有可能抛出的受检异常，这是强制要求的。非受检异常不要求必须通过throws声明，因为Error发生后对其无能为力，而如果有运行时异常，那么就是程序自身的问题，应该把时间花在修正程序的错误上，而不是说明程序发生的可能性上。

所以编写程序的时候throws关注点是受检异常，但是非受检异常也可以通过throws声明，只是不强制要求而已。若有多个受检异常，必须在throws中全部声明，如果方法没有声明所有可能的受检异常，编译器就会发出错误提示。

try-catch-finally用来捕获异常。

所有派生于Throwable类都可以通过 catch 捕获，try 中放可能存在异常的方法，如果在 try语句块中的任何代码抛出了一个在 catch 子句中说明的异常类，那么程序将跳过 try 语句块的其余代码，并且执行 catch 子句中的处理器代码。如果在 try 语句块中的代码没有抛出任何异常，那么程序将跳过 catch 子句，如果方法中的任何代码抛出了一个在 catch 子句中没有声明的异常类型，那么这个方法就会立刻退出。

在一个 try 语句块中可以捕获多个异常类型，并对不同类型的异常做出不同的处理。可以为每个异常类型使用一个单独的 catch 子句。需要注意，如果多个catch中的异常非继承关系，那么catch顺序不影响结果，如果catch异常存在类继承关系，那么子类的catch应该放在前面，父类的在后面。

如下图，catch的判断是从上到下，如果父类在前，那么派生于父类的都会被捕获，执行父类的处理代码，这样导致后面子类的处理代码永远不会被执行。

```
try
{
   code that might throw exceptions
}
catch (FileNotFoundException e)
{
   emergency action for missing files
}
catch (UnknownHostException e)
{
   emergency action for unknown hosts
}
catch (IOException e)
{
   emergency action for all other I/O problems
}
```

finally一般用来关闭所占用的资源。如果代码抛出异常，就会终止剩余代码的处理，并且退出这个方法。

这样可能会导致一些程序占用的系统并不能被正确的释放。而不管是否有异常被捕获，finally中子句的代码都会被执行，可以在这里正确的释放资源。

抓抛模型

所有的异常，一定是先有一个实例然后抛出。这个可以JVM完成，也可以手动执行。

这个实例是整个异常处理的源头。然后调用类可以利用try-catch-finally对异常进行捕获，也可以在方法首部通过throws继续向上层抛出。

try-catch-finally是抓，throw/throws是抛。一个类遇到异常，要么捕获后处理，要么继续向上抛出，让调用者进行处理。

catch中可以为空，这样程序就会忽略掉哪些异常。不进行处理本身就是一种处理方式。在catch中也可能抛出异常，也可以手动抛出。这样可以改变异常的类型，使用异常的包装技术，原先的异常时新异常产生的原因，可以让用户抛出子系统中的高级异常，而不会丢失原始异常的细节。

try-catch-finally可以灵活组合。

可以try-catch、try-finally或者try-catch-finally。可以分为以下几种情况

1、try中执行正常，忽略catch，最后执行finally。

2、try中出现异常，且异常在catch中声明，执行catch,最后执行finally.

3、try中出现异常，但catch中未声明，不执行catch，最后执行finally.

4、try中出现异常，且异常在catch中声明，执行catch时出现异常，停止catch中代码，执行finally并且抛出catch中新出现的异常。

```
try {
    System.out.println(1);
    IOException e = new IOException();
    throw e;
    } catch (IOException e){
        System.out.println(2);
        RuntimeException e1 = new  ArrayIndexOutOfBoundsException();
        throw e1;
    } finally {
        System.out.println(3);
    }
```

执行结果：

1
2
3
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException

finally

finally不管是try或者catch中没有由异常，最后大概率都会执行。

这是因为编译器会讲 finally 块中的代码复制两份并分别添加在 try 和 catch 的后面。

但是如果try或者catch中出现了 System.exit()语句，则会直接退出，并不会执行finally模块。

finally中没有return语句 最后返回值为2

```
int i = 1;
try {
    i = 2;
    return i;
} catch (Exception e) {

} finally {
    i = 3;
}
```

finally中有return语句 最后返回值为3

```
int i=1;
try {
    i = 2;
    return i;
} catch (Exception e) {
} finally {
    i = 3;
    return i;
}
```

上面两个代码只是在finally中是否存在return语句的区别，但直接结果却并不相同。

可以看到如果finally中没有return语句，程序就会把finally中的操纵数据忽略掉。

其实finally中的数据操作也是执行了的，但是并没有返回。这是因为在return语句返回之前，虚拟机会将待返回的值压入操作数栈，等待返回。即使 finally 语句块对 i 进行了修改，但是待返回的值已经确实的存在于操作数栈中了，所以不会影响程序返回结果。

在try中的数据处理完，检测到有return语句时，会先将数据压入操作数栈等待返回，然后去执行finally语句，如果finally语句没有将新的结果压入操作数栈，那么只可能返回原先的结果。

从这个角度理解，即使finally中对数据处理，但是返回的依旧时try中的“脏数据”。

finally并不是没有执行，而是执行了却没有没返回。添加return语句则会将finally处理过的数据压入操作数栈返回，原先的“脏数据”失效。

注意这里指的是基本变量，如果是引用类型不受此影响。因为不管在哪里进行运算，处理的都是引用背后的“实体”。

Throwable

Throwable是顶层的异常类，下面是 Throwable 类的主要方法：

1、public String getMessage()

返回关于发生的异常的详细信息。这个消息在Throwable 类的构造函数中初始化了

2、public Throwable getCause()

返回一个Throwable 对象代表异常原因

3、public String toString()

使用getMessage()的结果返回类的串级名字

4、public void printStackTrace()

打印toString()结果和栈层次到System.err，即错误输出流

5、public StackTraceElement [] getStackTrace()

返回一个包含堆栈层次的数组。下标为0的元素代表栈顶，最后一个元素代表方法调用堆栈的栈底

6、public Throwable fillInStackTrace()

用当前的调用栈层次填充Throwable 对象栈层次，添加到栈层次任何先前信息中

自定义异常

Java 的异常机制中所定义的所有异常不可能预见所有可能出现的错误，某些特定的情境下，则需要我们自定义异常类型来向上报告某些错误信息。

一般地，用户自定义异常类都是RuntimeException的子类

自定义异常类通常需要编写几个重载的构造器

自定义异常需要提供 serialVersionUID 序列化唯一ID，方便调试

自定义异常最重要的是异常类的名字，当异常出现时，可以根据 名字判断异常类型

注意事项

当子类重写父类带有throws声明的函数时，其声明的异常范围必须要在父类的支持范围内，即范围不能比父类大。只能保持范围不变或者更精确，不能变大。如果父类没有throws，那么子类也不能有throws，受检异常必须在子类内部捕获处理。

Java程序可以是多线程的。每一个线程都是一个独立的执行流，独立的函数调用栈。如果程序只有一个线程，那么没有被任何代码处理的异常 会导致程序终止。如果是多线程的，那么没有被任何代码处理的异常仅仅会导致异常所在的线程结束。也就是说，Java中的异常是线程独立的，线程的问题应该由线程自己来解决，而不要委托到外部，也不会直接影响到其它线程的执行。

异常处理不能代替简单的测试。捕获异常所花费的时间要远远的超过了测试的时间，所以在可能的情况下，先对执行条件进行判断，而不要等执行出错之后捕获异常。

接口

1. 接口的概念

1、接口中所有方法自动为 public 方法，定义接口时，不必提供关键字 public

2、接口可以定义常量

3、接口没有实例字段

4、JAVA8之前，接口中不能实现方法，之后可以

5、实现接口时，必须把方法默认声明为 public，否则编译器报错

6、Comparable接口文档建议 compareTo 方法应当与 equals 方法兼容。

当 x.equals(y) 时 x.compareTo(y) 就应当返回 0

Java API 大多实现 Comparable 接口的类都遵从了，除了BigDecimal，跟精度有关系 1.00 和 1.0

7、Arrays.sort()

要求数组中的元素必须属于实现了 Comparable 接口的类，且元素间必须可比较

2. 接口的属性

1、可以像使用 instanceof 检查一个对象是否属于某个特定类一样，也可以使用 instanceof 检查一个对象是否实现了某个特定的接口

```
if(object1 instanceof Comparable) {...}
```

2、与接口中的方法自动被设置成 public 一样，接口中的字段总是

```
public static final
```

3、每个类只能有一个超类，但是可以实现多个接口，使用逗号将要实现的各个接口分隔开

3. 静态与私有方法

在 Java 9 中，接口中的方法可以使 private，这个方法可以是 静态方法 或者 实例方法

由于私有方法只能在接口本身的方法中使用，所以他只能用于接口中其他方法的辅助方法

4. 默认方法

可以为接口方法提供一个默认实现，必须用 **default** 修饰符标记这个方法

```
default void remove() {throw new
UnsupportedOperationException("remove")}
```

默认方法可以调用其他方法

作用1:

迭代器中的remove方法，实现迭代器需要提供 hasNext 和 next 方法，这些方法没有默认实现，它们依赖于遍历的数据结构

作用2:

“接口演化”，保证源代码兼容，默认方法可以在为以前的接口增加方法时候使用，这样以前继承了这个接口的类，就不需要修改，因为新加入的方法是 默认方法，自动有默认实现

如果在一个接口中将一个方法定义为默认方法，然后又在超类或另一个接口中定义同样的方法

在Java中：

超类优先 （如果超类提供了一个具体方法，同名而且有相同参数类型的默认方法会被忽略）

接口冲突 （如果一个接口提供了一个默认方法，另一个接口也提供了一个同名参数相同的方法，**必须覆盖这个方法来解决冲突**）

如果一个类扩展了一个超类，同时实现了一个接口，并从超类和接口继承了相同的方法

遵循，**类优先**，即只会考虑类方法

5. 对象克隆

1、浅拷贝：

默认的克隆操作，只拷贝基本字段，不拷贝克隆对象中引用的其他对象

如果克隆对象的子对象是**不可变的**，或者子对象**没有更改器方法**，那么就是安全的

2、深拷贝：

重新定义 clone 方法，**克隆所有子对象**

Cloneable 接口出现和接口的使用没有关系，因为 clone 方法是Object类继承而来的

Cloneable接口是 **标记接口**，不含任何方法，唯一的作用就是允许在类型查询中视同 instanceof

3、所有数组类型都有一个公共的 clone 方法，不是受保护的

可以用这个方法建立一个新数组，包含原数组所有元素的副本

```
int[] arr = {1,2,3,4,5};
int[] arr2 = arr.clone();
```

Java集合

前言

何集合框架都包含三大块内容：对外的接口、接口的实现和对集合运算的算法。

接口：

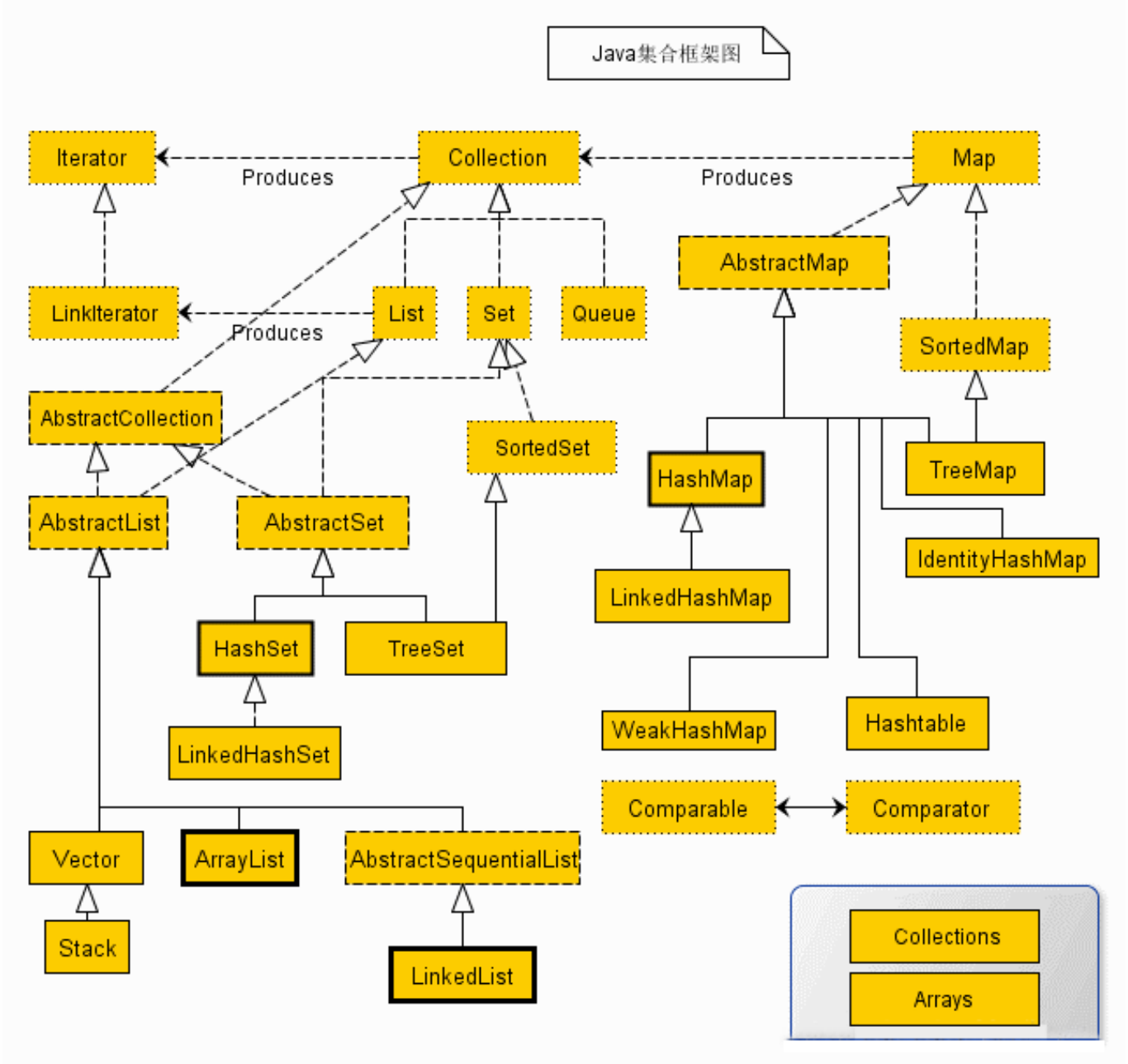
是代表集合的抽象数据类型。例如 Collection、List、Set、Map 等。之所以定义多个接口，是为了以不同的方式操作集合对象

实现（类）：

是集合接口的具体实现。从本质上讲，它们是可重复使用的数据结构，例如：ArrayList、LinkedList、HashSet、HashMap。

算法：

是实现集合接口的对象里的方法执行的一些有用的计算，例如：搜索和排序。这些算法被称为多态，那是因为相同的方法可以在相似的接口上有着不同的实现。



总述

理解Java的集合体系可以从三个层次，分别是最上层的接口、中间的抽象类、最后的实现类。

最上层接口：

表示不同的类型集合，Collection、Map、List、Set、Queue、Deque等，以及辅助性质的接口Iterator、LinkIterator、Comparator、Comparable这些接口是为迭代和比较元素而准备。

中层抽象类：

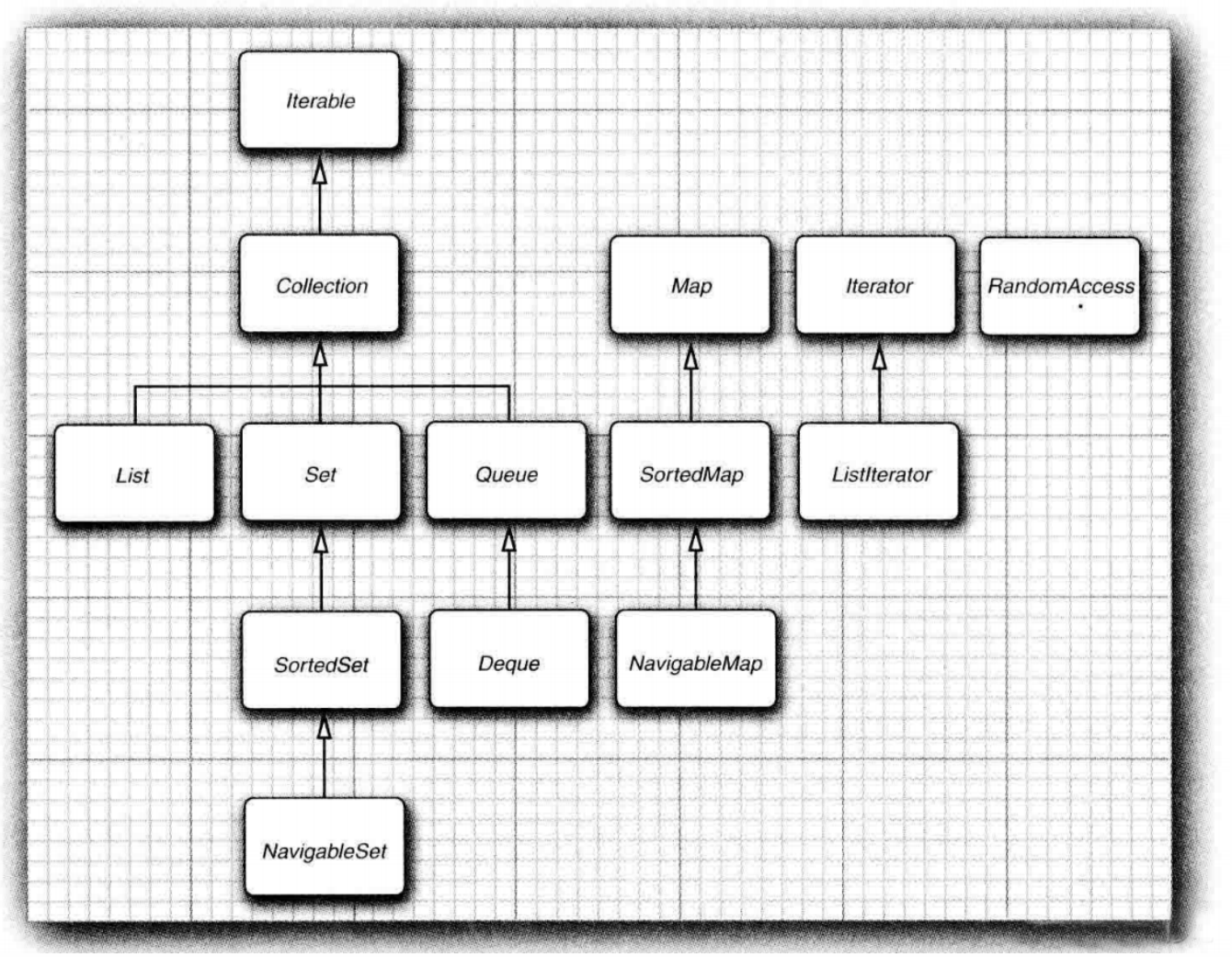
在这里实现大多数的接口方法，继承类只需要根据自身特性重写部分方法或者实现接口方法即可。

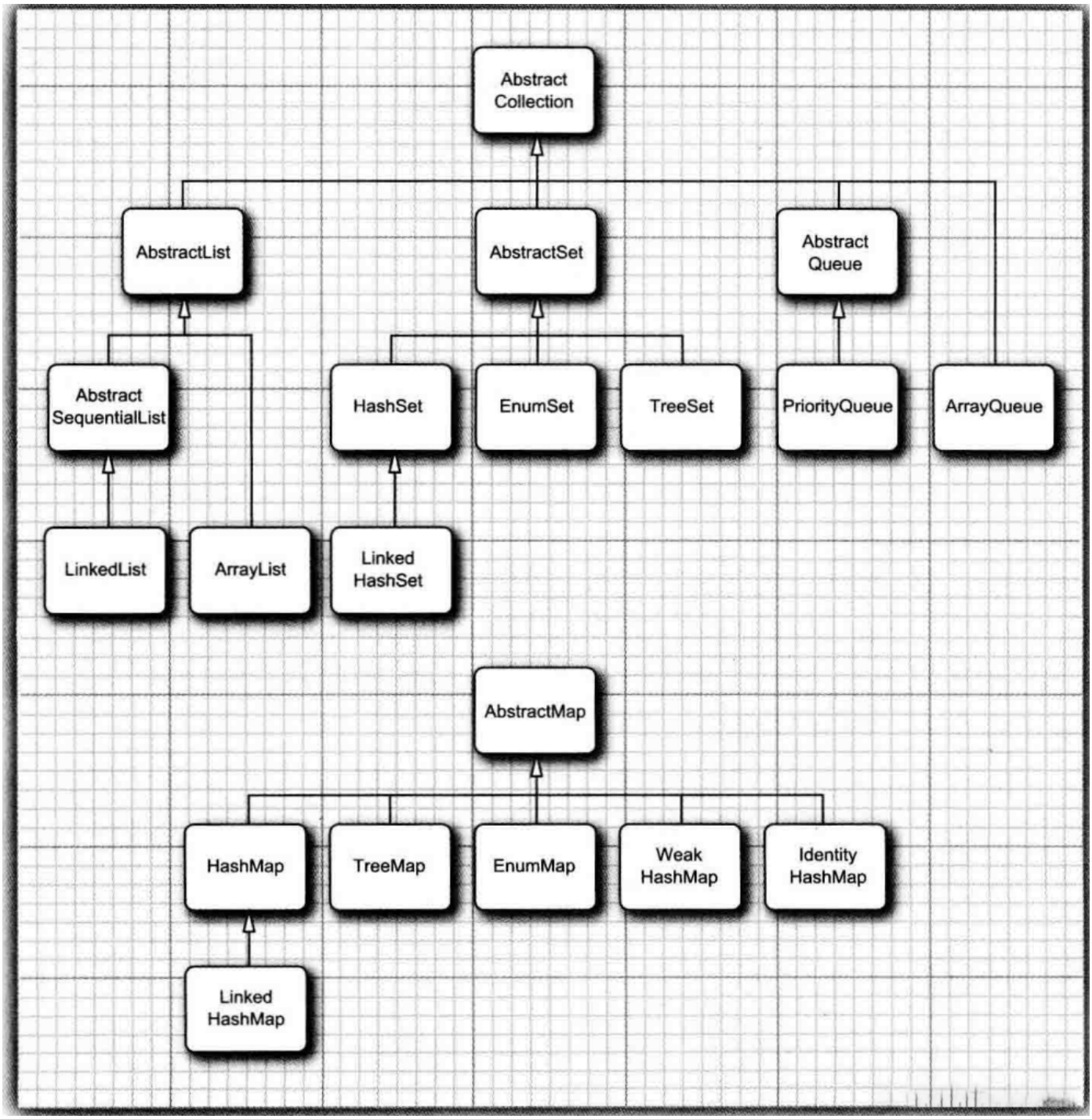
最底层实现类：

对接口的具体实现，主要常用的有：ArrayList、LinkedList、HashMap、HashSet、TreeMap等等。

除此之外，还有Collections和Arrays类用来提供各种方法，方便开发。

接口和实现类都比较熟悉，在敲代码时经常碰到，但是中间的抽象类一般看不到，但是各种各样的抽象类却有着承上启下的作用。

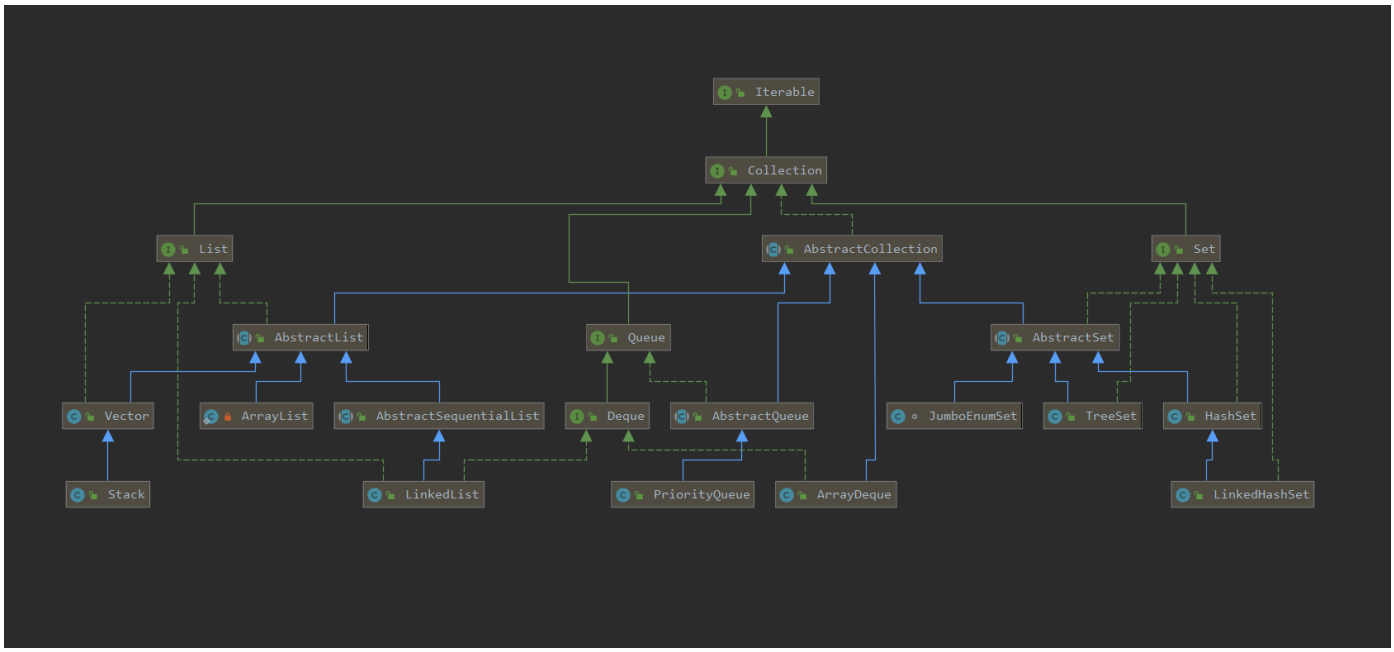




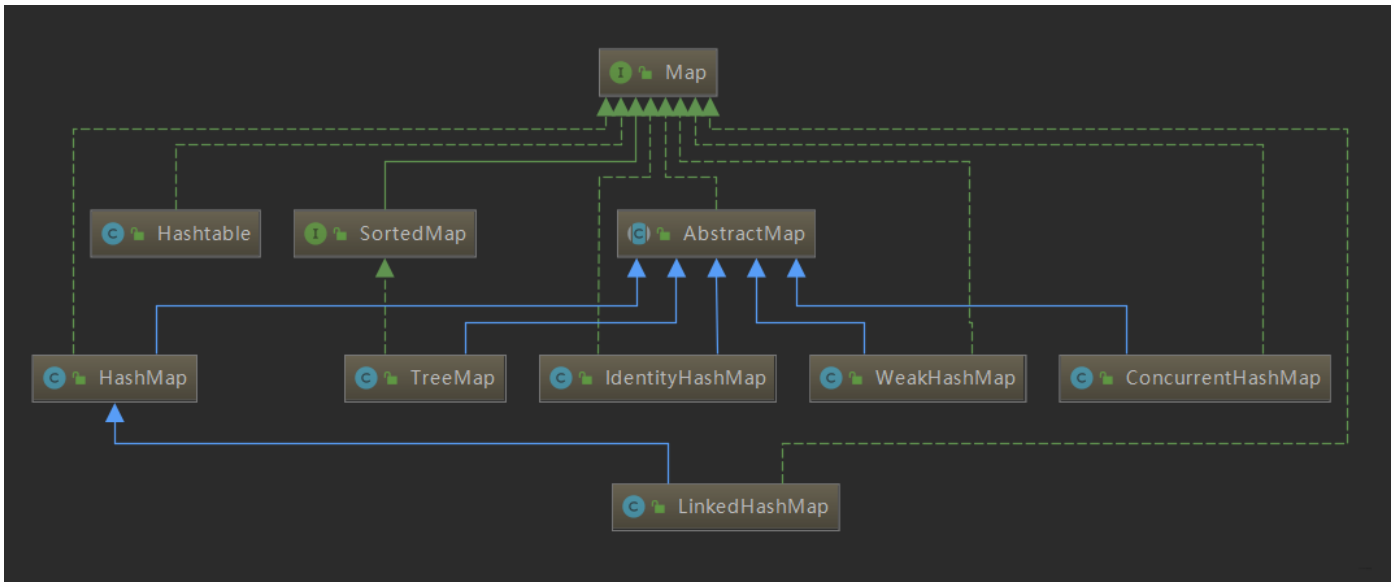
所有的集合都位于java.util包下，Java集合的基本接口是Collection接口和Map接口，所有的实现类都是这两个接口派生出来的。

Collection一次存一个元素，是单列集合，Map一次存一对元素，是双列集合。Map存储的一对元素：键–值，键（key）与值(value)间有对应(映射)关系。

单列集合继承关系图：

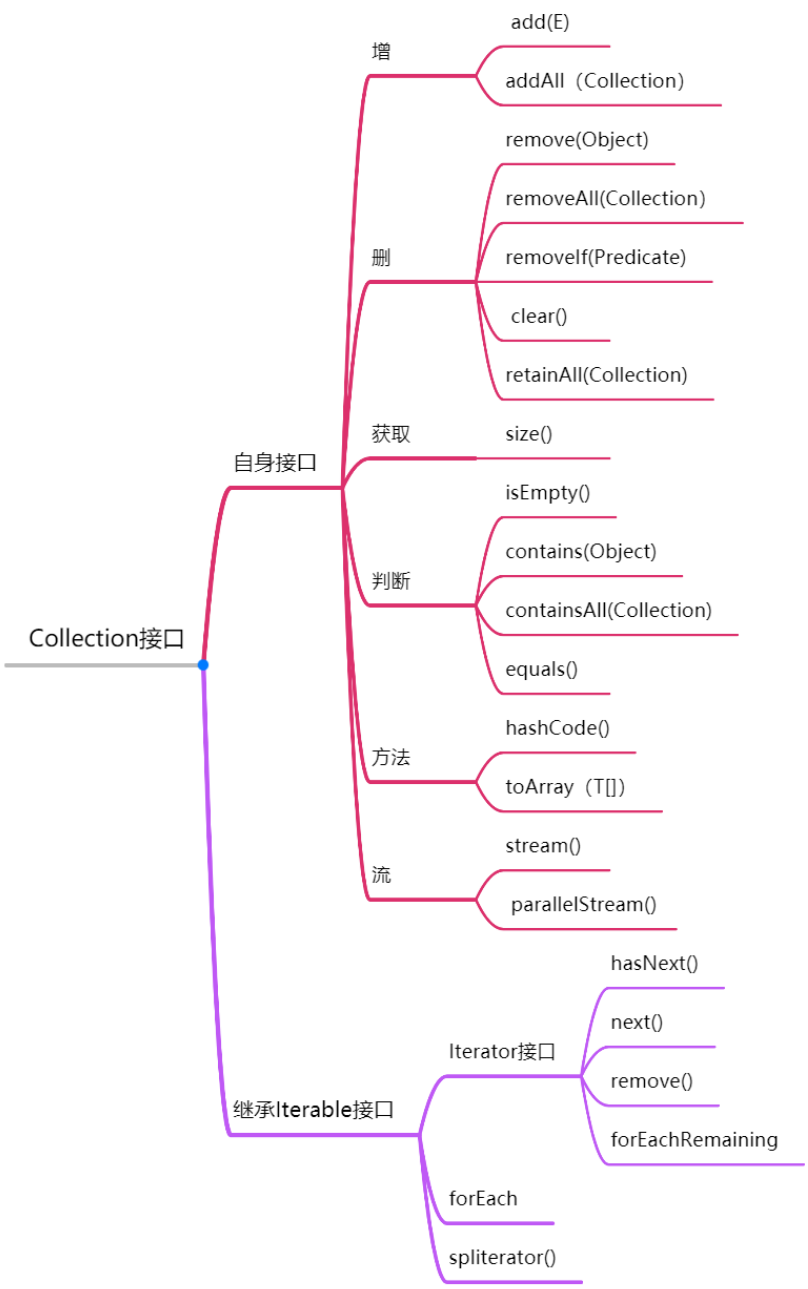


双列集合继承图：



Collection接口

思维导图：



上面的这些方法在实现类中都必须提供，而每一个类都要提供如此多的例行方法将是一件很烦人的事情，为了能够让实现者更容易地实现这个接口，Java 类库提供了一个类 AbstractCollection，它将基础方法 size 和 iterator 抽象化，但是实现了其他的方法。参照上面单列集合继承关系图，Collection下的所有实现类都是继承自AbstractCollection。AbstractCollection下又派生出AbstractList、AbstractSet、AbstractQueue、ArrayQueue。

List下的全部继承自AbstractList，而Set下的继承自AbstractSet。List 是一个有序集合，元素会加到容器的特定位置，并且允许重复元素。而Set是无序集合，并且不允许元素重复。Set要保证没有重复，就需要equals()和hashCode的正确重写。

Java中经常使用的语法糖"for each"循环可以与任何实现了Iterable接口的对象一起工作。Iterable包含了Iterator对象，可以调用iterator()方法，返回一个Iterator对象。这就是常用的迭代器，其中包含hasNext()、next()、remove()、forEachRemaining(Consumer)4个方法

Iterator()的功能简单，只能单向移动：

1、调用iterator()方法返回一个Iterator对象。

2、第一次调用Iterator的next()方法，返回序列的第一个元素。此后每调用一个next(),会返回序列的下一个元素。调用next()方法前最好先调用hasNext()方法，判断序列后面是否还有元素。

3、remove()删除上次next()返回的对象，remove()只能在next()之后使用，并且一个remove()匹配一个next()，不能重复调用.

4、使用hasNext()判断序列中是否还有元素

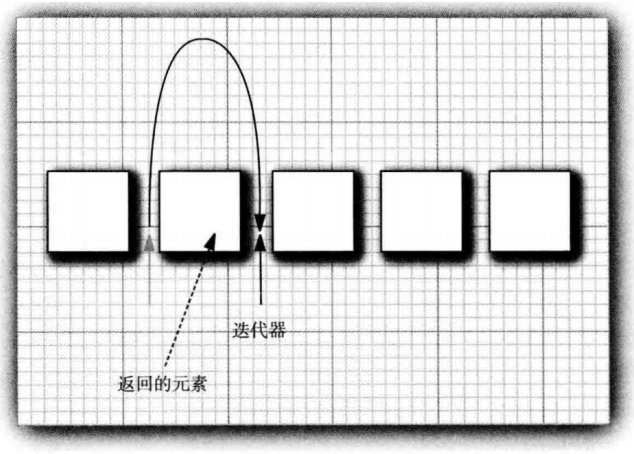
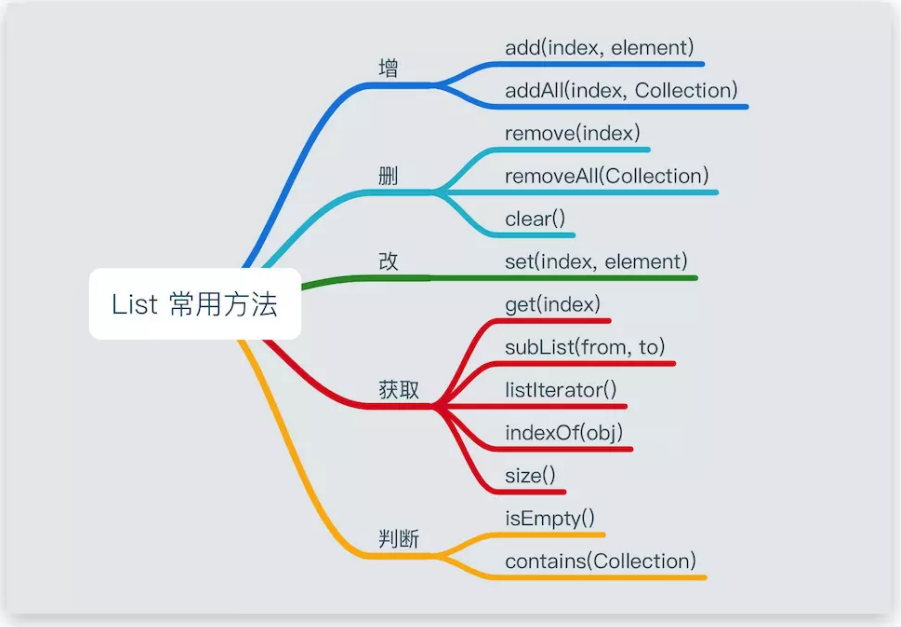


图 9-3　向前移动迭代器

List接口

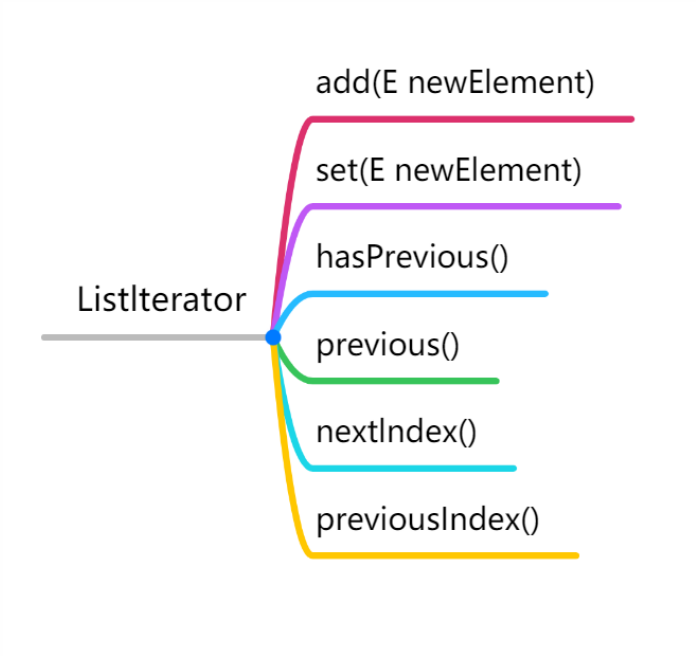
思维导图：



List接口常用的实现类有ArrayList、LinkedList

List接口继承自Collection。List中的方法和Collection中的大致相同，多出sort()、get()、listIterator()等方法，listIterator返回一个链表迭代器。

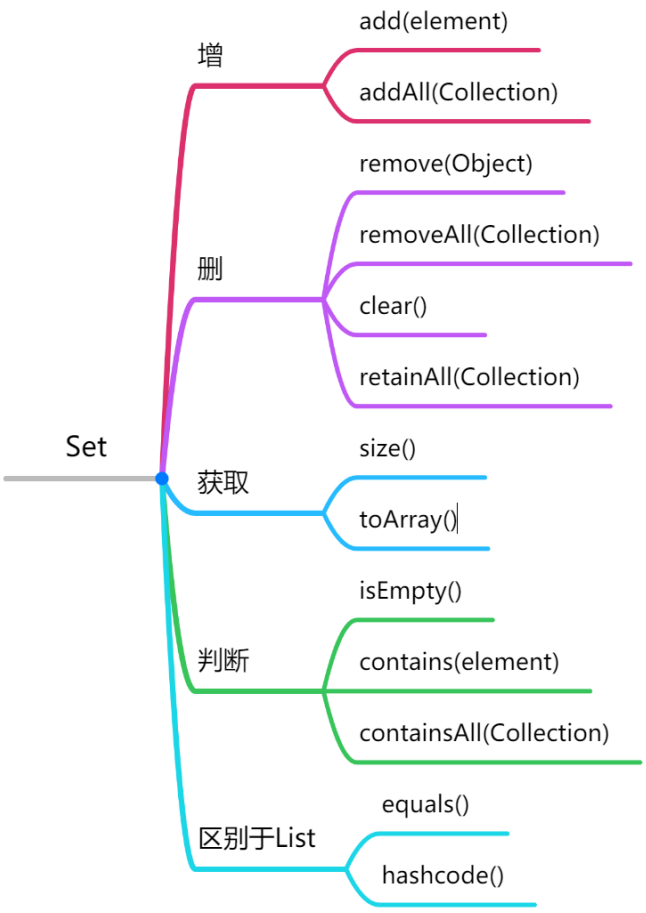
List有着特殊的迭代器接口ListIterator，继承Iterator接口。Iterator是单向的，但是ListIterator是双向的，增加了previous()和hasPreviousO方法用来反向遍历，ListIterator是一个功能更加强大的迭代器。注意只能List使用，其他接口无法使用。



Set接口

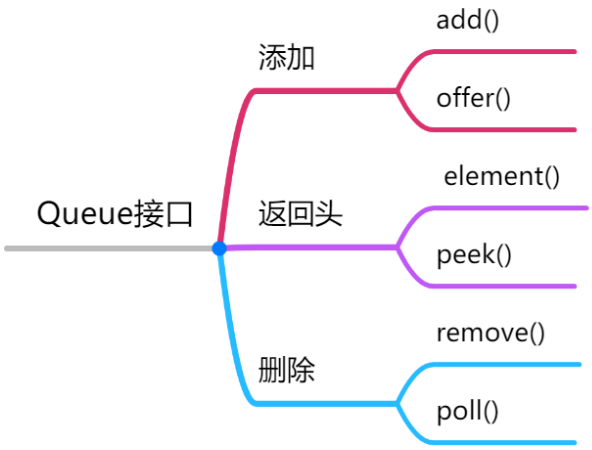
Set继承Collection,其方法的行为有更严谨的定义。Set不允许添加重复元素，为了实现这一点就要适当的定义equals方法和hashcode()方法。

Set常用的实现类有：HashSet、TreeSet、LinkedHashSet

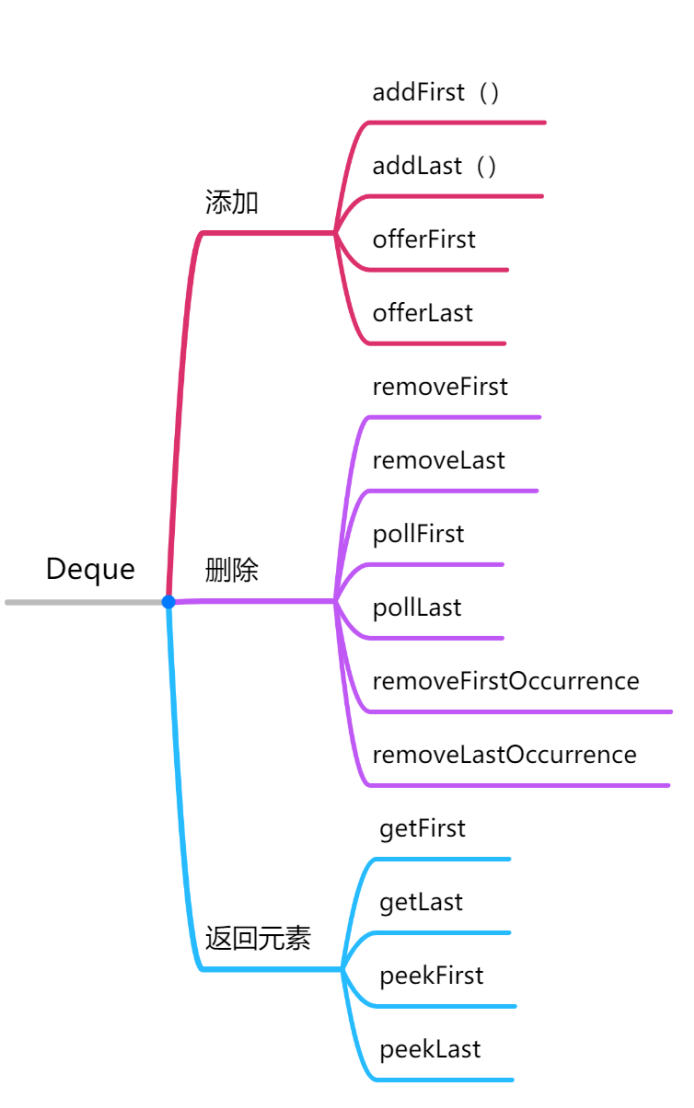


Queue接口

Queue接口也继承自Collenction。很多实现类既实现类List接口，又实现类Queue接口，如LinkedList,也有只实现了一个多，比如ArrayList就没有实现Queue接口，而PriorityQueue也没有实现List接口。这些交叉实现的类最容易混淆。

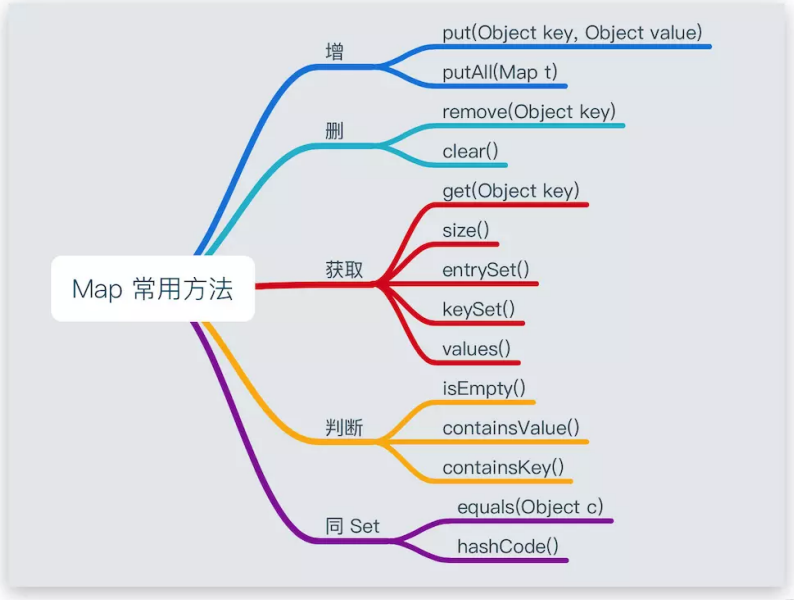


Deque接口继承自Queue,在Deque的基础上更加细化，可以选择从头部或者尾部操作。另外如果Deque中使用Queue的方法，一定要清楚实现类是从头部还是尾部操作的。



Map接口

思维导图：



Map是一组键值对，它是独立于Collection接口的。常用的实现类有：HashMap、TreeMap、HashTable、LinkedHashMap、ConcurrentHashMap等

由双列集合继承关系图可知，Map是顶层接口，sortedMap接口提供排序选项。所有实现类都派生于AbstractMap抽象类。

集合工具类Collections

类中方法都是静态方法，不需要创建对象就可直接使用。

排序

```
// 对指定 List 集合元素进行逆向排序。
void reverse(List list);

// 对 List 集合元素进行随机排序（shuffle 方法模拟了“洗牌”动作）。
void shuffle(List list);

// 根据元素的自然顺序对指定 List 集合的元素按升序进行排序。
void sort(List list);

// 根据指定 Comparator 产生的顺序对 List 集合元素进行排序。
void sort(List list, Comparator c);

// 将指定 List 集合中的 i 处元素和 j 处元素进行交换。
void swap(List list, int i, int j);

// 当 distance 为正数时，将 list 集合的后 distance 个元素“整体”移到前面；
// 当 distance 为负数时，将 list 集合的前 distance 个元素“整体”移到后面。
// 该方法不会改变集合的长度。
void rotate(List list, int distance):
```

查找、替换

```
// 使用二分搜索法搜索指定的 List 集合，以获得指定对象在 List 集合中的索引。
// 如果要使该方法可以正常工作，则必须保证 List 中的元素已经处于有序状态。
int binarySearch(List list, Object key);

// 根据元素的自然顺序，返回给定集合中的最大元素。
Object max(Collection coll);

// 根据 Comparator 指定的顺序，返回给定集合中的最大元素。
Object max(Collection coll, Comparator comp);

// 根据元素的自然顺序，返回给定集合中的最小元素。
Object min(Collection coll);

// 根据 Comparator 指定的顺序，返回给定集合中的最小元素。
Object min(Collection coll, Comparator comp);

// 使用指定元素 obj 替换指定 List 集合中的所有元素。
void fill(List list, Object obj);

// 返回指定集合中指定元素的出现次数。
int frequency(Collection c, Object o);

// 返回子 List 对象在父 List 对象中第一次出现的位置索引；
// 如果父 List 中没有出现这样的子 List，则返回 -1。
int indexOfSubList(List source, List target);

// 返回子 List 对象在父 List 对象中最后一次出现的位置索引；
// 如果父 List 中没有出现这样的子 List，则返回 -1。
int lastIndexOfSubList(List source, List target);

// 使用一个新值 newVal 替换 List 对象的oldVal
boolean replaceAll(List list, Object oldVal, Object newVal);
```

复制

```
void copy(List dest,List src)
```

数组工具类Arrays

类中方法都是静态方法，不需要创建对象就可直接使用。

```
// 对数组array的元素进行升序排序
void sort(array);

// 查询元素值val在数组array中的下标
int binarySearch(array,val);

// 该方法将会把一个数组array转换成字符串
String toString(array);

// 比较两个数组是否相等
boolean equals(array1,array2);

// 把数组array所有元素都赋值为val
void fill(array,val);

// 把数组array复制成长度为length的新数组
copyof(array,length);

// 查看数组array中是否有特定的值val1
Arrays.asList(array).contains(val1);

// 基于指定数组的内容返回哈希码
Arrays.hashCode(array);
```

Java 8 新特性

Lambda 表达式

将代码块传递到某个对象，这个对象将会在某个时间调用（**延迟执行**）

1. 语法

1、即使lambda表达式没有参数，仍然要提供空括号，类似于无参方法

```
() -> {....}
```

2、如果可以推导出一个lambda表的参数类型，则可以忽略其类型

```
(s1, s2) -> {...}
```

3、如果方法只有一个参数，而且这个参数类型可以推导出，那么可以省略小括号

```
a -> {....}
```

4、如果一个lambda表达式只在一些分支返回值，这是不合法的

如(int x) -> {if (x > 0) return 1;}

2. 函数式接口

1、只有一个抽象方法的接口，可以提供一个lambda表达式

2、在Java中，对lambda表达式所能做的也就是转换为 函数式接口

3、ArrayList类有一个 removeIf 方法，它的参数就是一个 **Predicate，这个接口用来传递 lambda 表达式**

如，下面的语句将从一个数组列表删除所有null值

```
list.removeIf(e -> e == null)
```

4、供应者（**supplier**）没有参数，调用时会生成一个T类型的值，用于实现懒计算

```
LocalDate hire = Objects.requireNunNullOrElseGet(day,
new LocalDate(1970,1,1));
```

这种情况下，每次都会构建默认的LocalDate，而day 为 null 的情况很少，因此，可用通过supplier实现延迟这个计算

```
LocalDate hire = Objects.requireNunNullOrElseGet(day, ()->new LocalDate(1970,1,1));
```

此时只有在需要值时，才会调用供应者

3. 方法引用

```
System.out::println;
```

1、指示编译器生成一个函数式接口的实例

覆盖这个接口的抽象方法来调用给定的方法

2、类似于lambda表达式，方法引用也不是一个对象

3、要用 **::**运算符分隔方法名 与 对象 或 类名

主要有三种情况：

（1）对象**::**实例方法 lambda参数作为方法的显示参数传入

（2）类**::**实例方法 String::trim ，lambda表达式会成为隐式对象

（4）类**::**静态方法 Integer::valueOf，lambda表达式会传递到这个静态方法

只有当那个lambda表达式的体只调用一个方法而不做其他操作时，才能把lambda表达式重写成方法引用

4、方法引用不能独立存在，总是会转换为函数式接口的实例

5、包含对象的方法引用 与 等价的lambda表达式还有一个细微差别

如果对象为空，方法引用会直接抛出异常，而lambda表达式只有在调用时才会抛出异常

6、可以在方法引用中使用 this 、super参数

4. 构造器引用

1、与方法引用类似，不过方法名为 new

`Person::new`这就是Person构造器的一个引用，具体哪一个构造器呢，取决于上下文

2、数组的构造器引用`Integer[]::new`

5. 变量作用域

1、lambda表达式可以“捕获”外围作用域中变量的值，只要确保所捕获的值时明确定义的

（1）**lambda表达式中，只能引用值不会改变的外部变量**
（2）lambda表达式中捕获的变量必须是 final 变量

2、**lambda表达式的与 嵌套块 有相同的作用域**，在lambda表达式中声明一个与局部变量同名的参数或局部变量是不合法的

3、lambda表达式中的 this 含义与外面一致

Java 源码学习

Linklist

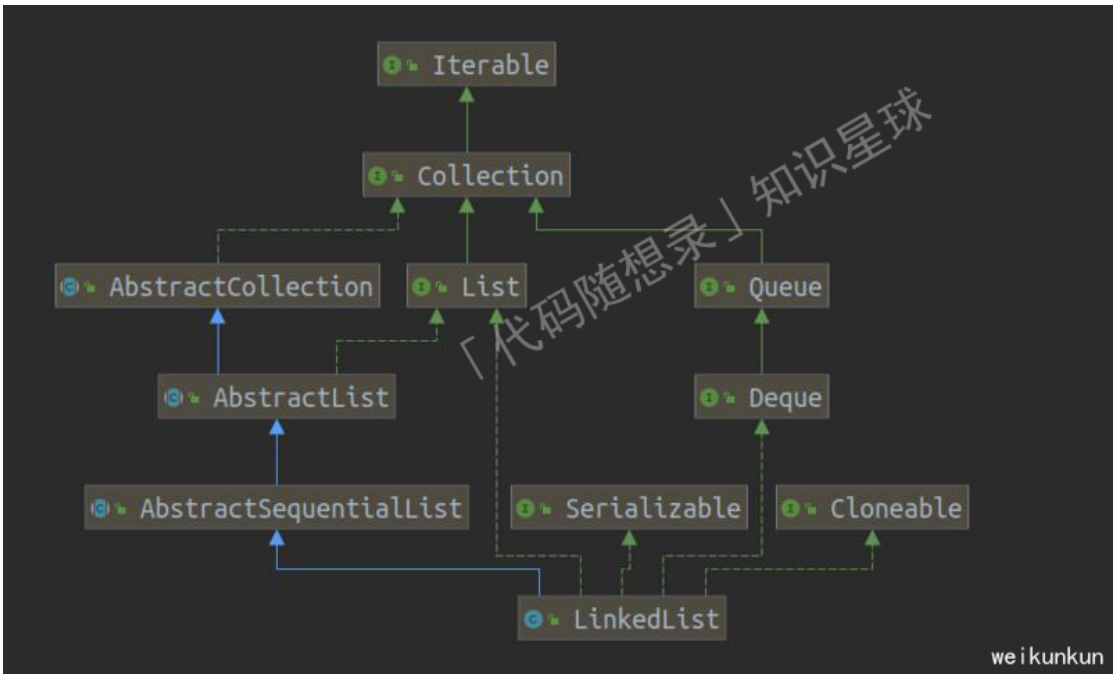
1. 概述

LinkedList底层数据结构是链表，是实现了List接口和Deque接口的双端链表;
其能够高效的实现插入删除操作, 而且也拥有了队列（简单的队列、双端队列）所拥有的特性。

和ArrayList相比，因为其没有实现RandomAccess，是以下标进行访问元素，
所以对于元素访问不及ArrayList，随机访问元素慢。

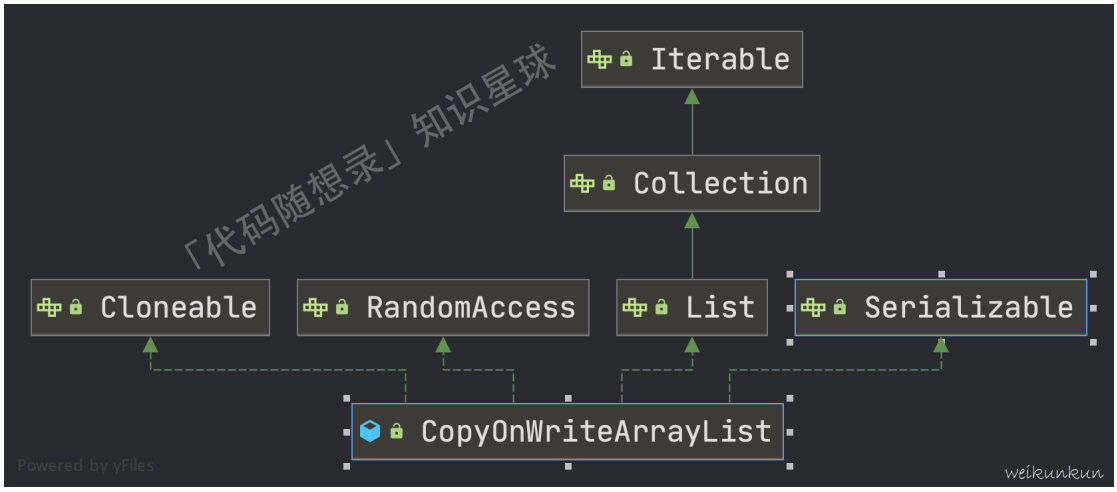
2. 细节

1. 对于普通的单向链表而言，按照下标查找查找的时间复杂度为O(n)，但是对于双向链表按照下标查找，可以利用其特性，选择从头部查找还是从尾部查找，所以更精确来讲，双向链表按照下标查找是O(n/2)，这应该也是LinkedList使用双向链表实现的一个原因吧，在源码里面也能够发现绝大多数的CRUD的API都是先按照下标找到对节点，再进行后续操作。
2. 默认的插入方式是尾部插入，不过也可以明确指定插入方式（头部插入、中间插入、尾部插入），remove方法同理，有这么多插入、删除的形式，为了就是能够在其基础上，实现栈、队列等数据结构。
3. 具有fail-fast机制。



CopyOnWriteArrayList

1. 概述



CopyOnWriteArrayList可以理解成是ArrayList的线程安全的版本，内部也是使用数组实现；

每次对数组的修改都完全拷贝一份新的数组来修改，修改后再替换原来的老数组，这样子只阻塞的了写操作，不阻塞读操作，实现读写分离。

有一问题是，其无法保证实时的一致性，只能保证最终的一致性，所以适用于对实时性要求不高，读多写少的场景，譬如黑白名单。

2. 部分细节

1. 每次对原数组的修改操作，都先加锁，然后copy一份新的数组，在新数组上做修改
2. 锁使用的是ReentrantLock，独占的不公平锁
3. 整个数组使用volatile修饰保证了可见性，结合锁之后，也就确保了单个api操作的线程安全
4. 内部无size属性，直接通过获取当前数组大小得到对应的元素的个数
5. 需要注意的是，因为写时会复制一个近乎等大小的数组，所以需要考虑内存空间和集合使用的业务场景
6. 看完源码之后，加深了System.arraycopy、Arrays.copyOf的理解

3. 相似的思想

使用fork创建子进程，避免了内核直接将整个父进程的虚拟内存的数据（堆、栈、数据段、执行代码段）内容整个复制并分配给子进程。

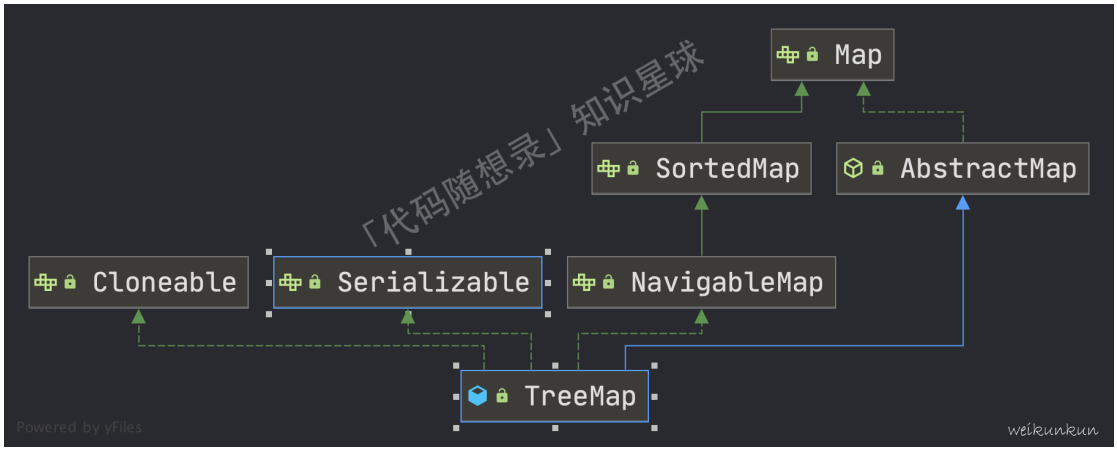
刚开始是让父子进程共享同一个副本，只有在需要写入的时候，数据才会被复制，从而父子进程才拥有各自的副本，在此之前都属于共享的读。

这样之后，对父进程实际物理内存中页的复制被推迟到发生写入的时候。

并且有的时候，共享的副本可能都不会被写（fork之后，调用exec）。

通过写时复制之后，fork的实际开销就变成了复制父进程的页表和给子进程创建PCB。

treemap



概述：

TreeMap底层基于红黑树实现，可以保证在log(n)的时间复杂度内完成containsKey、get、put、remove操作。

同时因为其由红黑树构成，也就是说明了能够维持内部元素的有序性，关于支持内部元素有序性的集合还有LinkedHashMap。

并没有对红黑树做很深的了解，只是跟着整体的思路，使用AVL树实现了一遍。

部分细节：

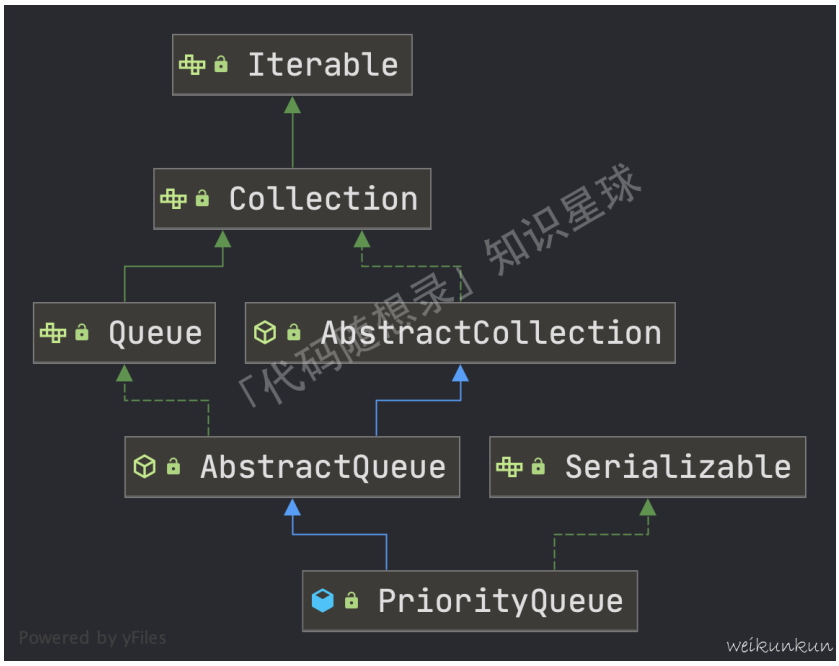
1. 红黑树相比于AVL树，牺牲了部分平衡性，以换取删除/插入操作时少量的旋转次数，整体来说，性能优于AVL树，但是做了性能测试，发现优化了的AVL树和红黑树相比差不了太多。

2. AVL树为了维护严苛的平衡条件，在破坏了平衡之后（插入、删除），需要执行旋转操作。共分为四种：左单旋、先左后右旋、右单旋、先右后左旋。
3. TreeMap内部无扩容的概念，因为使用的是树的链式存储结构
4. 支持范围查找，查找最近的元素
5. 以为内部是按照key进行排序的，所以不支持key为null
6. 排序依据，根据存放的对象是否实现Comprable接口。若实现了，则依据其自定义的compareTo方法，否者需要自定义外部比较器（Comparator），若是都为实现，则报错。

相关练习题：(Leetcode)

285. 二叉搜索树中的中序后继节点(plus会员)
108. 将有序数组转换为二叉搜索树
110. 平衡二叉树
450. 删除二叉搜索树中的节点
270. 最接近的二叉搜索树值(plus会员)

PriorityQueue



概述：

优先队列，是0个或者多个元素构成的集合，集合中按照某种排序方式（元素自身的权重）进行排序，不保证内部元素整体有序，但是每次弹出的元素的优先级最高/低。

内部的数据结构是堆，因为堆底层的结构是完全二叉树，对于树的存储包含有链式存储或者顺序存储，PriorityQueue使用的是顺序存储，所以使用的是Object[]数组，然后利用完全二叉树的性质，解决父子节点关系问题。

默认实现是小根堆

部分细节：

1. 未指定初始化容量大小，默认为11，感觉对于底层使用数组实现的集合，默认大小的规定好要没有啥规律可循

2. 扩容比其他集合多了一步，在数组长度＜64 时，扩容为原先的两倍+2，超过64时，扩容为原先的1.5倍，同时做了放溢出处理，支持最大元素个数Integer.MAX_VALUE;
3. 删除和插入操作均会破坏当前的堆结果，所以每次都需要调用siftUp、siftDown动态调整
4. 插入操作是插入当前堆的末尾，调用siftUp，自底向上调整
5. 删除操作弹出堆顶元素，然后将堆最后一个元素置于堆顶，调用siftDown，自顶向下调整
6. 同样具有fast-fail机制

相关题目：

题目的话，感觉还是根据具体情况使用，对于子序列DP问题也可以引入优先队列来优化暴力搜索的过程

(1) 频率相关问题，结合map使用
(2) TopK问题（海量数据处理）

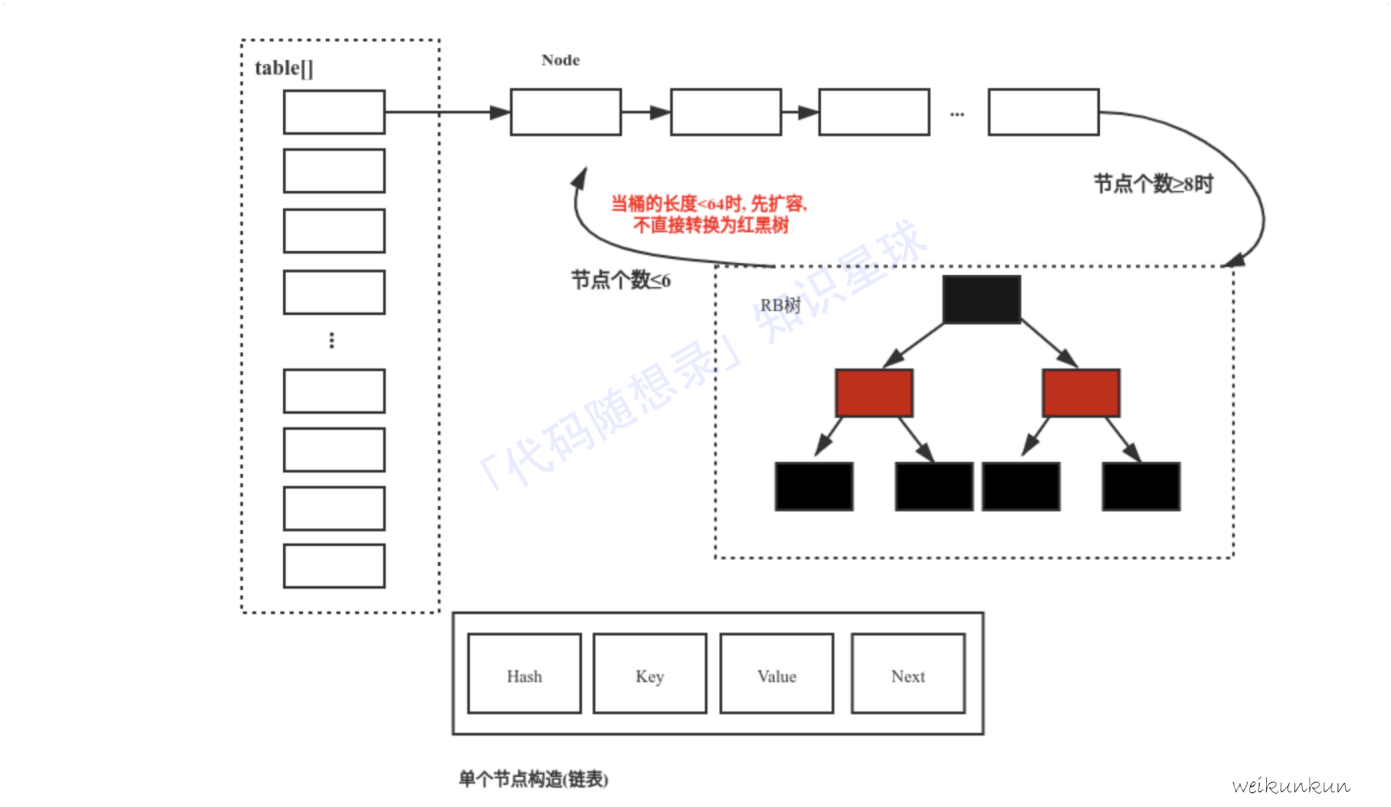
HashMap

概述：

HashMap从宏观来看是基于Hash表实现的，hash冲突的解决方式是拉链法。

具体实现上，在JDK1.8时有做过优化。引入了红黑树，避免因为大量的冲突导致某个位置链表长度过长，使得某些元素的查询效率变低。

同时将链表插入操作的头插法改为了尾插法，保证了插入顺序的同时，也避免了并发操作下的一些异常（并发操作，肯定要想办法解决线程安全嘛，不然出现异常才是正常现象吧）。同时扩容后的rehash的过程也做了优化。



部分细节：

1. 内部采用Node数组来表示Hash表，数组长度为2的幂次，主要是为了能够通过位运算获取key的索引位置，提升计算的效率
2. 链表长度>8时会进行树化，也就是转变为红黑树。大前提是整个Node数组容量>64。同时当树节点数量小于6时又会变为链表
3. 既然是一个集合类，肯定涉及到动态扩容，HashMap内部的扩容机制，默认是扩容为原数组长度的2倍，其次在初始化是若是没有指定容量大小，直接从0扩容为16。如果初始化时指定了容量大小，则整个Hash表的容量大小为最接近指定容量且大于指定容量的2的幂次
4. 扩容过程也伴随着rehash的过程，1.8之前是扩容后，对所有元素rehash，之后映射到对应的位置。1.8时做了优化，巧用的运用了2次幂的扩展(指长度扩为原来2倍)之后，元素的位置要么是在原位置，要么是在原位置再移动2次幂的位置这个特点，避免了rehash的过程，而是直接将key的hash值和oldCap相与，如果为0，则保持原位，如果为1，则放入到原位+oldCap的位置。
5. 内部没有capacity变量，Node数组的容量大小是在扩容时确定的
6. 默认的负载因子为0.75，和Redis中的一样，ThreadLocal的为黄金分割点
7. 对key取hash值，采用的是高地位异或求值，从hash分布的情况来看，离散型更好。
8. Redis中的rehash过程和HashMap1.8之前的实现很像，不过是渐进式的，同时其内部是有新旧两个hash表的，在删除、查找、更新操作时，会操作两个hash表，不过在新增操作时，操作的是新的hash表。

相关题目：

1. 用于dfs、递归中的缓存操作，防止超时
2. 用于查找、频率统计和查重，需要明确key和value的分别代表什么
3. 映射关系，譬如判断给定的字符是否属于ABA这种格式

Java 编译过程

前言

“说下Java代码编译和执行过程？”

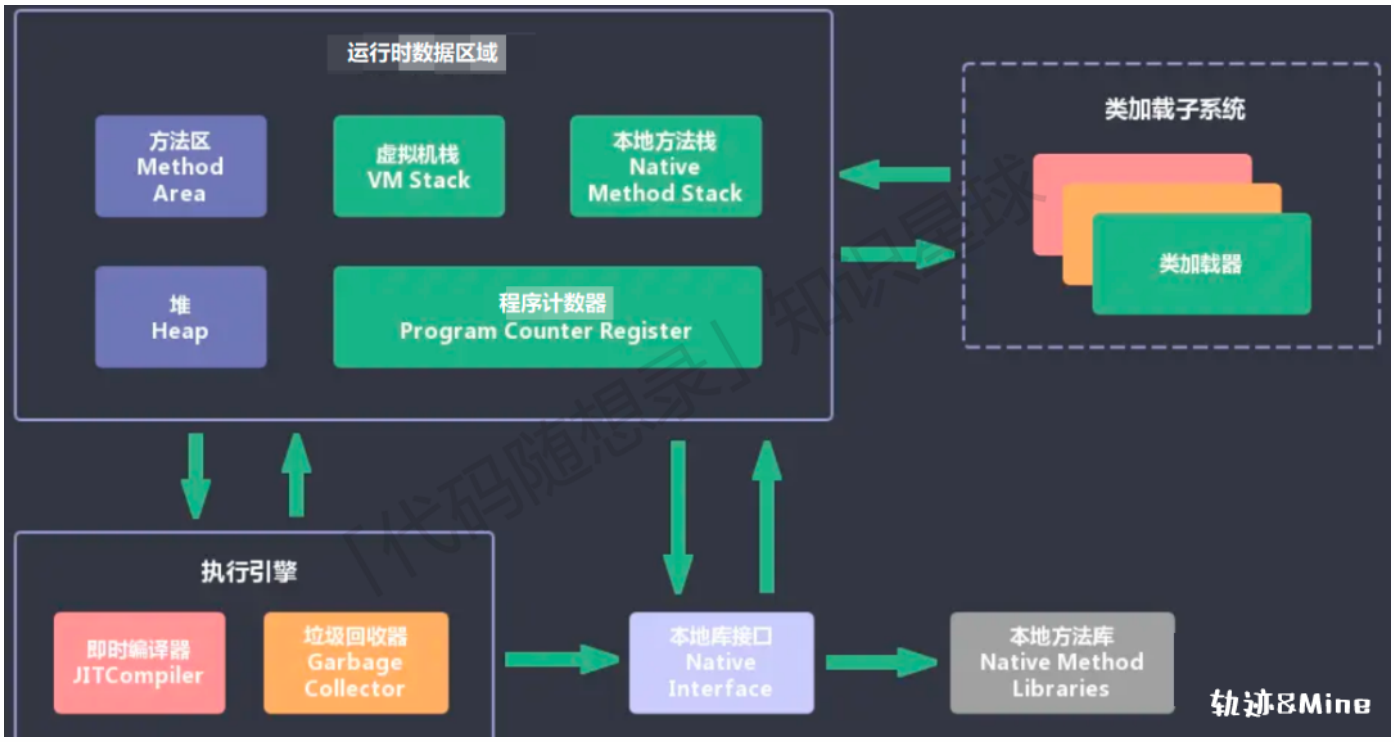
“为什么需要执行引擎，它的工作过程是怎样的？”

“什么是解释器，什么是JIT编译器，它们有什么区别？”

“为什么有了JIT编译器还需要解释器？”

面试过程中，你是否被问到过这些问题，如果不能完全回答出来，那么这篇文章一定要看下去。

执行引擎



从上图可以看出，JVM包含两个子系统和两个组件。

两个子系统：

1. 类加载子系统：负责将字节码文件装载内存成为一个对象。
2. 执行引擎：包含解释器、JIT编译器、垃圾收集器（GC）

两个组件：

1. 运行时数据区
2. 本地接口

执行引擎概述

执行引擎是JVM核心组成部分之一。这里的执行引擎特制虚拟机（VM)中的执行引擎，其实物理机中也存在执行引擎。物理机中执行引擎是直接建立在处理器、缓存、指令集和操作系统层面上的，而虚拟机中的执行引擎是由软件自行实现的。因此，可以不受物理条件制约地定制指令集与执行引擎的结构体系，能够执行那些不被硬件直接支持的指令集格式。那么，什么是不被硬件直接支持的指令集格式呢，别急，我们把这个问题放一放，后面自有解答。

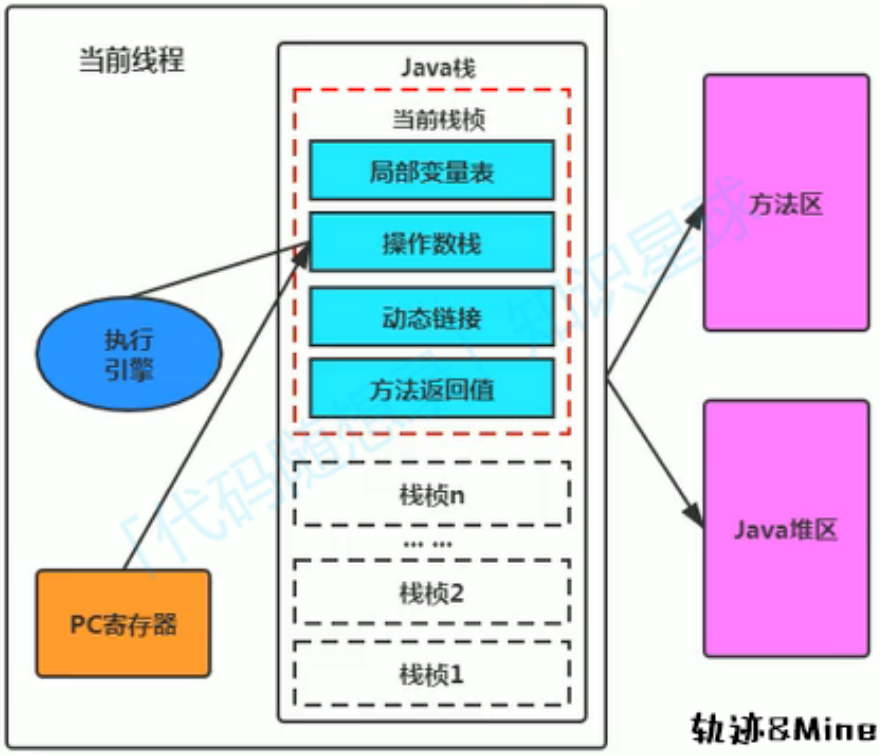
工作流程

为什么需要执行引擎呢？我们知道，jvm主要任务是装载字节码到其内部，但字节码不能直接运行在操作系统上，因为它内部仅仅包含一些能被jvm所识别的字节码指令、符号表以及其他辅助信息。所以，想让java程序执行起来，需要执行引擎将字节码指令解释/编译为对应平台上的本机机器指令。

执行引擎的执行过程：

如下图所示，执行引擎执行过程中执行什么指令依赖于程序计数器（PC寄存器）中的地址，它会根据该地址找到对应的指令。执行完成一条指令操作后，程序计数器会更新下一条需要被执行的指令地址。

在方法执行过程中，执行引擎也可能根据虚拟机栈中局部变量表中存储的对象引用找到堆中对应的对象实例信息，还可能通过对象头中的元数据指针定位到目标对象的类型信息。



指令集

包括：机器码&指令&汇编&字节码

了解代码编译和执行过程之前，先回顾下一些基础知识。

机器码：

各种用二进制编码方式表示的指令

任何用机器码编写的程序，cpu可以直接读取并运行，速度最快，不过可读性差

指令和指令集：

方便人们记忆，将机器码中的0、1序列简化表示为对应的指令。

由于硬件平台不同，执行同一个操作，对应的机器码可能不同，因此不同的硬件平台的同一个指令，对应 的机器码可能不同（如mov）

机器指令与CPU 紧密相关，不同的机器支持不同的指令。

每个平台所支持的指令称为指令集，如x86架构平台的机器支持x86指令集，ARM架构的机器支持ARM指令集

汇编：

由于指令的可读性还是太差，于是人们又发明了汇编语言。

在汇编语言中，用助记符（Mnemonics）代替机器指令的操作码，用地址符号（Symbol）或标号（Label）代替指令或操作数的地址。

在不同的硬件平台，汇编语言对应着不同的机器语言指令集，通过汇编过程转换成机器指令。

由于计算机只认识指令码，所以用汇编语言编写的程序还必须翻译成机器指令码，计算机才能识别和执行。

字节码：

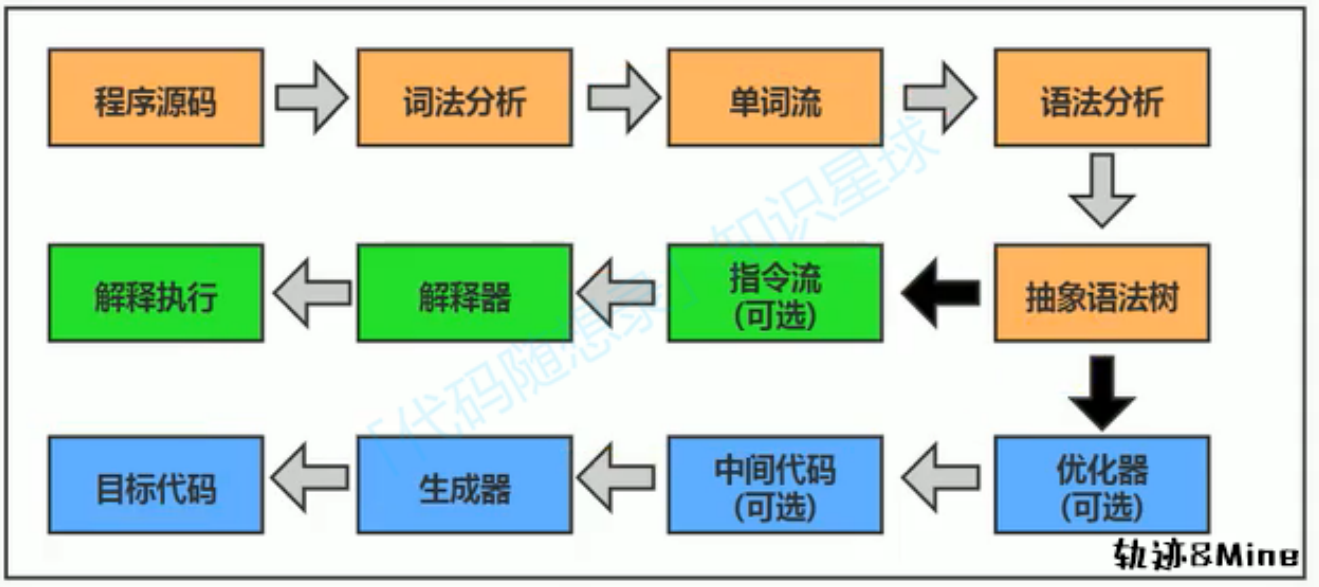
一种中间状态的二进制代码（文件）,它比机器码更加抽象，需要直译器转译后才能转为机器码。

主要是为了实现跨平台的特点，使得特定软件运行和软件环境、与硬件环境无关。

实现方式是通过编译器和虚拟机器。编译器将源码编译为字节码，特定平台上的虚拟机器将字节码转译为自己能识别并执行的指令。

Java代码执行与编译

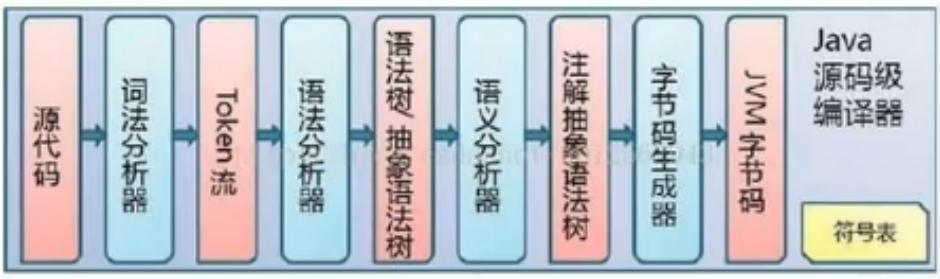
首先我们需要明确的是，大部分程序代码在转换为物理机器能理解的指令集之前，会经过以下步骤：



其中，橙色部分由javac编译器完成，生成线性的字节码指令。通过 javac 编译器，我们可以很方便地将 java 源文件编译成字节码文件。相对于JIT编译器，因为其处于编译的前期，因此又被称为前端编译器。

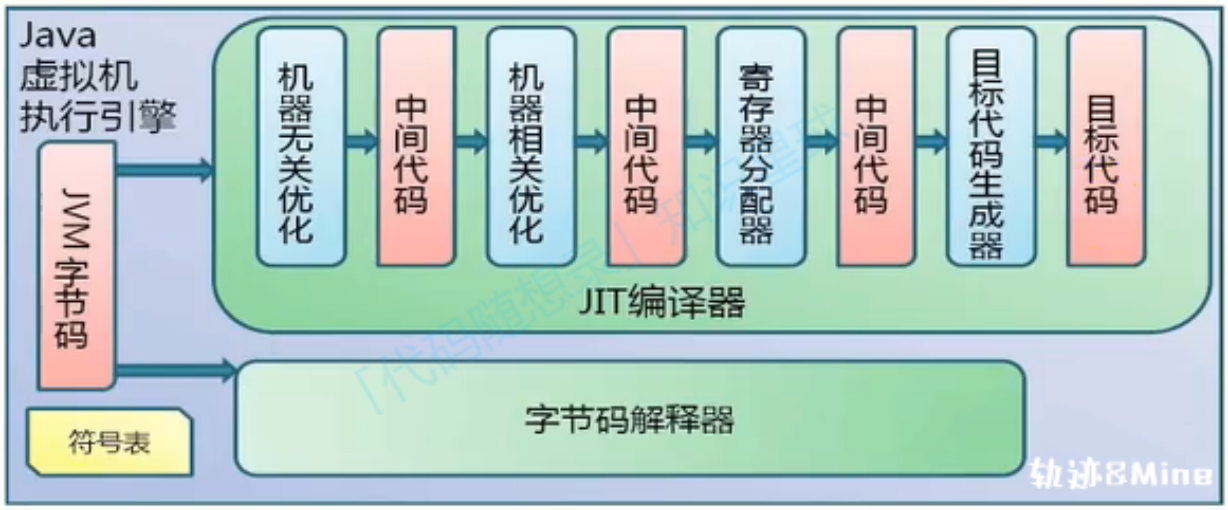
这个过程和JVM无关，因为对于 Java 虚拟机来说，其实际输入的是字节码文件，而不是 Java 文件。

javac编译过程如下所示：



执行步骤中的绿色部分由解释器完成，这也说明了java语言的半解释性。

蓝色对应的是如下的编译过程：



综上，当JVM启动时会根据预定义的规范对字节码进行逐行解释的方式执行，将字节码文件中的内容“翻译”为操作系统能理解的指令。

那么问题来了，为什么说Java是半解释型半编译型语言呢?

实际上，在jdk1.0时，java是解释执行的，后来做了优化。执行引擎在执行字节码指令时既可以选择解释器也可以选择编译器。这样JVM在执行java代码时，可以将解释执行和编译执行结合起来进行。

解释器

问题引出

为什么需要字节码作为中介，不直接将源代码编译为机器能识别的机器指令呢?

这是因为Java设计者的初衷是为了实现跨平台，因此避免采用类似C、C++那种静态编译的方式直接生成机器指令，从而诞生了通过编译器在运行时逐行解释字节码指令从而执行程序的想法。所以，解释器真正意义上所承担的角色就是一个运行时“翻译者”，将字节码文件中的内容“翻译”为对应平台的本地机器指令执行。

解释器的分类

字节码解释器

需要在执行代码时通过纯软件代码模拟字节码的执行，效率十分低

模板解释器

将每一条字节码和一个模板函数相关联，模板函数中直接产生这条字节码执行时的机器码，能提高性能
Hotspot中，解释器主要有Interpreter模块和Code模块组成。

Interpreter模块:

实现了解释器的核心功能

Code模块:

用于管理Hotspot在运行时生成的本地机器指令

解释器的现状

虽然解释器在设计和实现上很简单，但基于它执行已经沦为低效的代名词。针对该问题，jvm平台支持一种称为及时编译的技术。它的目的是避免函数被解释执行，而是将整个函数体编译为机器码，每次函数执行时，只执行编译后的机器码，实现提高程序的执行效率。

JIT编译器

JIT编译器，英文为Just In Time Compiler。根据它，JVM可将源代码直接编译为和本地机器平台相关的机器语言指令。既然根据及时编译的技术，可以提高程序的执行效率，那么为什么不把解释器抛弃掉呢？为什么要将JIT编译器和解释器并存呢？

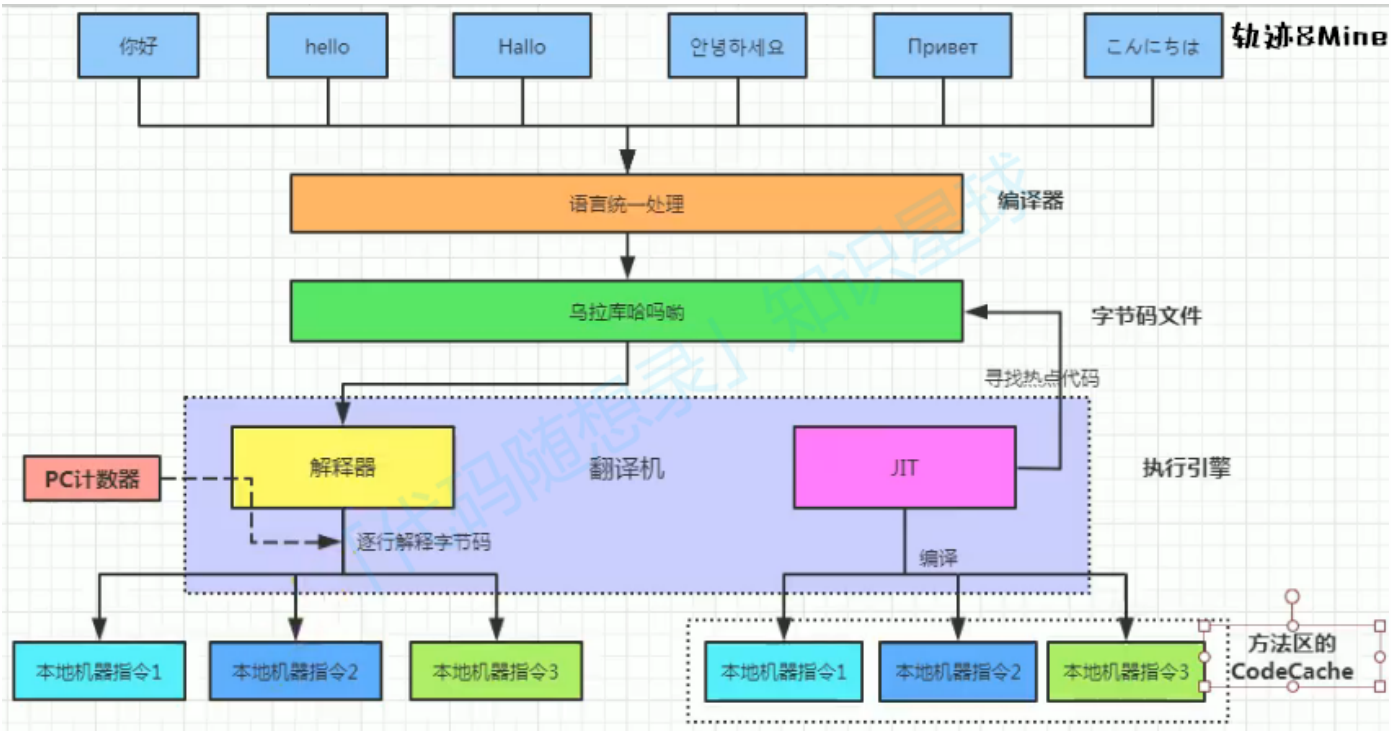
JVM执行代码主要有两种方式：

解释执行：

一段代码，解释一行执行一行。即源程序被编译为字节码文件后，解释器逐行将字节码解释成机器指令并执行。

编译执行：

事先已经被编译成机器码，直接执行，不用解释。源程序被编译为字节码文件后，JIT编译器将字节码编译为机器指令，再由cpu执行。



虽然JIT编译器能提高执行效率，但是在执行机器指令前，它需要对字节码进行编译，因此造成响应时间较长，而解释器可以直接对字节码进行解释执行。因此，Hotspot虚拟机采用二者并存的方式，在jvm执行过程中，二者相互协作，各自取长补短，尽全力去权衡编译本地代码的时间和直接解释执行代码的时间，保证执行效率。

在jvm启动时，解释器可以先发挥作用，而不必等待JIT编译器全部编译完成后再去执行，省去不必要的编译的时间。而随着程序运行时间的推移，即时编译器逐渐发挥作用，根据热点探测功能，将有价值的字节码编译为本地机器指令，以换取更高的执行效率。

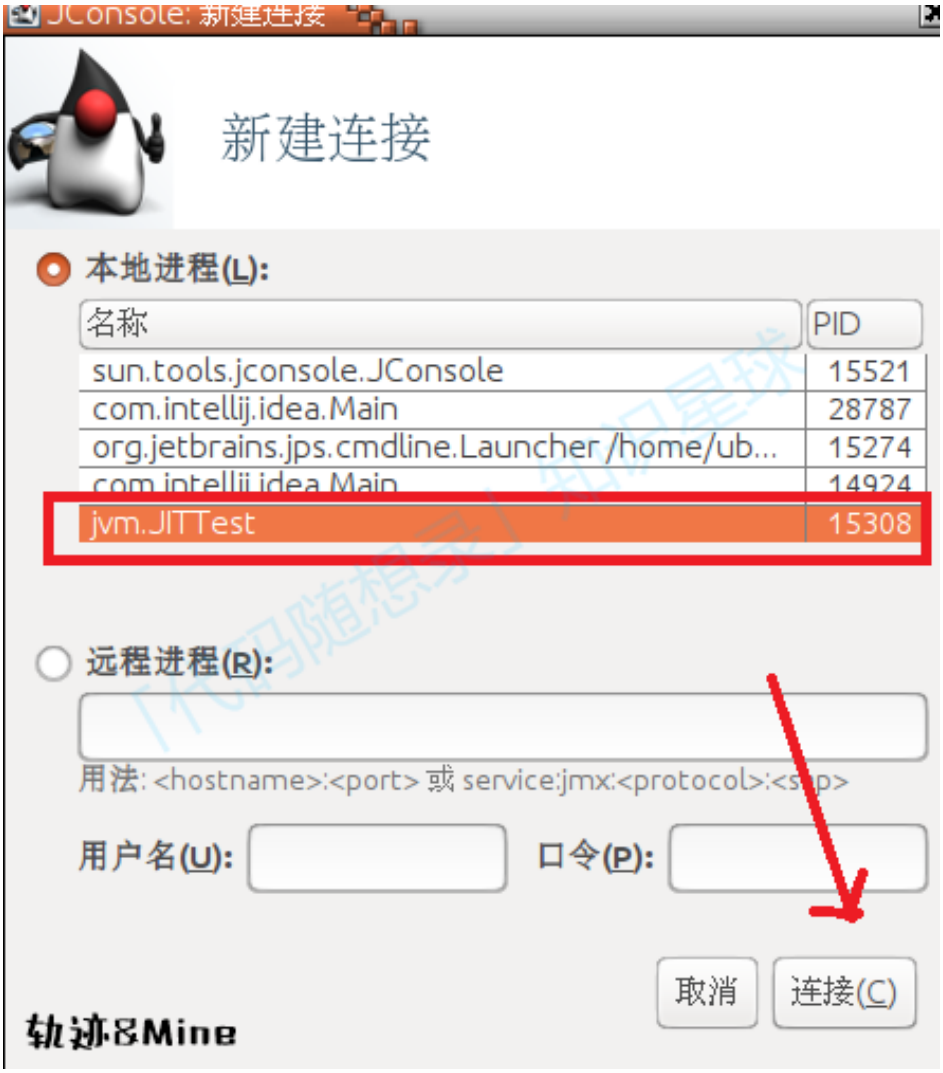
以下通过小例子查看JIT编译器的存在：

```java
package jvm;
/**
 * 运行后，打开jconsole，查看JIT编译器
 */

import java.util.ArrayList;

public class JITTest {
    public static void main(String[] args) {
        ArrayList list = new ArrayList();
        for (int i=0;i<100;i++){
            list.add("Hello!");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

运行以上程序过程中，打开jconsole工具，查看指定的Java进程：



在jconsole的VM概要一栏中，可以看到JIT编译器是确实存在的：

总结

关于执行引擎、解释器、编译器、JIT编译器，值得深入的点还有很多，比如热点探测、如何判断热点代码、热点检测方式，这篇文章只是做一个初步的介绍，待深入学习后，会及时更新相应的内容。

Java 后端编译优化

虚拟机最开始是通过解释器进行解释执行的，解释器是最基础的，但是没有优化，效率和速度都有待提高。但是只依靠解释器不进行优化也是可以的。即时编译器和提前编译器都是采取的优化措施，是可选择的。

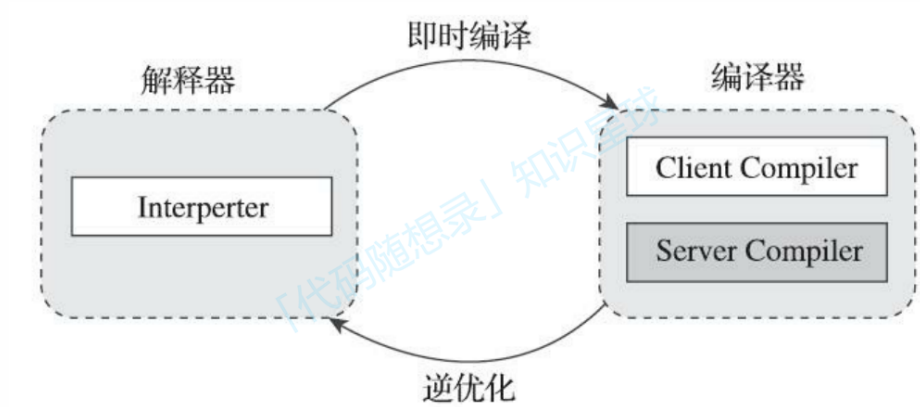
即时编译器

当虚拟机发现某个方法或代码块的运行特别频繁，就会把这些代码认定为“热点代码”（Hot Spot Code），为了提高热点代码的执行效率，在运行时，虚拟机将会把这些代码编译成本地机器码，并以各种手段尽可能地进行代码优化，运行时完成这个任务的后端编译器被称为即时编译器。

解释器与编译器

主流的虚拟机内部都同时包含解释器与编译器，二者各有优势：当程序需要迅速启动和执行的时候，解释器可以首先发挥作用，省去编译的时间，立即运行。当程序 启动后，随着时间的推移，编译器逐渐发挥作用，把越来越多的代码编译成本地代码，这样可以减少 解释器的中间损耗，获得更高的执行效率。

编译器是将代码转换为本机机器码保存下来，需要占用内存。所以内存资源不多的时候用解释器执行，反之可以用编译器来提高程序执行速度和效率。同时，编译器的优化不能保证完全成功，一些激进优化措施是有失败的风险，此时就不能再用编译器，要退回到解释器状态继续执行。



虚拟机有三种编译模式：

混合模式：解释器与编译器搭配使用

解释模式：解释器工作，编译器完全不工作

编译模式：优先采用编译器，但编译器无法进行时采用解释器

HotSpo内置的编译器有："客户端编译器（C1编译器）"和"服务端编译器（C2编译器）"

分层编译：

第0层：程序纯解释执行，并且解释器不开启性能监控功能（Profiling）。

第1层：使用客户端编译器将字节码编译为本地代码来运行，进行简单可靠的稳定优化，不开启性能监控功能。

第2层：仍然使用客户端编译器执行，仅开启方法及回边次数统计等有限的性能监控功能。

第3层：仍然使用客户端编译器执行，开启全部性能监控，除了第2层的统计信息外，还会收集如分支跳转、虚方法调用版本等全部的统计信息。

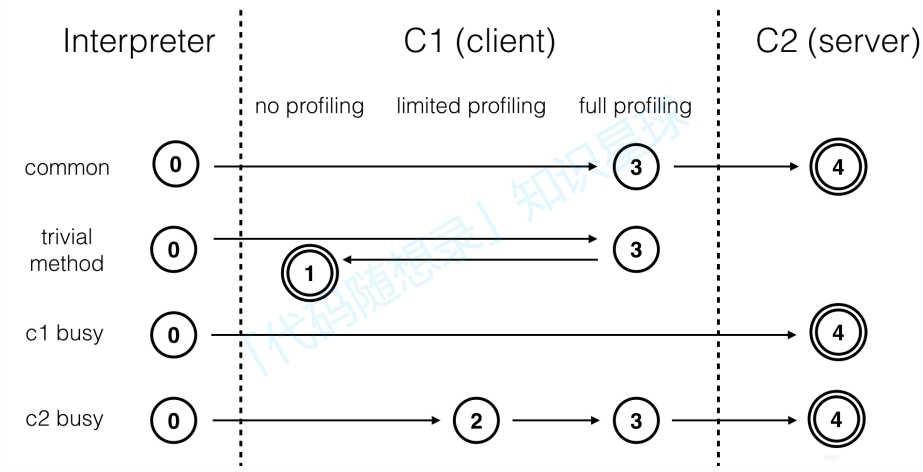
第4层：使用服务端编译器将字节码编译为本地代码，相比起客户端编译器，服务端编译器会启用更多编译耗时更长的优化，还会根据性能监控信息进行一些不可靠的激进优化。

解释器执行的启动速度是比编译器快的，所以程序先再第0层用解释器执行。

而编译器的启动速度也有区别，C1编译器启动要比C2编译器快，这是因为C2会进行更多的优化，所以C2编译出的代码要比C1的代码执行效率好很多。

从启动速度上看 解释器>C1编译器>C2编译器，从程序执行效率上看 4>1>2>3>0, 随着C1性能监控搜集的信息越来越多，性能开销也越大。第二层和第三层基本是为第4层服务的，减少第四层编译执行的时间。

程序最后会在1或者4编译，C2编译出的效果最好，所以最后要进入4，但是如果一段程序本身就简单，可以优化的地方不多，那么就不值得花费时间去搜集程序执行信息，直接去第1层用C1编译就可以，这样编译的效果也很好。



通常情况下，热点代码会被第三层C1编译，然后交给第4层的C2编译。

在字节码较少的情况下，此时性能监控收集的数据很少，就交给第1层的C1进行编译。

在C1比较繁忙时，会在第0层解释执行收集程序执行状态数据，然后直接交给第4层的C2编译。

在C2繁忙的时候，先交给第2层的C1，再交给低3层的C1，最后交给第4层的C2。

热点代码

1. 被多次调用的方法，触发标准编译请求
2. 被多次执行的循环体，触发栈上编译请求

热点代码编译的都是整个方法体。即使是方法中的循环体被认定是热点代码，热点只是整个方法的一部分，但编译器依然必须以整个方法作为编译对象，只是执行入口（从方法第几条指令开始执行）有不同。

编译时会传入入口点字节码序号，这种编译方式因为 编译发生在方法执行的过程中因此称为"栈上替换（OSR）"，即方法还在方法栈上但是具体执行的指令已经被替换了，注意方法栈本身并没有改变也没有移动，只是执行到热点代码时去内存中读取编译后的方法。因为循环体是从某条指令开始执行，一轮完成之后会再次跳转到开始位置，所以只传入一个入口点序号就可以（入口点也是出口）。

热点探测：

判断一个方法是不是热点代码，需不需要出发即时编译。主流有两种方法

1、基于采样的热点探测：

虚拟机会周期性地检查各个线程的调用栈顶，如果发现某个（或某些）方法经常出现在栈顶，那这个方法就是"热点方法"。好处是实现简单，只需要采样当前的栈顶即可。缺点是精度不行，容易因为受到线程阻塞或别的外界因素的影响而扰乱热点探测。

2、基于计数器的热点探测：

虚拟机为每个方法建立计数器，统计执行次数，超过一定阈值则认为是热点代码。好处是结果精确，缺点是比较麻烦，为每个方法建立计数器。

HotSpot采用此种方法，给每个方法准备两类计数器：方法调用计数器和回边计数器。

方法调用计数器：

统计方法被调用的次数。方法调用时判断是否存在编译过的版本，如果存在则执行编译后的本地代码。如果不存在，方法计数器+1，和阈值比较判断是否触发即时编译。一旦超过，申请即时编译。

统计绝对次数：

统计方法所有时间段内执行次数，计数器值不会减少。只要程序执行足够多，一定会触发即时编译。

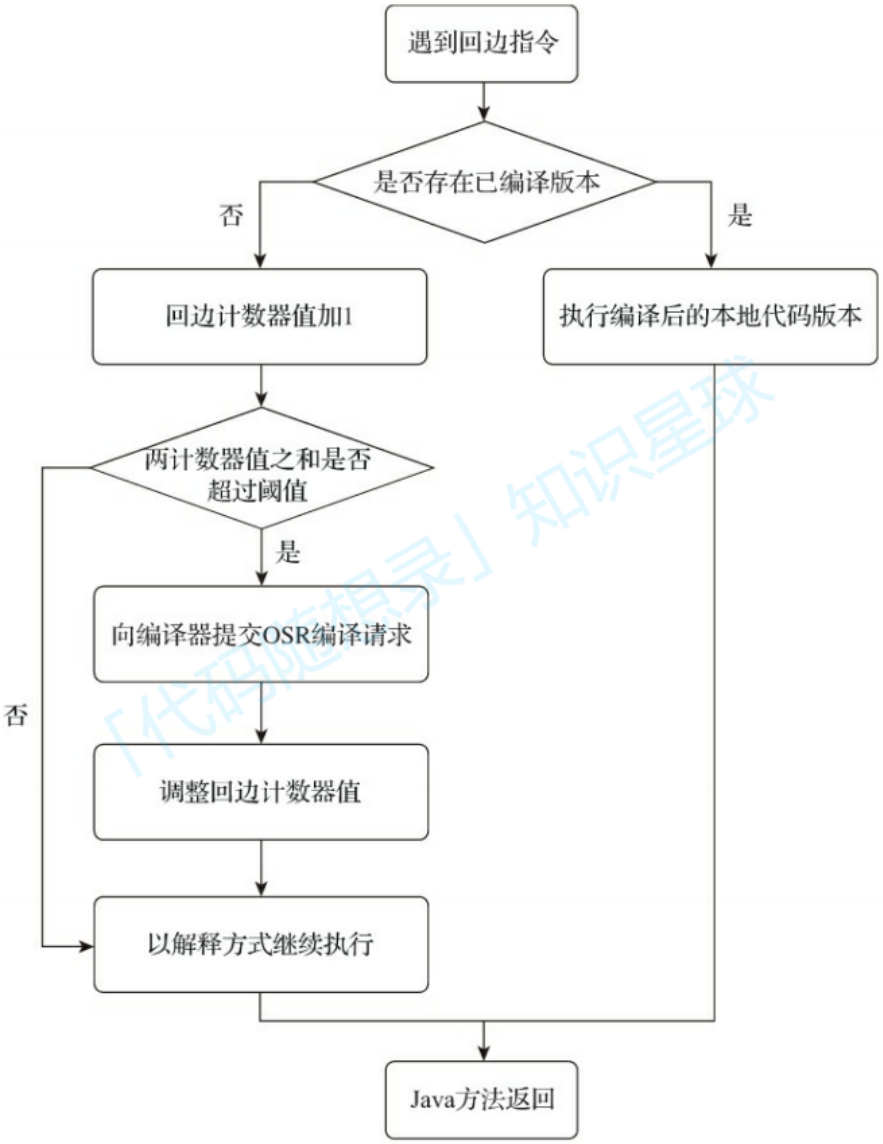
统计相对次数：

统计一段时间执行次数，如果超出时间还未能出发即时编译，统计次数减少一半。这个称为方法调用计数器热度的衰减，这段时间半衰周期。默认统计相对次数。

注意上面这两个方法只有在方法调用器种可以选择，回边调用器只能记录绝对次数。

回边计数器：

统计一个方法中循环体代码执行的次数，在字节码中遇到控制流向后跳转的指令就称为"回边"。当计数达到阈值后会触发栈上的替换操作（OSR），每个循环体都应该有一个回边计数器，所以一个方法内可能存在多个回边计数器。回边调用只记录绝对次数，触发的阈值可以间接设置。


遇到回边指令
是否存在已编译版本
否
是
回边计数器值加1
执行编译后的本地代码版本
两计数器值之和是否超过阈值
是
向编译器提交OSR编译请求
否
调整回边计数器值
以解释方式继续执行
Java方法返回

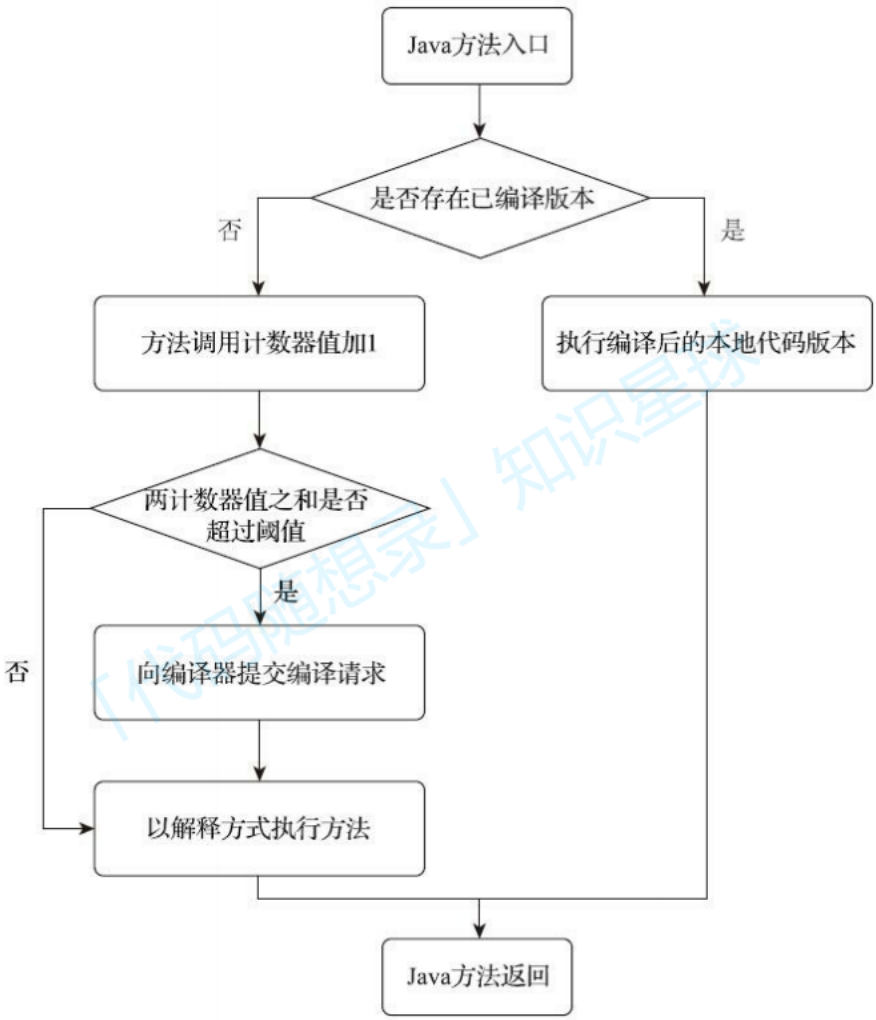


图11-3　方法调用计数器触发即时编译

热点代码经过编译后是放在虚拟机的内存中，这样以后调用时可以直接从内存中读取。

但是如果代码缓存空间用完了，虚拟机遇到热点代码也不会进行即时编译，而是启用解释器执行。

代码缓存占满后并不会抛出OOM异常，只是等待下一次GC清理出空间。当程序较大想要保存更多的编译后本地代码，应该扩大代码缓存空间。

提前编译器

1、在程序运行之前把程序代码编译成机器码的静态翻译工作

2、把原本即时编译器在 运行时要做的编译工作提前做好并保存下来，下次运行到这些代码（譬如公共库代码在被同一台机器 其他Java进程使用）时直接把它加载进来使用。目前主流虚拟机均支持此种方法。

即时编译是在程序运行时发生的，优化需要时间和运算资源，而提前编译是在程序运行之前就编译好，可以减少系统运行时优化消耗。

提前编译和即时编译优劣

提前编译的好处是省资源，避免了在运行时进行优化，节约了运算资源。而即时编译相对提前编译，有以下优点：

1. 性能分析制导优化
2. 激进预测性优化
3. 链接时优化

编译优化技术

方法内联：

把目标方法的代码原封不动地“复制”到发起调用的方法之中，避免发生真实的方法调用。这是编译器最重要的优化手段，被称为优化之母。

除了消除方法调用的成本之外，它更重要的意义是为其他优化手段建立良好的基础，没有内联，多数其他优化都无法有效进。

方法内联需要确定目标方法的具体版本，但是除了在编译期间进行解析的非虚方法和final修饰的方法，其他方法都是虚方法。

虚方法可能会进行重载或者重写，为了确定方法的版本Java虚拟机采用类型继承关系分析技术（CHA），这个可以确定在目前已加载的类中，某个接口是否有多于一种的实现、某个类是否存在子类、某个子类是否覆盖了父类的某个虚方法等信息。

守护内联：

如果内联的是非虚方法，那么直接进行内联就行，这样一定时安全的，要内联的方法版本不会发生改变。

但是如果时虚方法，CHA会查询此方法在当前程序状态下是否真的有多个目标版本可供选择，如果查询到只有一个版本，那就可以假设“应用程序 的全貌就是现在运行的这个样子”来进行内联，这种内联被称为守护内联。

如果后期类的关系没有变化，就一直按照内联优化的程序运行，如果加载了导致继承关系发生变化的新类，那么就必须抛弃已经编译的代码，退回到解释状态进行执行，或者重新进行编译。

内联缓存：

如果向CHA查询出来的结果是该方法确实有多个版本的目标方法可供选择,编译器将采用内联缓存的方式来减少方法调用开销。

在未发生方法调用之前，内联缓存为空。第一次调用方法时，记录下方法接受者的版本信息，随后的每次调用都先比较接收者版本信息，如果一致则认为调用的是同一种方法，就通过缓存来调用，称为单态内联缓存。

如果接收者信息不一样，就会退化为超多态内联缓存。单态内联缓存只是比普通内联调用多一次比较，速度还是很快，但是如果是超多态内联缓存，其速度相当于真正查找虚方法表来进行方法分派。

逃逸分析

分析对象动态作用域，当一个对象在方法里面被定义后，它可能被外部方法所引用，例如作为调用参数传递到其他方法中，这种称为方法逃逸；

甚至还有可能被外部线程访问到，譬如赋值给可以在其他线程中访问的实例变量，这种称为线程逃逸；

从不逃逸、方法逃逸到线程逃逸，称为对象由低到高的不同逃逸程度。

逃逸分析与类型继承关系分析一 样，并不是直接优化代码的手段，而是为其他优化措施提供依据的分析技术。

如果一个对象不会逃到方法和线程之外，即其他方法和线程无法通过任何手段访问到这个对象，或者逃逸程度低（只是方法逃逸）则可以为这个对象实现不同程度的优化：

栈上分配：

如果确定一个对象不会逃逸出线程之外，让这个对象在栈上分配内存，对象所占用的内存 空间就可以随栈帧出栈而销毁。因为不会逃逸出线程，所以其他线程是无法访问到的，那么在堆上创建会浪费GC资源，直接在此线程的栈上创建，随着栈帧出栈而销毁，减少GC压力。栈上分配可以支持方法逃逸，但不能支持线程逃逸

标量替换：

若一个数据已经无法再分解成更小的数据（int char long）来表示就称为标量，如果一个数据可以继续分解(Java对象)，那它就被称为聚合量。

如果把一个Java对象拆散，根据程序访问的情况，将其用到的成员变量恢复为原始类型来访问，这个过程就称为标量替换。

假如逃逸分析能够证明一个对象不会被方法外部访问，并且这个对象可以被拆散，那么程序真正执行的时候将可能不去创建这个对象，而改为直接创建它的若干个被这个方法使用的成员变量来代替。标量替换不允许对象逃逸出方法。

同步消除：

如果对象不会逃逸出线程，就不必做线程间的同步，对这个变量实施的同步措施也就可以安全地消除掉。

公共子表达式消除：

如果一个表达式E之前已经被计算过了，并且从先前的计算到现在E中所有变量的值都没有发生变化，那么E 的这次出现就称为公共子表达式。没有必要花时间再对它重新进行计算，直接用之前的计算结果代表E即可。

数组边界检查消除：

在Java语言中访问数组元素系统将会自动进行上下界的范围检查，对于拥有大量数组访问的程序代码，这必定是一种性能负担。

如果编译器只 要通过数据流分析就可以判定循环变量的取值范围永远在区间[0，length)之内，那么在循环中就可以把整个数组的上下界检查消除掉，这可以节省很多次的条件判断操作。

直白的说就是用过编译器更聪明的判断是否需要进行边界检查，消除不必要的检查节省时间。

Java 内存模型

先强调，Java内存模型（JMM）和Java运行时数据区的别搞混了。

Java内存模型不是介绍内存是怎么分配的，Java的多线程并发依赖Java内存模型。Java运行时数据区才是将内存分成了哪些部分，各部分分别放的是社么。JMM大概点可以分为基本的模型、3个同步原语的内存语句、happens-before等几块，理解了他们也就理解就JMM。

内存模型：为在特定的操作协议下，对特定的内存或高速缓存进行读写访问的过程抽象。

不同的硬件之间的内存模型有些许差异，Java内存模型屏蔽各种硬件和操作系统的内存访问差异，以实现让Java程序在各种平台下都能达到一致的内存访问效果。

通信和同步：

在并发编程中，线程间的通信有两种模型：共享内存和消息传递。

共享内存是指线程间共享，通过读写内存中的共功状态来实现隐式通信，消息传递线程间没有共享状态，线程间必须通过发送接受消息来进行显示通信。共享内存的同步是显示进行的，必须显示的指定某个方法或者某段代码在程序之间互斥执行，

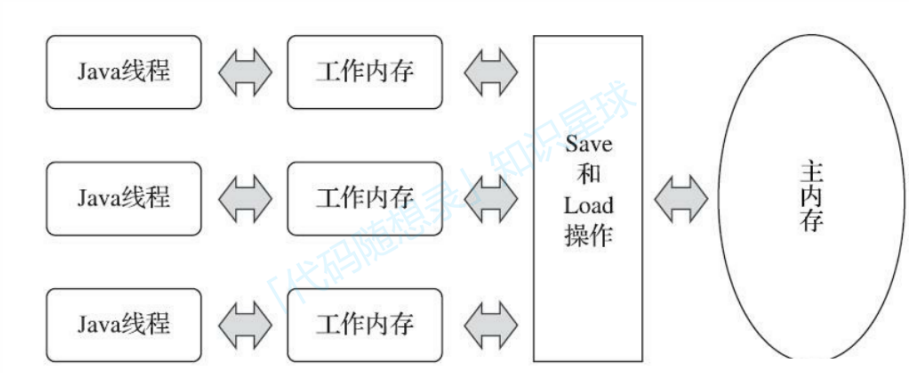
而消息传递的同步是隐式执行的，消息的发送一定在消息的接受之前嘛。

Java采用的是共享内存模型，隐式通信，显示同步。

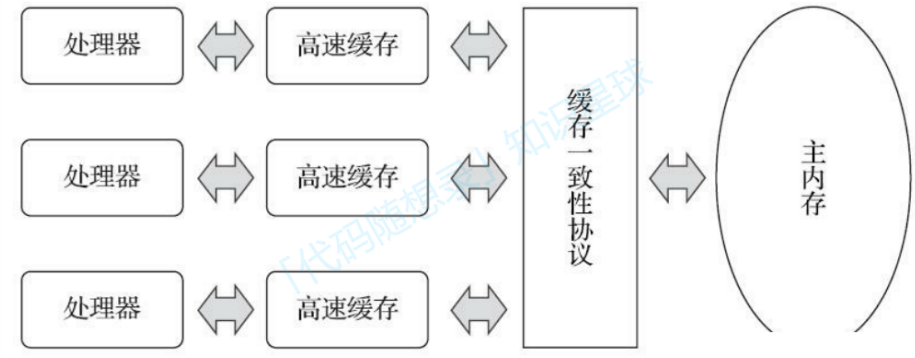
共享模型：

简单理解就是有一个主内存，每个线程有自己的工作内存。

线程工作时先从主内存中把需要的数据拷贝到工作内存中，然后线程从工作内存中读取相关数据进行处理。随后将处理完成的数据从工作内存写回到主内存中。这样当另一个线程从内存中读取数据时，得到的就是之前线程处理过的，两个线程之间完成了通信（隐式通信）。但如何确保线程从主内存中读取的先后顺序，比如后面的线程一定是等之前的线程将数据处理完之后再从主内存读取，这需要在程序中显示的指定互斥执行（显示同步）。



其实工作内存并不是一块真正的内存，它是缓存、写缓冲区、寄存器以及其他的硬件和编译器优化等等一系列的抽象。在硬件层面，真正流程是下图：



主内存中存放的数据应该是线程间共享的。

这里的主内存对应于Java堆中的对象实例数据部分，在Java中，所有实例域、静态域和数组元素都存放堆内存中，堆内存在线程间共享。而局部变量表和方法参数等是线程私有的，并不会被共享，局部变量表和方法参数是分配在虚拟机栈上的，并不是堆。

高速缓存：

处理器首先需要先读取数据，然后才能进行各种运算。但是处理器的速度要比内存的速度高出几个数量级，要是从内存读一个数然后处理器运算一个指令，处理器的时间都被浪费在等待内存上面了。

所以现代计算机系统都不得不加入一层或多层读写速度尽可能接近处理器运算速度的高速缓存（Cache）来作为内存与处理器之间的缓冲：将运算需要使用的数据复制到缓存中，让运算能快速进行，当运算结束后再从缓存同步回内存之中，这样处理器就无须等待缓慢的内存读写了。

工作内存概念不仅有高速缓存，还包括寄存器、硬件和编译器优化等待。

可见性在并发编程中，会出现各种各样的可见性问题。简单的说就是程序员认为的逻辑，和代码真正执行的逻辑并不一致，就会造成可见性问题。程序员不断的检查自己程序逻辑，程序逻辑正确但是运行结果却错误，造成各种认为的“玄学”问题。而只有真正了解计算机是如何处理代码的，才能破解“玄学”。

可见性其实就是一个线程执行的结果，其他的线程是否可以正确的访问到。假设线程A处理完后，将结果放入自己的工作内存，但并没有从工作内存刷回到主内存，其他线程是看不到这个结果的。或者在线程 A 执行之前，其他线程就去主内存中查看，这种肯定也看不到。JMM 通过控制主内存与每个线程的本地内存之间的交互，来为 Java 程序员提供内存可见性保证。

重排序：

在执行程序时，为了提高性能，编译器和处理器会对指令做重排序。分为三类：

1、编译器优化的重排序：

编译器在不改变单线程语义的前提下，可以重新安排语句执行顺序。

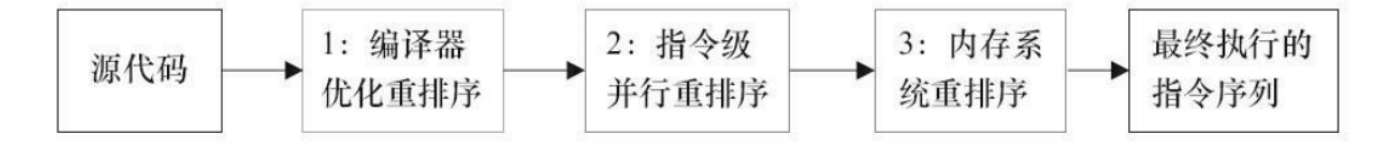
2、指令级别重排序：

如果不存在数据依赖，处理器可以改变语句对机器执行指令的顺序。

3、内存系统的重排序：

由于处理器使用缓存和读写缓冲区，这使的加载和存储操作看上去是乱序执行。

从源代码到最后执行的指令需要分别经历三次排序



数据依赖性：

如果两个操作访问同一个变量，且这两个操作中有一个为写操作，此时这两个操作之间就存在数据依赖性。

数据依赖分为 写后读、写后写、读后写三种情况（只要前后任意操作涉及到写，就存在数据依赖性），只要重排序两个操作的执行顺序，程序的执行结果就会被改变。编译器和处理器在重排序时，会遵守数据依赖性，编译器和处理器不会改变存在数据依赖关系的两个操作的执行顺序。

但此处的数据依赖性只是单个处理器和单个线程中的操作，不同处理器和不同线程之间的数据依赖不被编译器和处理器考虑。

as-if-serial 语义：

不管怎么重排序，单线程程序的执行结果不能被改变。各种重排序必须遵守as-if-serial 语义。比如上面的数据依赖性，并不会重排存在数据依赖性的操作，因为这样会改变结果。单线程程序无论怎样重拍，最后的结果一定是正确的。as-if-serial很好理解，单线程如果都不能保证，那整个程序就乱套啦。它使单线程程序员无需担心重排序会干扰程序，也无需担心内存可见性问题。

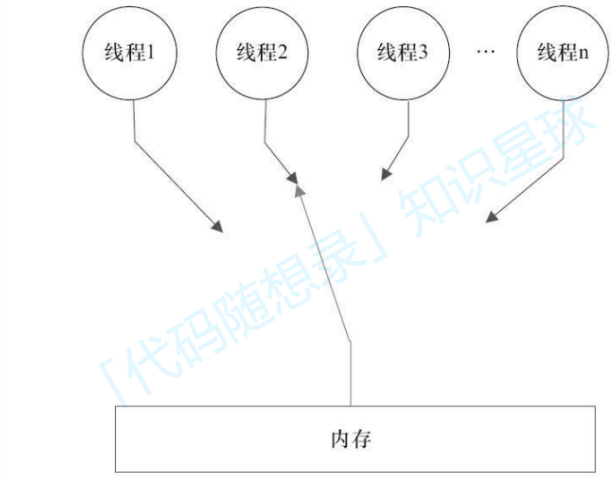
但as-if-serial 隐含的意思时，只要重排序不影响最后的结果，那么你怎样重排都是可以的。单线程不会有影响，但多线程就会出现可见性问题。此时就需要正确的同步。

顺序一致性模型：

顺序一致性模型是一种理想化理论参考模型

1、一个线程中的所有操作必须按照程序的顺序来执行。

2、（不管程序是否同步）所有线程都只能看到一个单一的操作执行顺序。在顺序一致性内存模型中，每个操作都必须原子执行且立刻对所有线程可见。



JMM并没有实现顺序一致模型，因为那样的成本太大了，各种优化措施都不能用。

JMM只保证：如果程序是正确同步的，程序的执行将具有顺序一致性-----即程序的执行结果与该程序在顺序一致性内存模型中的执行结果相同。

注意是执行结果相同，并不是按照一致性执行，JMM允许重排序且只有在正确同步下才能保证可见性。对于未同步或未正确同步的多线程程序，JMM 只提供最小安全性。

顺序一致性模型中，所有操作完全按程序的顺序串行执行。

JMM中，临界区内的代码可以重排序（但是不允许临界区内的代码“逃逸”，那样会破坏互斥性，并不是正确的同步）。

JMM 会在退出临界区和进入临界区这两个关键时间点做一些特别处理，使得线程在这两个时间点具有与顺序一致性模型相同的内存视图。

因为同步的互斥性，使得其他线程根本看不都临界区内重排序，线程只能在公共节点出看到相同的内存视图。

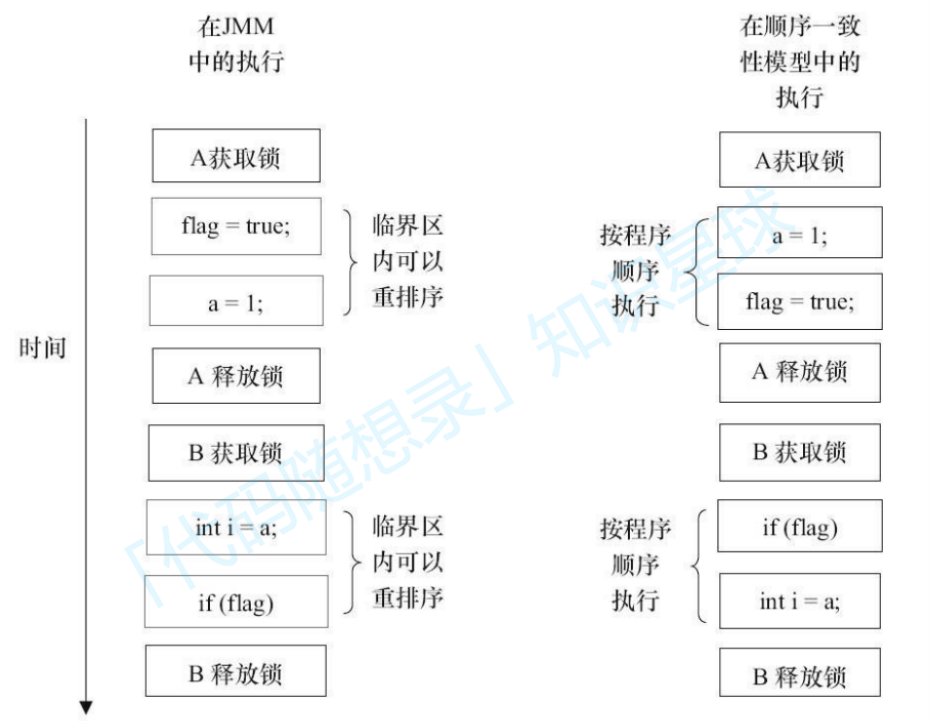


图 3-13　两个内存模型中的执行时序对比图

JMM的基本方针：在不改变正确同步的程序执行结果前提下，尽可能地为编译器和处理器的优化打开方便之门。

未正确同步的程序，JMM提供最小安全性：线程读取到的值，要么是默认值（0，null等），要么是其他线程写入的值。

注意，保证其他是其他线程写入的值并不是说这个值就是正确的，最小安全性只是确保读到的值不是胡乱产生的而已。假设64位数据，先写入低32位后，此时其他线程来读取此值，此种情况虽然读取到的64位数不正确（一半写入一半没写），但是依然符合最小安全性。因为即使这个64位数是拼凑起来的，但它也是之前的值（以前线程写入）和后32位（现在线程写入），都是由线程写入的，并不是胡乱产生的。

JMM 不保证未同步程序的执行结果与该程序在顺序一致性模型中的执行结果一致。不同步JMM认为这两个线程间没有关系，各自执行优化。

volatile 的内存语义

volatile 变量自身具有下列特性：

1、可见性：

对一个 volatile 变量的读，总是能看到（任意线程）对这个 volatile 变量最后的写入。

2、原子性：

对任意单个 volatile 变量的读/写具有原子性，但类似于 volatile++这种复合操作不具有原子性。

3、volatile 写的内存语义：

当写一个 volatile 变量时，JMM 会把该线程对应的本地内存中的共享变量值刷新到主内存。

4、volatile 读的内存语义：

当读一个 volatile 变量时，JMM 会把该线程对应的本地内存置为无效。线程接下来将从主内存中读取共享变量。

从内存语义的角度讲，volatile写和释放锁具有相同的内存效果，把本线程处理的共享变量的刷回到主内存，其他线

程便可看见。volatile读和获取锁具有相同的内存效果，从主内存中读取最新的共享变量。

线程 A 写一个 volatile 变量，实质上是线程 A 向接下来将要读这个 volatile 变量的某个线程发出了（其对共享变量所做修改的）消息。

线程 B 读一个 volatile 变量，实质上是线程 B 接收了之前某个线程发出的（在写这个 volatile 变量之前对共享变量所做修改的）消息。

线程 A 写一个 volatile 变量，随后线程 B 读这个 volatile 变量，这个过程实质上是线程 A 通过主内存向线程 B 发送消息。

volatile 内存语义的实现

对于主内存和工作内存之间的交互，JMM规定了8种操作：

lock（锁定）：作用于主内存的变量，它把一个变量标识为一条线程独占的状态。

unlock（解锁）：作用于主内存的变量，它把一个处于锁定状态的变量释放出来，释放后的变量 才可以被其他线程锁定。

read（读取）：作用于主内存的变量，它把一个变量的值从主内存传输到线程的工作内存中，以 便随后的load动作使用。

load（载入）：作用于工作内存的变量，它把read操作从主内存中得到的变量值放入工作内存的 变量副本中。

use（使用）：作用于工作内存的变量，它把工作内存中一个变量的值传递给执行引擎，每当虚 拟机遇到一个需要使用变量的值的字节码指令时将会执行这个操作。

assign（赋值）：作用于工作内存的变量，它把一个从执行引擎接收的值赋给工作内存的变量， 每当虚拟机遇到一个给变量赋值的字节码指令时执行这个操作。

store（存储）：作用于工作内存的变量，它把工作内存中一个变量的值传送到主内存中，以便随 后的write操作使用。

write（写入）：作用于主内存的变量，它把store操作从工作内存中得到的变量的值放入主内存的 变量中。

同时还有许多规则，比如unlock必须在lock之后，read和load、store和write两两成对，不允许单独出现等等。和volatile相关的只需要记住两个，load和store。标注volatile变量会限制一部分重排序：

是否能重排序	第二个操作		
第一个操作	普通读 / 写	volatile 读	volatile 写
普通读 / 写			NO
volatile 读	NO	NO	NO
volatile 写		NO	NO

表 3-5　volatile 重排序规则表

1. 第二个操作是volatile写，不管第一个操作是啥都不允许重排序。确保 volatile 写之前的操作不会被编译器重排序到 volatile 写之后
2. 第一个操作时volatile读，不管第二个操作是啥都不允许重排序。确保 volatile 读之后的操作不会被编译器重排序到 volatile 读之前。
3. 第一个是volatile写第二个是volatile读不允许重排序。

为了实现volatile限制重排序的功能，编译器在生成字节码时会插入内存屏障来静止特定类型的处理器重排序。而因为每个处理器重排序规则都不一样，JMM采取了保守策略插入内存屏障，保证JMM在不同处理器上最后重拍限制都相同。

在每个 volatile 写操作的前面插入一个 StoreStore 屏障
在每个 volatile 写操作的后面插入一个 StoreLoad 屏障

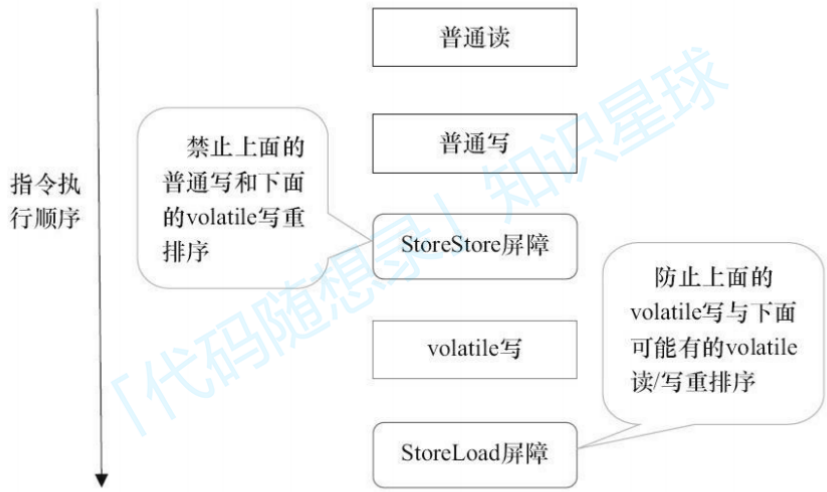


图 3-19　指令序列示意图

在每个 volatile 读操作的后面插入一个 LoadLoad 屏障
在每个 volatile 读操作的后面插入一个 LoadStore 屏障

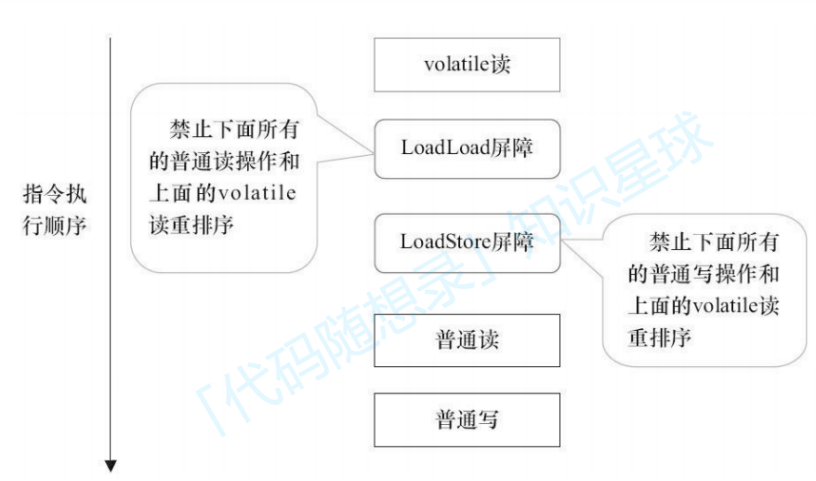


图 3-20　指令序列示意图

并不是严格的要求必须插入这些内存屏障，上述只是最保守的策略。在实际执行时，只要不该改变volatile写-读的内存语义，编译器可以优化掉不必要的内存屏障。

上面提到，store和load分别是存储到主内存和从主内存读取。在内存屏障的两侧，各有一些数据，内存屏障会保证在执行后面的执行之前，之前的相关指令已经全部完成。比如storestore屏障，在后面数据刷回到主内存之前，前面的数据已经全部刷回主内存。LodaStore屏障保证在后面数据刷回到主内存之前，已经从主内存中载入数据

表 3-3　内存屏障类型表

屏障类型	指令示例	说　明
LoadLoad Barriers	Load1; LoadLoad; Load2	确保 Load1 数据的装载先于 Load2 及所有后续装载指令的装载
StoreStore Barriers	Store1; StoreStore; Store2	确保 Store1 数据对其他处理器可见（刷新到内存）先于 Store2 及所有后续存储指令的存储
LoadStore Barriers	Load1; LoadStore; Store2	确保 Load1 数据装载先于 Store2 及所有后续的存储指令刷新到内存
StoreLoad Barriers	Store1; StoreLoad; Load2	确保 Store1 数据对其他处理器变得可见（指刷新到内存）先于 Load2 及所有后续装载指令的装载。StoreLoad Barriers 会使该屏障之前的所有内存访问指令（存储和装载指令）完成之后，才执行该屏障之后的内存访问指令

锁的内存语义

当线程释放锁时，JMM 会把该线程对应的本地内存中的共享变量刷新到主内存中。当线程获取锁时，JMM 会把该线程对应的本地内存置为无效。从而使得被监视器保护的临界区代码必须从主内存中读取共享变量。锁的实现也采用了 volatile内存语义。

在ReentrantLock 中，依赖于AQS框架实现锁，分为公平锁和非公平锁。

公平锁语义的实现：加锁方法首先读 volatile 变量 state(AQS的state)，在释放锁的最后写 volatile 变量 state。

非公平锁的实现：在非公平锁时，会采用CAS设置state.CAS具有 volatile 读和写的内存语义。

final内存语义

对于 final 域，编译器和处理器要遵守两个重排序规则。

1. 在构造函数内对一个 final 域的写入，与随后把这个被构造对象的引用赋值给一个引用变量，这两个操作之间不能重排序。
2. 初次读一个包含 final 域的对象的引用，与随后初次读这个 final 域，这两个操作之间不能重排序。

final内存语义实现：

写final:

JMM 禁止编译器把 final 域的写重排序到构造函数之外

编译器会在 final 域的写之后，构造函数 return 之前，插入一个 StoreStore 屏障。这个屏障禁止处理器把 final 域的写重排序到构造函数之外。

读final:

在一个线程中，初次读对象引用与初次读该对象包含的final 域，JMM 禁止处理器重排序这两个操作。编译

器会在读 final 域操作的前面插入一个 LoadLoad 屏障

final为引用类型(前提为final引用不能从构造函数溢出)：

在构造函数内对一个 final 引用的对象的成员域的写入，与随后在构造函数外把这个被构造对象的引用赋值给一个引用变量，这两个操作之间不能重排序。

happens-before

happens-before 是 JMM 最核心的概念

对于程序员：JMM 向程序员提供的 happens-before 规则能满足程序员的需求，也向程序员提供了足够强的内存可见性 保证。

对于处理器和编译器：

只要不改变程序的执行结果（指的是单线程程序和正确同步的多线程程序），编译器和处理器怎么优化都行。例如锁消除和volatile消除。

定义： 此处的定义分别呼应上面的两点

1. 如果一个操作 happens-before 另一个操作，那么第一个操作的执行结果将对第二个操作可见，而且第一个操作的执行顺序排在第二个操作之前。
2. 两个操作之间存在 happens-before 关系，并不意味着 Java 平台的具体实现必须要按照 happens-before 关系指定的顺序来执行。如果重排序之后的执行结果，与按happens-before 关系来执行的结果一致，那么这种重排序并不非法。

happens-before 规则

1. 程序顺序规则：一个线程中的每个操作，happens-before 于该线程中的任意后续操作。
2. 监视器锁规则：对一个锁的解锁，happens-before 于随后对这个锁的加锁。
3. volatile 变量规则：对一个 volatile 域的写，happens-before 于任意后续对这个volatile 域的读。
4. 传递性：如果 A happens-before B，且 B happens-before C，那么 A happens-before C。
5. start()规则：如果线程 A 执行操作 ThreadB.start()（启动线程 B），那么 A 线程的ThreadB.start()操作 happens-before 于线程 B 中的任意操作。
6. join()规则：如果线程 A 执行操作 ThreadB.join()并成功返回，那么线程 B 中的任意操作 happens-before 于线程 A 从 ThreadB.join()操作成功返回。

Java 线程安全的实现

Java中线程安全的实现方式---战略性观望

1、阻塞同步：

说白了也就是使用锁实现，具体采用什么锁，有两种选择：内置锁也就是synchronized关键字，JUC下具体锁的实现

2、非阻塞同步：

使用锁带来的主要问题，是频繁的线程阻塞、唤醒操作以及用户态内核态的切换带来的性能问题。

可能这些额外的操作带来的时间消耗远大于线程自身的业务执行执行时间。所以引入非阻塞同步，也就是基于CAS操作。

最直接的实现就是JUC下的各种原子类的实现。虽然CAS避免了锁带来的性能开销，不过其仅适用于少部分同步场景，没有阻塞同步更加具有普适性。

缺点：

1. 未获取同步资源的线程陷入自旋状态，所以对于CPU的消耗很高
2. 仅能操作单个共享资源，对于组合类型还是需要加锁处理，或者重新组合为一个共享资源
3. ABA问题

3、无同步方案:

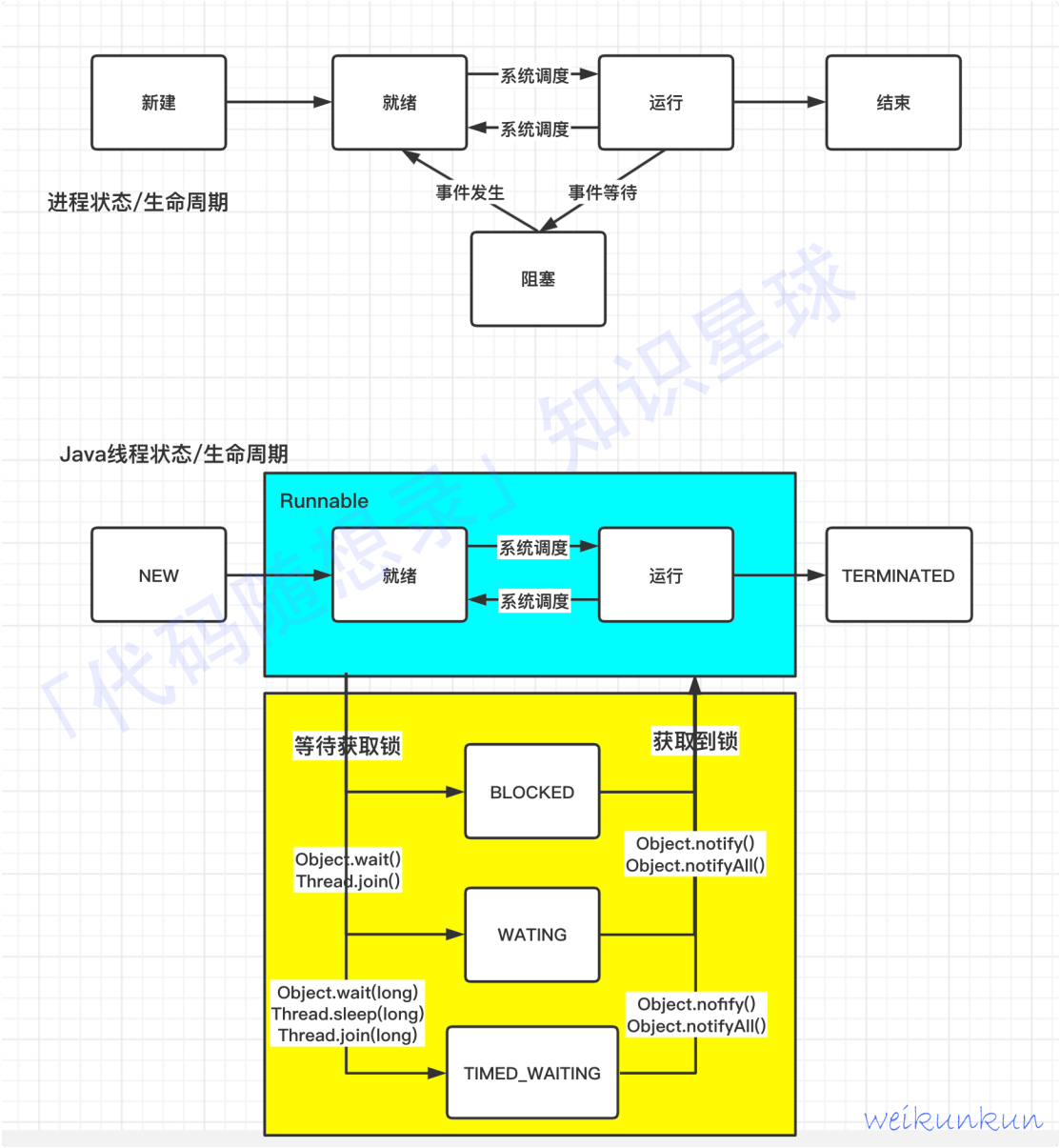
线程的本地存储，主要是用于对于一个共享资源都尽可能的在同一个线程中执行，使用场景:

1. 譬如生产者-消费者模型中，消费者消费消费一个内容
2. web服务中，一个请求对用于一个服务线程（也属于生产者-消费者模型）
3. 链路跟踪中动态采集方法的执行信息

Java 线程的状态转换

Java线程的状态转换和操作系统中进程的状态转换关系

Java中线程共有6种状态，也可以理解成其生命周期，同理操作系统的进程也有自己的生命周期，然后看Java并发编程的艺术关于线程生命周期的图，有些凌乱，然后结合操作系统，再结合优质的博客内容，重新理解了下。



Java中将操作系统中的

1. 就绪状态/运行状态转化为一个状态Runnable
2. 阻塞状态 细分为了三种
    1. BlOCKED
    2. WAITING
    3. TIMED_WAITING

Java 并发

Synchronized

1、作用

Java中的锁分为显示锁和隐式锁。隐式锁由synchronized关键字实现，而显示锁是由实现了Lock接口和AQS框架等等类来实现。

Java中的synchronized，通过使用内置锁，来实现对共享变量的同步操作，进而解决了对共享变量操作的原子性、保证了其他线程对共享变量的可见性、有序性，从而确保了并发情况下的线程安全。

同时synchronized是可重入的锁，避免了同一个线程重复请求自身已经获取的锁时出现死锁问题（请求于保持、不可剥夺感觉都有体现）

2、基础用法

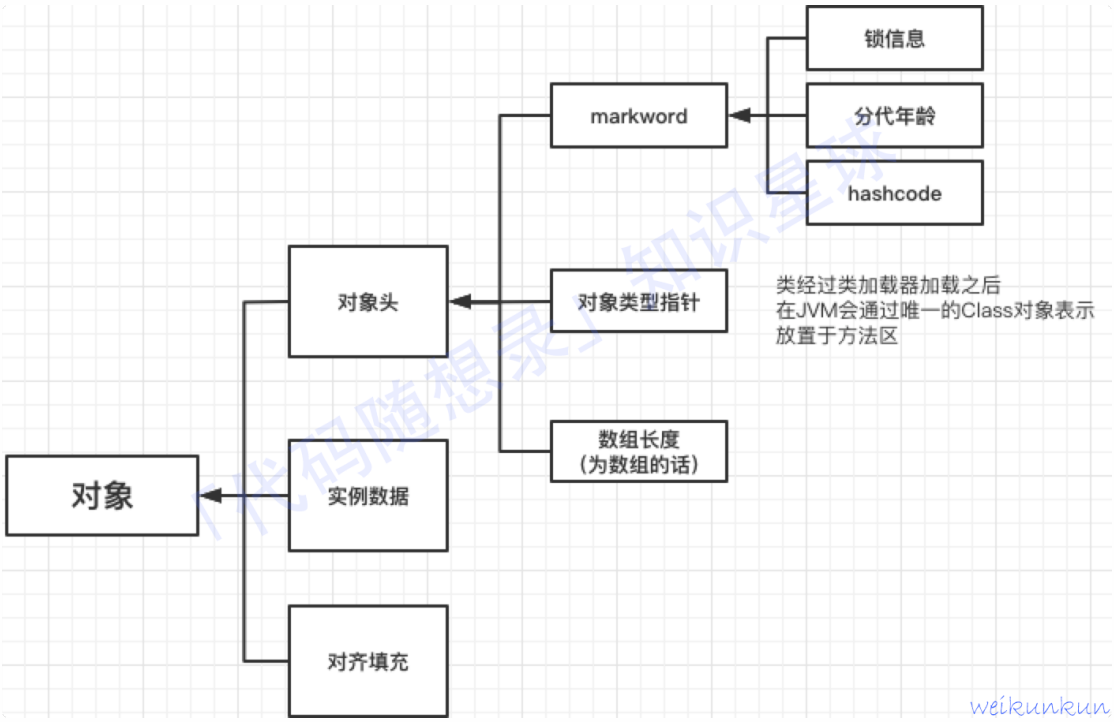
普通同步方法、静态同步方法、同步代码块儿

3、什么是内置锁

在Java中，每个对象都有一把锁，放置于对象头中，用于记录当前对象被哪个线程所持有。

相对于实例数据，对象头属于额外开销，所以被设计的极小来提高效率（一个对象在堆中的分布：对象头、实例数据、对齐填充）。

对象头中的markword更加体现了这一点。markword非结构化的，这样在不同的锁状态下，能够复用相同的bit位。markword中就有存储锁的信息的部分。



4、内置锁又是通过什么实现

synchronized被编译之后，使用的是monitorEnter和monitorExit两个字节码指令(同步代码块儿)，而这两个字节码指令实质上是依赖于操作系统中的mutex lock实现。

因为使用的是互斥锁，需要CPU的参与，涉及到线程的唤醒、同时切换需要记录线程的私有数据、寄存器等不共享的数据，可能这些操作带来的时间开销远大于线程自身执行的时间消耗，所以JDK1.6开始，引入了无锁、偏向锁、轻量级锁、重量级锁的锁优化过程，进而优化synchronized的性能

5、锁的分类

从宏观上看，锁的分类有多种不同划分。可以分为乐观锁和悲观锁，可以分为共享锁和排他锁，还可以分为可重入锁和不可重入等等。

乐观锁：

乐观锁就是对数据冲突保持乐观点态度，认为不会有其他线程同时修改数据。因此乐观锁不会上锁，只是在更新数据都时候判断是否有其他线程更新，如果没有其他线程修改则跟新数据，有其他线程修改则放弃数据，重新读取数据处理。

悲观锁：

悲观锁被数据冲突持悲观的态度，认为总是发生数据冲突。因此它以一种预防的态度，先行把数据锁住，知道操作完成才释放锁，在此期间其他线程无法操作数据。

6、synchronized关键字

Java 中的每一个对象都可以作为锁，有三种加锁的方式：

（1）对于普通同步方法，锁是当前实例对象。

（2）对于静态同步方法，锁是当前类的 Class 对象。

（3）对于同步方法块，锁是 Synchonized 括号里配置的对象。

7、对象头

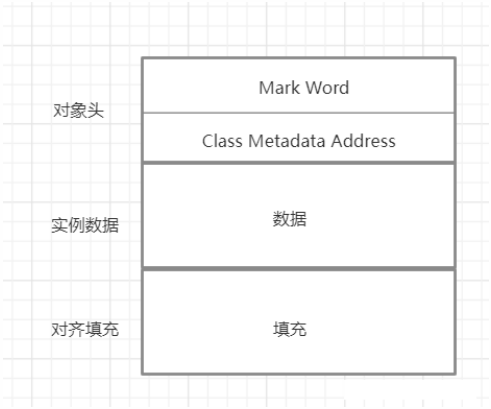
synchronized关键字的实现，依赖于Java的对象头。

一个对象由三部分组成：对象头、实体数据、对齐填充。对象头的长度不是固定的，如果是数据类型则对象头占12个字节，非数组类型对象头占8个字。

非数组类型的对象头分两部分，Mark Word 和对象类型指针，数组类型对象会多一部分来存储数组的长度。而synchronized关键字的实现，就和对象头中的Mark Word密切相关。

表 2-2　Java 对象头的长度

长　　度	内　　容	说　　明
32/64bit	Mark Word	存储对象的 hashCode 或锁信息等
32/64bit	Class Metadata Address	存储到对象类型数据的指针
32/32bit	Array length	数组的长度（如果当前对象是数组）



Java 对象头里的 Mark Word 里默认存储对象的 HashCode、分代年龄、是否偏向锁以及锁标记位。32位 JVM 的 Mark Word 的默认存储结构如下

锁状态	25bit	4bit	1bit 是否是偏向锁	2bit 锁标志位
无锁状态	对象的 hashCode	对象分代年龄	0	01

为了提高虚拟机空间的使用效率，Mark Word被设计成一个非固定的动态数据结构，以便存储更多的信息。不同状态下对象头Mark Word存储的信息如下图

锁状态	32bit				
	25bit		4bit	1bit	2bit
	23bit	2bit		偏向模式	标志位
未锁定	对象哈希码		分代年龄	0	01
轻量级锁定	指向调用栈中锁记录的指针				00
重量级锁定（锁膨胀）	指向重量级锁的指针				10
GC 标记	空				11
可偏向	线程 ID	Epoch	分代年龄	1	01

由图可知分为未锁定、偏向所、轻量级锁、重量级锁、GC，但是在轻量级锁升级为重量级锁时，还可能进行自旋。

synchronized锁的状态被分为4种，级别从低到高依次是：无锁、偏向锁、轻量级锁、重量级锁。

注意在锁标志中并没有出现自旋锁，它仅仅是锁可能存在的一种状态，是暂时性的，并没有官方的标志。32位虚拟机下，Mark Word的最后3位就可以判断锁的状态。

无锁和偏向锁标志位都是01，只是偏向锁时偏向模式会被置为1。锁可以升级但不能降级，意味着偏向锁升级成轻量级锁后不能降级成偏向锁。这种锁升级却不能降级的策略，目的是为了提高获得锁和释放锁的效率。

难道锁的不同仅仅是标志不同吗?

肯定不是，不同的锁各自实现的途径不同，适合的场景也各不相同。理解各种锁是怎样实现的，也就理解了synchronized关键字，理解了锁优化过程。

8、无锁

按最后3位标识位来判断，无锁应该是001状态，偏向标志为0，锁标志为01。

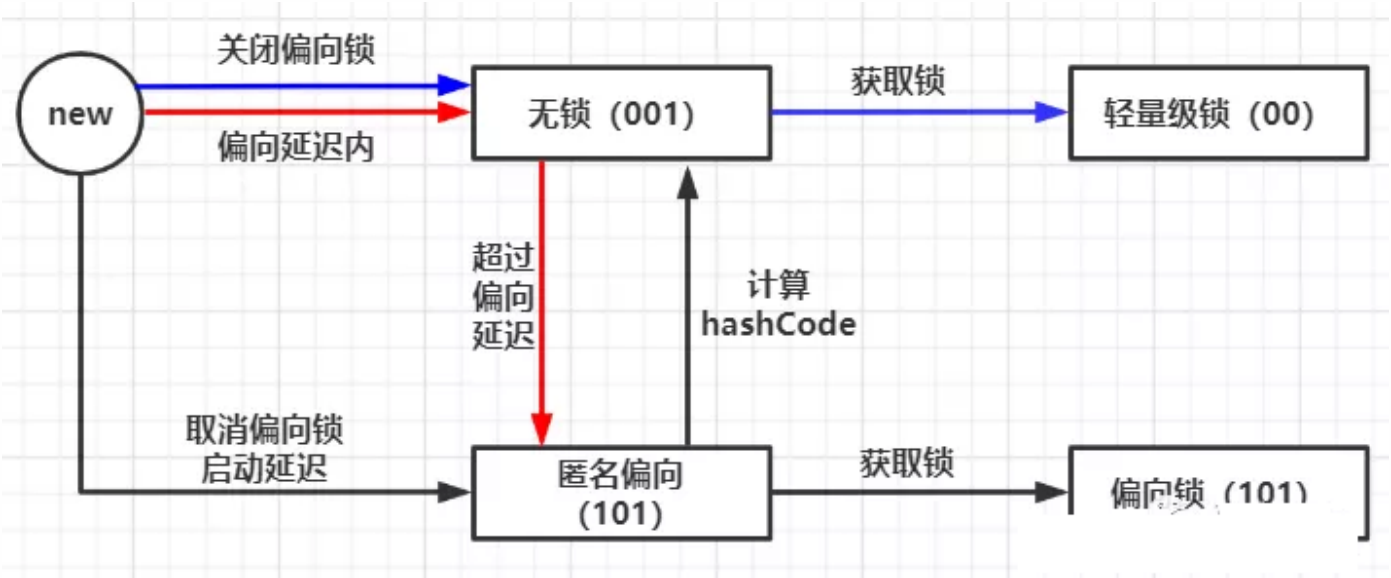
更准确的讲001状态是无锁不可偏，还有一种是101状态，无锁可偏（又叫匿名偏向）。无锁不可偏状态下遇到同步，会直接升级为轻量级锁，而不会变为偏向锁（看名字也知道，不可偏嘛）。只有在无锁可偏的状态下，才可能变成偏向锁。匿名偏向状态下虽然标识码是101，但是线程ID部分全部0，意味着没有线程实际获得偏向锁。

为啥要分无锁可偏和无锁不可偏呢?

因为所有偏向锁的起点就是匿名偏向状态，无锁不可偏状态会直接变为轻量级锁。

首先如果JVM设置取消偏向锁，那么无锁状态只可能是无锁不可偏。JDK8默认启动了偏向锁，但是偏向锁时在JVM启动几秒（默认4秒，但可以设置更改）之后才会启动，此时能设置为匿名偏向的会全部设置为匿名偏向，因此匿名偏向是偏向锁的起点。

而JVM为啥会在几秒之后才会启用偏向锁，这是因为JVM内部的代码会使用synchronized，这些类里有很多激烈线程竞争，如果采用锁升级策略这些锁会浪费很多时间。所以JVM索性先不开启偏向锁，先执行这些库类，等过几秒差不多都执行完了再开启偏向锁。至于JDK8为啥默认4秒，这是个经验值，4秒大多数类都启动完了，此数值可以修改。



9、偏向锁

大多数情况下，锁不仅不存在多线程竞争，而且总是由同一线程多次获得，为了让线程获得锁的代价更低而引入了偏向锁。

这个锁永远会偏向于获得它的线程，如果在获得锁之后并没有其他线程获取，则获得偏向锁的线程永远不需要同步，减少锁带来的时间消耗。

9.1、偏向锁的获取

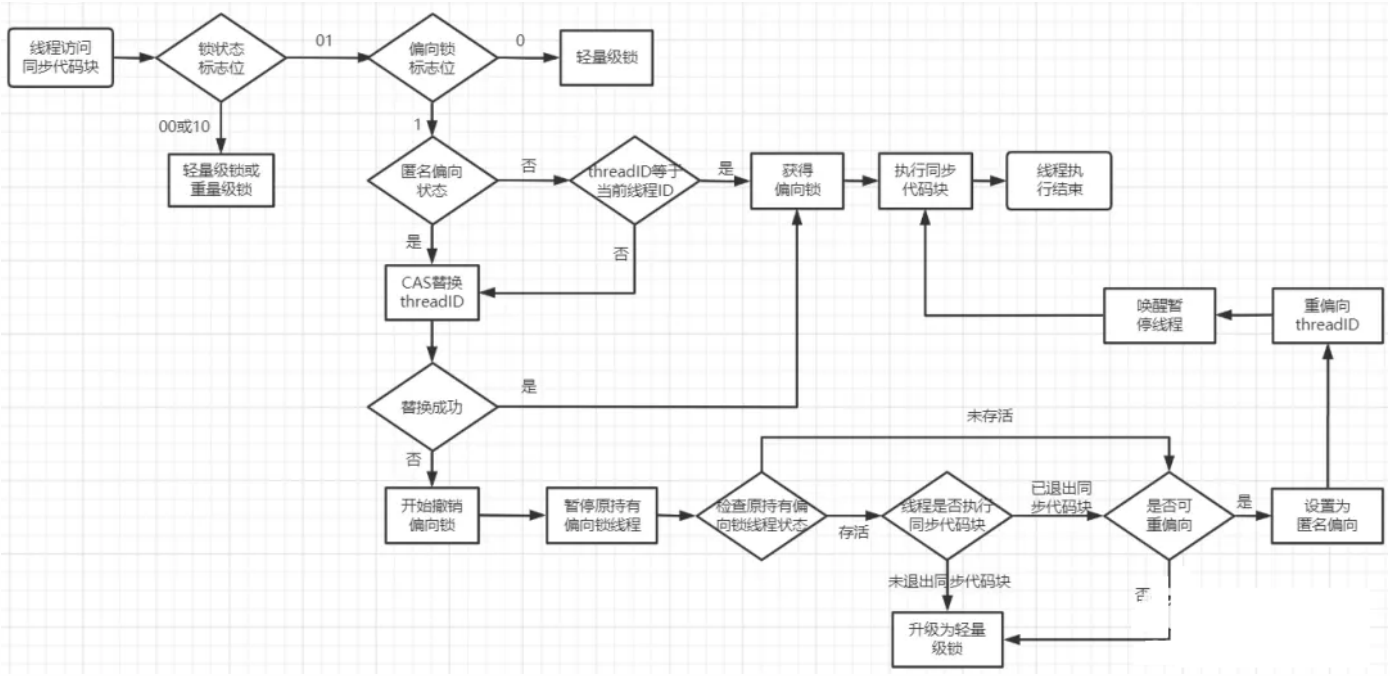
偏向锁对象头将不在存放Hash值，而在此位置上存放线程ID（23bit）+Epoch(2bit),一共25bit,其他部分保持不变。Epoch是一个时间戳，用来判断线程ID是否过时。具体获得锁流程如下：

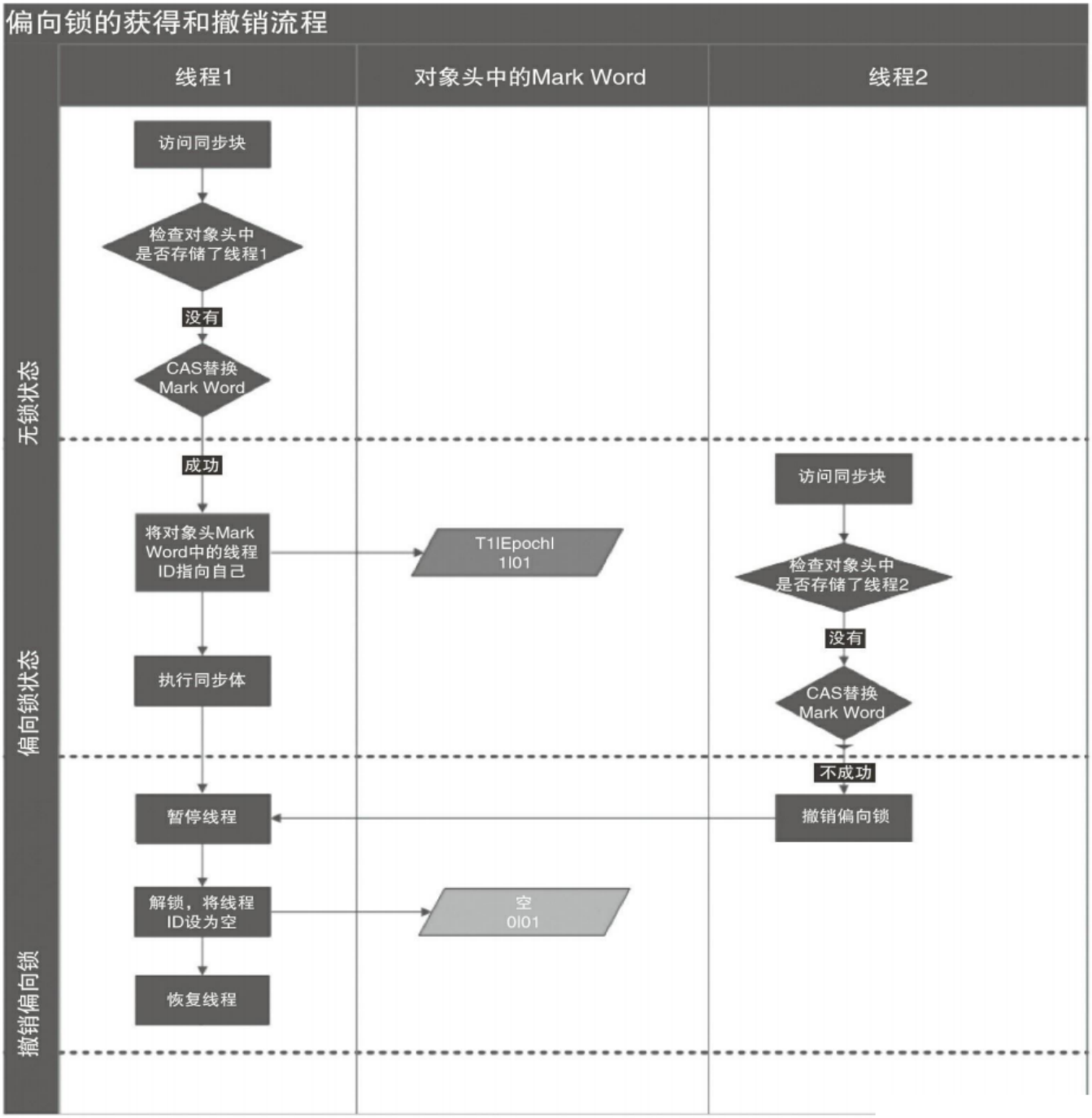
匿名偏向是偏向锁的初始状态，所以先判断锁标志，再判断偏向锁标志位，只有最后三位是101才开始，否则直接走其他的锁。如果是匿名状态，线程ID为0，采用CAS去将当前线程写入，如果成功则获得锁，不成功表示存在竞争。线程ID不为0，此前已经有偏向，判断此值是否和当前线程相同，若一致则表示线程之前就获得了锁，不一致就尝试CAS替换。同意，替换成功获得锁，替换失败存在竞争。未获得锁时将会等待安全点（STW），安全点会进行偏向锁的撤销。

安全点是JVM在进行垃圾GC时为了保证引用关系不会发生变化而设置的安全状态（GC Roots的确定就在此时），STW(STOP THE WORLD)此时将暂停所有线程的工作。

在STW会检测持有偏向锁的线程是否还存活，如果存活则升级轻量级锁，如果线程未存活或者已经退出来同步代码块，将会判断是否可重偏向，否则直接升级为轻量级锁。允许重偏向时会先设置为匿名重偏向，在使用CAS偏向线程。

判断是否可重偏向需要用到Epoch,偏向锁中有一个Epoch，对应的Class类中也有一个Epoch。在进入全局安全点之后，首先会对Class类中的Epoch进行增加，得到新的Epoch_new，然后扫描所有持有Class类实例的线程，根据线程信息判断是否锁住了该对象。如果锁住了说明此对象还在使用，将Epoch_new更新给它，如果未锁住则说明不需要加锁，不进行更新。如果对象的Epoch和类的Epoch相同，则表示它是被更新过的，需要锁，不能重偏向。而如果不相同，则表示已经不需要加锁了，此对象可以重偏向到其他线程。





9.2、偏向锁的释放

从偏向锁的获取过程可以看到，等到竞争出现的时候才会释放。如果没有出现竞争，它不会去改变Mark Word的相关字段。就算是线程已经执行完同步代码块，不需要加锁了，也不会去修改对象头，那个锁依旧存在，依旧保持偏向。

只是在其他线程需要偏向，出现了竞争的时候会进行判断，如果以前偏向的线程不需要了，那么对象首先会被设置为匿名偏向，然后CAS替换尝试加锁。如果以前偏向的线程还需要加锁，升级为轻量级锁。

所以线程不会主动的将偏向锁设置为匿名偏向状态，不会主动的去释放锁。

9.3、批量偏向与批量撤销

偏向锁有三个参数：

BiasedLockingBulkRebiasThreshold：偏向锁批量重偏向阈值，默认为20次

BiasedLockingBulkRevokeThreshold：偏向锁批量撤销阈值，默认为40次

BiasedLockingDecayTime：重置计数的延迟时间，默认值为25000毫秒（即25秒）

批量重偏向是以class而不是对象为单位的，每个class会维护一个偏向锁的撤销计数器，每当该class的对象发生偏向锁的撤销时，该计数器会加一，当这个值达到默认阈值20时，jvm就会认为这个锁对象不再适合原线程，因此进行批量重偏向。而距离上次批量重偏向的25秒内，如果撤销计数达到40，就会发生批量撤销，如果超过25秒，那么就会重置在[20, 40)内的计数。

当一个线程建立了大量的对象，并对他们都加了偏向锁。而此时若另一个线程也来获取这些对象，此时发生了竞争理论上都会升级轻量级锁。但是因为批量偏向的存在，并不会全部升级。

假设线程A建立了10个对象，全部加偏向锁，随后线程B同样也对这40个对象加锁。线程B对每一个对象进行加锁是，都会导致撤销一次偏向锁，升级为轻量级锁。当这个数值变为20后，JVM会认为其余的对象也不适合线程A，当后面的对象遇到需要同步的时候，会先被重置为可偏向状态，以便快速冲偏向。这样线程B对后面的对象加锁就不会升级为轻量级锁，而是偏向了线程B。

当线程C再来加锁的时候，前20个对象竞争轻量级锁，直接竞争，后20个锁是偏向线程B的，是偏向锁。此时不会触发批量重偏向，所以后20个也升级为轻量级锁。升级为轻量级锁就需要撤销偏向锁，加上之前的20次，一共40次。达到40次撤销偏向锁，会触发批量撤销机制，将偏向锁升级为轻量级锁，并且此类新建的对象都不是无锁不可篇状态，不会出现偏向锁。

9.4、偏向锁的优缺点

优点：

在只有单一线程访问对象的时候，偏向锁几乎没有影响。只有第一次需要CAS操作替换，随后的只要比较线程ID即可，比较方便快速。

缺点：

如果有多个线程访问，就会出现竞争，竞争需要等到安全点时，并且进行一系列分析比较耗费时间。另外，偏向锁存放线程ID和Epoch后，对象头中不存在Hash值，如果程序需要Hash值需要调用HashCode,这会导致偏向锁退出。

如果对象需要调用Object方法，会启用对象的minoter内置锁.。此时会直接由偏向锁退出进入重量级锁。

10、轻量级锁

轻量级锁是相对于使用操作系统互斥量来实 现的传统锁而言的，因此传统的锁机制就被称为“重量级”锁。轻量级锁并不是用来代替重量级锁的，它设计的初衷是在没有多线程竞争的前提下，减少传统的重量级锁使用操作系统互斥量产生的性能消耗。

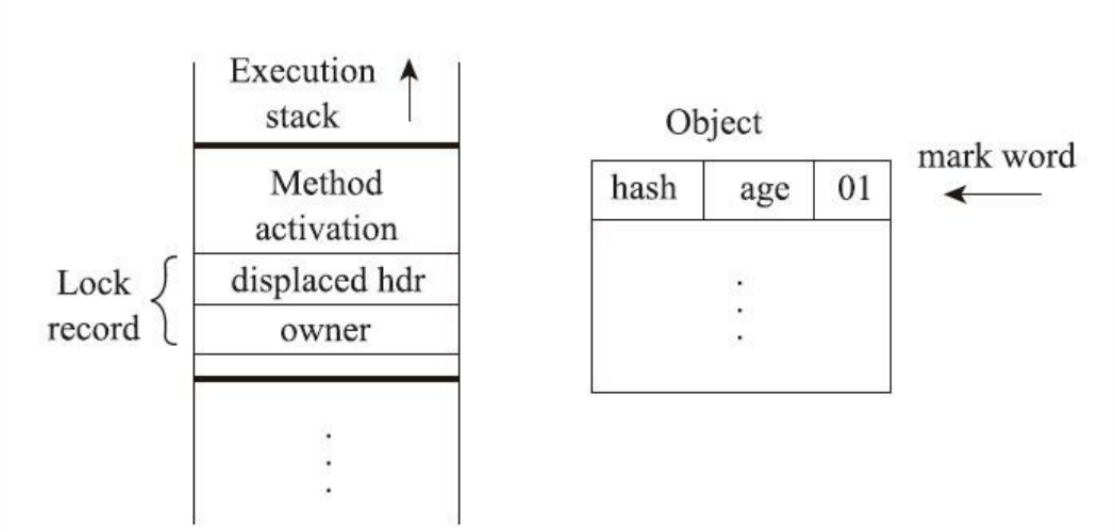
10.1、轻量级锁的获取

虚拟机首先将在当前线程的栈帧中建立一个名为锁记录（Lock Record）的空间，此空间包含两部分：

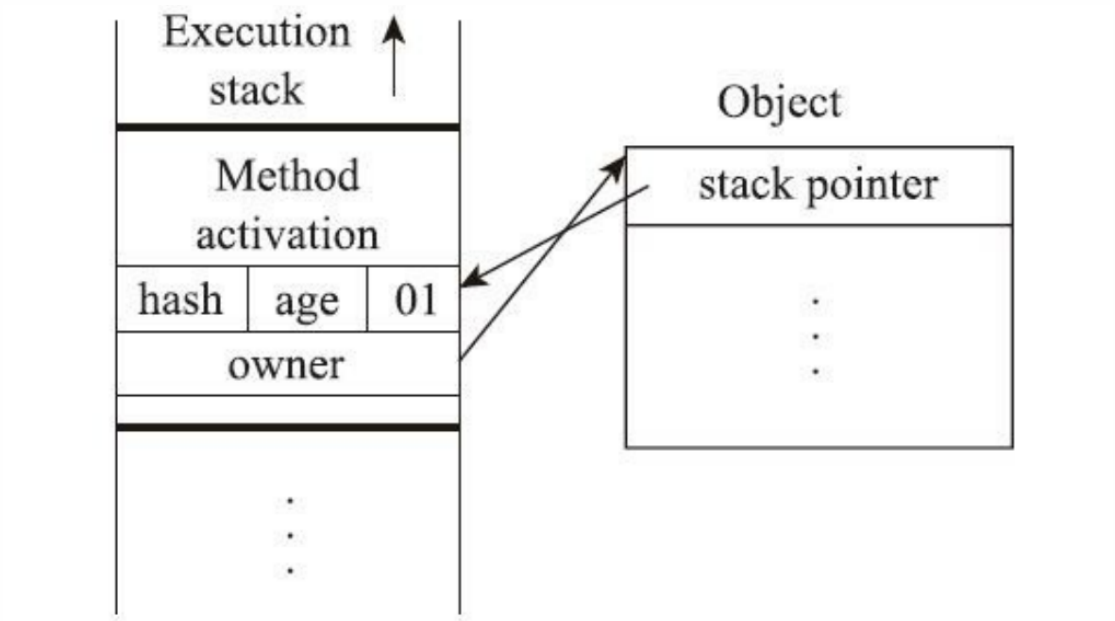
displaced mark word：用于存储锁对象目前的Mark Word的拷贝

owner:指向当前的锁对象的指针

虚拟即首先会将对象的Mark World拷贝到栈帧中的Lock Record，此时owner为空。



然后虚拟机使用CAS操作尝试把对象的Mark Word更新为指向Lock Record的指针（此空间占30bit, 除了最后两位锁标志，Mark Word前30bit均用来存放指针），如果操作成功则代表线程拥有了这个对象锁，对象锁标志位变为00. 如失败了则会会判断对象Mark Word存储的指针是否指向自己，如果是则表示拥有锁，执行同步代码，如果不是则意味着其他线程抢占，出现激烈竞争需要升级为重量级锁。



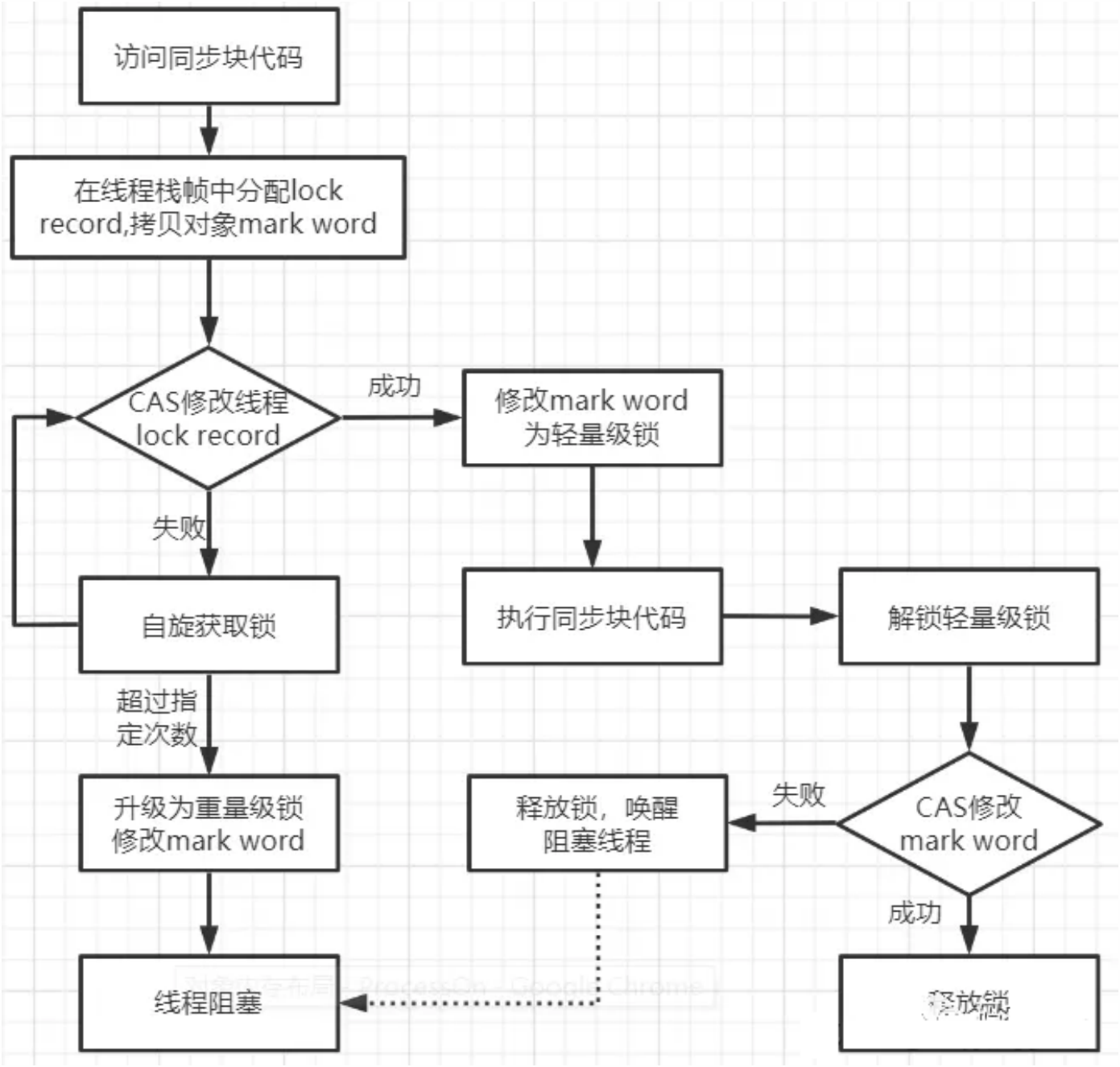
10.2、轻量级锁的释放

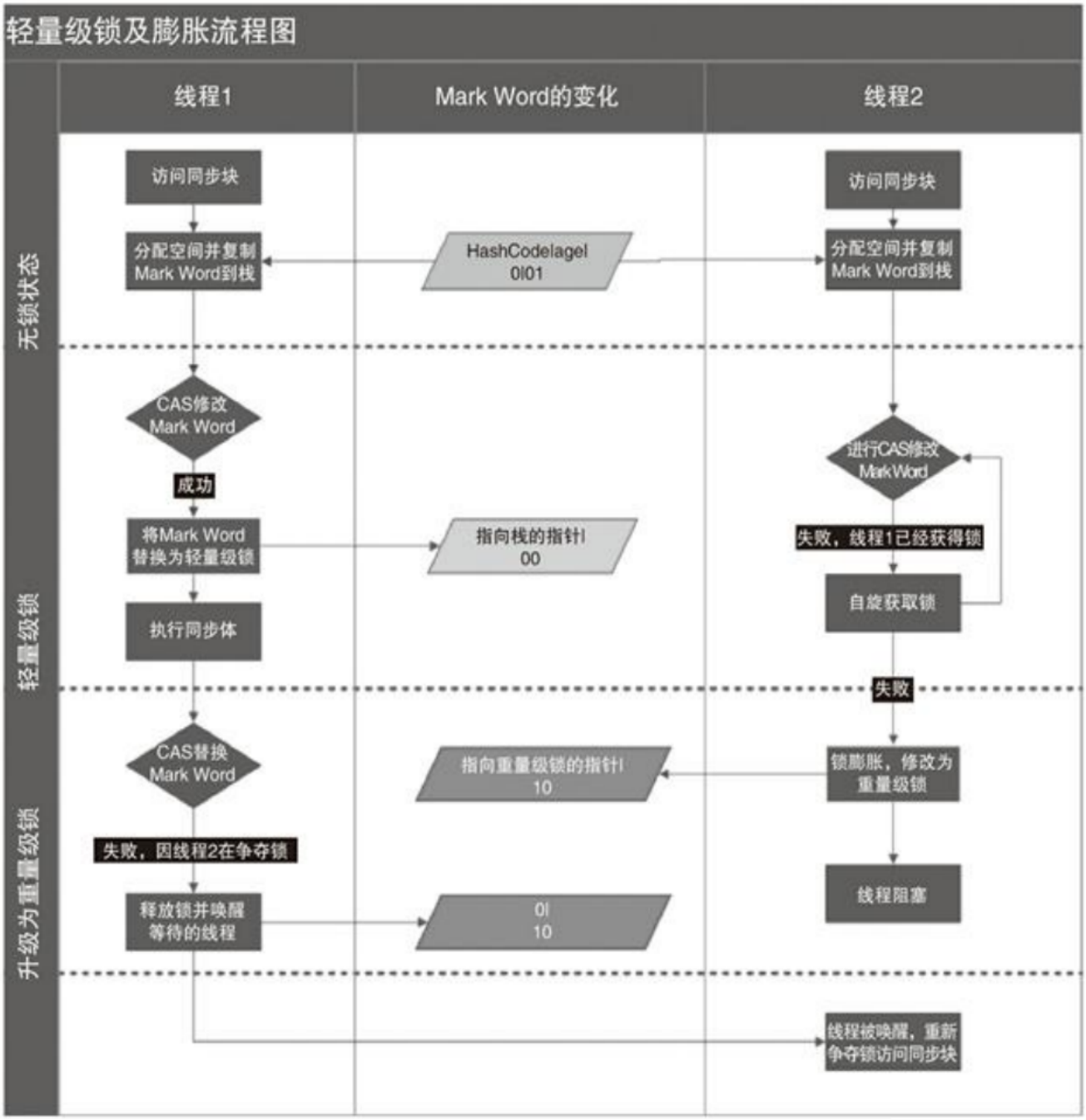
使用CAS尝试将Lock Record中的displaced mark word替换回去，需要检查对象头中的指针是否指向当前线程。

如果替换成功，表示没有竞争，锁成功释放

如果替换失败会进行自旋，如果自旋之后仍未获得锁表示存在竞争并升级为重量级锁。

当其他线程竞争轻量级锁时，并不会对已经持有轻量级锁的线程发送什么，而是对象头的锁标志被修改，同时竞争线程自身被挂起。因此如果CAS替换失败，原本持有锁的线程除了释放锁之外，还需要唤醒被挂起的线程。


访问同步块代码
在线程栈帧中分配lock record,拷贝对象mark word
CAS修改线程 lock record
成功
修改mark word 为轻量级锁
失败
自旋获取锁
执行同步块代码
解锁轻量级锁
超过指定次数
升级为重量级锁 修改mark word
释放锁，唤醒阻塞线程
失败
CAS修改 mark word
成功
线程阻塞
释放锁

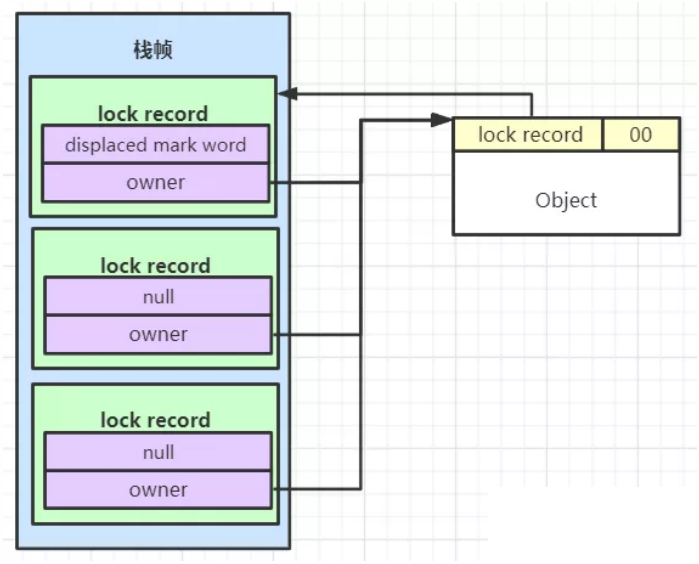


10.3、轻量级锁的重入

轻量锁的每一次重入，都会在栈中生成一个Lock Record。只是只有第一次会拷贝Mark Word，随后的加锁Displaced Mark Word区域为NULL,owner区域统一指向对象头。

线程第一次获得锁时，将对象的Mark Word拷贝到本线程Lock Record，同时将对象的指针指向自己，Lock Record中的owner也指向对象。随后的加锁对象中存储的是指向本线程的指针，并没有Mark Word，也就不需要拷贝。只是将owner指向对象即可。

每加一次锁帧栈中多一个Lock Record，Lock Record的个数也就是加锁的次数。释放锁时也要一个一个释放，只有解锁次数等于加锁次数，才会真正释放锁。释放锁时，如果是重入锁则直接删掉一个Lock Record，如果不是重入则采用CAS替换对象的Mark Word。



10.4、轻量级锁自旋

当轻量级出现竞争以后，会尝试进行自旋。自旋就是CPU空转，线程没有挂起依然在执行，等过一段时间后再去加锁。

这是因为如果升级为重量级锁，是通过操作系统来实现，涉及到内核态和用户态之间的切换，这个操作的比较耗时。

如果竞争没有那么激烈，锁住的同步代码块执行的时间还没有切换上下文花的时间多，反而得不偿失。因此采用自旋锁，出现竞争之后等一等再去尝试，可能前面获得锁的线程已经执行完了，再次加锁。这样就免去了升级重量锁带来的消耗。

自旋不能一直进行。自旋时CPU是空转，这就浪费了处理器资源。上面的情况是竞争不激烈，但假设竞争激烈那么自旋完全是浪费时间，还不如直接升级到重量级锁省资源。以前的自旋次数默认是10，如果10次之后依然不行说明竞争很激烈，需要升级到重量级锁。

JDK1.6以后加入了自适应自旋：

对于某个锁对象，如果自旋等待刚刚成功获得过锁，并且持有锁的线程正在运行中，那么虚拟机就会认为这次自旋也是很有可能再次成功，进而允许自旋等待持续相对更长时间

对于某个锁对象，如果自旋很少成功获得过锁，那在以后尝试获取这个锁时将可能省略掉自旋过程，直接阻塞线程，避免浪费处理器资源。

除此之外，JVM还会根据CPU的负载进行优化：

如果平均负载小于CPUs则一直自旋

如果有超过(CPUs/2)个线程正在自旋，则后来线程直接阻塞

如果正在自旋的线程发现Owner发生了变化则延迟自旋时间（自旋计数）或进入阻塞

如果CPU处于节电模式则停止自旋

自旋时间的最坏情况是CPU的存储延迟（CPU A存储了一个数据，到CPU B得知这个数据直接的时间差）

自旋时会适当放弃线程优先级之间的差异

10.5、轻量级锁自旋

优点：

适合多个线程不同时访问同步对象场景。偏向锁的撤销必须在安全点才可以进行，假设多个线程交替访问某个对象，处理完成后又释放锁，这种情况下偏向锁也可以完成但是效率会很低，每次撤销再加锁必须等到安全点，同时还要进行Epoch的分析，因此偏向锁才会有批量撤销机制，撤销偏向锁次数过多则意味偏向锁不适用，不再新增偏向锁而直接变成轻量级锁。轻量锁则很方便，直接CAS替换就可以，对这种多线程竞争不激烈的场景很适用。锁的自旋也是为此设计。

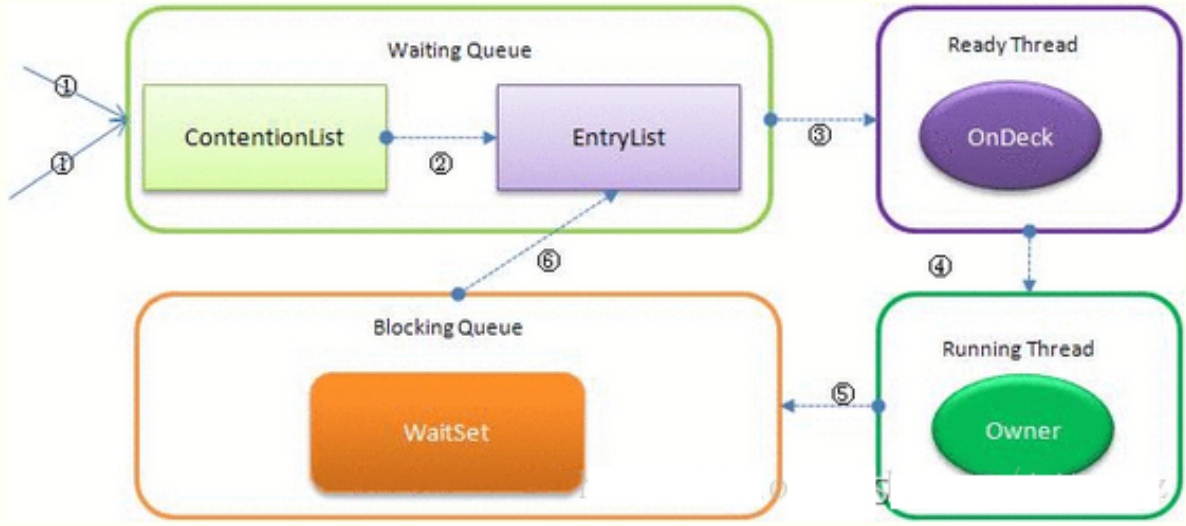
缺点：

竞争激烈的场景下不适用，此时进行自旋就是再浪费CPU资源。竞争激烈时可能进行很多次的自旋都不回获得锁，这种浪费的代价比上下文切换的代价要大，所以引入自适应自旋来裁决。

11、重量级锁

重量级锁的实现依赖于ObjectMonitor，而ObjectMonitor又依赖于操作系统底层的Mutex Lock（互斥锁）实现。

Monitor可以理解为一个同步工具或一种同步机制，通常被描述为一个对象。每一个Java对象就有一把看不见的锁，称为内部锁或者Monitor锁。主要包含以下几部分：



Contention List：竞争队列，所有请求锁的线程首先被放在这个竞争队列中；

Entry List：Contention List中那些有资格成为候选资源的线程被移动到Entry List中；

Wait Set：哪些调用wait方法被阻塞的线程被放置在这里；

OnDeck：任意时刻，最多只有一个线程正在竞争锁资源，该线程被成为OnDeck；

Owner：当前已经获取到所资源的线程被称为Owner；

!Owner：当前释放锁的线程。

count：monitor的计数器，数值加1表示当前对象的锁被一个线程获取，线程释放monitor对象时减1

11.1、重量级锁的获取

锁升级为重量级之后，Mark Word中存储的指针不再指向线程，而是指向ObjectMonitor。当线程访问同步代码块时，每个线程都会被封装成一个ObjectWaiter对象进入monitor。

JVM每次从队列的尾部取出一个数据用于锁竞争候选者（OnDeck），但是并发情况下，ContentionList会被大量的并发线程进行CAS访问，为了降低对尾部元素的竞争，JVM会将一部分线程移动到EntryList中作为候选竞争线程。

Owner线程会在unlock时，将ContentionList中的部分线程迁移到EntryList中，并指定EntryList中的某个线程为OnDeck线程（一般是最先进去的那个线程）。

Owner线程并不直接把锁传递给OnDeck线程，而是把锁竞争的权利交给OnDeck，OnDeck需要重新竞争锁。这样虽然牺牲了一些公平性，但是能极大的提升系统的吞吐量，在JVM中，也把这种选择行为称之为“竞争切换”。

OnDeck线程获取到锁资源后会变为Owner线程，而没有得到锁资源的仍然停留在EntryList中。如果Owner线程被wait方法阻塞，则转移到WaitSet队列中，直到某个时刻通过notify或者notifyAll唤醒，会重新进去EntryList中。

处于ContentionList、EntryList、WaitSet中的线程都处于阻塞状态，该阻塞是由操作系统来完成的（Linux内核下采用pthread_mutex_lock内核函数实现的）。

Synchronized是非公平锁。 Synchronized在线程进入ContentionList时，等待的线程会先尝试自旋获取锁，如果获取不到就进入ContentionList，这明显对于已经进入队列的线程是不公平的，还有一个不公平的事情就是自旋获取锁的线程还可能直接抢占OnDeck线程的锁资源。

为啥采取非公平锁，主要还是考虑到性能。采取非公平锁下一次获得锁线程可能还是原来持有锁的线程，这样就避免了内核和用户态的切换，节省时间。

11.2、重量级锁的重入

线程如果获取到锁，会判断是否重入，如果是重入锁，count计数+1，释放锁时count数值减1。

11.3、重量级锁的优缺点

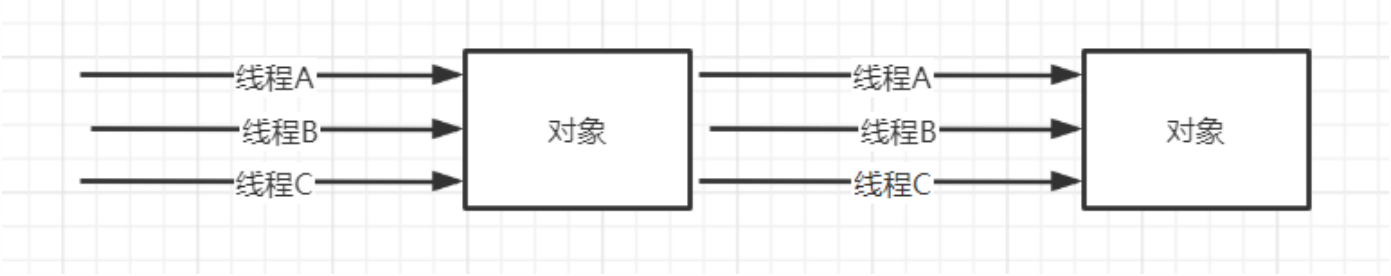
重量级锁需要内核态和用户态的切换，这个代价很大。所以把它放在最后，经过偏向和轻量级之后才是它。

但是只有它能应对竞争激烈的场景，也算是JVM最后的杀手锏了。

Synchronized 反向理解

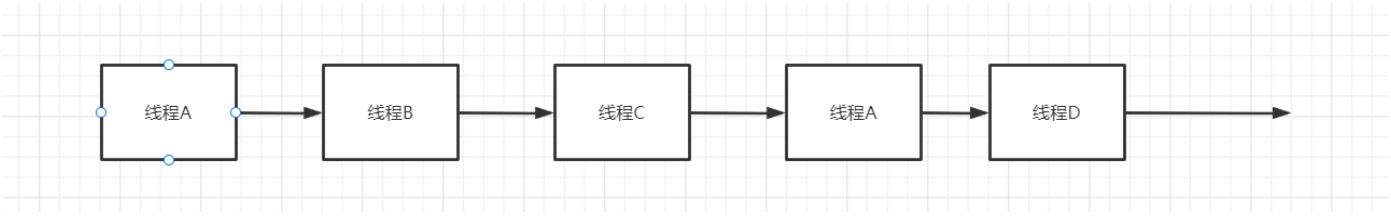
在总结前文吃透synchronized关键字中，锁升级介绍是由低到高，一步一步的出发。其实也可以反其道而行之，想想怎么锁降级，这样能更好的理解锁升级的思路。

原先synchronized关键字就是现在的重量级锁，不论遇到何种情况都是重量级锁，需要内核态和用户态之间的相互切换。在JDK1.6之后引入了锁升级，分为无锁-偏向锁-轻量级锁-重量级锁。从开发人员角度想，为啥要这么做呢?



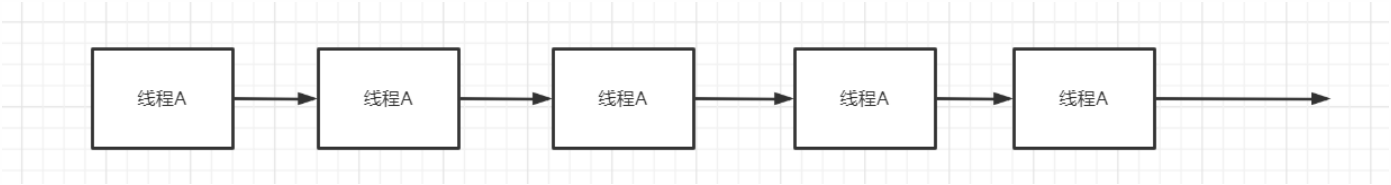
无锁状态和重量级锁肯定好理解，一个最轻，一个最重。记住无论何时何地，只要出现两个线程一直争夺同一个锁，必然是用重量级锁。不管原先是无锁、偏向还是轻量级，出现两个线程竞争同一个锁，最后一定是启动重量级锁。但不是任何时候都会有多个线程来竞争同一个锁，像这种竞争激烈场景在全部程序中并不太多。

假设同一个对象，一会线程A得到，执行完成之后释放，之后线程B得到，执行完成之后释放，甚至可能还有线程CDEFG。根本没有竞争，只是单纯的加个锁然后释放，这种情况下用重量级锁就浪费了，杀鸡用牛刀，不划算。那就锁简化一下，不用重量级的等待阻塞队列，也不用内核态和用户态切换了。也不需要对象头里指向mionter的指针了，直接把这个存放线程的指针。在线程的帧栈中加个Lock Record，方便轻量级锁的判断。设么？用轻量级锁有些时候还会出现竞争，还只是短暂地？切换线程多不划算，轻量级锁自旋吧，多等一会说不定就没竞争了。万一这不是短暂地竞争呢？最初是限制自旋次数，又来用适应性自旋，更灵活的判断。



绝大多数连竞争也没有？单个线程访问某个对象，这个对象到死都没见过其他线程。线程每次加锁又释放，费那么大力气建Lock Reocrd，还采用CAS的方式。杀鸡用牛刀，这可太亏了。把锁再简化一点，轻量级里还复制Mark Word，干脆啥都不要了。直接在对象头也写线程ID，线程访问的时候对比一下和自己的一样不一样就行，多简单。锁干脆不释放了，就算不用了也懒得改，等下次那个线程要偏向自己去对象头里把线程ID写成它的就行。

不对啊，单线程访问的省心了，但还是有其他情况啊。万一锁多个对象，其中一部分已经可以释放了，另一部分还必须拿着，偏向锁咋区分这种？简单，给个Epoch值，进入STW的时候看看那些对象还需要，给他们新的Epoch，不需要的就不管。Epoch是不能重偏向，旧的就可以重偏向，哪个线程需要了就可以改过去。万一线程A和线程B交替访问一个对象，或者现场A锁住大量对象后线程B也来锁这些对象，这压根不是偏向锁设计的初衷，引出批量重偏向和批量撤销。



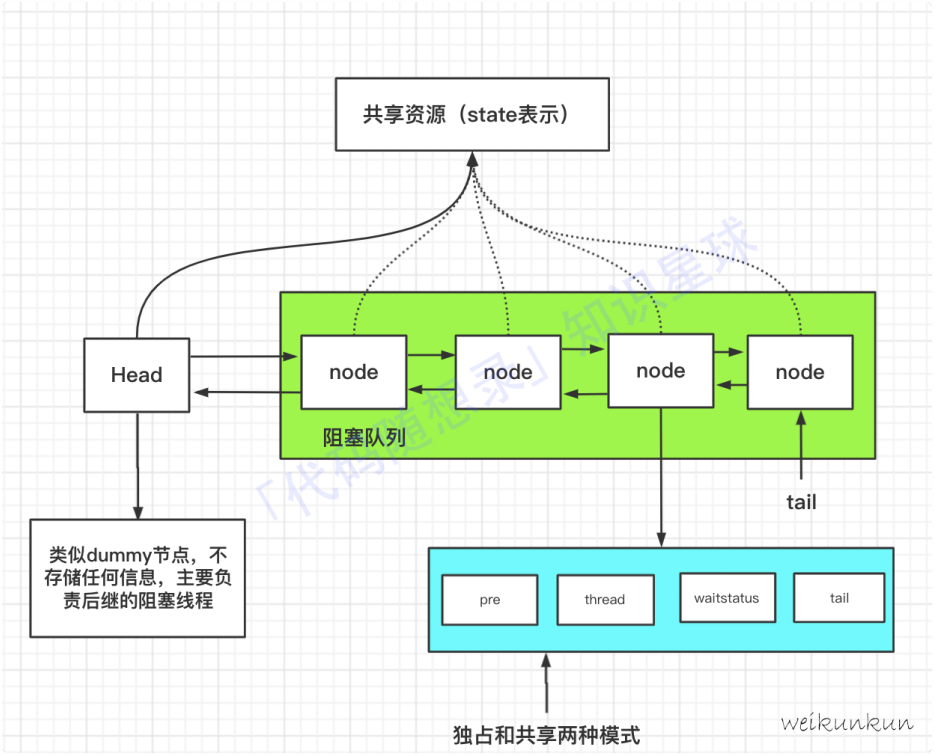
另外，出现竞争系统是怎么判断的？是某个线程想要加锁却被人占有，但是原先持有锁的线程并不知情。原先持有轻量锁的想要释放锁的时候一看，变重量级锁了，已经有一堆线程排着等了。锁释放掉，还得把队列里睡眠都线程唤醒，让他们竞争锁。偏向锁升级轻量级锁，需要到STW才能偏向撤销，还需要回复对象的Mark Word。

AQS

1、概念：

AQS，一个用来构建锁和同步器的框架，许多的经典的同步器都是基于AQS构建出来的，譬如CountDownLatch、Semaphored。

其内部使用同步队列帮我们解决了实现同步器时的一些细节问题，譬如何时阻塞线程，如何按照顺序唤醒线程。



2、核心思想：

AQS通过一个被volatile修饰的int类型的成员变量表示同步状态（共享资源），然后使用cas对这个同步状态进行修改，从而保证线程安全。

如果被请求的共享资源空闲，则将当前的请求线程设置为有效的工作线程，然后将对应的共享资源设置为锁定状态。

如果共享资源被占用，则通过CLH同步队列将暂时用不到的线程封装成一个节点加入到队列中，同时在适当的时候对其进行阻塞和唤醒。

基于AQS实现的应用（目前接触到的）

（1）lock

区别于synchronozed的锁，同时还支持了更加的灵活丰富操作，譬如可定时的尝试获取锁（避免了死锁的发生）、可中断的获取锁。

（2）CountDownLatch

一个或一组操作等其他操作执行结束之后，继续执行。

（3）Semaphore

用于多个共享资源的互斥，或者用于控制并发数量

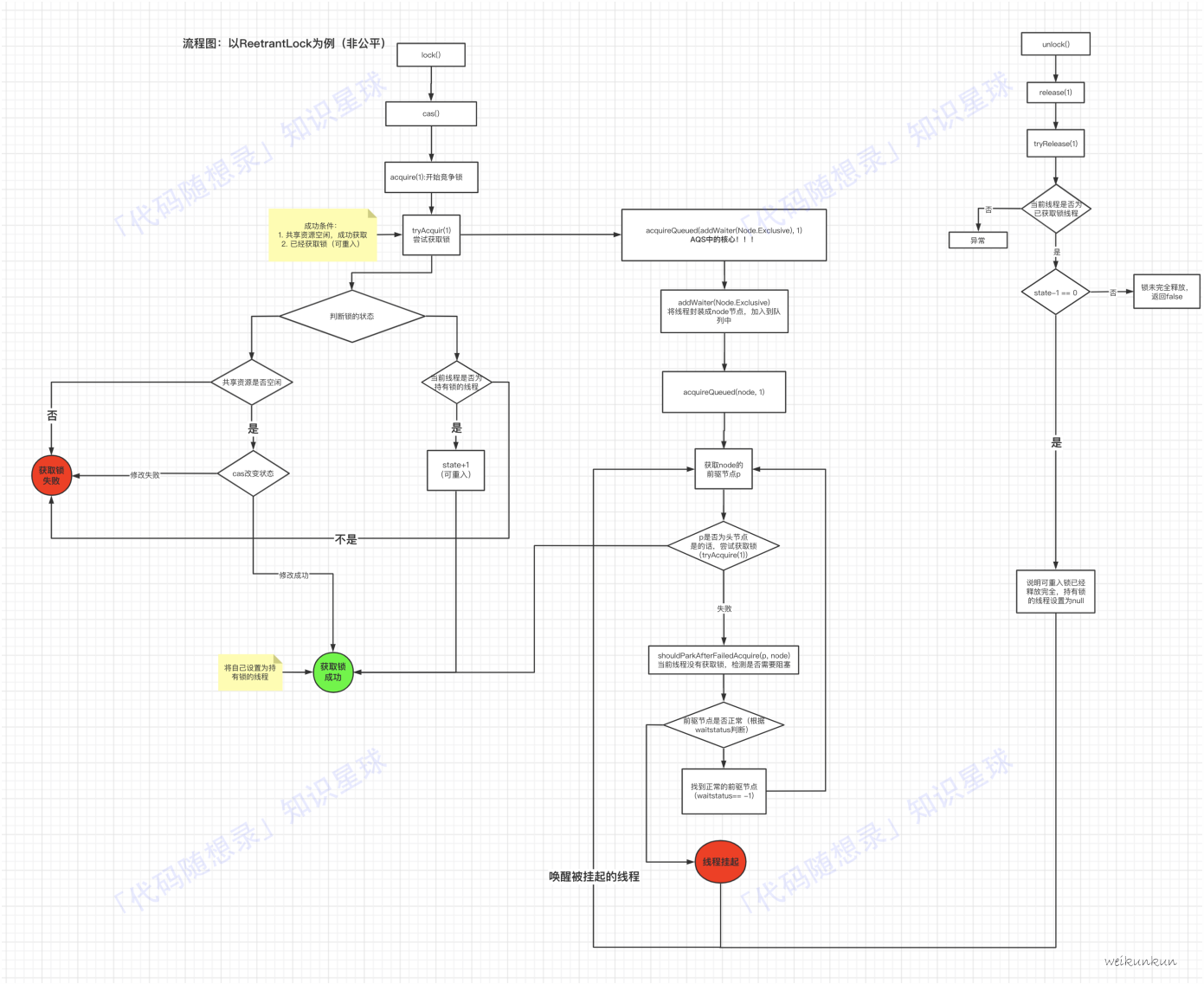
3、内部细节：

1. 目前理解是通过waitStatus来标志线程的阻塞、唤醒等状态，但是每个节点的waitStatus状态不是通过自身设置的，而是通过其后继节点设置的。
2. 内部的同步队列采用的数据结构是双向链表，同时Head节点属于dummy节点，不存储信息，仅表示当前持有锁的线程，同时还会负责后续阻塞线程的唤醒
3. exclusiveOwnerThread变量，标志当前持有锁的线程，防止错误的释放锁，这种思想在Redis分布式锁中也有体现

4. state是用volatile修饰的int型变量，这样其除了能够实现独占锁之外，还能实现Semaphore、CountDownLatch这样的同步工具
5. 对外提供了使用protected修饰的方法tryAcquire、tryRelease等，这样在实现的子类中，能够通过这些方法，实现一些我们自定义的内容，譬如锁的可重入，公平与非公平锁。

基于ReentrantLock的独占式共享锁的整个流程：

原图链接：「代码随想录」知识星球



JDK

java 并发 中的 ThreadLocal

有多个线程(孩子)，争抢同一个共享变量(玩具)，就会产生冲突，而程序的解决办法是加锁(父母说服，讲道理，轮流玩)，但加锁就意味着性能的消耗(父母比较累)。

所以有一种解决办法就是避免共享(让每个孩子都各自拥有一个变形金刚)，这样线程之间就不需要竞争共享变量(孩子之间就不会争抢)。

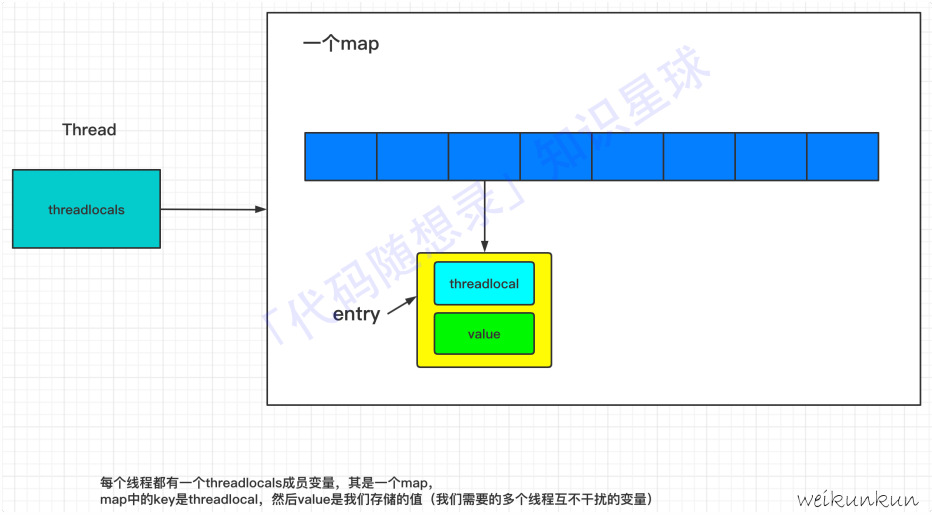
所以 ThreadLocal 的存在就是为了通过本地化资源来避免共享，避免了多线程竞争导致的锁等消耗。

在每个线程内部都有一个名为threadLocals的成员变量，该变量的类型为HashMap，其中key为我们定义的ThreadLocal变量的this 引用，value 则为我们使用set方法设置的值。每个线程的本地变量存放在线程自己的内存变量threadLocals中，如果当前线程一直不消亡，那么这些本地变量会一直存在，所以可能会造成内存溢出，因此使用完毕后要记得调用ThreadLocal的remove方法删除对应线程的threadLocals中的本地变量。

ThreadLocal

概述：

ThreadLocal提供了一种方式，让在多线程环境下，每个线程都可以拥有自己私有的数据结构，进而减少同一个线程内多个函数或者组件之间一些公共变量的传递的复杂度。但因为其自身的实现机制，使用之后记得及时remove，避免内存泄漏。



每个线程都有一个threadLocals变量，该变量的数据结构是threadLocalMap，而map中的每个entry中key是threadLocal，value是我们需要存储的本地变量

部分细节：

1. get操作，因为threadLocal是一个key，所以每次获取其映射的value时，首先先获取当前线程的threadLocalMap，然后再根据该map和key获取对用的value
2. set操作，同理也需要先根据threadLocal这个key获取map，然后通过map才能真正的将threadLocal变量和value进行映射
3. threadLocalMap和HashMap差不多，使用的数据结构都是hash表，不过对于hash冲突的解决方式，threadLocalMap使用的是线型探测，装载因子采用的是黄金分割点，然后hash算法使用的是斐波那契散列发，扩容方式均为原Entry数组的2倍
4. 因为threadLocalMap的中的key也就是threadLocal是弱引用，所以只要触发GC，key就会被回收，这样ThreadMap中的key就变成了null，但是value被Entry引用，Entry被ThreadLocalMap引用，ThreadLocalMap被Thread引用，这也就说明了只要，线程不终止，value的值一直无法被回收，所以可能会出现内存泄漏的现象，所以使用完之后，要及时调用remove方法。
5. 如果想子线程继承父线程的本地变量，可以采用InheritableThreadLocal，感觉这个在中间件里面用的会很多，没验证过。

JVM

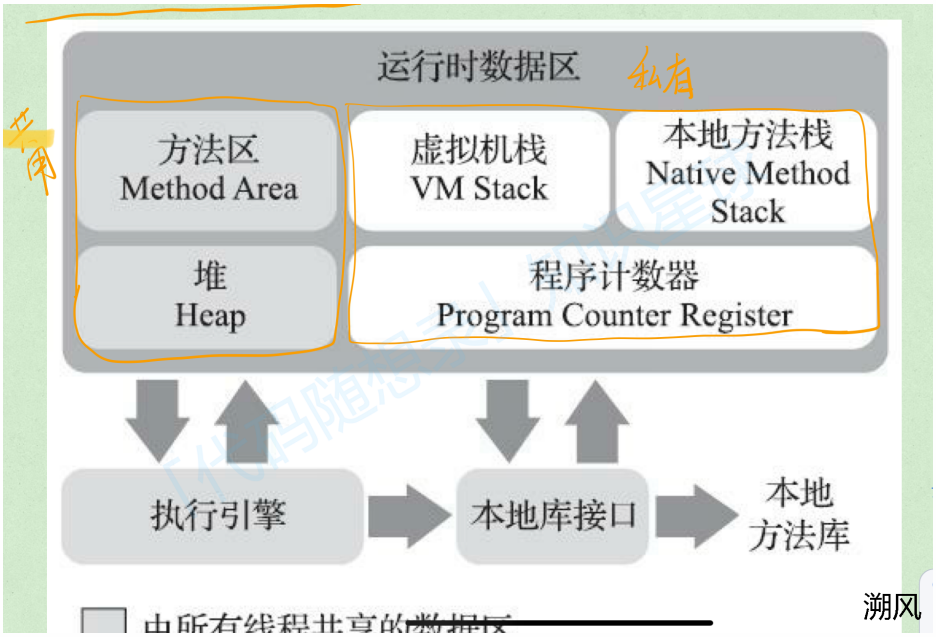
JVM内存管理

Java 运行时数据区组成部分？哪些是共享的，哪些是私有的？

有以下部分：

Java堆，方法区，Java 虚拟机栈，本地方法栈，程序计数器。

公有私有：



私有数据区

1、程序计数器

程序计数器，记录的是正的是正在执行的虚拟机字节码指令的地址。字节码解释器工作时，就是通过改变这个计数器的值来选取下一条需要执行的字节码指令，完成程序的流程控制。

在多线程情况下，每个线程都需要有一个独立的程序计数器，以便于切换回来后可以恢复到正确的执行位置。

2、Java 虚拟机栈

Java 虚拟机栈由一个个栈帧组成，每个方法被执行时，Java 虚拟机都会同步创建一个栈帧。

一个方法从开始被调用到执行完毕，对应的就是一个栈帧在虚拟机中入栈到出栈的过程。

栈帧 Stack Frame,用于存储 局部变量表，操作数栈，动态连接，方法出口等信息。

3、局部变量表

存放编译期间可知的 各种 Java 虚拟机基本数据类型，对象引用和 returnAddress 类型（指向了一条字节码指令的地址）。

这些数据类型在局部变量表中以局部变量槽来表示，局部变量表所需的内存空间在编译期间完成分配，方法运行期间，不会改变槽的数量，具体的内存空间还是由虚拟机来决定。

4、本地方法栈

为虚拟机使用到的本地方法 native 方法服务，其余的同 Java 虚拟机栈。

共享数据区

1、Java 堆

Java 堆 在虚拟机启动时创建，是垃圾收集器管理的内存区域。这里暂时就这么多，再往下细分永久代啥的，以后再来

2、方法区

方法区用于存储已经被虚拟机加载的类型信息，常量，静态变量，即时编译器编译后的代码缓存等数据。

方法区与永久代，可以说 方法区是接口，是规范，永久代是 HotSpot 虚拟机给出的实现，

3、运行时常量池

运行时常量池是方法区的一部分，包括类的描述信息和常量池表，

其中 类的描述信息包括类的版本，字段，方法，接口。

常量池表用于存储 编译期生成的各种字面量与符号引用。在类加载后，常量池表存放到方法区的运行时常量池中。

运行时常量池具有动态性，运行期间也可以将新的常量放入。

对象的创建

五步

1、类加载检查

当 Java 虚拟机遇到一条字节码 new 指令时，首先将去检查这个指令的参数是否能够在常量池中定位到一个类的符号引用，并且检查这个符号引用代表的类是否已经被加载，解析和初始化过，如果没有，那么就要先执行相应的类加载过程。

2、分配内存

在类加载完成后，对象所需要的内存大小完全确定，可以对新生对象分配内存。

分配内存有两种方式：

（1）内存规整

如果内存规整，就是指针碰撞，将指针向空闲空间方向移动与对象大小相等的距离

（2）内存不规整

如果内存不规整，那就是空闲列表，维护一个列表，记录哪些内存块可用，在分配时，从列表中找到一块足够大的空间划分给对象实例，并更新列表上的记录。

划分空间时的并发问题的解决方法:

CAS + 失败重试，另一种是 TLAB :先预先分配一块内存，称为本地内存缓冲 Thread Local Allocation Buffer，线程先在自己的 TLAB 中分配，用完了之后，分配新的缓冲区时，进行同步锁定。

3、初始化内存空间

分配到的内存空间（对象头除外）全部分配为 0 值，这步保证了对象的实例字段在 Java 代码中可以不赋值就直接使用，使得程序可以访问到这些字段的数据类型所对应的零值。

4、进行对象头的设置

在对象的对象头中保存一些必要的信息：这个对象是哪个类的实例，如何才能找到类的元数据信息， 对象的哈希码，对象的GC 分代年龄等信息。

5、执行构造函数

按照程序员的意愿初始化对象

对象的结构

一个对象分为以下几个部分： 对象头，实例数据，对齐填充

1、对象头

对象头包括两类数据：对象自身的运行时数据和类型指针。

（1）对象自身的运行时数据

对象自身的运行时数据叫做 Mark Word, 包括哈希码， GC 分代年龄，锁状态标志，线程持有的锁，偏向线程ID,偏向时间戳等。

Mark Word 是一个动态的数据结构，为的就是在极小的空间内存储尽量多的数据。

（2）类型指针

类型指针，对象指向元数据的指针，Java 通过这个来确定对象是哪个类的实例。

如果对象是数组类型，那么在对象头中还必须有一块用于记录数组长度的数据，因为虚拟机可以通过普通 Java 对象的元数据信息确定 Java 对象的大小，但是如果数据的长度不确定，将无法通过元数据中的信息判断出数组的大小。

2、实例数据

实例数据是对象真正存储的有效信息，也就是我们在程序代码中所定义的各种类型的字段内容，包括从父类继承的。 这部分的存储顺序会受到 HotSpot 虚拟机默认的分配策略参数 和 字段在 Java 源码中定义顺序的影响。

3、对齐填充

对齐填充其实不是必然的，作用就是在占位符，保证对象的起始地址是 8 字节的整数倍。

也就是说，任何对象的大小都必须是 8 字节的整数倍，这个是 HotSpot 虚拟机的自动内存管理系统的要求。

对象访问定位

对象的访问的两种方式？区别和优缺点

Java 通过栈上的 reference 数据来操作堆上的具体对象。

对象访问的两种方式为句柄和直接指针

1、句柄方式

Java 堆中将可能会划分除一块内存来作为 句柄池，然后 reference 储存句柄池的地址，句柄中包含了指向对象实例数据的指针，和对象类型数据的指针。

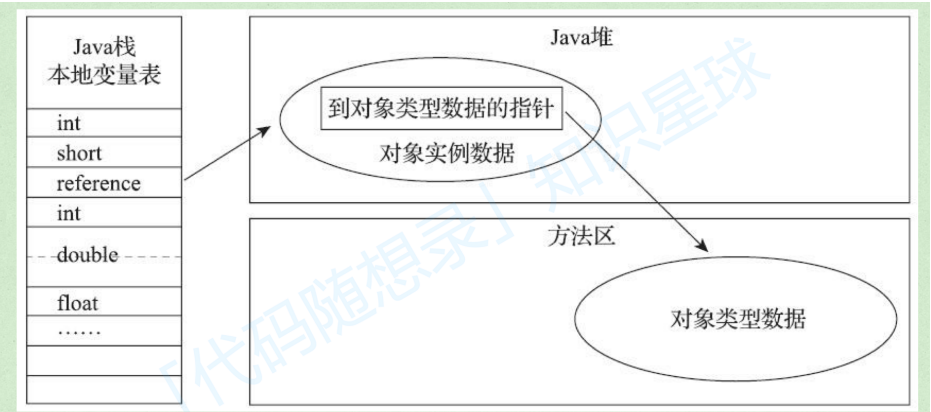


图2-3　通过直接指针访问对象　　溯风

2、直接指针

reference 中放的是对象地址

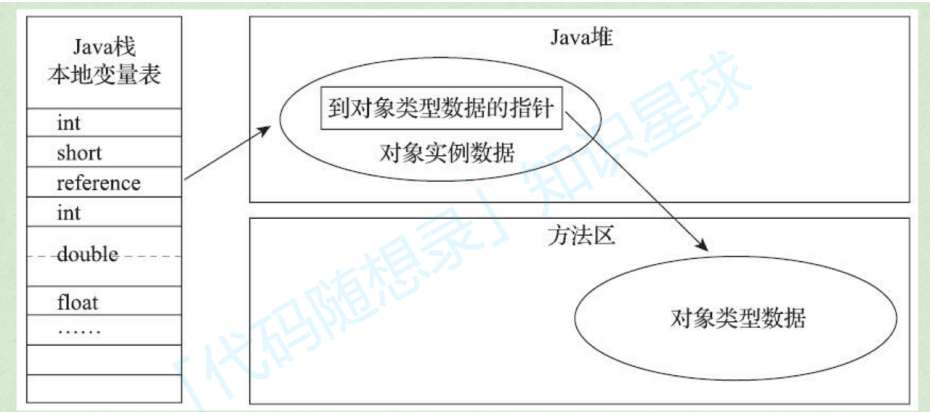


图2-3　通过直接指针访问对象　　溯风

3、优缺点

句柄方式比较稳定：对象被移动时，只会改变句柄中的实例数据指针，不需要修改 reference 本身。

直接指针的话，少了一次指针定位的开销，所以好处就是访问速度更快。

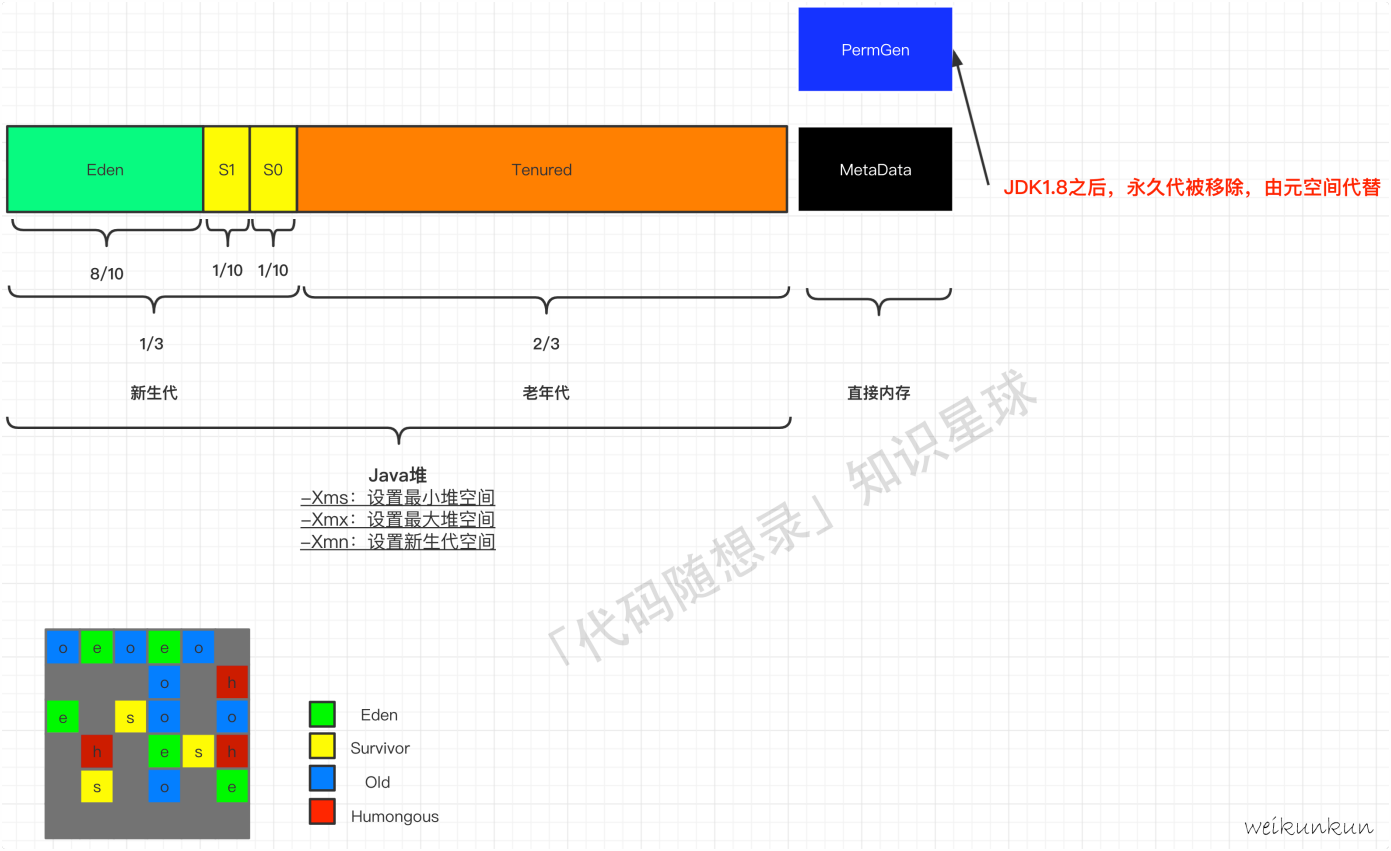
JVM之经典收集器---G1

全称Garbage-First，之前的收集器因为是直接将堆内存物理划分为新生代老年代

所以需要来两种收集器配合使用才能实现全代的回收，G1将整个堆内存划分为多个等大的region（内部连续）

然后每个region不同时间代表角色不固定，不过整体分为四种：Eden、Survioor、Old、Humongous（存储大对象）

对于G1来说整体采用的是标记-整理算法，然后从region的角度来看，采用的是复制算法



G1 垃圾收集过程主要分为4个阶段：

> 这里不同的地方，总结不同，最后还是选择以深入理解Java虚拟机为标准。

1、初始标记

1. 联GC Roots直接关联的对象
2. 触发STW

2、并发标记

和用户程序并发执行，标记出所有回收对象

3、最终标记

1. 处理并发标记后，新产生的对象
2. 触发STW

4、筛选回收

1. 先对Region的回收价值进行排序，然后根据期望暂停时间，选择性回收Region
2. 回收时采用标记复制，多条收集器线程并发执行
3. 不追求一次全部清理完
4. 触发STW

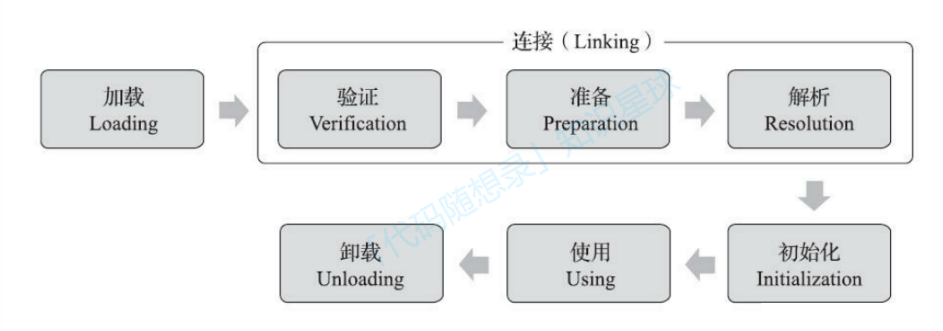
类加载机制

类加载机制：

类的数据从Class文件加载到内存，并对数据进行校验、转换解析和初始化，最 终形成可以被虚拟机直接使用的Java类型。

类的生命周期：

一个类型从被加载到虚拟机内存中开始，到卸载出内存为止，它的整个生命周期将会经历加载 （Loading）、验证（Verification）、准备（Preparation）、解析（Resolution）、初始化 （Initialization）、使用（Using）和卸载（Unloading）七个阶段，其中验证、准备、解析三个部分统称 为连接（Linking）。



其中解析也可以在初始化之后再开始，位置并不是唯一的。

加载

在加载阶段，虚拟机要完成三件事情

1、通过类的全限定名来获取定义此类的二进制字节流

2、将这个字节流所代表的静态存储结构转化为方法区的运行时数据结构

3、在内存中生成一个代表这个类的java.lang.class对象，作为方法区这个类的各种数据访问的入口

这个阶段需要用到加载器，“双亲委派模型”与此有关。

数组类对象是虚拟机直接在动态内存中构造出来，但这个数组的组件类型由加载器加载。比如`int[]`，`char[]`组件类型不是引用类型，由引导类加载器加载，而类似Integer[]这种组件为引用类型的，就按照本文说的步骤加载。

而关于第一点，获取类的二进制字节流。并没有规定必须从哪里获取，从ZIP压缩包读取，从网络中获取（例如Web Applet），运算时计算生成（如动态代理），其他文件生成（如JSP），从加密文件中获取等等。

连接

验证

Class文件可以由Java源代码编译而来，但它也可以由其他途径产生。毕竟Class文件只是一种格式，最底层的还是0/1，它可以使用包括靠键盘0和1直接在二进制编辑器中敲出 Class 文件在内的任何途径产。

Java虚拟机如果不检查输入的字节流，对其完全信任的话，很可能会因为载入了有错误或有恶意企图的字节码流而导致整个系统受攻击甚至崩溃。

1、文件格式验证：

检查是否符合Class文件的格式，如Class文件开头的魔数，主次版本号是否可以被虚拟机接受、常量池是否正确等等

2、元数据验证：

对类的元数据信息进行语义校验。比如是否有父类，是否继承了不被允许的类、是否实现了接口或者抽象类中的方法等等

3、字节码验证：

通过数据流分析和控制流分析，确定 程序语义是合法的、符合逻辑的。就是对将要执行的程序字节码验证（即Class中的Code属性），是否出现了逻辑性的错误。**字节码验证有问题，则程序一定不对，但字节码验证没问题，程序不一定能正确执行**。通过程序去校验程序逻辑是无法做到绝对准确的，不可能用程序来准确判定一段程序是否存在Bug。这一阶段很费时间，所以在编译器生成Class文件时就会生成StackMapTable，这样虚拟机直接检查StackMapTable记录是否合法就行。

4、符号引用验证：

验证将符号引用转化为直接引用时候合法，判断该类是否缺少或者被禁止访问它依赖的某些外部类、方法、字段等资源。这个动作实在解析阶段进行的，所以这7个步骤并不是严格的顺序，很多时候是交替进行的。

验证阶段有点类似查杀病毒，确保虚拟机执行的 Class 文件都是正确的。它很重要但并不是必须的，如果使用的 Class 类都是经过反复使用和验证的，那就可以关闭类验证措施，缩小类加载的时间。就像不开启防火墙，不下载一些恶意软件也能正常使用计算机。

准备

准备阶段是为类中静态变量分配内存并设置类变量初始值的阶段。

此处仅包括静态变量（static修饰），而不包括实例变量，实例变量实在对象实例化的时候随着对象一起分配在堆中。而且此处的初始值指的数据类型默认的零指。

比如`public static int a = 123`, 准备阶段会给 a 赋值为 0，而不是 123。a 变成 123 是类构造器完成的，所以是在初始化阶段才会执行。

也有例外，如果此处类字段属性存在 ConstantValue 属性，那么在准备阶段就会赋值。

例如`public static final int value = 123`，添加 final 关键字后类会有 ConstantValue 属性，此时在准备阶段 a 就会被赋值为 123。

解析

解析阶段是Java虚拟机将常量池内的符号引用替换为直接引用的过程。

符号引用是以一组符号来描述所引用的目标，它与虚拟机实现的内存布局无关，引用的目标并不一定是已经加载到虚拟机内存当中的内容。而直接引用是可以直接指向目标的指针、相对偏移量或者是一个能间接定位到目标的句柄。如果有了直接引用，那引用的目标必定已经在虚拟机的内存中存在。

初始化

类加载过程最后一步。进行准备阶段时，静态变量已经赋过一次系统要求的初始零值，而在初始化阶段，则会根据程序员通过程序编码制定的主观计划去初始化类变量和其他资源。

通俗的说初始化就是执行类构造器()方法的过程。()方法是由编译器自动收集类中的所有类变量的赋值动作和静态语句块（static{}块）中的语句合并产生的，编译器收集的顺序是由语句在源文件中出现的顺序决定的，静态语句块中只能访问到定义在静态语句块之前的变量，定义在它之后的变量，在前面的静态语句块可以赋值，但是不能访问。

```
public class Test {
    static  i = 0; // 给变量复制可以正常编译通过 可以给后面定义都赋值
    System.out.print(i); // 这句编译器会提示“非法向前引用” static int i=1在此之后才定义 无法访问}
    static int i = 1; //此处才定义
}
```

父类中的()方法在子类之前执行，保证在子类执行前父类的()已经执行完毕。()方法对于类或接口来说并不是必需的，如果一个类中没有静态语句块，也没有对变量的 赋值操作，那么编译器可以不为这个类生成()方法。

类加载器及加载机制

类加载器

在类加载流程第一步“加载”阶段，通过一个类的全限定名来获取描述该类的二进制字节流，实现这一功能的代码叫做类加载器。

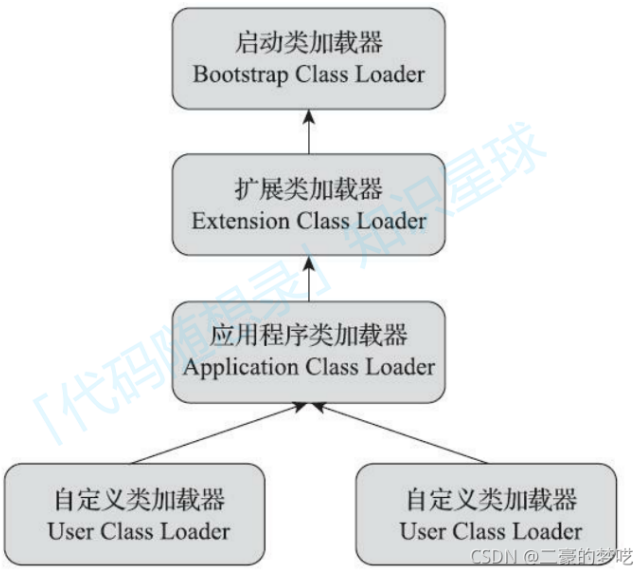
Java中的任意一个类，都必须由加载它的类加载器和这个类本身一起共同确立其在Java虚拟机中的唯一性，每一个类加载器，都拥有一个独立的类名称空间。

判断两个类是否相等，必须在这两个类是被同一个加载器加载的前提下才有意义。否则即使Class文件相同，被同一个虚拟机加载，但是使用不同的类加载器，那么两个类也是不同的。

双亲委派模型

如果一个类被不同的加载器加载，那虚拟机会认定是不同的类，这样Java体系最基础的行为也无从保证，应用程序会变得混乱。

所以有了双亲委派模型，双亲委派一共分了四层的加载器。求除了顶层的启动类加载器外，其余的类加载器都应有自己的父类加载器，子类使用“组合”关系来复用父类加载器的代码，而不是继承。



双亲委派模型一共分四层：

1、启动类加载器：

负责加载存放在 <JAVA_HOME>\lib目录，或者被-Xbootclasspath参数所指定的路径中存放的类

2、扩展类加载器：

它负责加载<JAVA_HOME>\lib\ext目录中，或者被java.ext.dirs系统变量所 指定的路径中所有的类库

3、应用类加载器：

负责加载用户类路径 （ClassPath）上所有的类库，开发者同样可以直接在代码中使用这个类加载器。如果应用程序中没有 自定义过自己的类加载器，一般情况下这个就是程序中默认的类加载器。

4、自定义加载器：

用户自行扩展，典型的如增加除了磁盘位置之外的Class文件来源，或者通过类 加载器实现类的隔离、重载等功能。

如果一个类加载器收到了类加载的请求，它首先不会自己去尝试加载这个类，而是把这个请求委派给父类加载器去完成，每一个层次的类加载器都是如此，因此所有的加载请求最终都应该传送到最顶层的启动类加载器中，只有当父加载器反馈自己无法完成这个加载请 求（它的搜索范围中没有找到所需的类）时，子加载器才会尝试自己去完成加载。

破坏双亲委派

亲委派很好地解决了各个类加载器协作时基础类型的一致性问题（越基础的类由越上层的加载器进行加载），但如果有基础类型又要调用回用户的代码就不得不破坏双亲委派模型，也就是父类加载器需要去请求子类加载器去完成需要的类。

典型例子如JNDI，JNDI的代码由启动类加载器完成加载，但是这个类需要应用程序ClassPath下的JNDI服务提供者接口，启动类加载器无法加载这些代码。为解决这个问题使用线程上下文类加载器，JNDI服务使用这个线程上下文类加载器去加载所需的SPI服务代码，这是一种父类加载器去请求子类加载器完成类加载的行为，这种行为实际上是打通了双亲委派模型的层次结构来逆向使用类加载器。

OSGi实现模块化热部署：在OSGi环境下，类加载器不再双亲委派模型推荐的树状结构，而是进一步发展为更加复杂的网状结构。

内部类

使用原因：

1. 内部类可以对同一个包中的其他类隐藏
2. 内部类方法可以访问定义这个类的作用域中的数据，包括私有数据

1. 使用内部类访问对象状态

一个内部类方法可以访问自身的数据字段，也可以访问创建它的外围对象的数据字段，所以，内部类的对象总有一个 隐式引用，指向创建它的外部类对象

外围类的引用在构造器中设置，编译器会修改所有内部类的构造器，添加提个对应的外围类引用的参数

只有内部类是可以是 private，而常规类只可以有 包可见性(default、protected) 或 公共可见性(public)

2. 内部类的特殊语法规则

1、外围类引用`OuterClass.this`

如：Person.this

2、内部类对象的构造器

```
outerObject.new InnerClass(construction param)
```

3、外围类的作用域之外引用内部类

```
OuterClass.InnerClass
```

4、内部类中**声明的所有静态字段都必须是 final**，并初始化为一个编译时常量

5、内部类不能有static方法，除了**访问外围类的静态字段和方法**的静态方法

3. 内部类与编译器

内部类是一个编译器现象，与虚拟机无关，编译器会讲内部类转换为常规的类文件，用 `$` 分隔外部类名和内部类名

4. 局部内部类

当这个类的对象只被一个方法创建一次时，在一个方法中的局部地定义这个类

1、声明局部类时

不能有访问说明符(private，public) 局部类的作用域被限定在声明这个局部类的块中

2、局部类的优势

对外部世界完全隐藏，**连外围类中的其他代码都不能访问它**

5. 由外部方法访问变量

与其他内部类相比较，局部类可以访问 **方法中的局部变量**
不过，这些变量**必须是 final 变量**

6. 匿名内部类

只创建这个类的一个对象，不需要为类指定名字

1、语法：`new superType(construcrion param) {...}`

superType可以是接口，内部类要实现这个接口，也可以是一个类，内部类就要扩展这个类

2、构造器名字必须与类名相同

匿名内部类没有类名，所以匿名内部类不能有构造器，实际上，构造参数要传递给超类的构造器。但，只要内部类实现一个接口，就不能有任何构造参数，不过仍然要小括号

尽管匿名内部类不能有构造器，但可以提供一个对象初始化块

`new Person("asd") {  {init}  ....}`

3、如果构造参数列表的结束小括号后面跟一个开始大括号，就是在定义匿名内部类

4、静态方法中要得到当前类名，不能使用 this，静态方法没有this，可以使用如下技巧

`new Object(){}.getClass().getEnclosingClass()`
上式，创建了一个匿名内部类，getEnclosingClass得到他的外围类

7. 双括号初始化

利用了内部类语法，使用匿名列表

```
ArrayList<Integer> array = new ArrayList<>(){{
    add(1);
    add(2);
    add(3);
}};
System.out.println(array.toString());
```

外层括号建立了ArrayList的一个匿名子类，内层括号则是一个对象初始化块

适用于这些数组列表不需要再使用的情况

Map，Deque，Set等集合也有类似技巧

8. 静态内部类

static声明的类

1. 静态内部类类似于其他内部类，不过静态内部类的独享没有生成它外围类对象的引用
2. 只要内部类不需要访问外围类对象，就应该使用静态内部类
3. 与常规内部类不同，静态内部类可以由静态字段和方法
4. 在接口中声明的内部类自动是 public 和 static

代理类

1. 创建代理对象

使用Proxy类的newProxyInstance方法，有三个参数

1. 一个类加载器
2. 一个Class对象数组
3. 一个调用处理器

```
Student s1 = new Student();
//动态代理增强s1对象
/*
三个参数:
```

```
 类加载器：真实对象.getClass().getClassloader()
 接口数组：真实对象.getClass().getInterfaces()
 处理器：new InvocationHandler()
*/
Student proxy_s1 = (Student)Proxy.newProxyInstance(s1.getClass().getClassLoader(),
s1.getClass().getInterfaces(), new InvocationHandler() {
   //代理逻辑编写的方法，代理对象调用的所有方法都会触发该方法的执行
   /*
    参数：proxy：代理对象
      method：代理对象调用的方法，被封装为对象
      args：代理对象调用实际方法时，传递的实际参数
   */
            @Override
            public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable {
             System.out.println("每次都会调用我");

    if(method.getName().equals("setName")){//如果调用setName()方法就增强，否则不增强
     String name = args[0];
     name = "李四";
     String obj = (String) method.invoke(s1,name);//使用真实对象调用该方法，改变参数调用，同
时改变返回值为String
                return obj+"和王五";
    }else{
     Object obj = method.invoke(s1,args);//使用真实对象调用该方法,原样调用
                return obj;
    }
            }
        });
      System.out.println(proxy_s1.setName("张三");)//输出“每次都会调用我”，“李四和王五”，
      //证明每次调用代理对象的真实对象的方法都会执行invoke()
      //并且method对象是代理对象调用的方法的被封装后的对象，args是传递的参数
```

2. 代理类的特性

代理类是在程序运行过程中动态创建的，一旦被创建，就成为了常规类

所有代理类都扩展Proxy类，一个代理类只有一个实例字段，调用处理器，在Proxy超类中定义，完成代理对象任务所需要的任何额外数据都必须存储在调用处理器中

代理类总是 public 或 final，如果代理类实现的所有接口都是 public 这个代理类就不输入任何特定的包，斗则所有非公共的接口都必须属于同一个包，代理类也属于这个包

Spring

IoC

IoC: 控制反转，意思就是将创建对象的控制权从自己硬编码new的一个对象反转到了第三方身上。

IoC的主要实现方式是依赖注入，Spring中的依赖注入方式有：构造方法注入、settter注入、接口注入。

目的：

帮助我们接耦各种有依赖关系的业务对象之间的绑定关系

IoC-Provider

虽然不需要我们自己来做绑定关系，但是这部分的工作还是需要有人来实现的，所以IoC Provider就担任了这个角色，同时IoC Provider的职责也不仅仅这些，其基础职责如下：

1、业务对象的构建管理：

IoC中，业务对象不需要关心所依赖的对象如何构建如何获取，这部分任务交由IoC Provider

2、业务对象之间的依赖绑定：

通过结合之前构建和管理的所有业务对象，以及各个业务对象之间可识别的依赖关系，将这些对象所依赖的对象注入绑定。从而保证每个业务对象在使用的时候，可以出狱就绪状态。

Spring的IoC容器

担任了IoC Provider的职责，同时在此基础上，还增加了对Bean生命周期的管理、AOP支持内容。

从整体来看Spring的IoC容器的作用，共分为两部分：

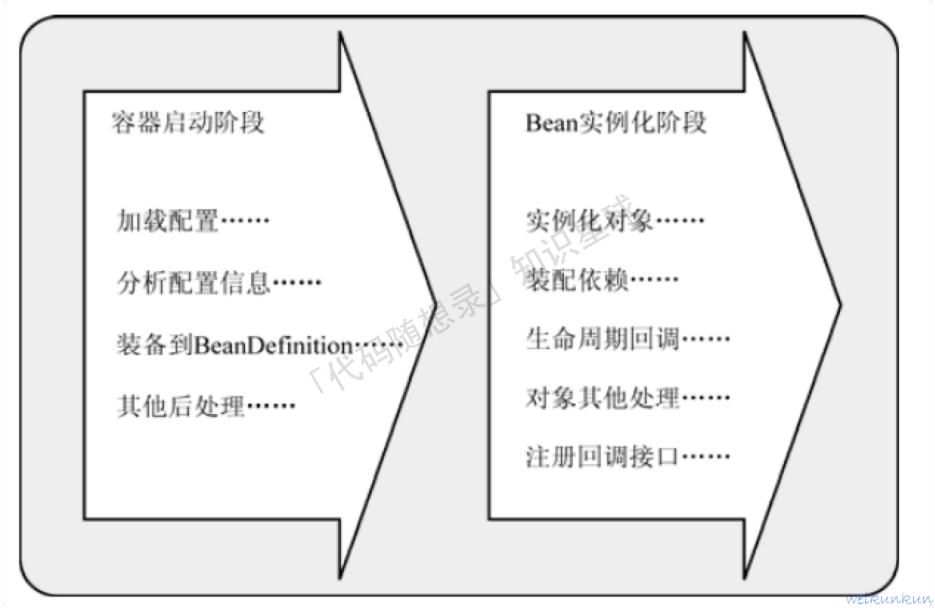
1、容器启动阶段：

以某种方式将配置的Bean信息（XML、注解、Java编码）加载如整个Spring应用

2、Bean实例化阶段：

将加载的Bean配置信息组装成应用需要的业务对象

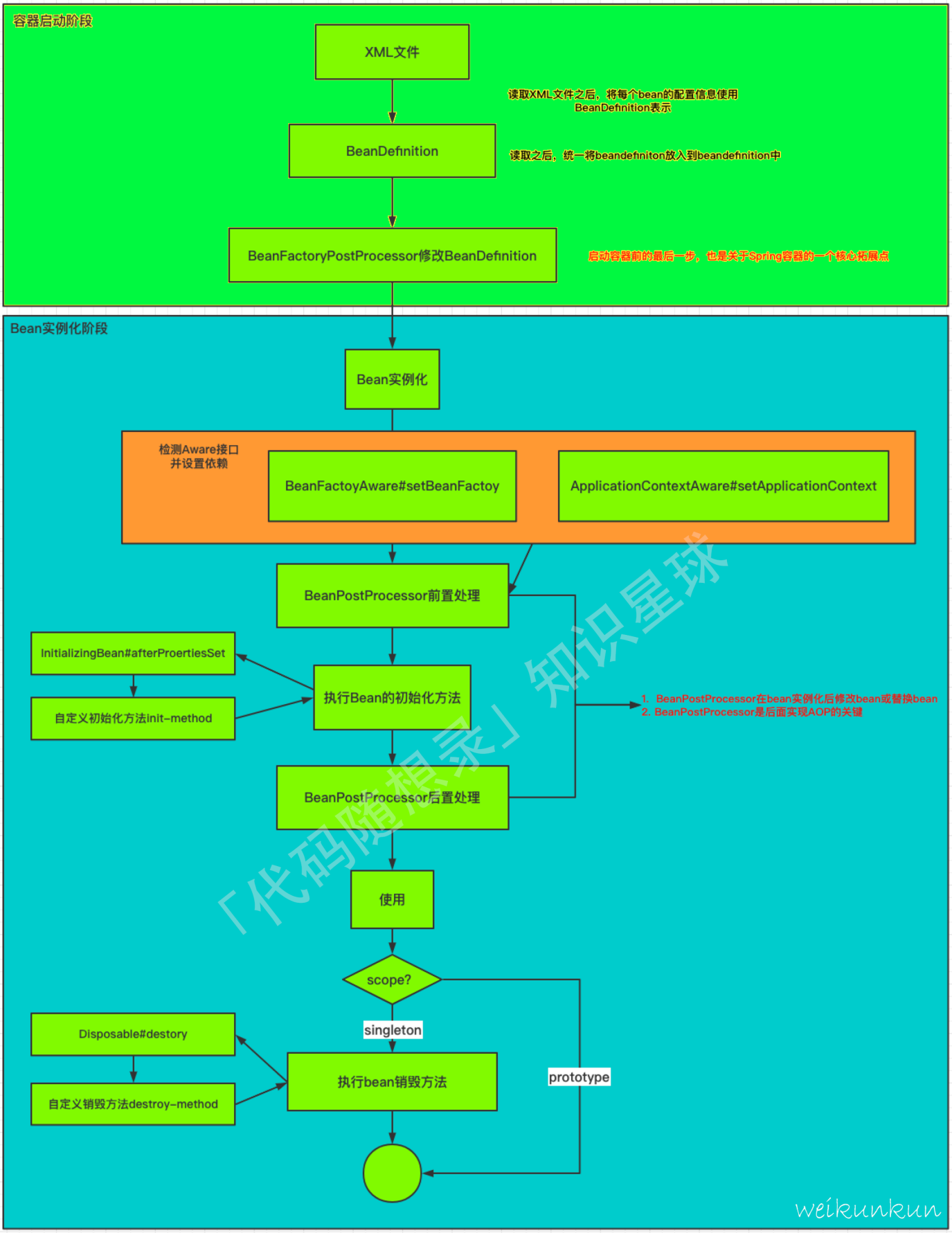
在此基础上，还充分运用了这两个阶段不同的特点，都预留了拓展钩子，供我们根据业务场景进行自定义拓展。



一些核心的接口、类介绍：

1. Resource: 用于解决 IoC 容器中的内容从哪里来的问题，也就是 配置文件从哪里读取、配置文件如何读取 的问题
2. BeanDefinition: 用于解决 Bean 的具体定义问题，包括 Bean 的名字是什么、它的类型是什么，它的属性赋予了哪些值或者引用，也就是 如何在 IoC 容器中定义一个 Bean，使得 IoC 容器可以根据这个定义来生成实例 的问题。
3. BeanFactoy: 用于解决 IoC 容器在 已经获取 Bean 的定义的情况下，如何装配、获取 Bean 实例 的问题。
4. ApplcationContext: 对上述的内容进行了功能的封装，解决 根据地址获取 IoC 容器并使用 的问题。

从Bean的角度来看，其整个生命周期如下：



Spring只帮我们管理单例Bean的生命周期，对于prototype类型的bean，Spring在创建好交给使用者使用之后，就不在管理其后续的生命周期了。

粗略涉及具体源码的整体流程：

1. BeanDefinitionReader读取Bean的配置信息（XML等），将读取到的每个Bean的配置信息使用BeanDefinition表示，同时注册到相应的BeanDefinitionRegistry（一个map）中 。
2. 通过实现了BeanFactoryPostProcessor的类，自定义修改BeanDefinition中的信息（如果有的话）
3. Bean的实例化:
   (1) 采用策略化bean的实例， 两种方式：cglib、反射
   (2) 获取Bean的实例之后，根据BeanDefinition中信息，填充Bean的属性、依赖
4. 检测各种Aware接口，BeanFactory的、ApplicationContext的等
5. 调用BeanPostProcessor接口的前置处理方法，处理符合要求的Bean实例
6. 如果实现了InitializingBean接口，执行对应的afterPropertiesSet()方法
7. 如果定义了init-method，执行对应的自定义初始化方法
8. 调用BeanPostProcessor接口的前置处理方法，处理符合要求的Bean实例
9. 使用
0. 判断Bean的Scope，如果是prototype类型，不再管理
1. 如果是单例类型，如果实现了DisposableBean接口，执行对应的destoy方法
2. 如果定义了destory-method，执行对应的自定义销毁方法

两个拓展点：

1、BeanFactoryPostProcess

Spring提供的容器扩展机制，允许我们在bean实例化之前修改bean的定义信息即BeanDefinition的信息

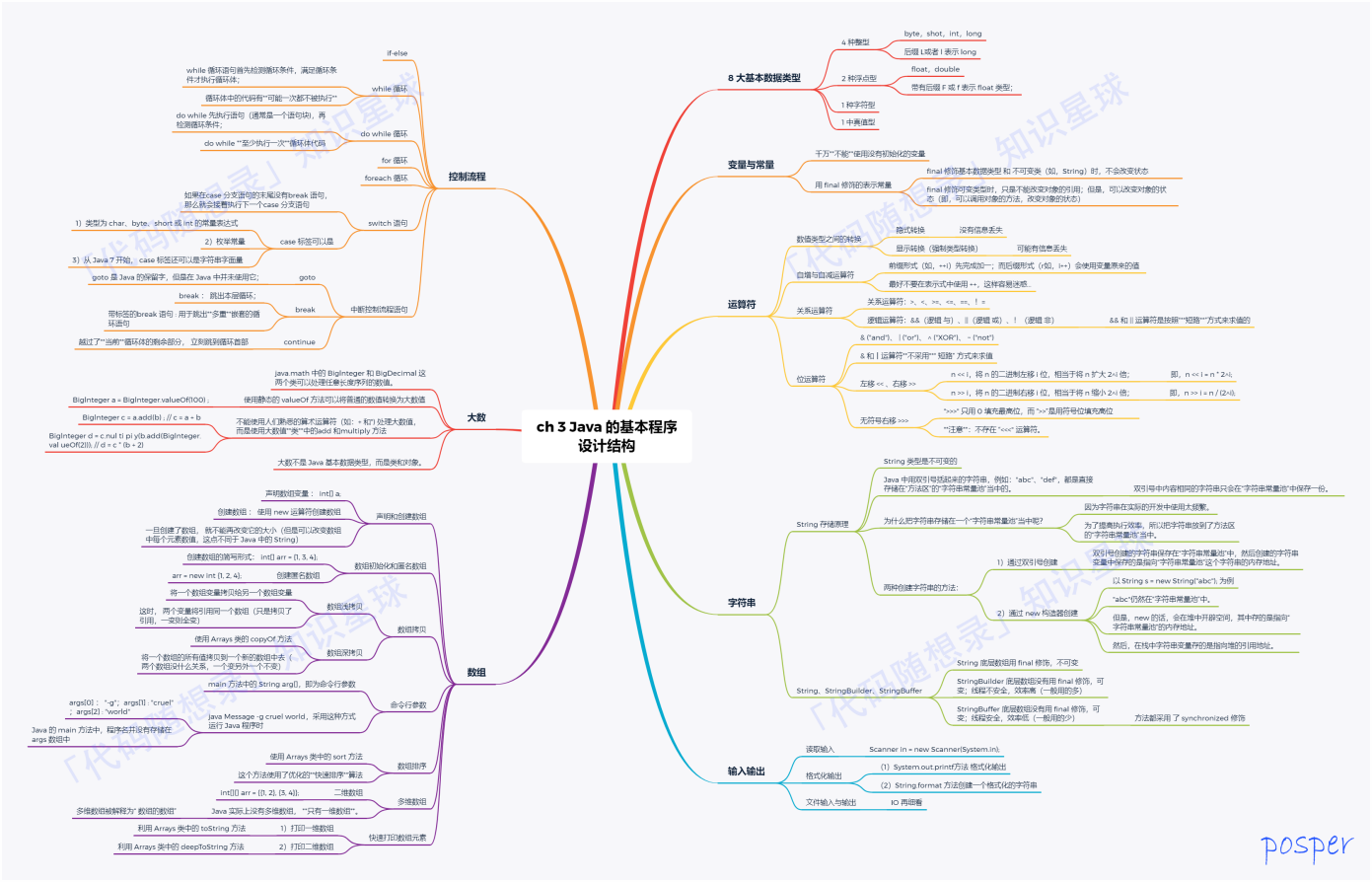
2、BeanPostProcessor

也是Spring提供的容器扩展机制，不同于BeanFactoryPostProcessor的是，BeanPostProcessor在bean实例化后修改bean或替换bean。BeanPostProcessor是后面实现AOP的关键。

Java知识导图

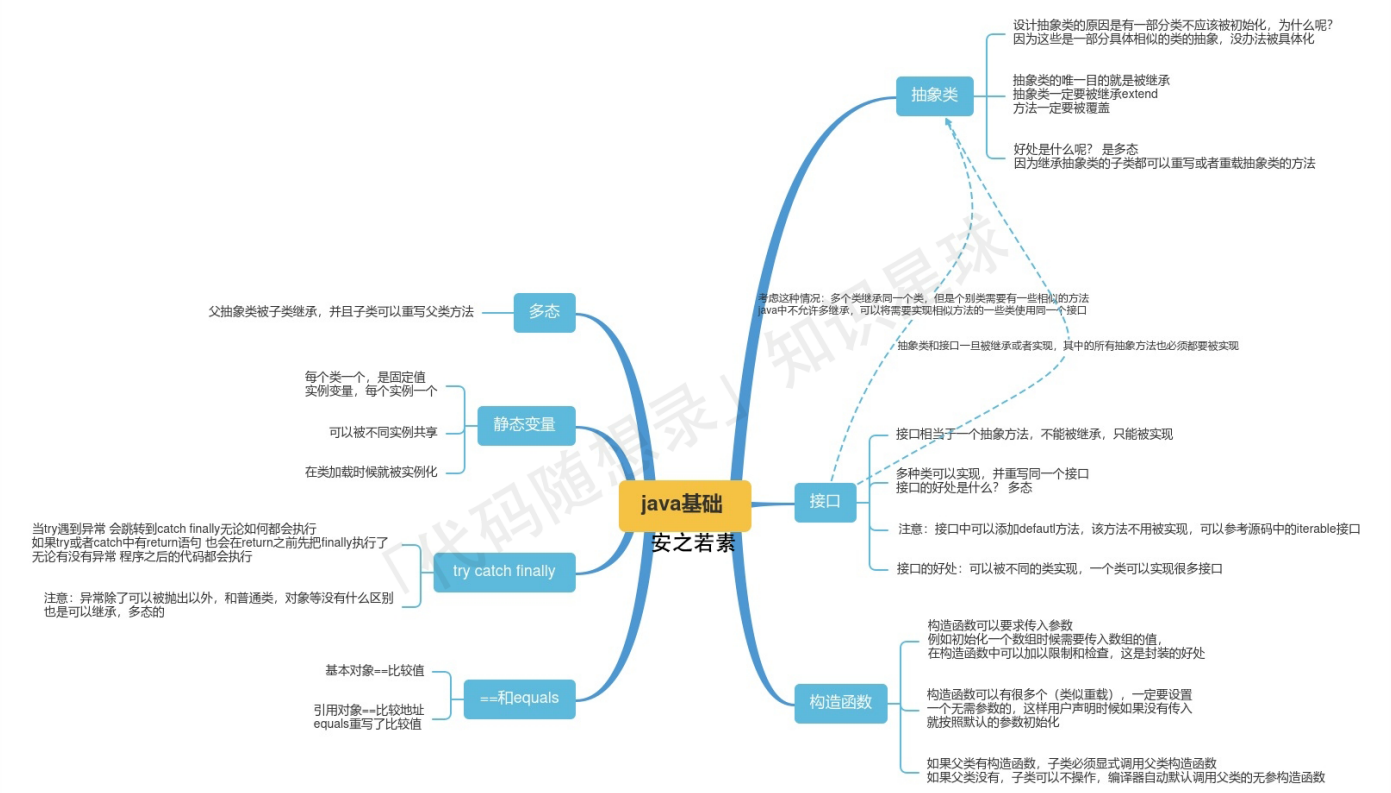
Java程序设计基本结构

原图链接：「代码随想录」知识星球



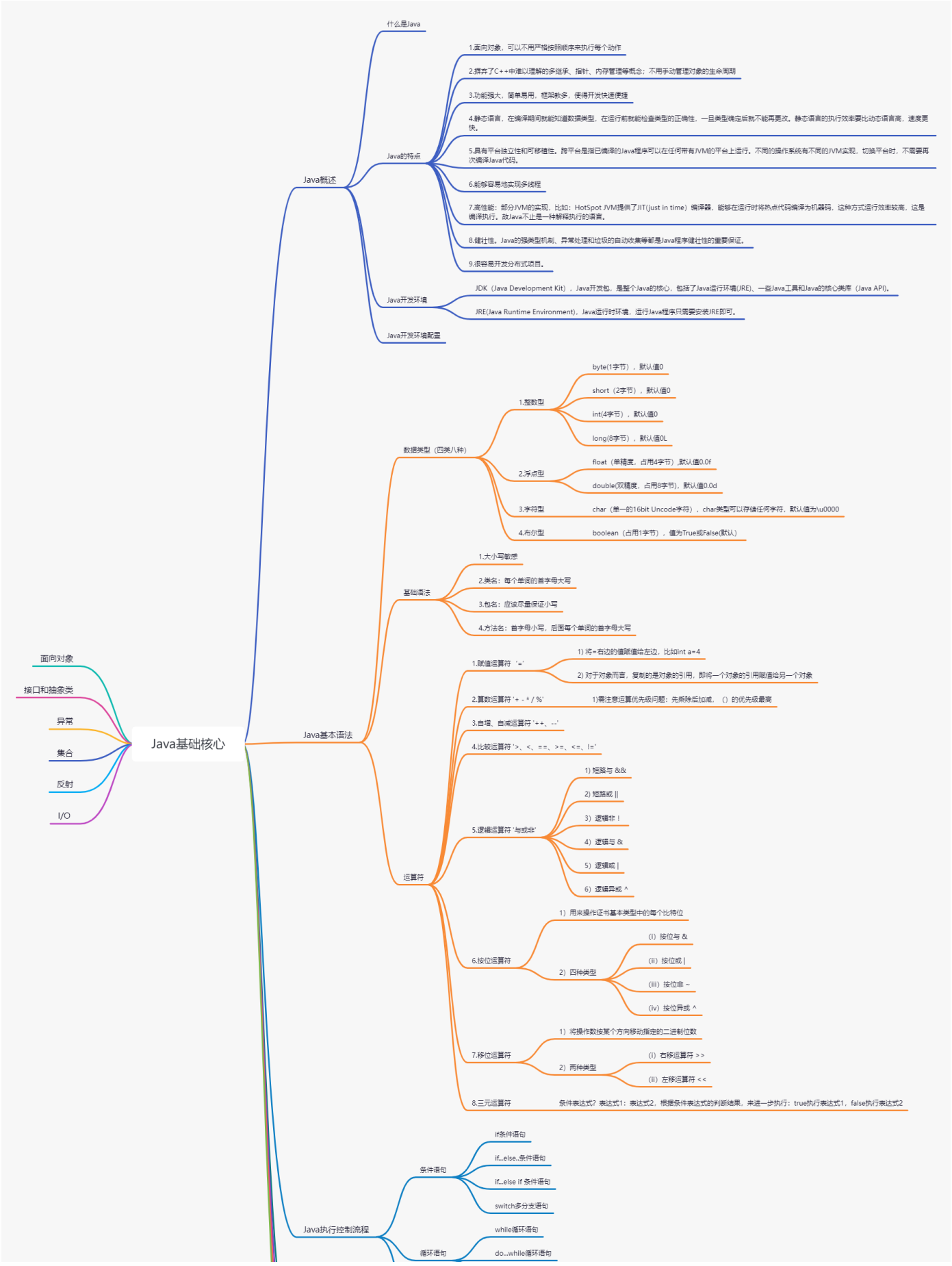
Java 基础总结

原图链接：「代码随想录」知识星球

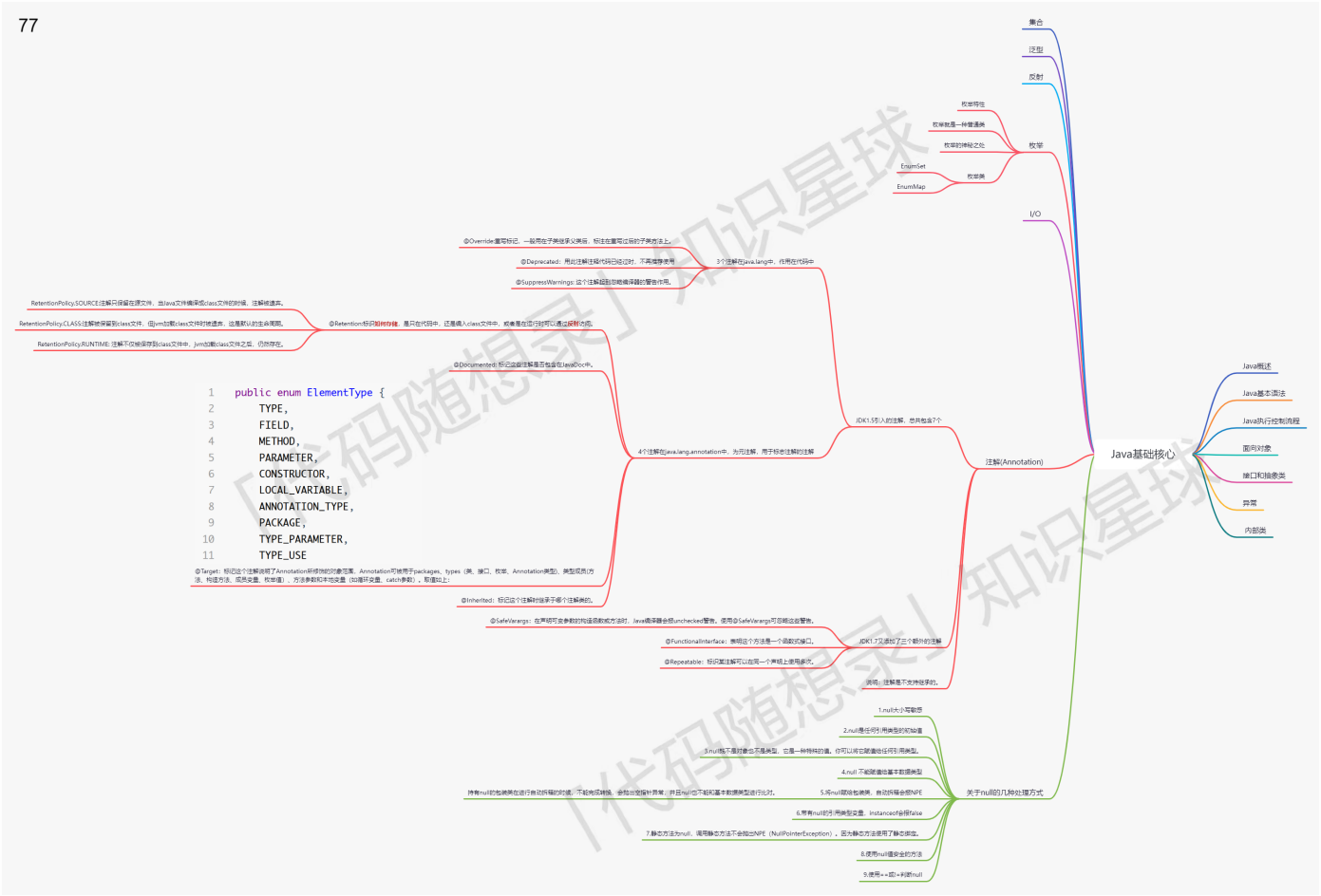


Java基础总结

原图链接：「代码随想录」知识星球

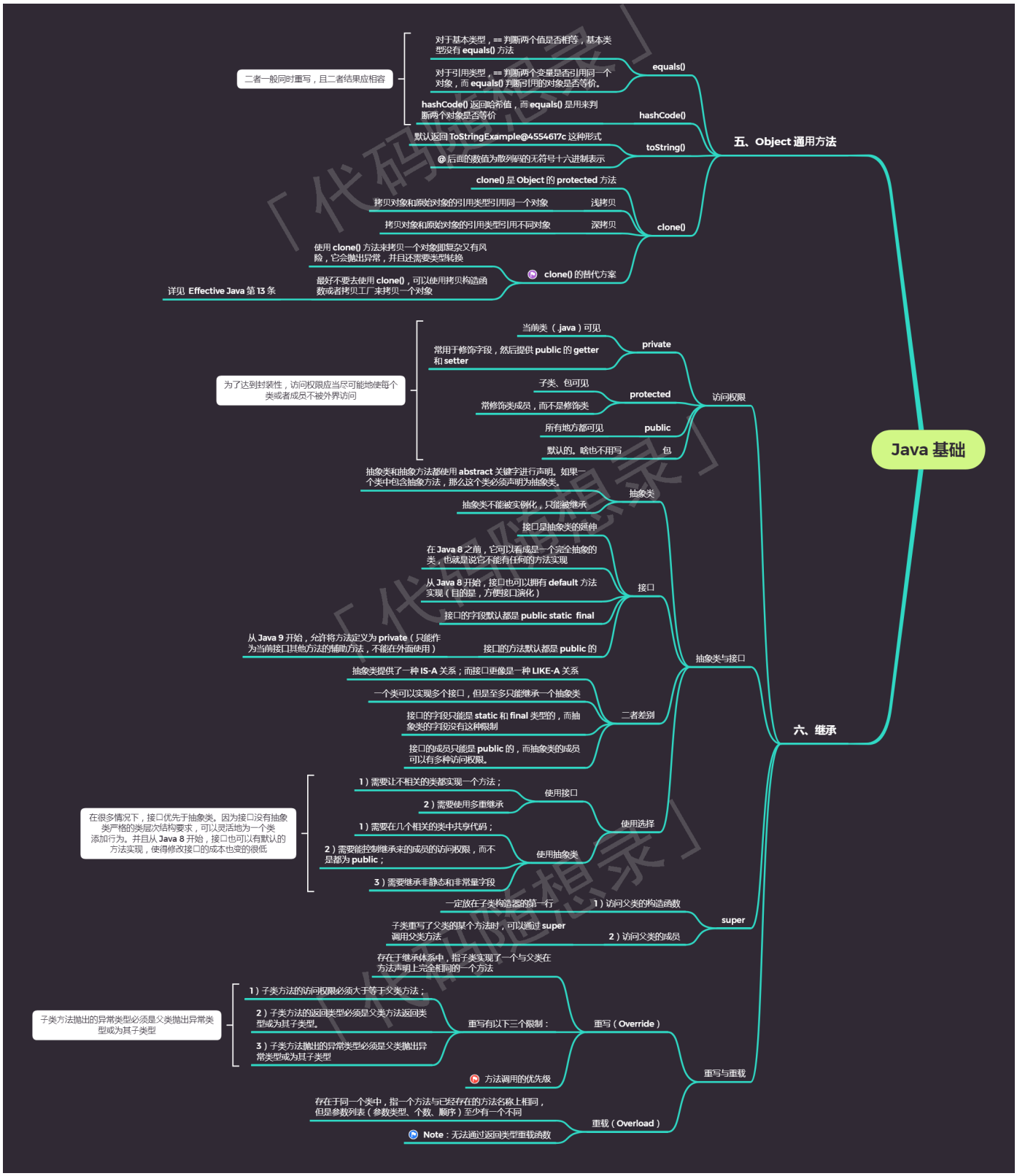


原图链接：「代码随想录」知识星球



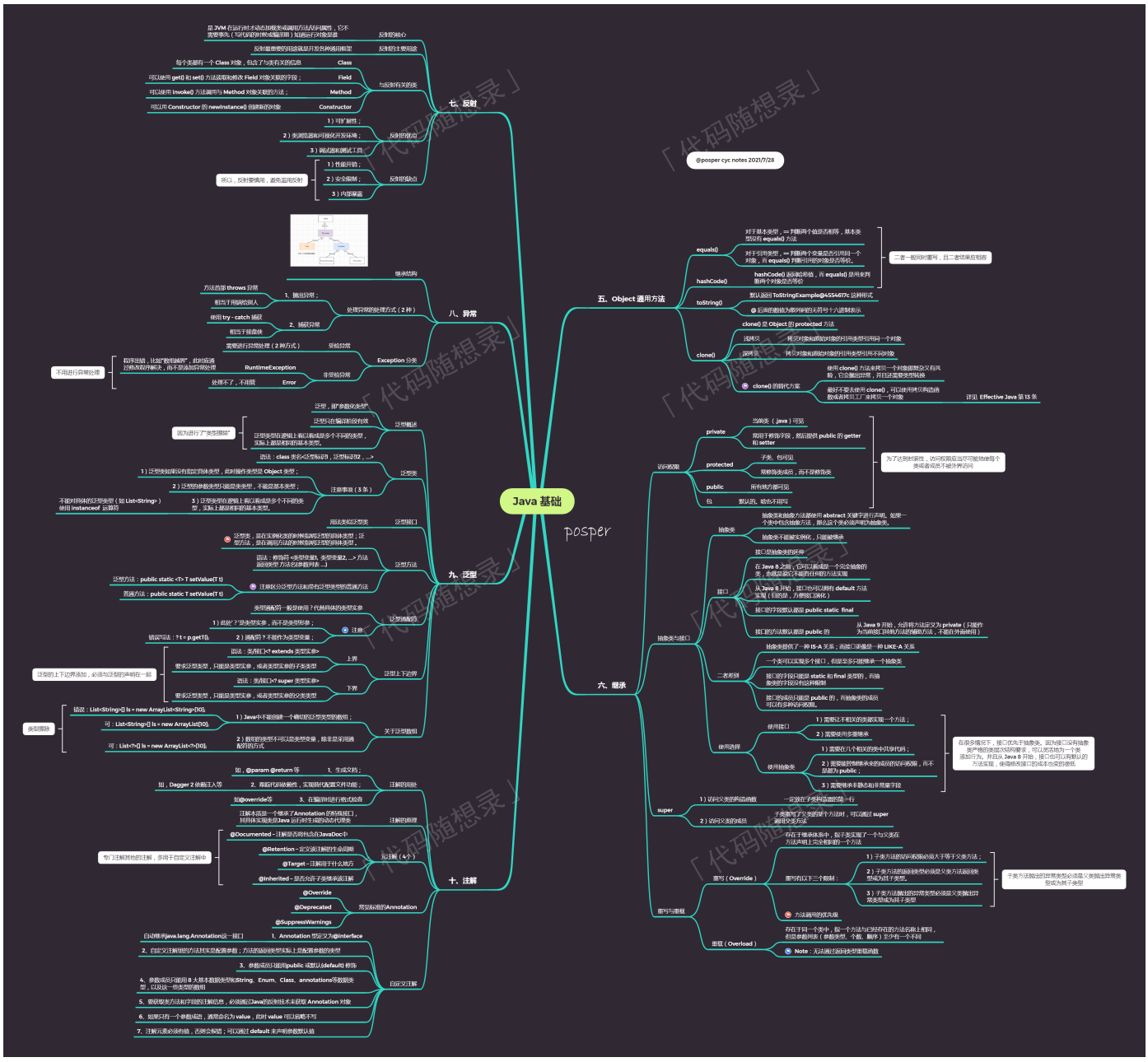
Java基础总结

原图链接：「代码随想录」知识星球


Java 基础
五、Object 通用方法
equals()
对于基本类型，== 判断两个值是否相等，基本类型没有 equals() 方法
对于引用类型，== 判断两个变量是否引用同一个对象，而 equals() 判断引用的对象是否等价。
二者一般同时重写，且二者结果应相容
hashCode()
hashCode() 返回哈希值，而 equals() 是用来判断两个对象是否等价
toString()
默认返回 ToStringExample@4554617c 这种形式
@ 后面的数值为散列码的无符号十六进制表示
clone()
clone() 是 Object 的 protected 方法
浅拷贝
拷贝对象和原始对象的引用类型引用同一个对象
深拷贝
拷贝对象和原始对象的引用类型引用不同对象
clone() 的替代方案
使用 clone() 方法来拷贝一个对象即复杂又有风险，它会抛出异常，并且还需要类型转换
最好不要去使用 clone()，可以使用拷贝构造函数或者拷贝工厂来拷贝一个对象
详见 Effective Java 第 13 条
六、继承
访问权限
为了达到封装性，访问权限应当尽可能地使每个类或者成员不被外界访问
private
当前类（.java）可见
常用于修饰字段，然后提供 public 的 getter 和 setter
protected
子类、包可见
常修饰类成员，而不是修饰类
public
所有地方都可见
包
默认的。啥也不用写
抽象类与接口
抽象类
抽象类和抽象方法都使用 abstract 关键字进行声明。如果一个类中包含抽象方法，那么这个类必须声明为抽象类。
抽象类不能被实例化，只能被继承
接口
接口是抽象类的延伸
在 Java 8 之前，它可以看成是一个完全抽象的类，也就是说它不能有任何的方法实现
从 Java 8 开始，接口也可以拥有 default 方法实现（目的是，方便接口演化）
接口的字段默认都是 public static final
接口的方法默认都是 public 的
从 Java 9 开始，允许将方法定义为 private（只能作为当前接口其他方法的辅助方法，不能在外面使用）
二者差别
抽象类提供了一种 IS-A 关系；而接口更像是一种 LIKE-A 关系
一个类可以实现多个接口，但是至多只能继承一个抽象类
接口的字段只能是 static 和 final 类型的，而抽象类的字段没有这种限制
接口的成员只能是 public 的，而抽象类的成员可以有多种访问权限。
使用选择
在很多情况下，接口优先于抽象类。因为接口没有抽象类严格的类层次结构要求，可以灵活地为一个类添加行为。并且从 Java 8 开始，接口也可以有默认的方法实现，使得修改接口的成本也变的很低
使用接口
1）需要让不相关的类都实现一个方法；
2）需要使用多重继承
使用抽象类
1）需要在几个相关的类中共享代码；
2）需要能控制继承来的成员的访问权限，而不是都为 public；
3）需要继承非静态和非常量字段
super
1）访问父类的构造函数
一定放在子类构造器的第一行
2）访问父类的成员
子类重写了父类的某个方法时，可以通过 super 调用父类方法
重写与重载
重写（Override）
存在于继承体系中，指子类实现了一个与父类在方法声明上完全相同的一个方法
重写有以下三个限制：
1）子类方法的访问权限必须大于等于父类方法；
2）子类方法的返回类型必须是父类方法返回类型或为其子类型。
3）子类方法抛出的异常类型必须是父类抛出异常类型或为其子类型
子类方法抛出的异常类型必须是父类抛出异常类型或为其子类型
方法调用的优先级
重载（Overload）
存在于同一个类中，指一个方法与已经存在的方法名称上相同，但是参数列表（参数类型、个数、顺序）至少有一个不同
Note：无法通过返回类型重载函数

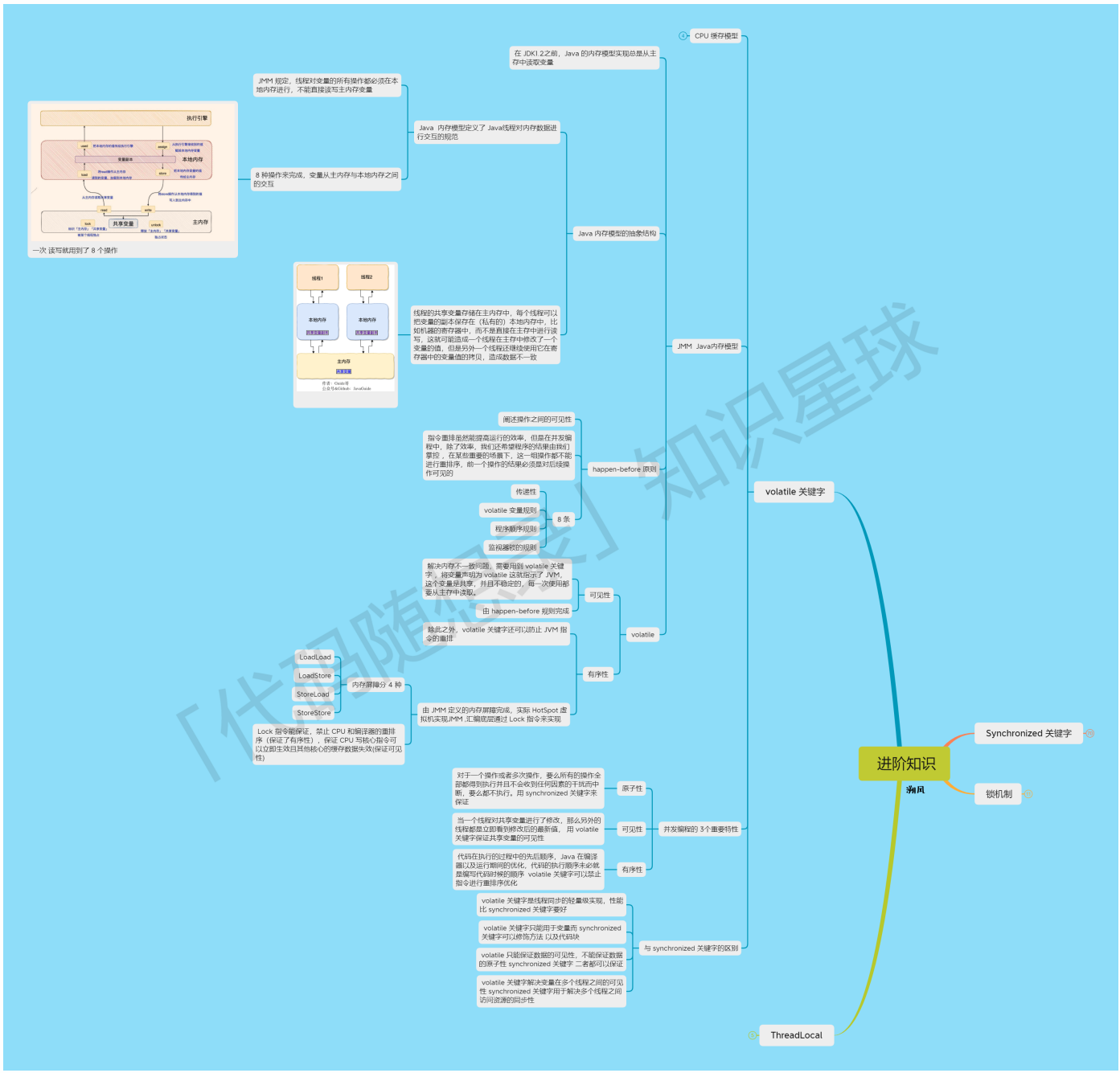
Java基础总结

原图链接：「代码随想录」知识星球



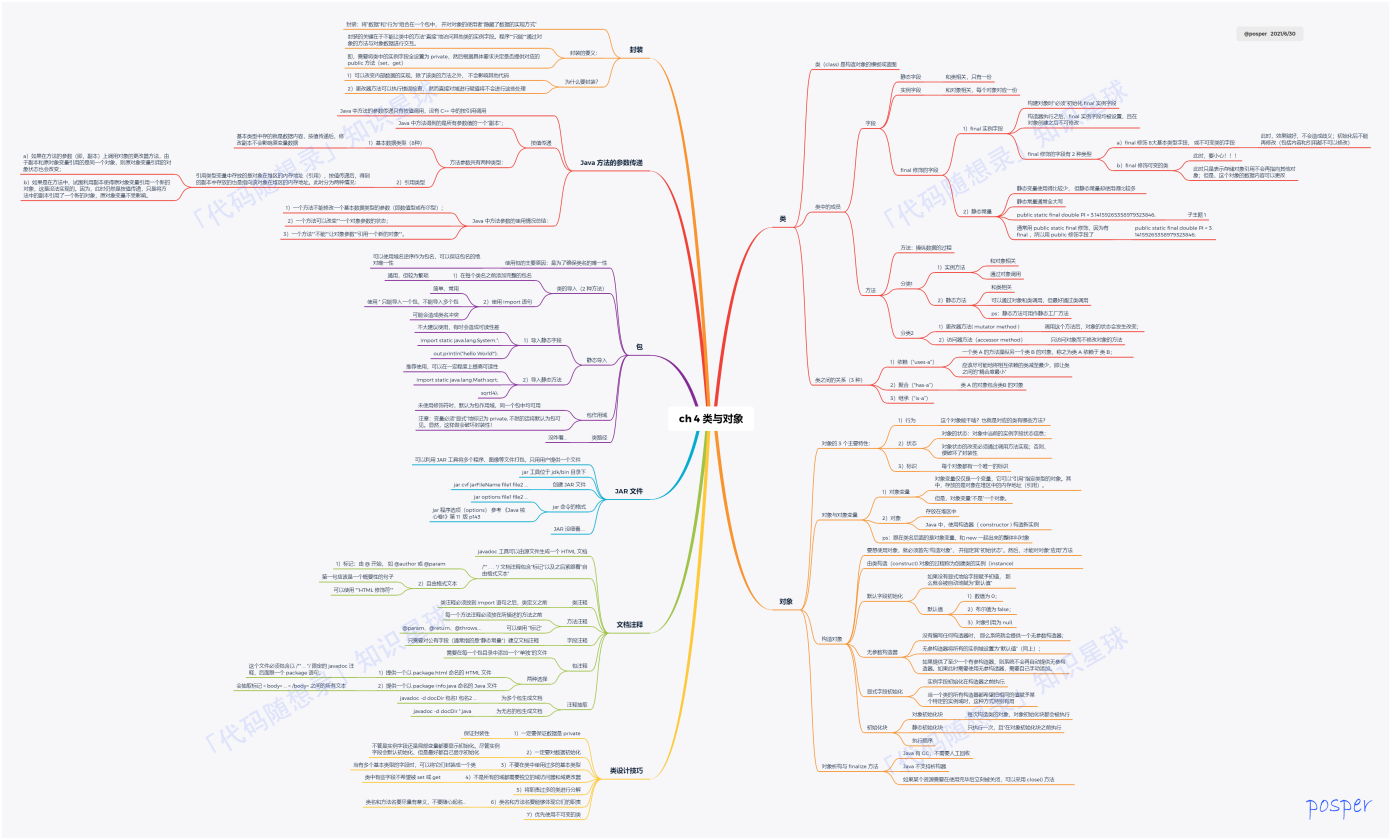
volatile关键字

原图链接：「代码随想录」知识星球



Java类和对象

原图链接：「代码随想录」知识星球



Java容器

原图链接：「代码随想录」知识星球

Java 容器

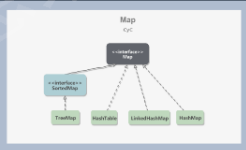
posper

一、概览

继承结构图

Collection

- Set
  - TreeSet
    - 无序，不可重复，自动排序
    - 存放在 TreeSet 中相当于存放到 TreeMap 的 key 部分
  - HashSet
    - 无序，不可重复，支持快速查找
    - 存放在 HashSet 中相当于存放到 HashMap 的 key 部分
  - LinkedHashSet
    - 基于双向链表实现，具有 HashSet 的查找效率
- List
  - ArrayList
    - 基于动态数组实现，支持随机访问
    - 增删元素通过移动元素实现，效率较低
  - Vector
    - 和 ArrayList 类似
    - 线程安全，但效率较低，现大多不用
  - LinkedList
    - 基于双向链表实现，增删元素效率高，但查询效率低（每次都要从头结点开始）
    - LinkedList 还可以用作栈、队列和双向队列
- Queue
  - LinkedList
    - 可以用它来实现双向队列
  - PriorityQueue
    - 基于堆结构实现，可以用它来实现优先队列



Map

继承结构图

- TreeMap
  - 基于红黑树实现，会根据 key 自动排序
- HashMap
  - 基于哈希表实现
- HashTable
  - 和 HashMap 类似
  - 线程安全。但它是遗留类，现大多不用；而是使用 ConcurrentHashMap 来支持线程安全
- LinkedHashMap
  - 使用双向链表来维护元素的顺序
  - 顺序为插入顺序或者最近最少使用（LRU）顺序

二、容器中的设计模式

迭代器模式

- Collection 继承了 Iterable 接口，其中的 iterator() 方法能够产生一个 Iterator 对象

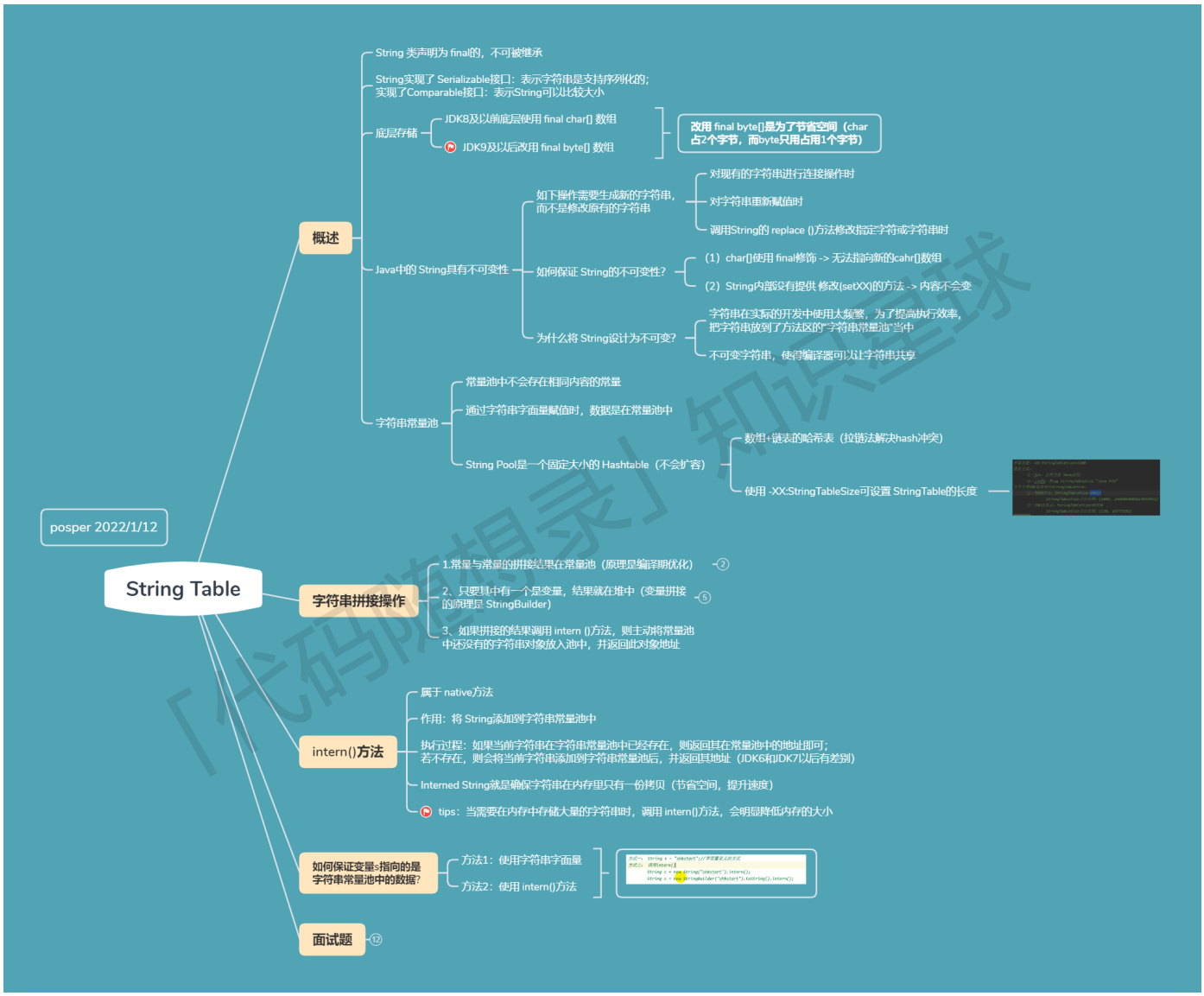
适配器模式

- java.util.Arrays#asList() 可以把数组类型转换为 List 类型

三、源码分析

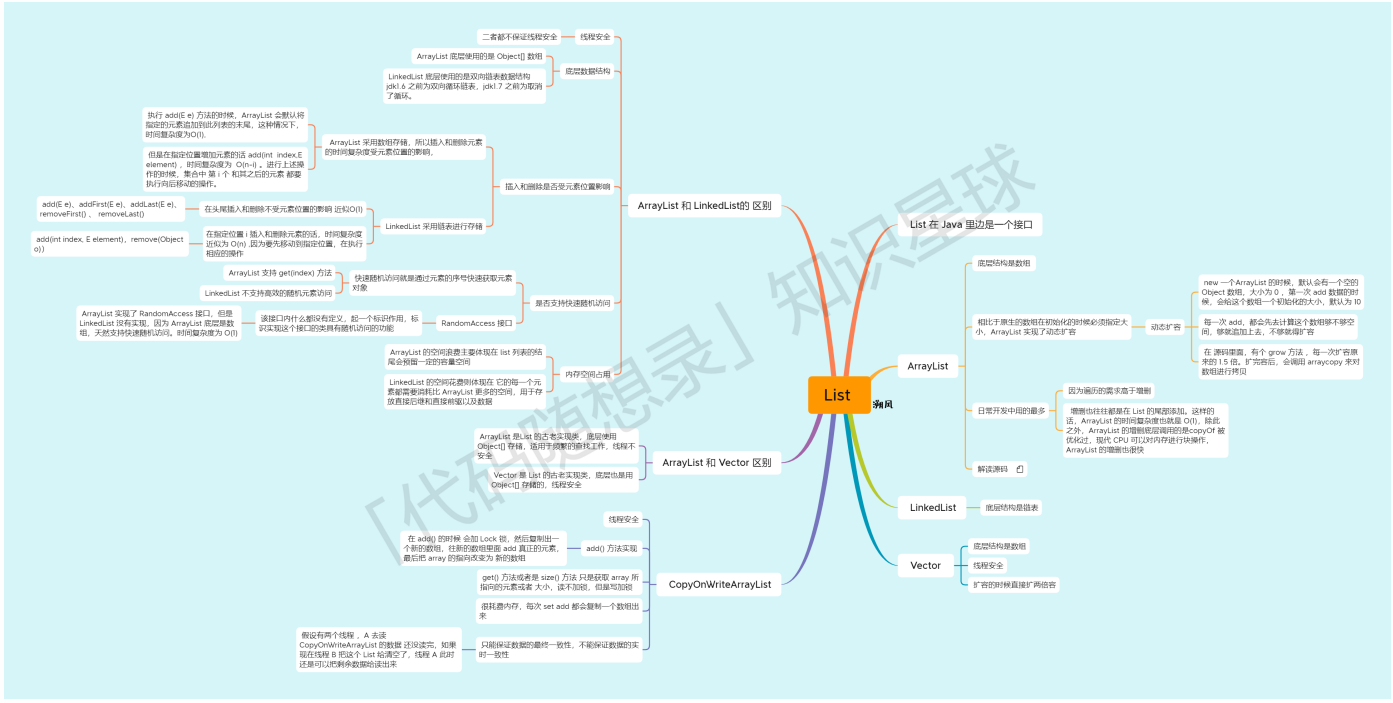
String Table

原图链接：「代码随想录」知识星球

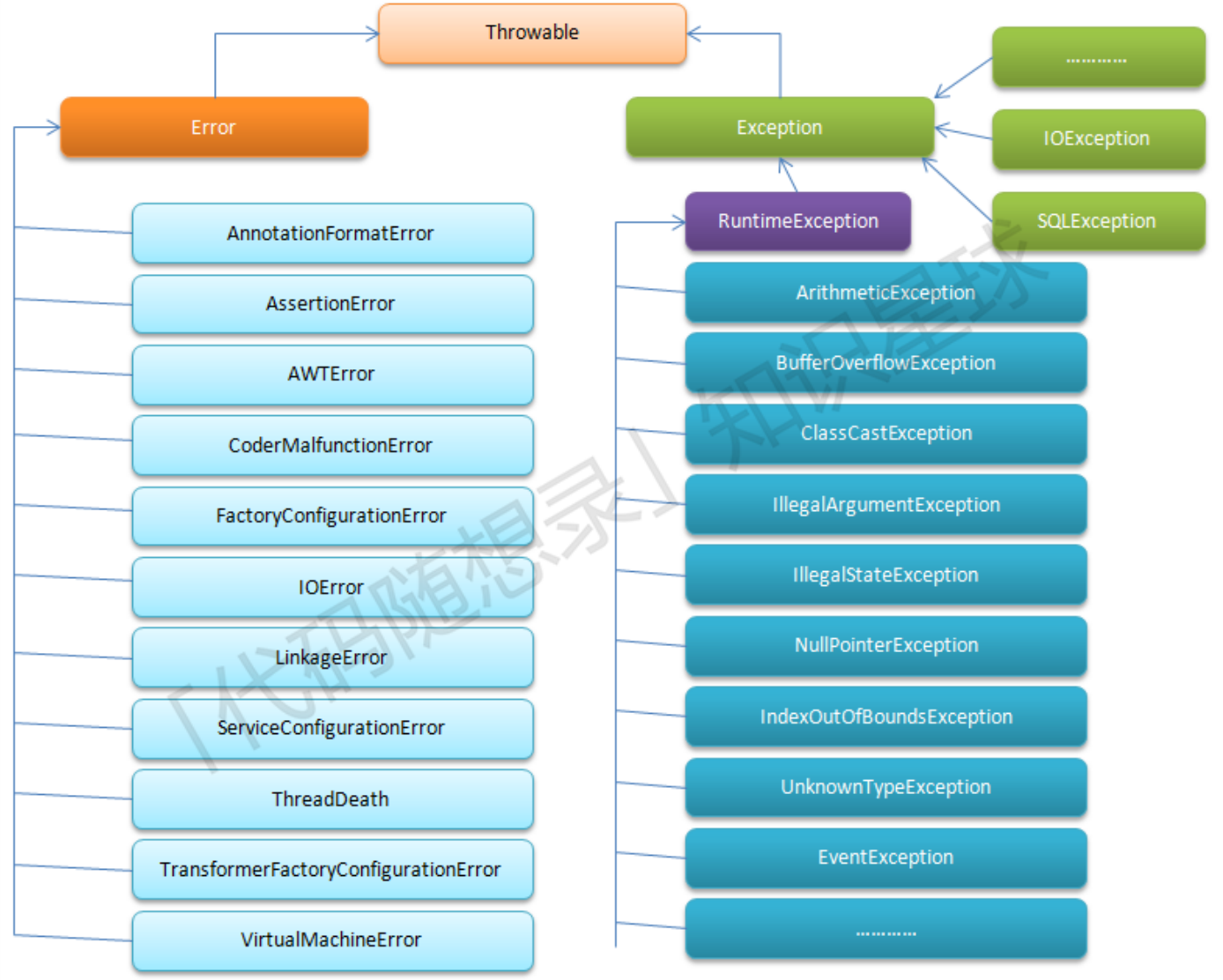


List

原图链接：「代码随想录」知识星球

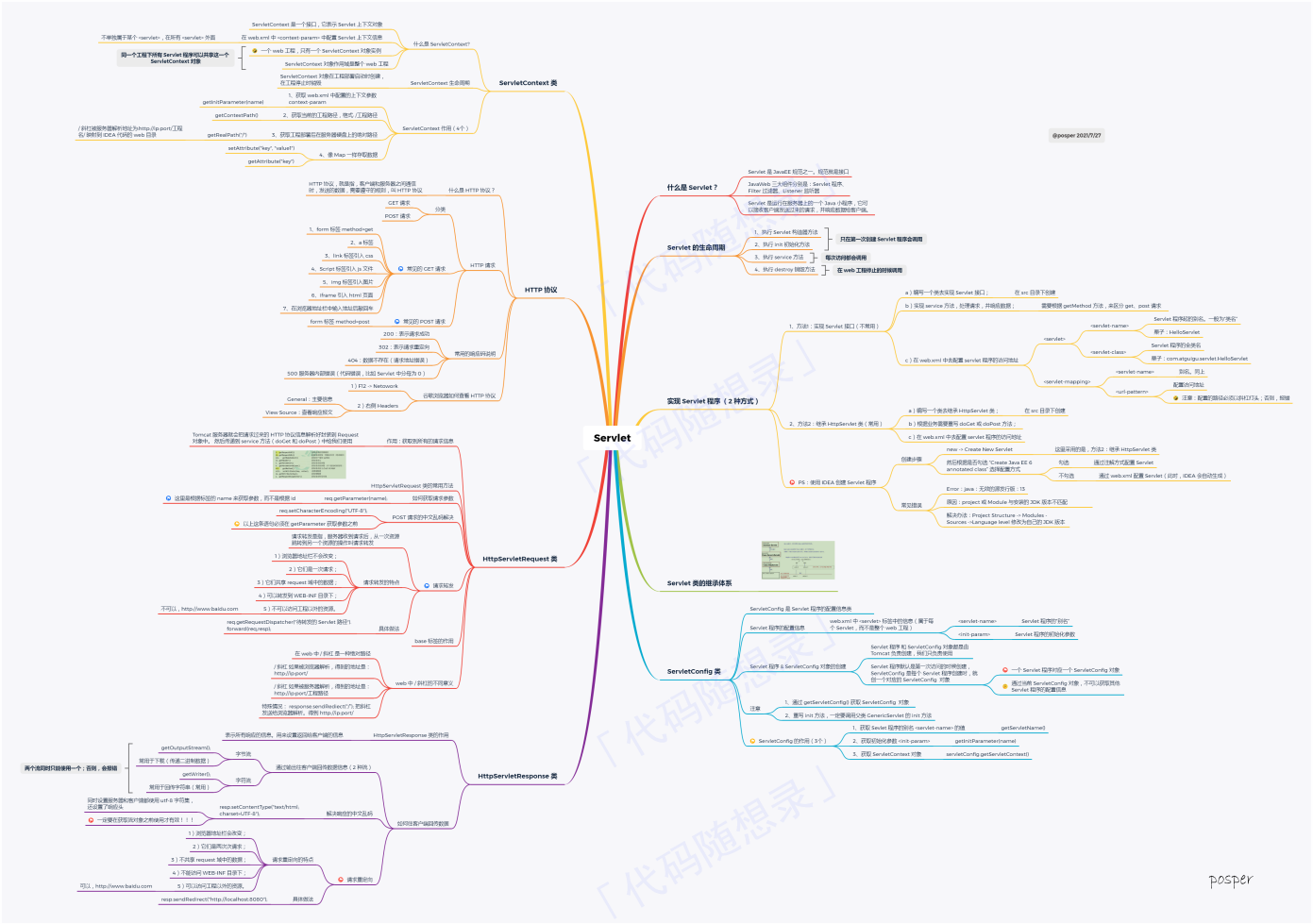

List
List 在 Java 里边是一个接口
ArrayList
底层结构是数组
相比于原生的数组在初始化的时候必须指定大小，ArrayList 实现了动态扩容
动态扩容
new 一个 ArrayList 的时候，默认会有一个空的 Object 数组，大小为 0 ，第一次 add 数据的时候，会给这个数组一个初始化的大小，默认为 10
每一次 add，都会先去计算这个数组够不够空间，够就追加上去，不够就得扩容
在 源码里面，有个 grow 方法，每一次扩容原来的 1.5 倍。扩完容后，会调用 arraycopy 来对数组进行拷贝
日常开发中用的最多
因为遍历的需求高于增删
增删也往往都是在 List 的尾部添加。这样的话，ArrayList 的时间复杂度也就是 O(1)。除此之外，ArrayList 的增删底层调用的是copyOf 被优化过。现代 CPU 可以对内存进行块操作，ArrayList 的增删也很快
解读源码
LinkedList
底层结构是链表
Vector
底层结构是数组
线程安全
扩容的时候直接扩两倍容
ArrayList 和 LinkedList的 区别
线程安全
二者都不保证线程安全
底层数据结构
ArrayList 底层使用的是 Object[] 数组
LinkedList 底层使用的是双向链表数据结构 jdk1.6 之前为双向循环链表，jdk1.7 之后为取消了循环。
插入和删除是否受元素位置影响
ArrayList 采用数组存储，所以插入和删除元素的时间复杂度受元素位置的影响。
执行 add(E e) 方法的时候，ArrayList 会默认将指定的元素追加到此列表的末尾，这种情况下，时间复杂度为O(1)。
但是在指定位置增加元素的话 add(int index,E element) ，时间复杂度为 O(n-i)。进行上述操作的时候，集合中 第 i 个 和其之后的元素 都要执行向后移动的操作。
LinkedList 采用链表进行存储
在头尾插入和删除不受元素位置的影响 近似O(1)
add(E e)、addFirst(E e)、addLast(E e)、removeFirst() 、 removeLast()
在指定位置 i 插入和删除元素的话，时间复杂度近似为 O(n) .因为要先移动到指定位置，在执行相应的操作
add(int index, E element)，remove(Object o))
是否支持快速随机访问
快速随机访问就是通过元素的序号快速获取元素对象
ArrayList 支持 get(index) 方法
LinkedList 不支持高效的随机元素访问
RandomAccess 接口
该接口内什么都没有定义。起一个标识作用，标识实现这个接口的类具有随机访问的功能
ArrayList 实现了 RandomAccess 接口，但是 LinkedList 没有实现。因为 ArrayList 底层是数组，天然支持快速随机访问。时间复杂度为 O(1)
内存空间占用
ArrayList 的空间浪费主要体现在 list 列表的结尾会预留一定的容量空间
LinkedList 的空间花费则体现在 它的每一个元素都需要消耗比 ArrayList 更多的空间，用于存放直接后继和直接前驱以及数据
ArrayList 和 Vector 区别
ArrayList 是 List 的古老实现类，底层使用 Object[] 存储，适用于频繁的查找工作，线程不安全
Vector 是 List 的古老实现类，底层也是用 Object[] 存储的，线程安全
CopyOnWriteArrayList
线程安全
add() 方法实现
在 add() 的时候 会加 Lock 锁，然后复制出一个新的数组，往新的数组里面 add 真正的元素，最后把 array 的指向改变为 新的数组
get() 方法或者是 size() 方法 只是获取 array 所指向的元素或者 大小，读不加锁，但是写加锁
很耗费内存，每次 set add 都会复制一个数组出来
只能保证数据的最终一致性，不能保证数据的实时一致性
假设有两个线程 ，A 去读 CopyOnWriteArrayList 的数据 还没读完，如果现在线程 B 把这个 List 给清空了，线程 A 此时还是可以把剩余数据给读出来

Java异常



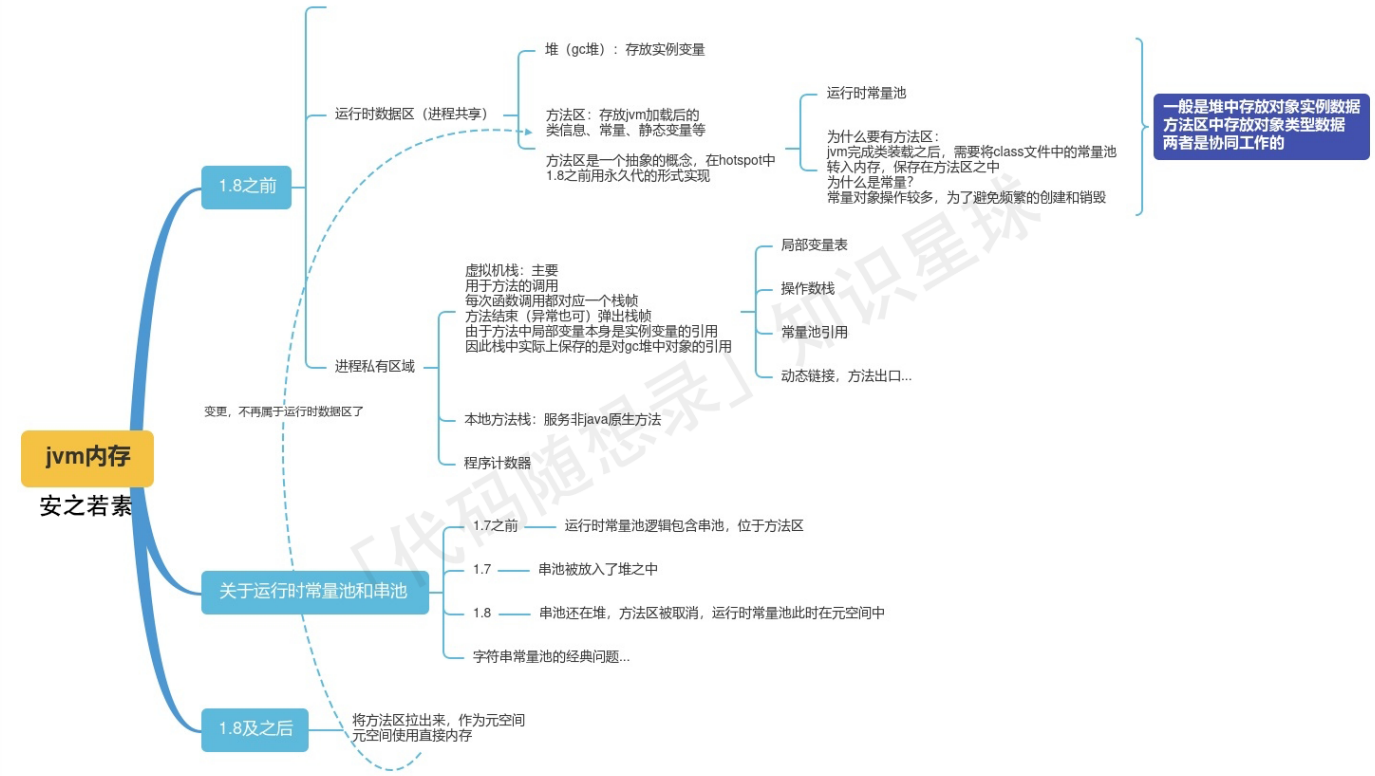
Servlet

原图链接：「代码随想录」知识星球



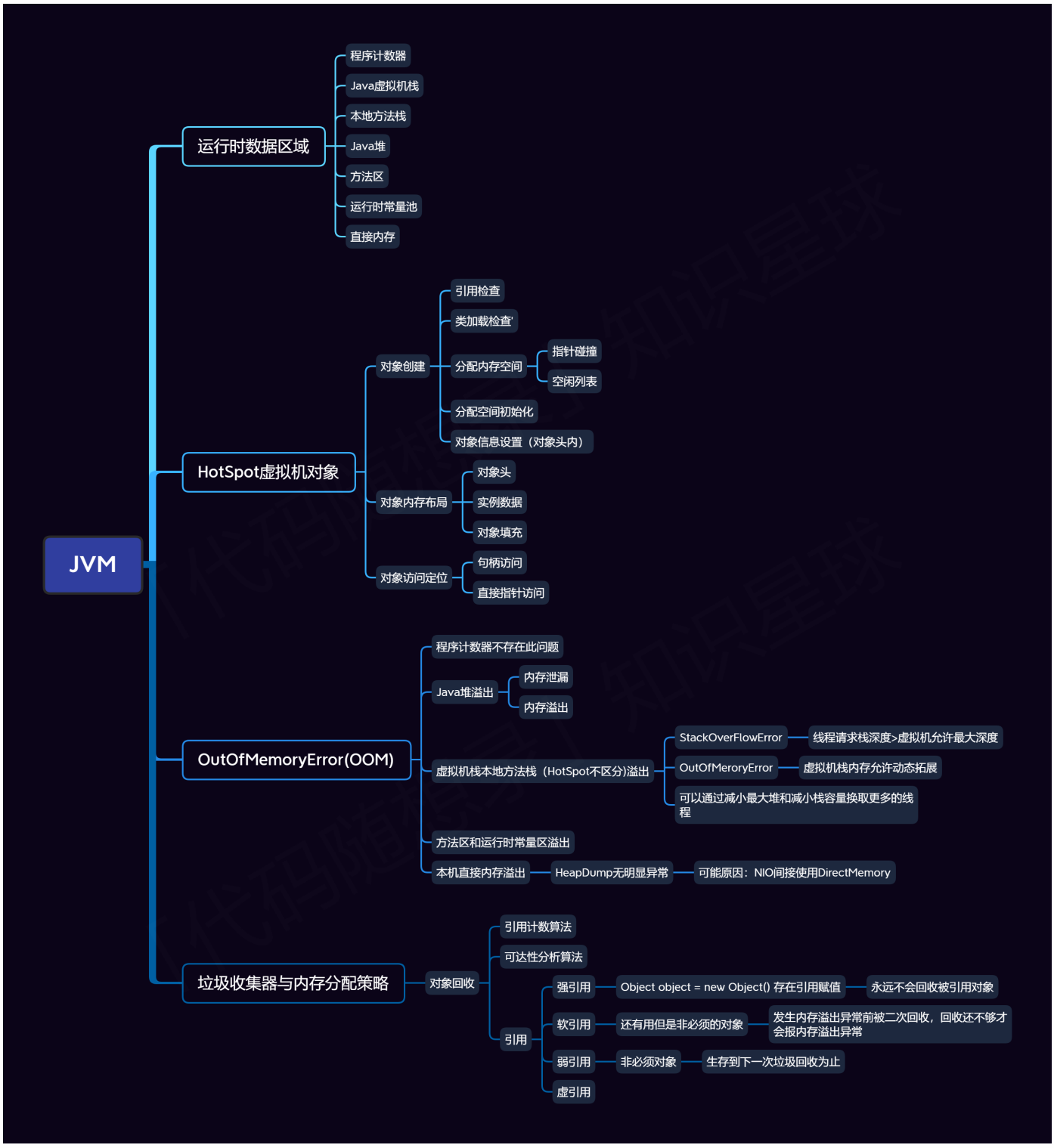
JVM内存

原图链接：「代码随想录」知识星球



JVM内存管理

原图链接：「代码随想录」知识星球



JavaWeb基础

原图链接：「代码随想录」知识星球

javaweb 前置

安之若素

web应用

- html css js
- 动态，可以和数据库交互，数据可以持久化
- jsp servlet
  - B/S架构（*）浏览器和服务器
  - C/S架构 客服端和服务器
- java程序
- jar
- properties

web服务器

- tomcat

http

- http1.0 http/1.0 客服端和服务器连接，只能获取一个资源
- http2.0 http/1.1 可以获得多个资源
- https
- 请求
  - 请求行
    - get 携带的参数比较少，大小有限制，url显示数据内容，不安全，但高效
    - post 与get相反
    - head...
  - 消息头
    - accept 浏览器支持的数据类型
    - accep-encoding
    - accep- language 支持的编码格式
    - cache- control
    - connection 断开还是连接
    - host 主机
- 响应
  - refresh 多久刷新一次
  - location 重定位
  - 状态码
    - 200
    - 4** 资源不存在
    - 3** 重定向
    - 5**
      - 服务器代码错误
      - 502 网关错误

面试题

- 本地能访问www.xxx.com吗
- 从浏览器地址栏输入地址，到返回页面，整个过程中发生了什么？

maven

- 在web开发中，使用大量的jar包，需要手动导入，如何让一个文件自动导入配置jar
- 核心思想：约定大于配置，不要违反约束
- 会导入目标jar包依赖的所有jar包
- maven资源导出问题 build中配置resource，防止资源导出失败
- maven父项目可以包括子项目，子项目可以直接调用父项目的所有依赖

servlet

- 开发动态web的技术，是一个接口，只需要两个步骤，编写一个类实现servlet接口，开发好的java类部署在web服务器
- mapping
  - 一个servlet可以指定一个或者多个或者通用前后缀（正则）路径
  - 优先级问题：匹配度高的路径优先级最高
- servlet context
  - sevlet1 url1
  - servlet2 url2
  - servlet3 url3
  - 保存多个servlet数据
  - 请求转发 地址不变，但是显示另一个页面的内容，相当于另一个页面是"代理" 307
  - 重定向 地址跳转 302
  - 读取资源文件

cookie session

- cookie客户端每次访问服务器都需要携带
- session可以保存用户的会话信息，可以把信息放在sesssion中
  - 服务器会对每一个用户（浏览器）创建一个session，一个session独占一个浏览器
  - 用户登陆之后，整个网站都可以访问
- session和cookie的区别
  - cookie把用户的数据写给用户（浏览器）
  - session把用户的数据写到用户独占的session中，服务器保存，只保存重要的信息

jsp：java server pages

- 和serverlet一样，用于开发动态web
- 语法和html差不多，但是jsp页面中可以嵌入java代码
- 服务器内部是如何工作的
  - tomcat中有一个work目录，发现页面转换成了java程序
  - 浏览器不管访问什么资源，都是访问servlet
  - jsp本质就是一个serblet，只要是是java代码，原封输出，如果是html代码，会装饰输出

九大内置对象

- 只在一个页面有效 存东西 pagecontext
- 只在一次请求有效 存东西 request
- response
- 从打开到关闭浏览器 存东西 session
- 服务器开机就一直存在 存东西 application(servlet context)
- config(servlet config)
- out
- 不用 page
- exception

Java Bean

- 实体类，有特定的写法
- 必须要有一个无参数构造
- 属性一定需要私有化
- 必须要有getset方法
- 一般用来和数据库字段做orm
  - java直接操作数据库
    - 表-类
    - 字段-属性
    - 行记录-对象

mvc三层架构

- model
  - 业务处理（逻辑）Service
  - 数据持久CRUD（Dao）
- view
  - 展示数据
  - 提供链接发起servlet请求
- controller（servlet）
  - 接受用户的请求（req，session，。）
  - 交给业务层处理
  - 控制试图跳转
- 架构图

filter

- web服务器调用sevlet时候经过filter，可以指定scope
- 实现权限拦截

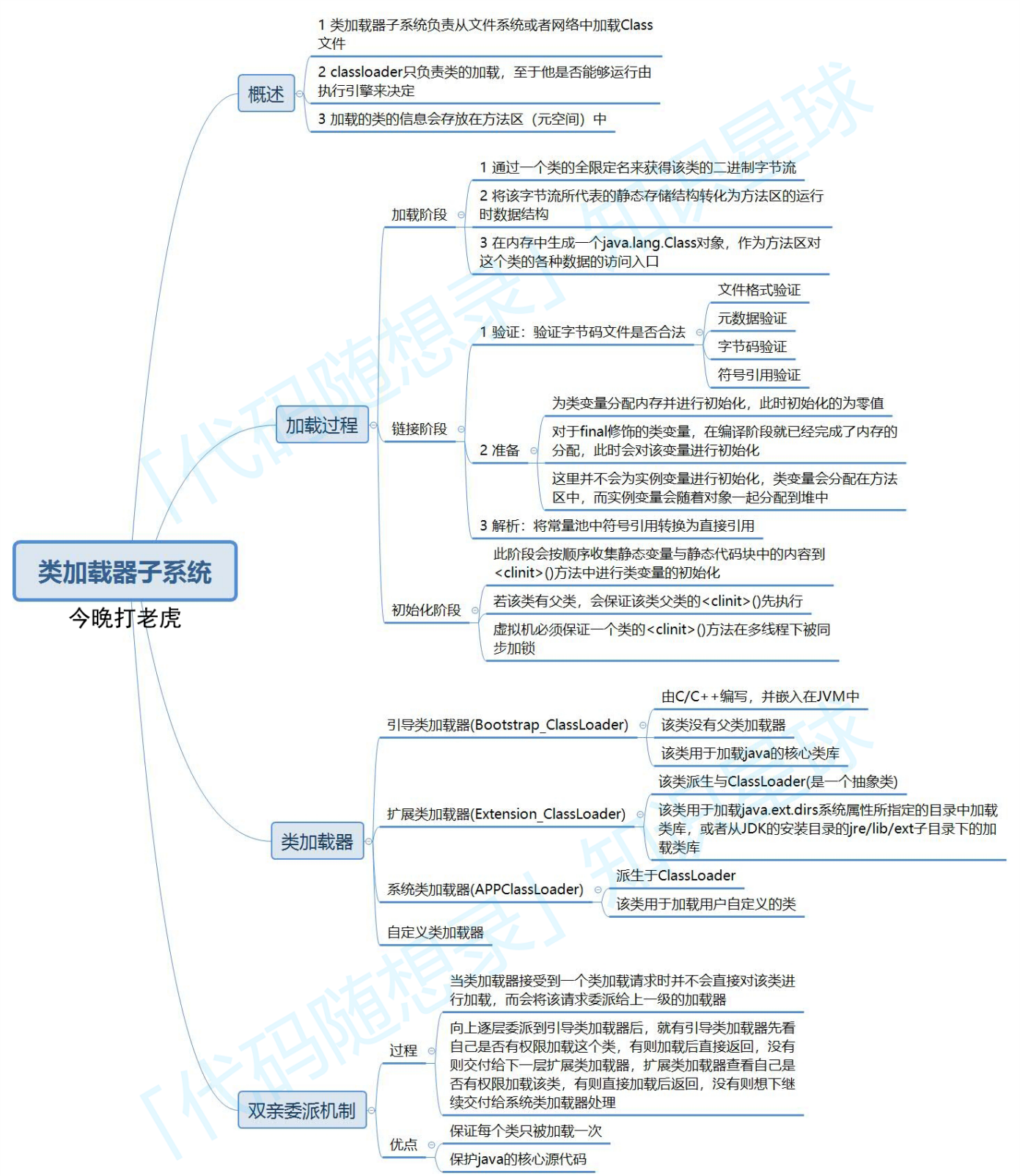
listener

jdbc

- 操作时候注意事务的几大特性ACID

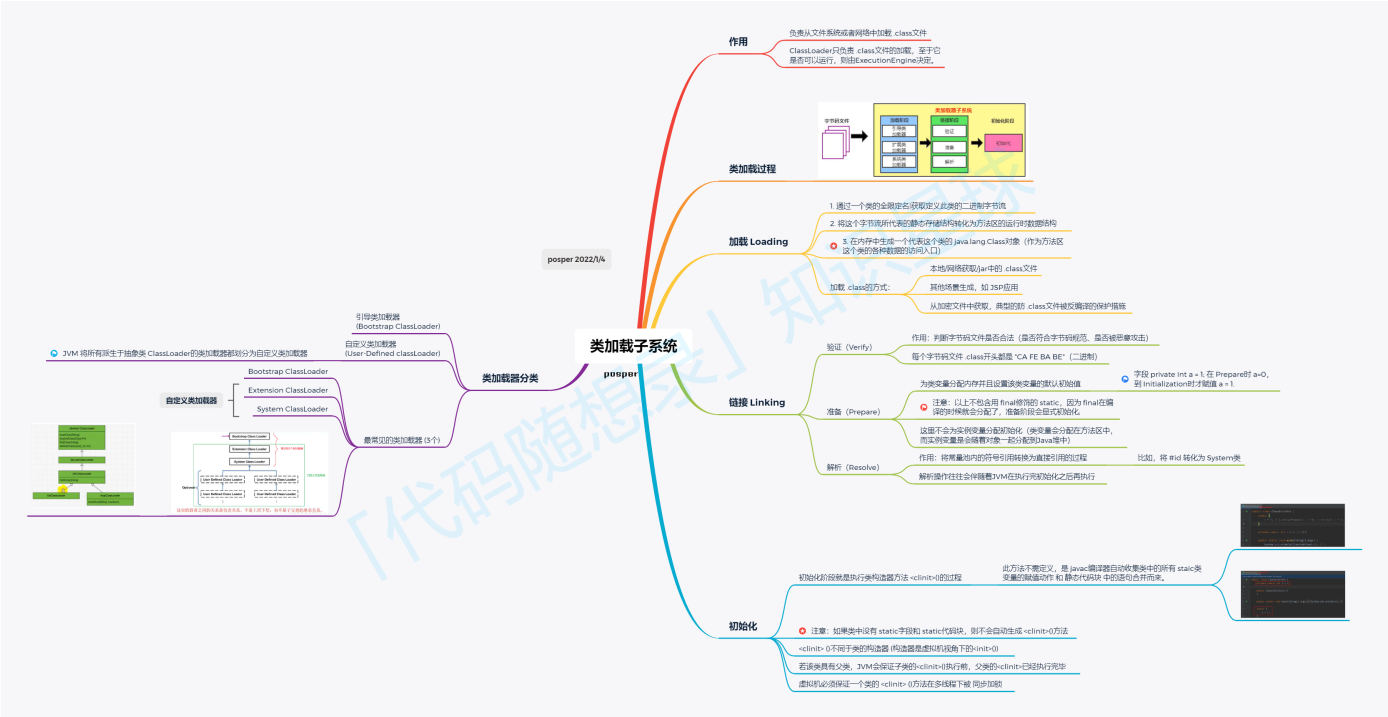
Java类加载子系统（1）

原图链接：「代码随想录」知识星球



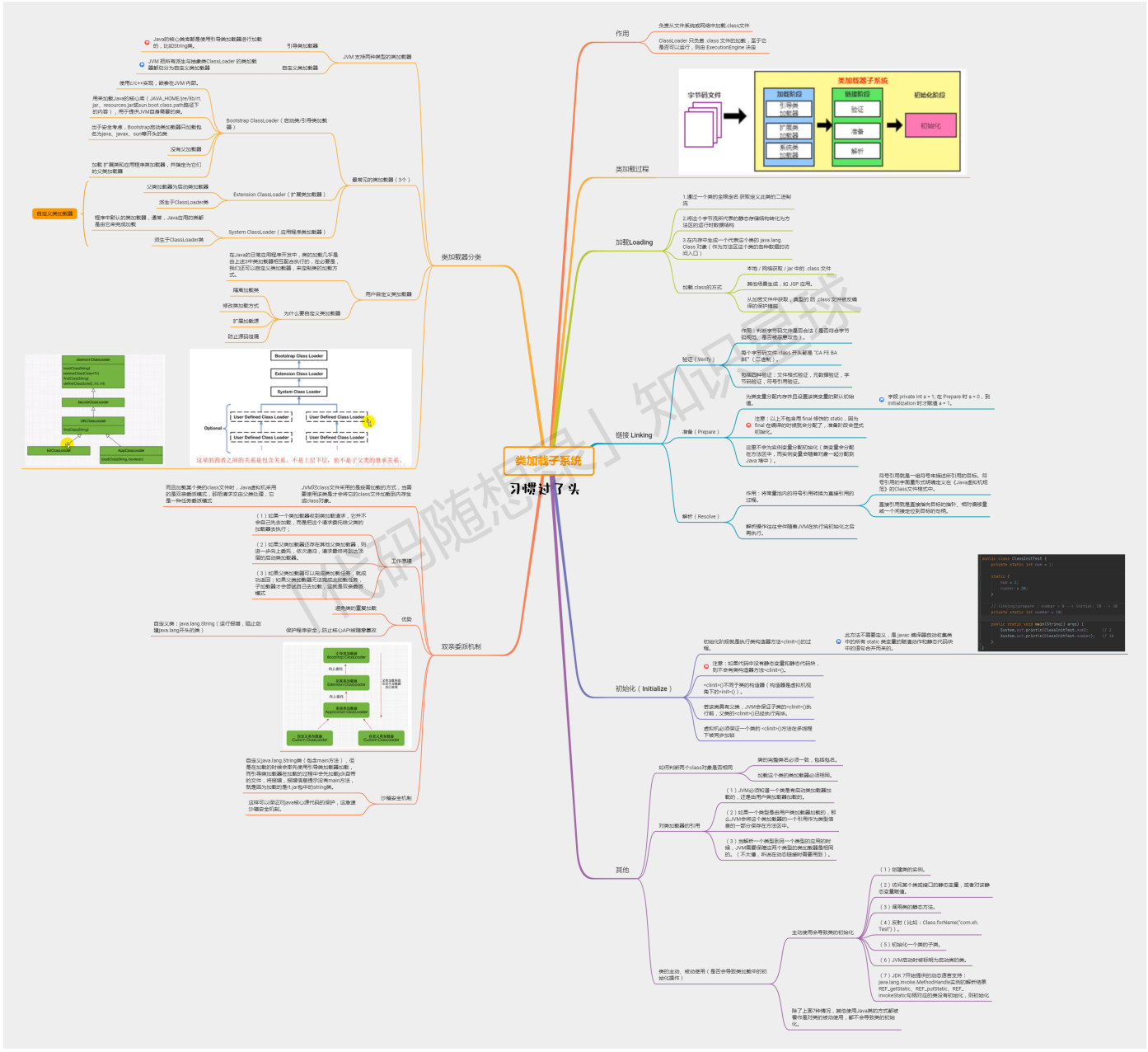
Java类加载子系统（2）

原图链接：「代码随想录」知识星球



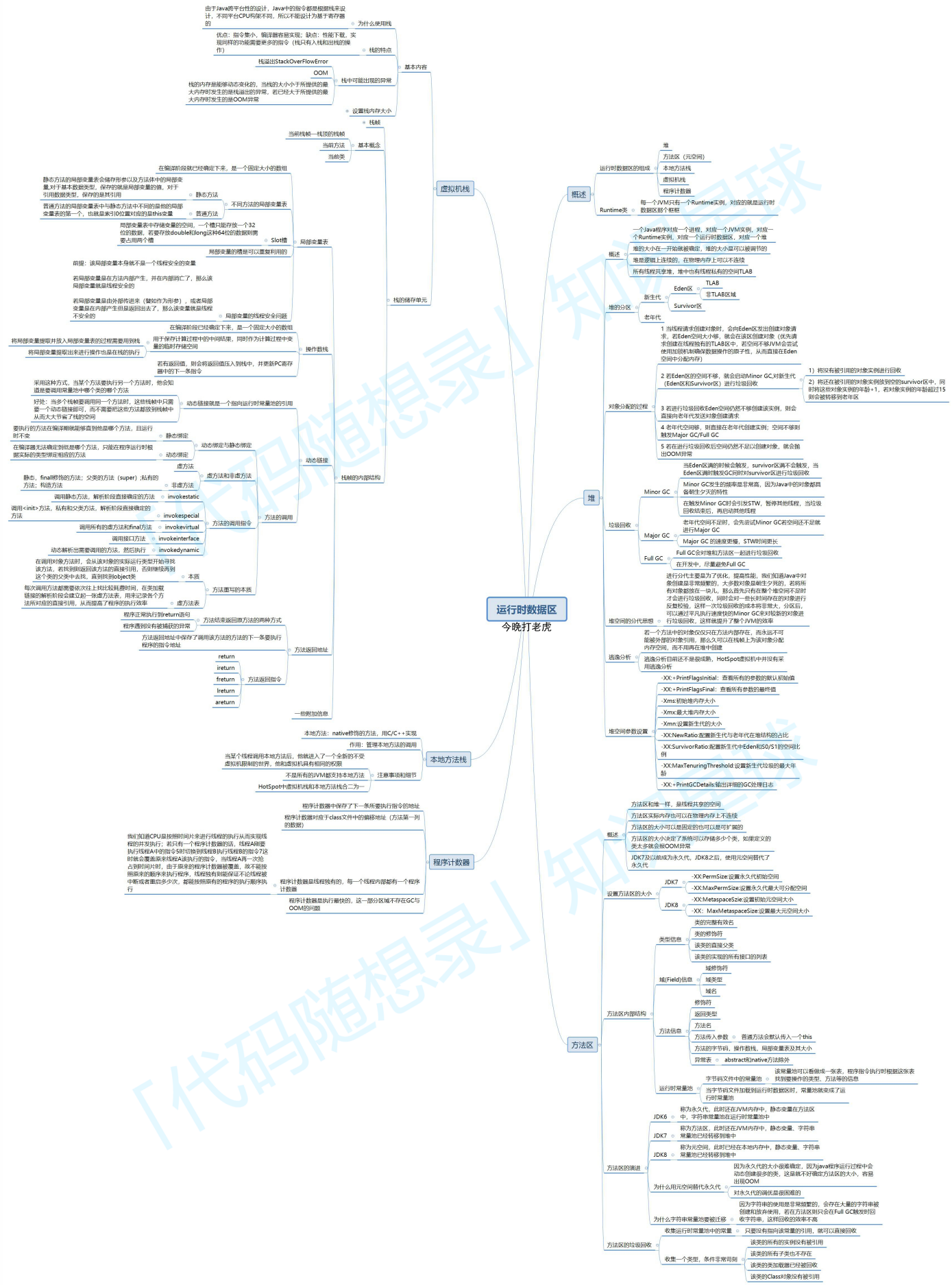
Java类加载子系统（3）

原图链接：「代码随想录」知识星球

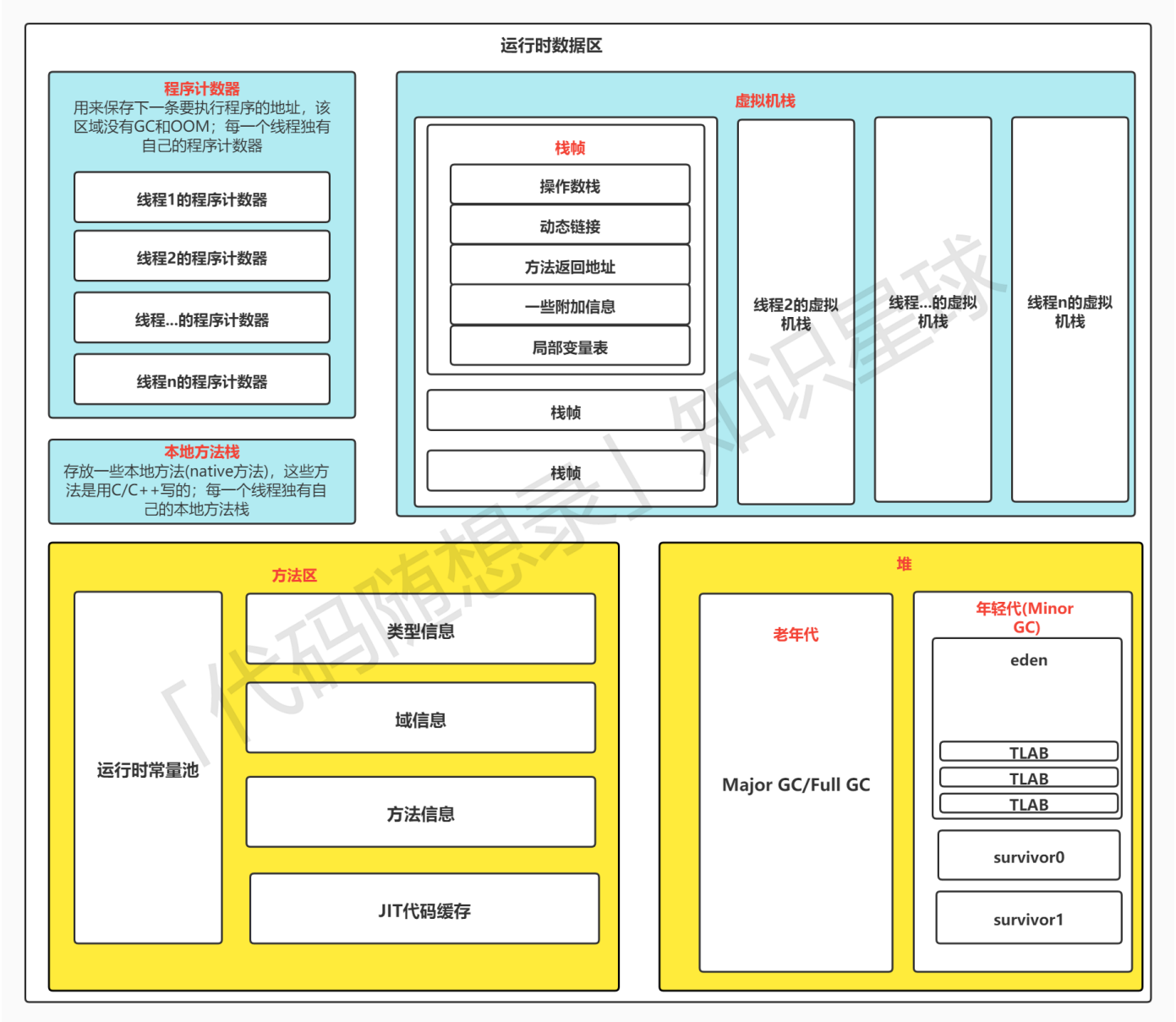


运行时数据区（1）

原图链接：「代码随想录」知识星球

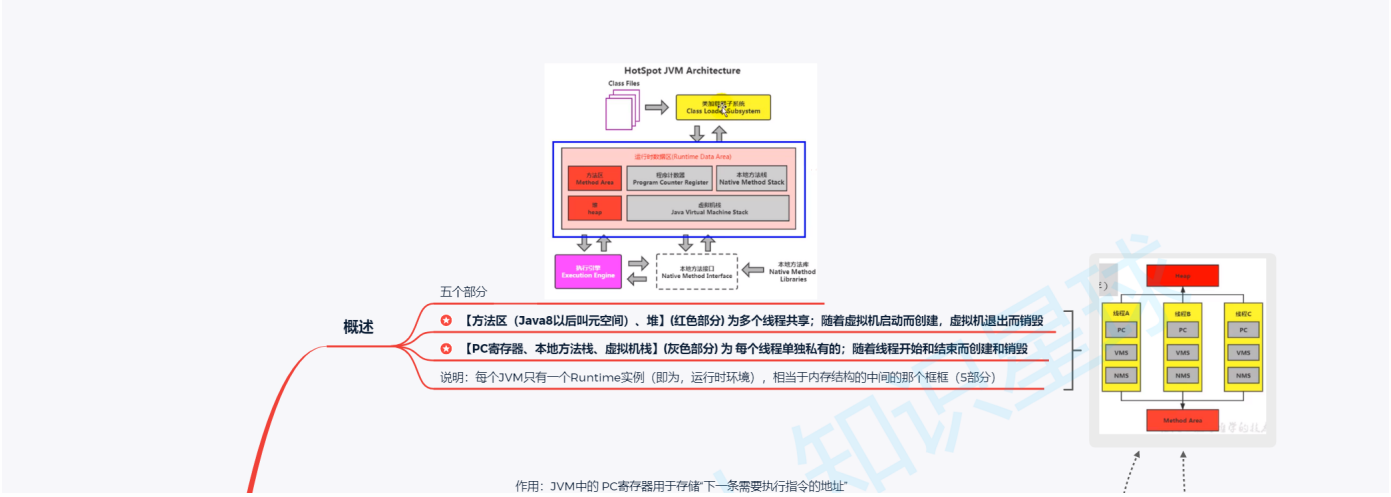


原图链接：「代码随想录」知识星球



运行时数据区（2）

原图链接：「代码随想录」知识星球



posper 2022/1/6

运行时数据区

PC寄存器（程序计数器）

（由执行引擎读取下一条指令代码）

每个线程都有自己的 PC寄存器（线程私有），记录该线程当前的执行位置

JVM中的 PC寄存器是对物理PC寄存器的抽象模拟，是软件层面上得

它是唯一一个在 JVM规范中没有规定任何 OutOfMemoryError（OOM）情况的区域（亦不用GC）

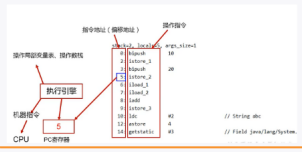
它是一块很小的内存空间，几乎可以忽略不记（只用放下条指令的地址）；也是运行速度最快的存储区域。

任何时间一个线程都只有一个方法在执行，也就是所谓的当前方法。程序计数器会存储当前线程正在执行的Java方法的JVM指令地址或者，如果是在执行native方法，则是未指定值（undefned）。

栗子

```
public static void main(String[] args) {
    int i = 10;
    int j = 20;
    int k = i + j;

    String s = "abc";
    System.out.println(i);
    System.out.println(j);
}
```



反编译命令：javap -v 文件名.class'

note：这里康师傅只是截取了上面代码的部分指令（不全）

两个常见问题

1、使用 PC寄存器存储字节码指令地址有什么用呢？（为什么使用PC寄存器记录当前线程的执行地址呢？）

因为"并发"执行时，CPU需要不停的切换各个线程。PC寄存器用于记录线程切换回来以后，从哪开始继续执行。

2、PC寄存器为什么会被设定为线程私有？

因为"并发"执行时，CPU需要不停的切换各个线程。每个线程执行的位置可能不同，所以需要为每个线程都分配一个 PC寄存器，使得各个线程之间可以进行独立计算，不会相互干扰。

虚拟机栈

概述

Java的指令都是根据"栈"（而不是寄存器）来设计的（不同平台 CPU架构不同，所以不能设计为基于寄存器的）

优点：跨平台，指令集小，编译器容易实现

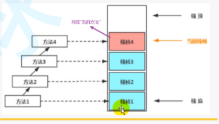
缺点：性能下降，实现同样的功能需要更多的指令

栈是运行时的单位，而堆是存储的单位

作用：主管 Java程序的运行，它保存方法的局部变量（八大基本类型、对象的引用）、部分结果，并参与方法的调用和返回

生命周期：生命周期和线程一致（每个线程在创建时都会创建一个虚拟机栈，是线程私有的）

内部存储的基本单位：虚拟机栈 内部保存的是一个个的栈帧(Stack Frame)【每个栈帧对应着一次 Java方法调用，栈顶的栈帧为"当前栈帧"，其对应的方法为"当前方法"】



特点

栈是一种快速有效的分配存储方式，访问速度仅次于 PC寄存器

【虚拟机栈】不存在垃圾回收问题

栈中可能出现的异常

如果采用固定大小的 Java虚拟机栈：当申请的栈空间超过规定的最大栈容量时，JVM会抛出 StackOverflowError异常（常出现于"无限递归"时）

如果Java虚拟机栈可以动态扩展：当无法申请到足够的栈空间时，JVM将会抛出一个 OutOfMemoryError异常

JVM规范允许Java栈的大小是动态的或者是固定不变的

设置栈内存大小

可以使用参数 -xss选项来设置线程的最大栈空间

以 IDEA为例：菜单栏 RUN -> Edit Configurations -> VM options

栈的存储单位

栈中存储什么?

JVM只对 Java栈进行两个操作，即 进栈和出栈

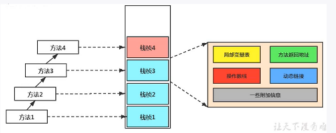
栈运行原理

在一条活动线程中，一个时间点上，只会有一个活动的栈帧（即，只有当前栈顶的"当前栈帧"是 当前有效的）

执行引擎运行的所有字节码指令只针对当前栈帧进行操作

如果在该方法中调用了其他方法，对应的新的栈帧会被创建出来，放在栈的顶端，成为新的当前帧

注意：不同线程中所包含的栈帧是不允许存在相互引用的，即不可能在一个栈帧之中引用另外一个线程的栈帧（即，只会套娃调用，不会交叉调用）【因为Java栈为 线程私有】



栈帧的内部结构（5部分）

局部变量表(Local variables)

操作数栈(operand stack)

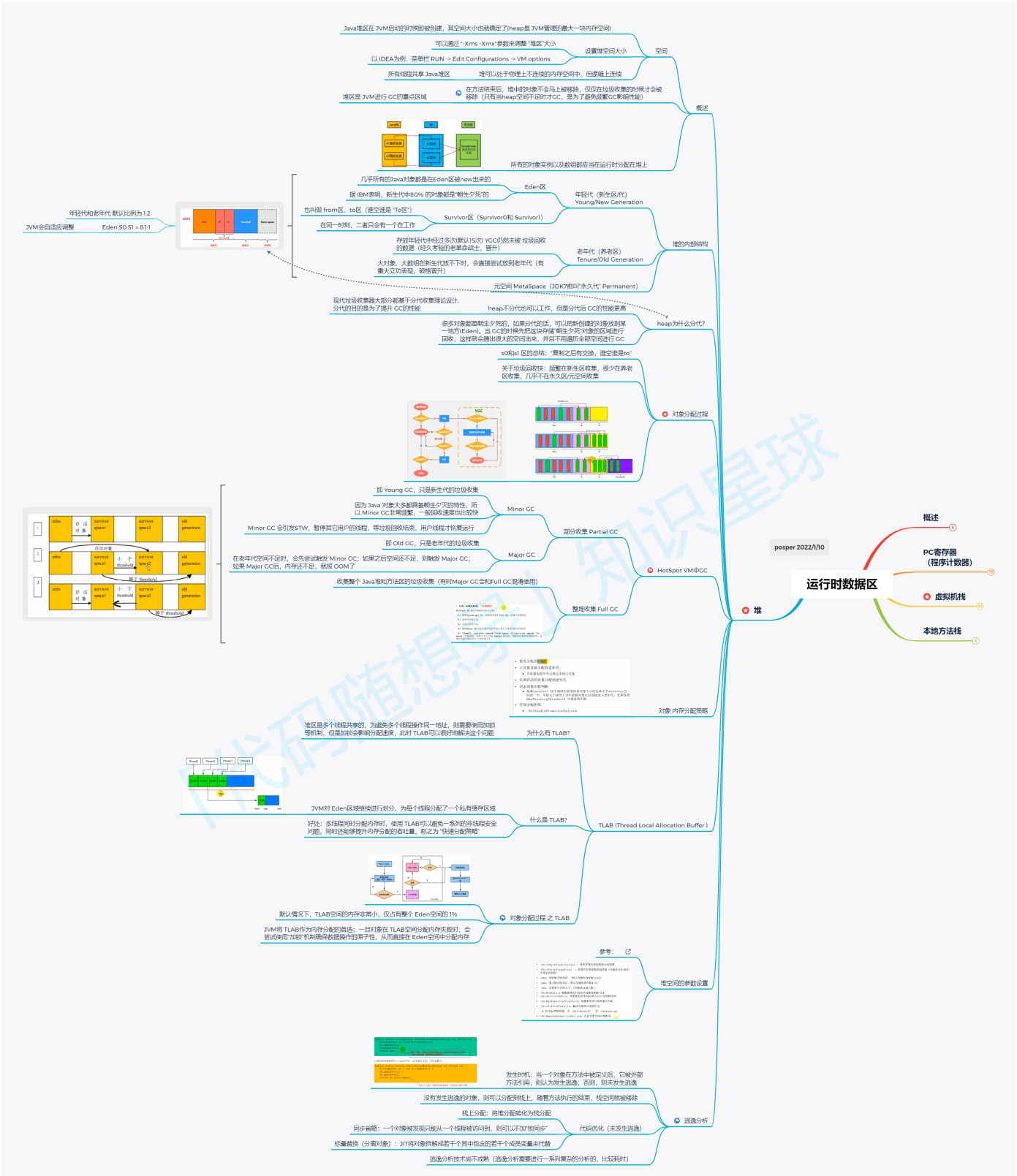
动态链接(Dynamic Linking)

方法返回地址（Return Address）

一些附加信息

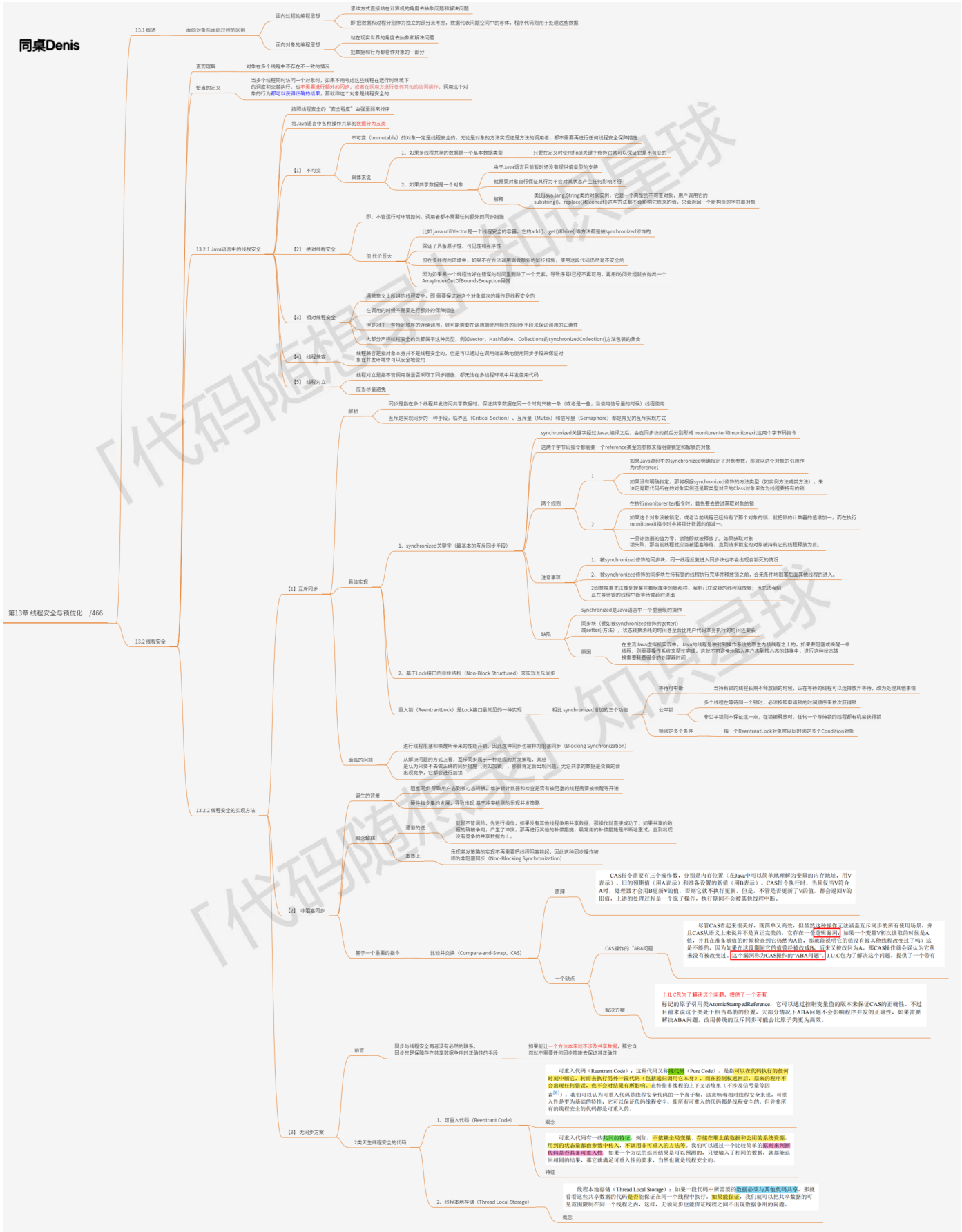
运行时数据区（3）

原图链接：「代码随想录」知识星球

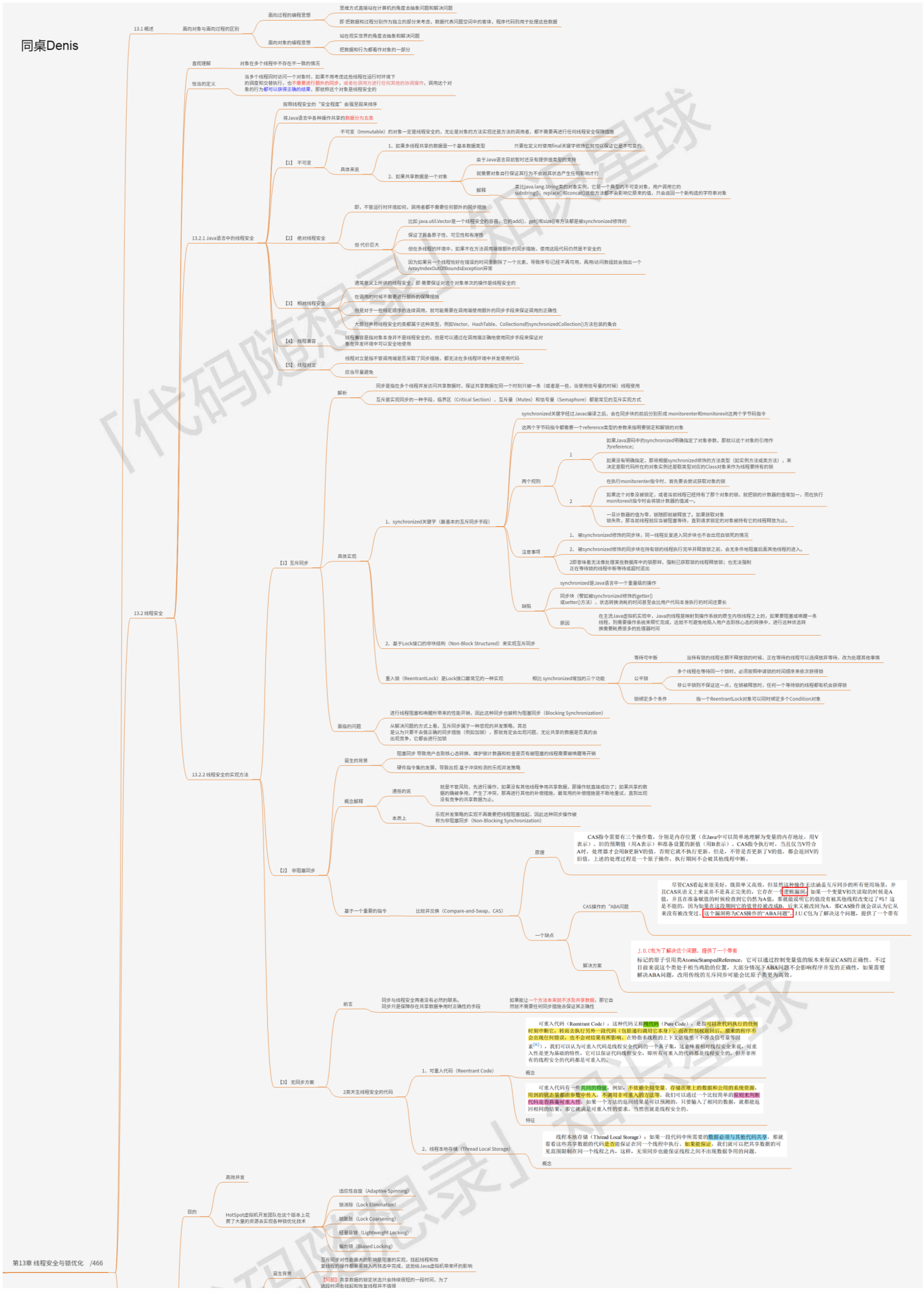


线程安全

原图链接：「代码随想录」知识星球

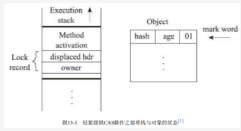
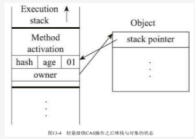
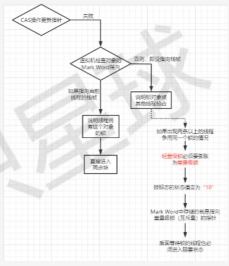
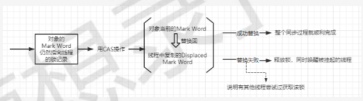
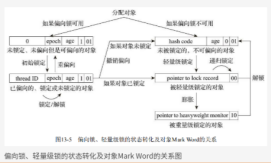


原图链接：「代码随想录」知识星球



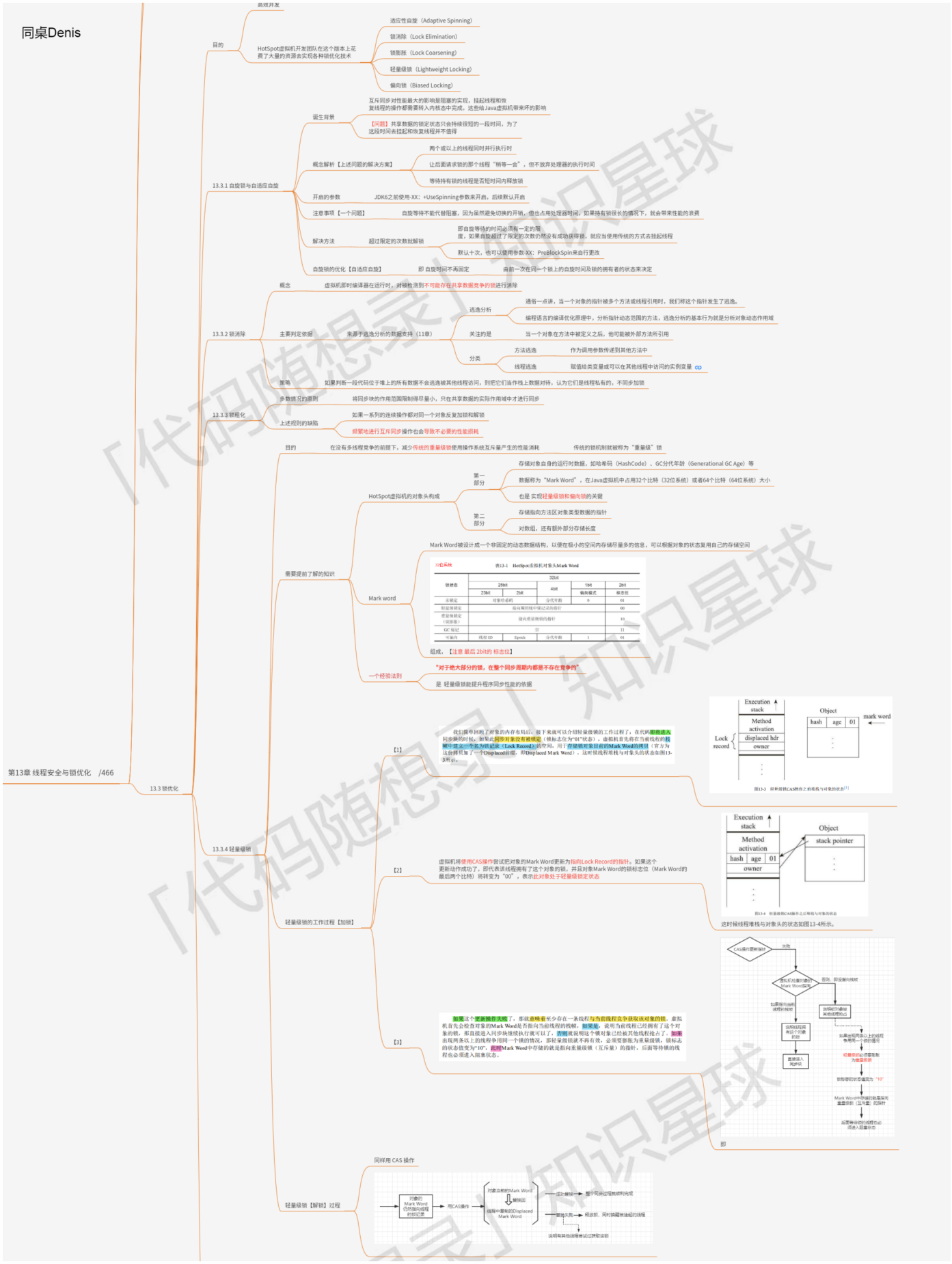
- 13.3 锁优化
  - 13.3.1 自旋锁与自适应自旋
    - 概念解析【上述问题的解决方案】
      - 两个或以上的线程同时并行执行时
      - 让后面请求锁的那个线程“稍等一会”，但不放弃处理器的执行时间
      - 等待持有锁的线程是否短时间内释放锁
    - 开启的参数
      - JDK6之前使用-XX：+UseSpinning参数来开启，后续默认开启
    - 注意事项【一个问题】
      - 自旋等待不能代替阻塞，因为虽然避免切换的开销，但也占用处理器时间，如果持有锁很长的情况下，就会带来性能的浪费
    - 解决方法
      - 超过限定的次数就解锁
        - 即自旋等待的时间必须有一定的限度，如果自旋超过了限定的次数仍然没有成功获得锁，就应当使用传统的方式去挂起线程
        - 默认十次，也可以使用参数-XX：PreBlockSpin来自行更改
    - 自旋锁的优化【自适应自旋】
      - 即 自旋时间不再固定
      - 由前一次在同一个锁上的自旋时间及锁的拥有者的状态来决定
  - 13.3.2 锁消除
    - 概念
      - 虚拟机即时编译器在运行时，对被检测到不可能存在共享数据竞争的锁进行消除
    - 主要判定依据
      - 来源于逃逸分析的数据支持（11章）
        - 逃逸分析
          - 通俗一点讲，当一个对象的指针被多个方法或线程引用时，我们称这个指针发生了逃逸。
          - 编程语言的编译优化原理中，分析指针动态范围的方法，逃逸分析的基本行为就是分析对象动态作用域
        - 关注的是
          - 当一个对象在方法中被定义之后，他可能被外部方法所引用
        - 分类
          - 方法逃逸
            - 作为调用参数传递到其他方法中
          - 线程逃逸
            - 就值给类变量或可以在其他线程中访问的实例变量
    - 策略
      - 如果判断一段代码位于堆上的所有数据不会逃逸被其他线程访问，则把它们当作栈上数据对待，认为它们是线程私有的，不同步加锁
  - 13.3.3 锁粗化
    - 多数情况的原则
      - 将同步块的作用范围限制得尽量小，只在共享数据的实际作用域中才进行同步
    - 上述规则的缺陷
      - 如果一系列的连续操作都对同一个对象反复加锁和解锁
      - 频繁地进行互斥同步操作也会导致不必要的性能损耗
  - 13.3.4 轻量级锁
    - 目的
      - 在没有多线程竞争的前提下，减少传统的重量级锁使用操作系统互斥量产生的性能消耗
      - 传统的锁机制就被称为“重量级”锁
    - 需要提前了解的知识
      - HotSpot虚拟机的对象头构成
        - 第一部分
          - 存储对象自身的运行时数据，如哈希码（HashCode）、GC分代年龄（Generational GC Age）等
          - 数据称为“Mark Word”，在Java虚拟机中占用32个比特（32位系统）或者64个比特（64位系统）大小
          - 也是 实现轻量级锁和偏向锁的关键
        - 第二部分
          - 存储指向方法区对象类型数据的指针
          - 对数组，还有额外部分存储长度
      - Mark word
        - Mark Word被设计成一个非固定的动态数据结构，以便在极小的空间内存储尽量多的信息，可以根据对象的状态复用自己的存储空间
        - 32bit系统 表13-1 HotSpot虚拟机对象头Mark Word

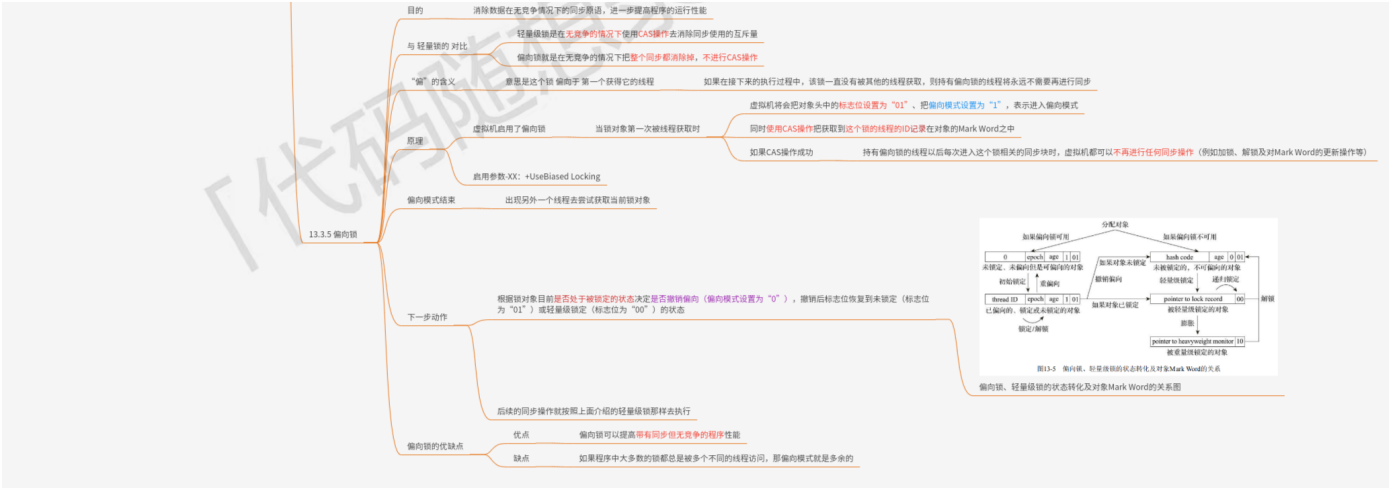
锁状态	32bit				
	25bit		4bit	1bit	2bit
	23bit	2bit		偏向模式	锁标志位
未锁定	对象哈希码		分代年龄	0	01
轻量级锁定	指向栈中锁记录的指针				00
重量级锁定（膨胀锁）	指向重量级锁的指针				10
GC 标记	空				11
可偏向	线程 ID	Epoch	分代年龄	1	01

        - 组成，【注意 最后 2bit的 标志位】
      - 一个经验法则
        - “对于绝大部分的锁，在整个同步周期内都是不存在竞争的”
        - 是 轻量级锁能提升程序同步性能的依据
    - 轻量级锁的工作过程【加锁】
      - 【1】
        - 我们简单回顾了对象的内存布局后，接下来就可以介绍轻量级锁的工作过程了，在代码即将进入同步块的时候，如果此同步对象没有被锁定（锁标志位为“01”状态），虚拟机首先将在当前线程的栈帧中建立一个名为锁记录（Lock Record）的空间，用于存储锁对象目前的Mark Word的拷贝（官方为这份拷贝加了一个Displaced前缀，即Displaced Mark Word），这时候线程堆栈与对象头的状态如图13-3所示。
        - 
        - 图13-3 轻量级锁CAS操作之前堆栈与对象的状态[1]
      - 【2】
        - 虚拟机将使用CAS操作尝试把对象的Mark Word更新为指向Lock Record的指针。如果这个更新动作成功了，那代表该线程拥有了这个对象的锁，并且对象Mark Word的锁标志位（Mark Word的最后两个比特）将转变为“00”，表示此对象处于轻量级锁定状态
        - 
        - 图13-4 轻量级锁CAS操作之后堆栈与对象的状态
        - 这时候线程堆栈与对象头的状态如图13-4所示。
      - 【3】
        - 如果这个更新操作失败了，那就意味着至少存在一条线程与当前线程竞争获取该对象的锁。虚拟机首先会检查对象的Mark Word是否指向当前线程的栈帧，如果是，说明当前线程已经拥有了这个对象的锁，那直接进入同步块继续执行就可以了，否则就说明这个锁对象已经被其他线程抢占了。如果出现两条以上的线程争用同一个锁的情况，那轻量级锁就不再有效，必须要膨胀为重量级锁，锁标志的状态值变为“10”，此时Mark Word中存储的就是指向重量级锁（互斥量）的指针，后面等待锁的线程也必须进入阻塞状态。
        - 
        -
        - 即
    - 轻量级锁【解锁】过程
      - 同样用 CAS 操作
      - 
      -
  - 13.3.5 偏向锁
    - 目的
      - 消除数据在无竞争情况下的同步原语，进一步提高程序的运行性能
    - 与 轻量级锁的 对比
      - 轻量级锁是在无竞争的情况下使用CAS操作去消除同步使用的互斥量
      - 偏向锁就是在无竞争的情况下把整个同步都消除掉，不进行CAS操作
    - “偏”的含义
      - 意思是这个锁 偏向于 第一个获得它的线程
      - 如果在接下来的执行过程中，该锁一直没有被其他的线程获取，则持有偏向锁的线程将永远不需要再进行同步
    - 原理
      - 虚拟机启用了偏向锁
        - 当锁对象第一次被线程获取时
          - 虚拟机将会把对象头中的标志位设置为“01”，把偏向模式设置为“1”，表示进入偏向模式
          - 同时使用CAS操作把获取到这个锁的线程的ID记录在对象的Mark Word之中
          - 如果CAS操作成功
            - 持有偏向锁的线程以后每次进入这个锁相关的同步块时，虚拟机都可以不再进行任何同步操作（例如加锁、解锁及对Mark Word的更新操作等）
      - 启用参数-XX：+UseBiased Locking
    - 偏向模式结束
      - 出现另外一个线程去尝试获取当前锁对象
    - 下一步动作
      - 根据锁对象目前是否处于被锁定的状态决定是否撤销偏向（偏向模式设置为“0”），撤销后标志位恢复到未锁定（标志位为“01”）或轻量级锁定（标志位为“00”）的状态
      - 后续的同步操作就按照上面介绍的轻量级锁那样去执行
      - 
      -
      - 图13-5 偏向锁、轻量级锁的状态转化及对象Mark Word的关系
      - 偏向锁、轻量级锁的状态转化及对象Mark Word的关系图
    - 偏向锁的优缺点
      - 优点
        - 偏向锁可以提高带有同步但无竞争的程序性能
      - 缺点
        - 如果程序中大多数的锁都总是被多个不同的线程访问，那偏向模式就是多余的

锁优化

原图链接：「代码随想录」知识星球





Synchronized关键字

原图链接：「代码随想录」知识星球

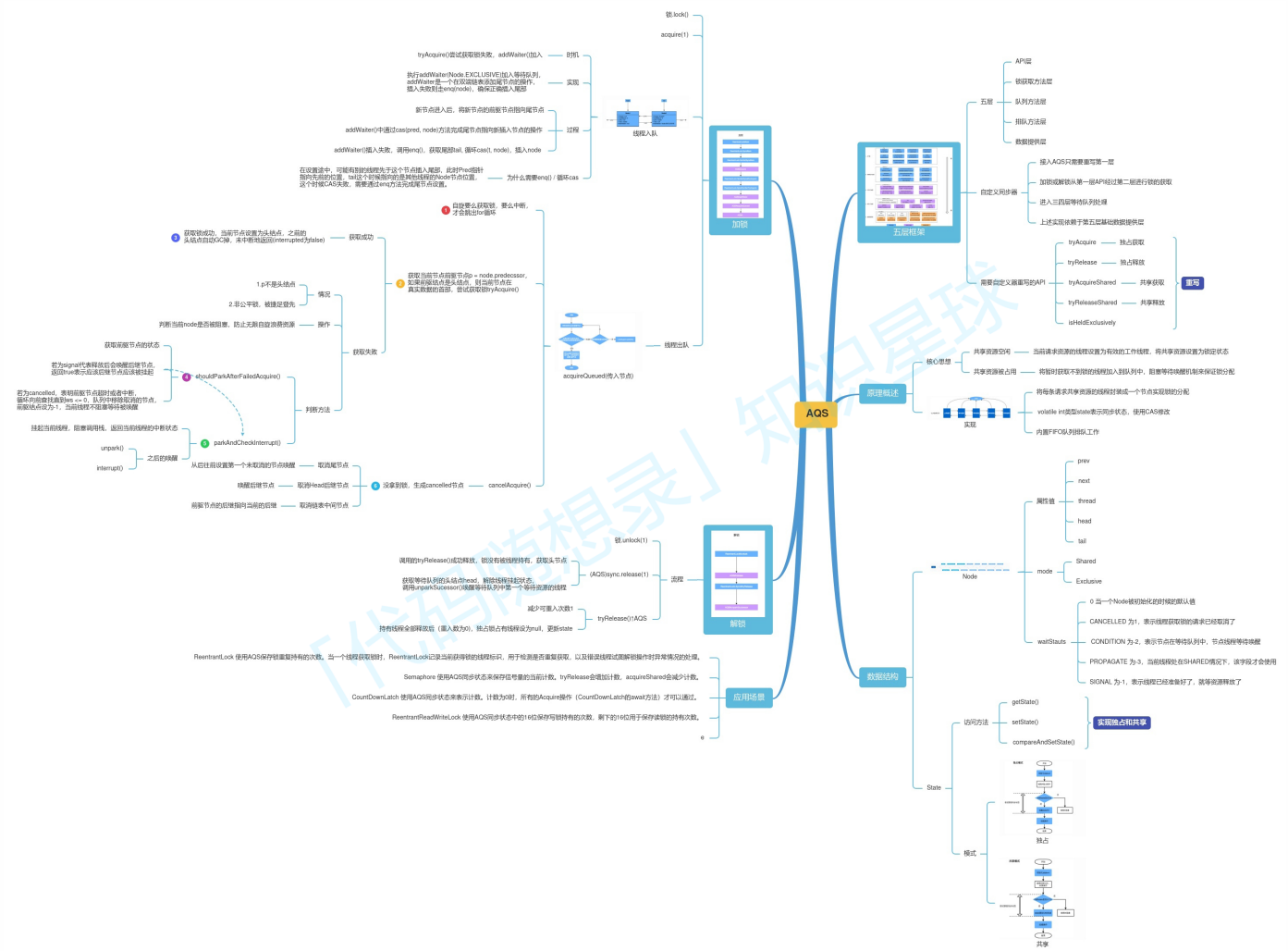
Java 并发

- ThreadLocal
  - 简介
    - 解决每一个线程都有自己专属的本地变量，让每个线程绑定自己的值，可以将ThreadLocal 类 比喻成存放数据的盒子，盒子中可以存储每个线程的私有数据
    - 创建了一个 ThreadLocal 变量，那么访问这个变量的每个线程都会有这个变量的本地副本，每个线程可以调 get() 和 set() 方法来获取默认值或者将其值更换为当前线程所存的副本的值，从而避免了线程安全问题
  - 举例子
  - 原理
- Synchronized 关键字
  - 对 Synchronized 关键字的了解
    - synchronized 关键字解决的是多个线程之间访问资源的同步性
    - synchronized关键字可以保证被它修饰的方法或者代码块在任意时刻只能有一个线程执行，在 Java 1.5 之前中，synchronized 属于重量级锁，效率低下
  - synchronized 的使用方式
    - 修饰实例方法：作用于当前对象实例 this 加锁，进入同步代码前需要先获得该对象实例的锁
      - synchronized void method() { //业务代码 }
    - 修饰静态方法：也就是给当前的类的Class 对象加锁，会作用于类的所有对象实例，进入同步代码块之前要获得当前 class 的锁。因为静态成员不属于任何一个实例对象，而是类的成员。
      - 所以如果一个线程 A 调用 一个实例对象的非静态synchronized 方法，而线程 B 需要调用这个实例对象所属类的静态synchronized 方法，并不会发生互斥现象。因为访问静态 synchronized 方法占用的锁是当前类的锁，而访问 非静态synchronized 方法占用的锁是当前实例对象锁
      - synchronized static void method() { //业务代码 }
      - 总结：synchronized 关键字加到 static 静态方法和 synchronized(class) 代码块上都是给 Class 类上锁。synchronized 关键字加到实例方法上是给对象实例上锁。尽量不要使用 synchronized(String a) 因为 JVM 中，字符串常量池具有缓存功能！
    - 修饰代码块：指定加锁对象，对给定对象/类加锁。synchronized(this|Object) 表示进入同步代码块前要获得给定对象的锁。 synchronized(类.class) 表示进入同步代码块之前要获得当前 class 的锁
      - public void test(){ SynchronizedTest test = new SynchronizedTest(); synchronized (test){ System.out.println("hello world"); } }
  - 一个不会的面试题：手写单例模式！ 解释一下双重检验锁方式实现单例模式的原理"
  - 构造方法不可以使用 synchronized 关键字修饰，因为构造方法本身就是线程安全的。
  - 底层原理：对象监视器monitor 的获取
    - Monitor
      - 它内置在每一个对象中，任何一个对象都有一个monitor与之关联，synchronized在JVM里的实现就是基于进入和退出monitor来实现的
      - 简单来说可以想象成为一个只能容纳一个对象的房间，然后想要获取对象锁的线程就是想要进入这个房间的客人。
    - 修饰同步块
      - monitorenter 和 monitorexit 两个字节码指令
        - synchronized 被编译后生成 monitorenter 与 monitorexit 指令，其中 monitorenter 指令指向同步代码块的开始位置，monitorexit 指令则指明同步代码块的结束位置。在 JVM 中实现进入和退出monitor
        - monitorenter，如果当前monitor的进入数为0时，线程就会进入monitor，并且把进入数+1，那么该线程就是monitor的拥有者(owner)。如果该线程已经是monitor的拥有者，又重新进入，就会把进入数再次+1，也就是可重入的。
        - 执行monitorexit 的线程必须是monitor 的拥有者，执行 monitorexit 指令后，monitor 的进入数减一，如果减一后进入 数为0 表明锁被释放。其他被阻塞的线程就可以尝试去获取monitor 的所有权 。
    - 修饰 方法
      - 一个标志位ACC_SYNCHRONIZED
        - 作用就是一旦执行到这个方法时，就会先判断是否有标志位，如果有这个标志位，就会先尝试获取monitor，获取成功才能执行方法，方法执行完后再释放monitor。在方法执行期间，其他线程都无法获取同一个monitor。
  - 性能问题
    - monitor 的下层是依靠操作系统的 Mutex Lock 来实现的，Java 线程事实上是对操作系统线程的映射，所以每次进行挂起或者唤起一个线程都要切换到操作系统的内核态，比较消耗性能
  - JDK1.6 之后的优化
    - 自旋锁
      - 线程如果没有争取到 monitor 时，就会进入阻塞状态，当持有锁的线程释放了锁之后，才可以再去竞争。这样的话，会浪费大量的性能在阻塞和唤醒的切换上（如果线程占用锁的时间很短，那么切换的时间可能会超过线程执行任务的时间）。
      - 为了避免阻塞和唤醒的切换，在没有获得锁的时候就不进入阻塞，而是不断的循环检测锁是否被释放，这就是自旋。
      - 但是线程一直在循环检测锁的状态，就会占用 CPU资源，如果锁被占用的时间较长，那么自旋的次数就会变多，占用 CPU 时间变长，导致性能变差。
        - 通过参数 -XX: PreBlockSpin 来设置只需自旋的次数，自旋一定的次数后就挂起。
    - 自适应自旋锁
      - 自旋次数不固定，而是由前一次在同一个锁上的自旋时间以及锁的拥有者的状态来决定。
      - 如果这一次自旋成功了，很可能下一次也能够成功，于是允许自旋的次数就会更多，反之，自旋的次数就减少。
    - 锁消除
      - 在 JVM 编译时，通过对运行时上下文的扫描，去除不可能存在共享资源竞争的锁，节约无意义的请求时间。比如说：StringBuffer 的 append() 方法
        - public synchronized StringBuffer append(String str) { toStringCache = null; super.append(str); return this; }
        - 在实例方法中使用StringBuffer作为局部变量使用 append 方法，此时StringBuffer 不可能存在共享资源竞争的，因此会自动将锁消除
          - public String add(String s1, String s2) { //sb属于不可能共享的资源，JVM会自动消除内部的锁 StringBuffer sb = new StringBuffer(); sb.append(s1).append(s2); return sb.toString(); }
    - 锁粗化
      - 将多个连续加锁，解锁的操作链接在一起，扩展成为一个范围更大的锁
    - 在用户空间内去完成锁的获取与释放，避免进入重量级锁的状态，那么将会调用内核空间，产生较大的开销
    - 轻量级锁
      - 无锁
        - 无需对资源进行任何同步保护，可以给各个线程随意调用
        - 资源会被竞争。但是不使用操作系统同步原语对共享资源进行锁定，而是通过一些其他的机制来控制资源，比如 CAS
      - 偏向锁
        - 为单线程情况下设计，在大多数情况下，锁不仅不存在多线程竞争，而且总是由同一线程多次获得。也就是假设上是多线程的场景，实际上是单线程。
        - Mark Word中，当锁的标志位 为 01，那么判断倒数第三个 bit 是否为 1，为 1 则代表当前对象的锁状态为偏向锁，于是再去读 Mark Word 的前 23 个 bit，也就是线程 ID，通过这个 ID 来判断是否是 同一个线程再次进入，如果是，则不再进行同步操作。省去了大量的锁申请的操作，提高性能
        - 当偏向多个线程竞争时，锁的标志位为00。
        - 线程想要获得某个对象的锁，假如看到锁的标志位为 00，那么就知道它是轻量级锁，线程会在自己的虚拟机栈中开辟一块被称为 Lock Record 的空间，用于存放 对象头 的Mark Word 的副本以及Owner 指针。
        - 线程会将复制来对象的 Mark Word 到虚拟机栈中的 Lock Record 中，并且将 Lock Record 中的Owner 指针指向该对象锁。
        - 对象的 Mark Word 中的前 30 bit 将生成一个指针，指向持有该对象锁的线程的虚拟机栈中的 Lock Record。
        - 一旦有自旋等待的线程数超过 1 个，那么轻量级锁将会升级
        - 其他的线程想要获取这个对象，将会自旋等待，自旋等待的过程完全在用户空间解决，不需要进行系统中断和现场恢复
      - 重量级锁
        - 锁竞争激烈，让线程进入阻塞状态，进入阻塞队列，以减少 cpu 消耗。
    - 偏向锁
      - 在 Java 中每个对象都拥有一把锁，这把锁存在于对象头中，记录了当前对象被哪一个线程所占用，对象头中的 Mark Word 就是这个用处
      - 锁主要存在四种状态，依次是：无锁状态、偏向锁状态、轻量级锁状态、重量级锁状态，他们会随着竞争的激烈而逐渐升级。注意锁可以升级不可降级，这种策略是为了提高获得锁和释放锁的效率。
  - synchronized 和 ReentrantLock 的区别
    - 1.二者都是可重入锁
      - “可重入锁”指的是自己可以再次获取自己的内部锁。比如一个线程获得了某个对象的锁，此时这个对象的锁还没有释放，当其再次想要获取这个对象的锁的时候还是可以获取的，如果不可重入的话，就会造成死锁。同一个线程每次获取锁，锁的计数器都自增 1，所以要等到锁的计数器下降为 0 时才能释放锁
    - 2. synchronized 依赖于 JVM 而 ReentrantLock 依赖于 API
      - synchronized 是依赖于 JVM 实现的，前面我们也讲到了 虚拟机团队在 JDK1.6 为 synchronized 关键字进行了很多优化，但是这些优化都是在虚拟机的层面实现的，并没有直接暴露给我们。ReentrantLock 是 JDK 层面实现的（也就是 API 层面，需要 lock() 和 unlock() 方法配合 try/finally 语句来实现），所以我们可以通过查看它的源代码，来看它是如何实现的。
    - 3. ReentrantLock 比 synchronized 增加了一些高级功能
      - 等待可中断：ReentrantLock提供了一种能够中断等待锁的线程的机制，通过 lock.lockInterruptibly() 来实现这个机制。也就是说正在等待的线程可以选择放弃等待，改为处理其他事情。
      - 可实现公平锁：ReentrantLock可以指定是公平锁还是非公平锁。而synchronized只能是非公平锁。所谓的公平锁就是先等待的线程先获得锁。ReentrantLock默认情况是非公平的，可以通过 ReentrantLock类的ReentrantLock(boolean fair)构造方法来制定是否是公平的。
      - 可实现选择性通知（锁可以绑定多个条件）：synchronized关键字与wait()和notify()/notifyAll()方法相结合可以实现等待/通知机制。ReentrantLock类当然也可以实现。但是需要借助于Condition接口与newCondition()方法。
- 锁机制
- volatile 关键字

「代码随想录」知识星球

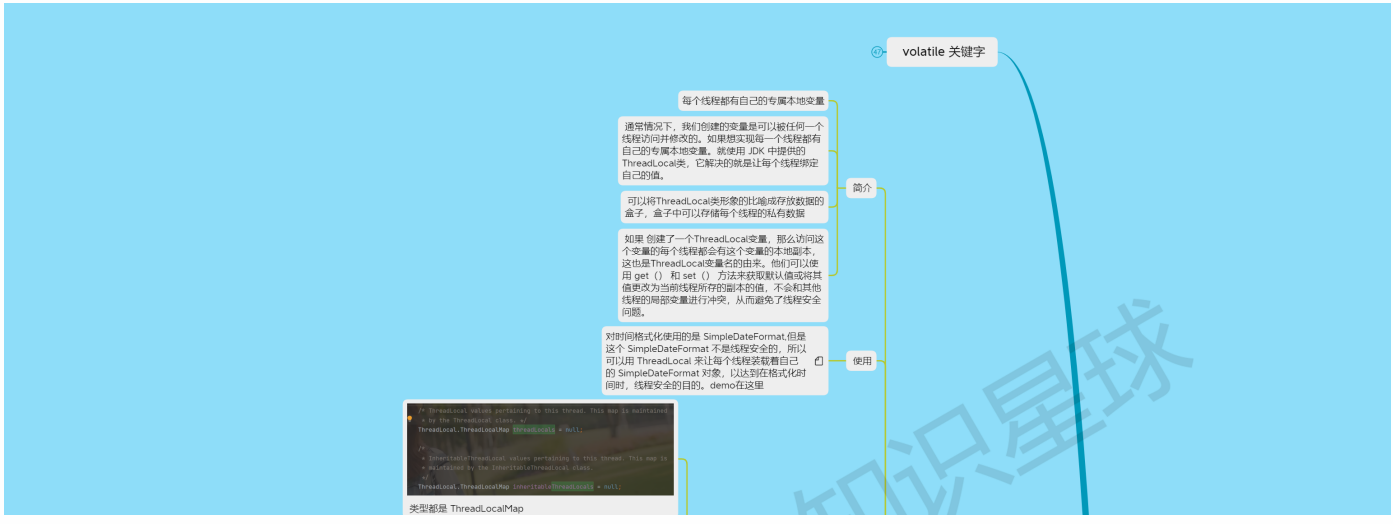
AQS

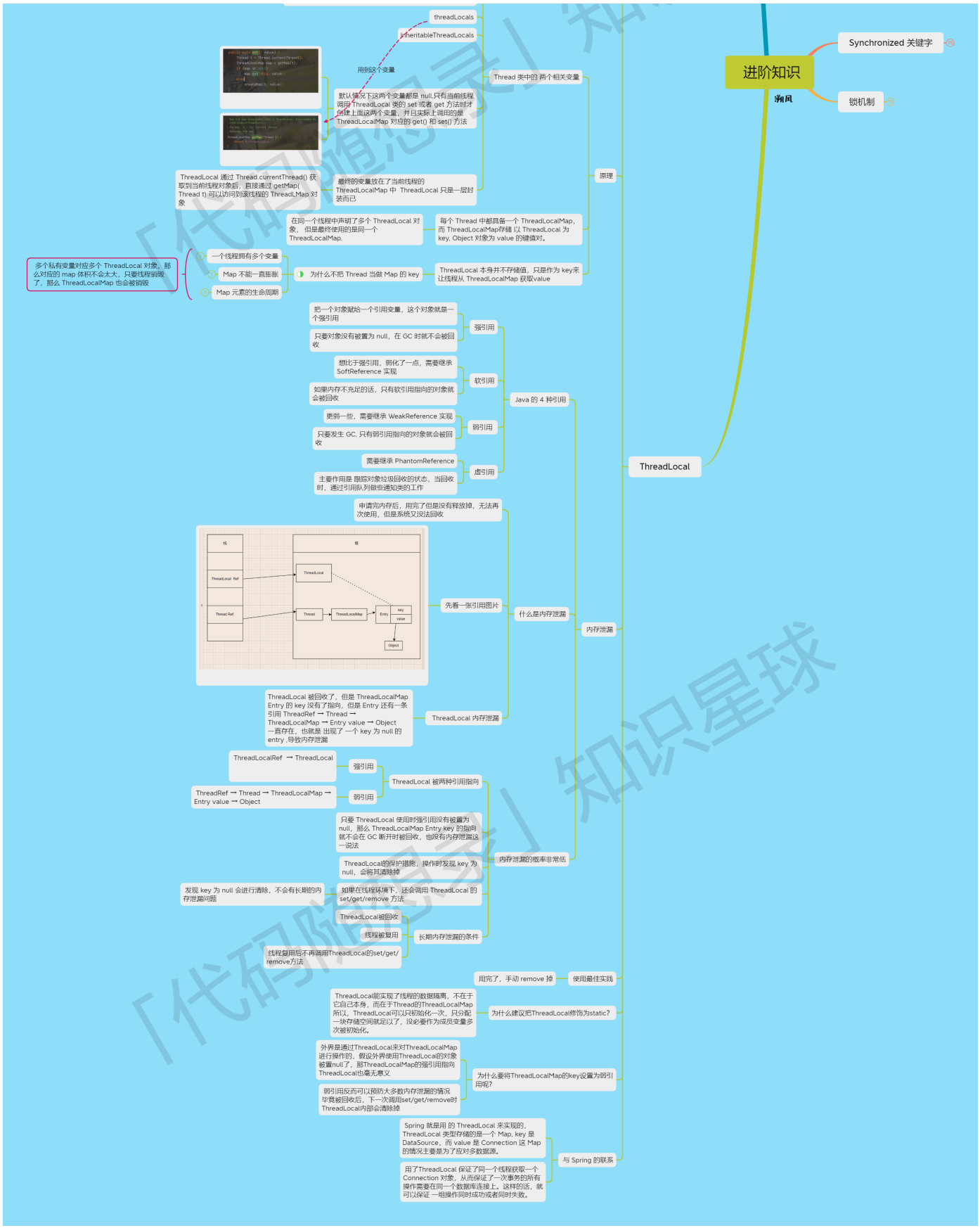
原图链接：「代码随想录」知识星球



ThreadLocal

原图链接：「代码随想录」知识星球



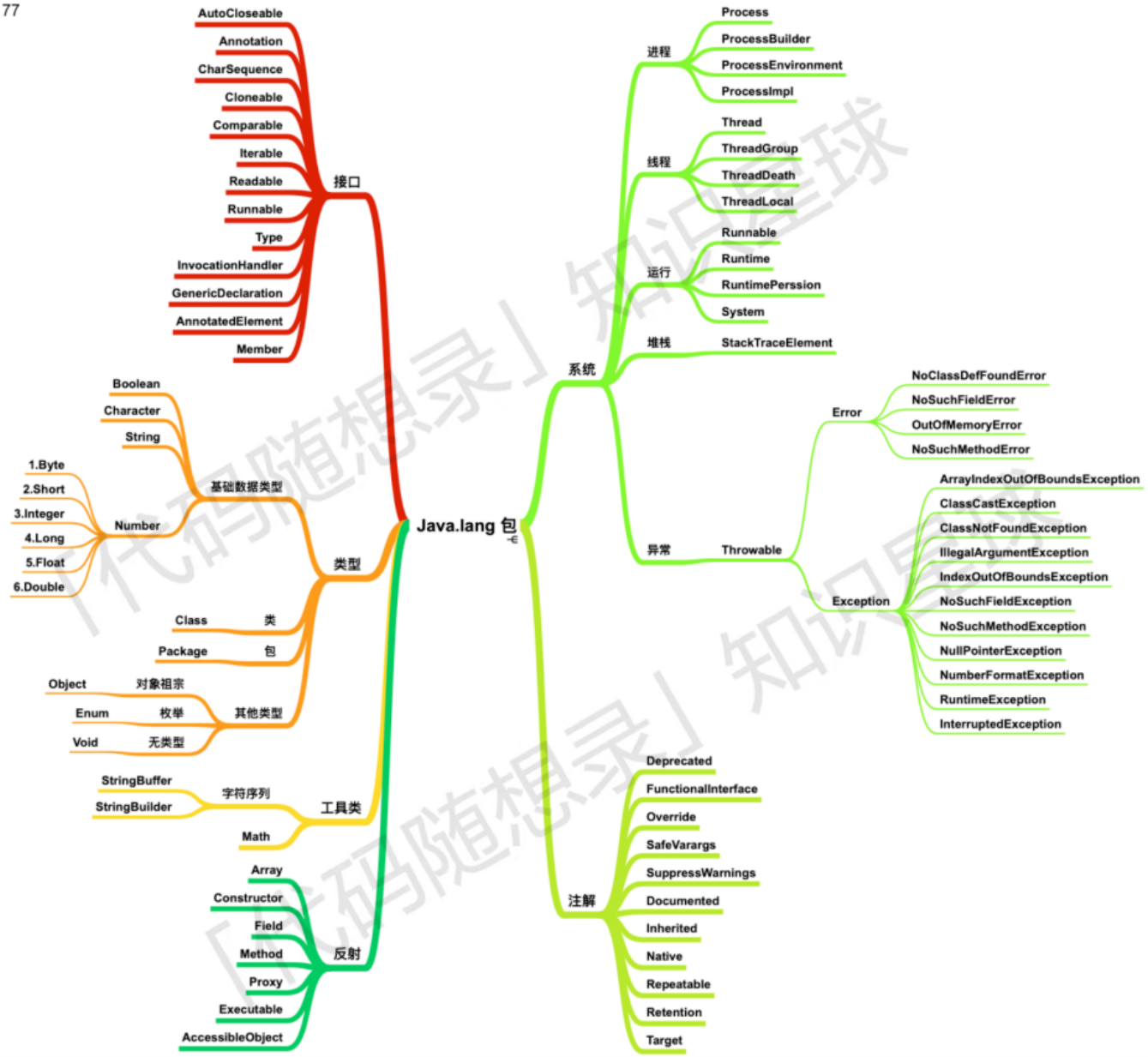

进阶知识
Synchronized 关键字
锁机制
ThreadLocal
原理
Thread 类中的 两个相关变量
threadLocals
inheritableThreadLocals
用到这个变量
默认情况下这两个变量都是 null,只有当前线程调用 ThreadLocal 类的 set 或者 get 方法时才创建上面这两个变量,并且实际上调用的是 ThreadLocalMap 对应的 get() 和 set() 方法
最终的变量放在了当前线程的 ThreadLocalMap 中 ThreadLocal 只是一层封装而已
ThreadLocal 通过 Thread.currentThread() 获取到当前线程对象后,直接通过 getMap(Thread t) 可以访问到该线程的 ThreadLMap 对象
每个 Thread 中都具备一个 ThreadLocalMap,而 ThreadLocalMap存储 以 ThreadLocal 为 key, Object 对象为 value 的键值对。
在同一个线程中声明了多个 ThreadLocal 对象,但是最终使用的是同一个 ThreadLocalMap.
ThreadLocal 本身并不存储值,只是作为 key来让线程从 ThreadLocalMap 获取value
为什么不把 Thread 当做 Map 的 key
一个线程拥有多个变量
Map 不能一直膨胀
Map 元素的生命周期
多个私有变量对应多个 ThreadLocal 对象,那么对应的 map 体积不会太大,只要线程销毁了,那么 ThreadLocalMap 也会被销毁
Java 的 4 种引用
强引用
把一个对象赋给一个引用变量,这个对象就是一个强引用
只要对象没有被置为 null,在 GC 时就不会被回收
软引用
想比于强引用,弱化了一点,需要继承 SoftReference 实现
如果内存不充足的话,只有软引用指向的对象就会被回收
弱引用
更弱一些,需要继承 WeakReference 实现
只要发生 GC,只有弱引用指向的对象就会被回收
虚引用
需要继承 PhantomReference
主要作用是 跟踪对象垃圾回收的状态,当回收时,通过引用队列做些通知类的工作
内存泄漏
什么是内存泄漏
申请完内存后,用完了但是没有释放掉,无法再次使用,但是系统又没法回收
先看一张引用图片
栈
堆
ThreadLocal Ref
Thread Ref
ThreadLocal
Thread
ThreadLocalMap
Entry
key
value
Object
ThreadLocal 内存泄漏
ThreadLocal 被回收了,但是 ThreadLocalMap Entry 的 key 没有了指向,但是 Entry 还有一条引用 ThreadRef → Thread → ThreadLocalMap → Entry value → Object 一直存在,也就是 出现了 一个 key 为 null 的 entry ,导致内存泄漏
内存泄漏的概率非常低
ThreadLocal 被两种引用指向
ThreadLocalRef → ThreadLocal
强引用
ThreadRef → Thread → ThreadLocalMap → Entry value → Object
弱引用
只要 ThreadLocal 使用时强引用没有被置为 null,那么 ThreadLocalMap Entry key 的指向就不会在 GC 断开时被回收,也没有内存泄漏这一说法
ThreadLocal的保护措施,操作时发现 key 为 null,会将其清除掉
如果在线程环境下,还会调用 ThreadLocal 的 set/get/remove 方法
发现 key 为 null 会进行清除,不会有长期的内存泄漏问题
长期内存泄漏的条件
ThreadLocal被回收
线程被复用
线程复用后不再调用ThreadLocal的set/get/remove方法
使用最佳实践
用完了,手动 remove 掉
为什么建议把ThreadLocal修饰为static?
ThreadLocal能实现了线程的数据隔离,不在于它自己本身,而在于Thread的ThreadLocalMap 所以,ThreadLocal可以只初始化一次,只分配一块存储空间就足以了,没必要作为成员变量多次被初始化。
为什么要将ThreadLocalMap的key设置为弱引用呢?
外界是通过ThreadLocal来对ThreadLocalMap进行操作的,假设外界使用ThreadLocal的对象被置null了,那ThreadLocalMap的强引用指向ThreadLocal也毫无意义
弱引用反而可以预防大多数内存泄漏的情况毕竟被回收后,下一次调用set/get/remove时ThreadLocal内部会清除掉
与 Spring 的联系
Spring 就是用 的 ThreadLocal 来实现的,ThreadLocal 类型存储的是一个 Map, key 是 DataSource,而 value 是 Connection 这 Map 的情况主要是为了应对多数据源。
用了ThreadLocal 保证了同一个线程获取一个 Connection 对象,从而保证了一次事务的所有操作需要在同一个数据库连接上。这样的话,就可以保证 一组操作同时成功或者同时失败。

Java工具包

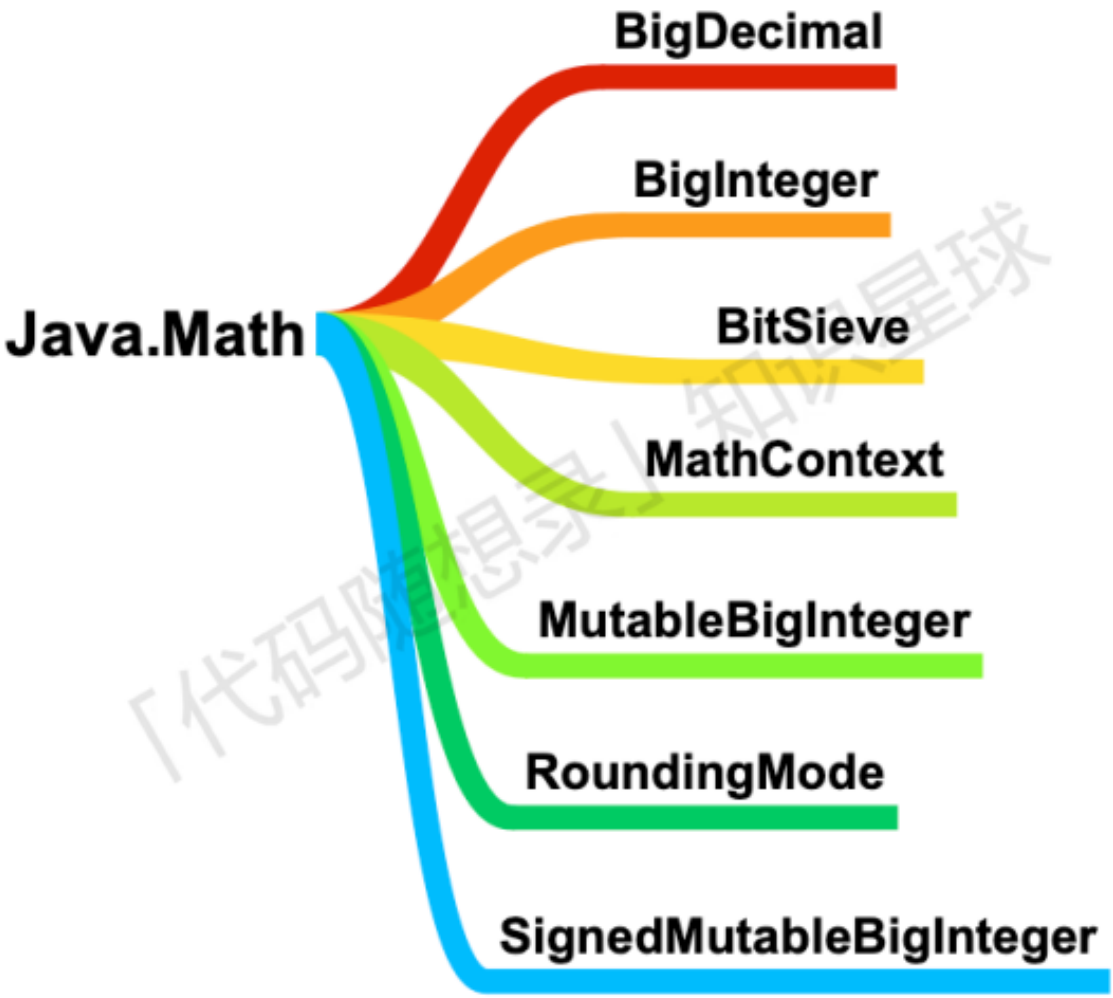
原图链接：「代码随想录」知识星球

(1) Java.IO

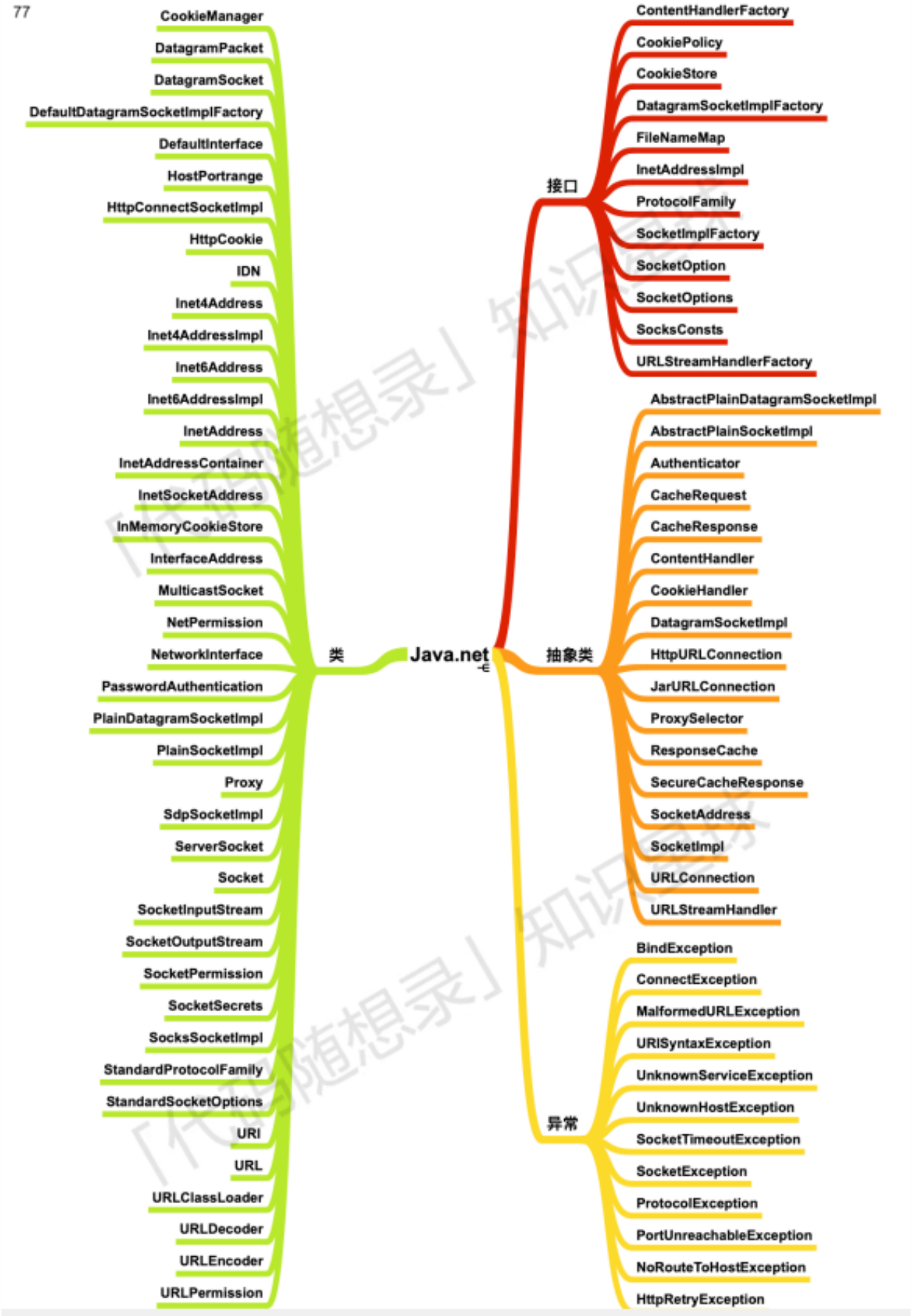
(2) Java.lang


Java.lang 包
接口
AutoCloseable
Annotation
CharSequence
Cloneable
Comparable
Iterable
Readable
Runnable
Type
InvocationHandler
GenericDeclaration
AnnotatedElement
Member
类型
基础数据类型
Boolean
Character
String
Number
1.Byte
2.Short
3.Integer
4.Long
5.Float
6.Double
Class 类
Package 包
其他类型
Object 对象祖宗
Enum 枚举
Void 无类型
工具类
字符序列
StringBuffer
StringBuilder
Math
反射
Array
Constructor
Field
Method
Proxy
Executable
AccessibleObject
系统
进程
Process
ProcessBuilder
ProcessEnvironment
ProcessImpl
线程
Thread
ThreadGroup
ThreadDeath
ThreadLocal
运行
Runnable
Runtime
RuntimePerssion
System
堆栈
StackTraceElement
异常
Throwable
Error
NoClassDefFoundError
NoSuchFieldError
OutOfMemoryError
NoSuchMethodError
Exception
ArrayIndexOutOfBoundsException
ClassCastException
ClassNotFoundException
IllegalArgumentException
IndexOutOfBoundsException
NoSuchFieldException
NoSuchMethodException
NullPointerException
NumberFormatException
RuntimeException
InterruptedException
注解
Deprecated
FunctionalInterface
Override
SafeVarargs
SuppressWarnings
Documented
Inherited
Native
Repeatable
Retention
Target

(3) Java.math

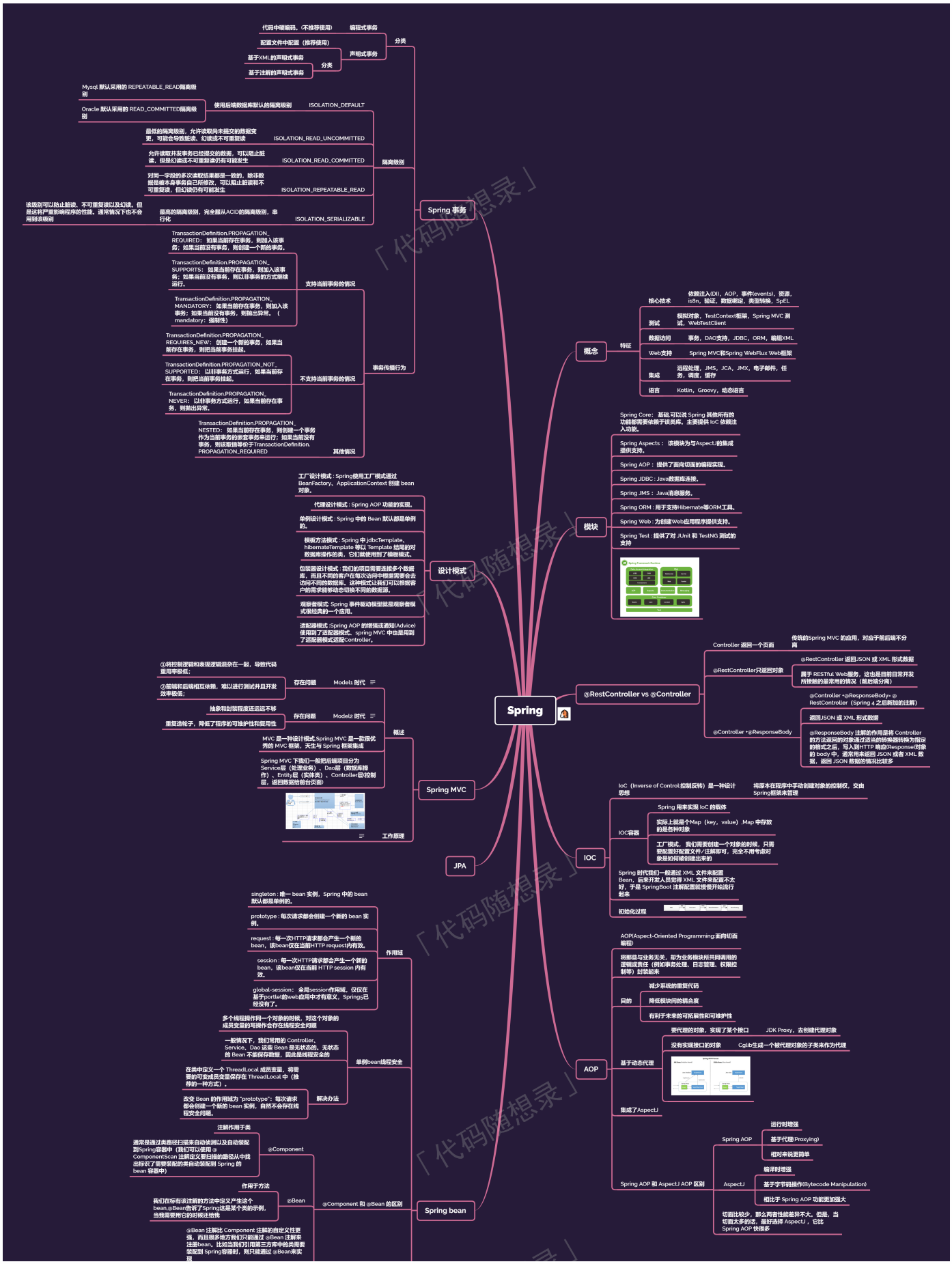

Java.Math
BigDecimal
BigInteger
BitSieve
MathContext
MutableBigInteger
RoundingMode
SignedMutableBigInteger

(4) Java.net


Java.net
类
CookieManager
DatagramPacket
DatagramSocket
DefaultDatagramSocketImplFactory
DefaultInterface
HostPortrange
HttpConnectSocketImpl
HttpCookie
IDN
Inet4Address
Inet4AddressImpl
Inet6Address
Inet6AddressImpl
InetAddress
InetAddressContainer
InetSocketAddress
InMemoryCookieStore
InterfaceAddress
MulticastSocket
NetPermission
NetworkInterface
PasswordAuthentication
PlainDatagramSocketImpl
PlainSocketImpl
Proxy
SdpSocketImpl
ServerSocket
Socket
SocketInputStream
SocketOutputStream
SocketPermission
SocketSecrets
SocksSocketImpl
StandardProtocolFamily
StandardSocketOptions
URI
URL
URLClassLoader
URLDecoder
URLEncoder
URLPermission
接口
ContentHandlerFactory
CookiePolicy
CookieStore
DatagramSocketImplFactory
FileNameMap
InetAddressImpl
ProtocolFamily
SocketImplFactory
SocketOption
SocketOptions
SocksConsts
URLStreamHandlerFactory
抽象类
AbstractPlainDatagramSocketImpl
AbstractPlainSocketImpl
Authenticator
CacheRequest
CacheResponse
ContentHandler
CookieHandler
DatagramSocketImpl
HttpURLConnection
JarURLConnection
ProxySelector
ResponseCache
SecureCacheResponse
SocketAddress
SocketImpl
URLConnection
URLStreamHandler
异常
BindException
ConnectException
MalformedURLException
URISyntaxException
UnknownServiceException
UnknownHostException
SocketTimeoutException
SocketException
ProtocolException
PortUnreachableException
NoRouteToHostException
HttpRetryException

Spring

原图链接：「代码随想录」知识星球



@Autowired 注解自动装配 bean

@Component：通用的注解，可标注任意类为 Spring 组件。如果一个Bean不知道属于哪个层，可以使用@Component 注解标注

@Repository：对应持久层即 Dao 层，主要用于数据库相关操作

@Service：对应服务层，主要涉及一些复杂的逻辑，需要用到 Dao层

@Controller：对应 Spring MVC 控制层，主要用于接受用户请求并调用 Service 层返回数据给前端页面

类声明为Spring的 bean 的注解

生命周期

「代码随想录